Software Requirement

Scope: SW-Requirements for B-Sample

Objective: to create a baseline for the B-Sample SW-Requirements

1 General . . . 6
1.1 General assumptions . . . 6
1.2 Software build information . . . 6
1.2.1 Software build information - build date . . . 6
1.2.2 Software build information - software version . . . 6
2 Heating . . . 6
2.1 General requirements . . . 6
2.2 Heating configuration . . . 7
2.3 Heating modes . . . 8
2.3.1 Heating Control - getCamControl . . . 8
2.3.2 Heating Control - setCamControl . . . 9
2.3.3 Heating Control - getDataBlock . . . 9
3 Shutter . . . 9
3.1 General Requirements . . . 9
3.2 Shutter configuration . . . 10
3.3 Shutter modes . . . 10
3.3.1 Shutter Control - getCamControl . . . 11
3.3.2 Shutter Control - setCamControl . . . 11
4 PMIC . . . 11
5 Ethernet . . . 12
5.1 Ethernet Configuration . . . 12
5.2 Remark / Note . . . 13
5.3 Client DHCP with selected FQDN features . . . 13
5.3.1 Dynamic Host Configuration Protocol (DHCP) Client . . . 13
5.3.2 Host Name Option . . . 14
5.3.3 Fully Qualified Domain Name (FQDN) Option . . . 14
5.3.4 DHCP address renewal (FORCE RENEW) . . . 14
5.4 Ethernet Config for gPTP Messages . . . 14
5.5 Ethernet MAC Requirements . . . 14
6 DHCP . . . 15
6.1 DHCP - Hostname . . . 15
6.2 DHCP - IpAddress . . . 15
7 SOME/IP . . . 16
7.1 General . . . 16
7.1.1 Network configuration . . . 16
7.1.2 Data structures . . . 16
7.1.3 Message validation . . . 17
7.2 Service Discovery . . . 18
7.2.1 Network configuration . . . 18
7.2.2 Startup behaviour . . . 19
7.2.2.1 Delay Values . . . 20
7.2.3 OfferService Message . . . 20
7.2.4 Eventgroup Messages . . . 22
7.3 Camera Service Methods . . . 23
7.3.1 Method getCamIdentification . . . 23
7.3.2 Method getDataSheet . . . 24
7.3.3 Method setCamID . . . 25
7.3.4 Method setCamExclusive . . . 25
7.3.5 Method eraseCamExclusive . . . 26
7.3.6 Method getCamControl . . . 26
7.3.7 Method setCamControl . . . 27
7.3.8 Method getDataBlock . . . 28
7.3.9 Method setDataBlock . . . 28
7.3.10 Method subscribeRoiVideo . . . 29
7.3.10.1 subscribeRoiVideo - Payload Validation . . . 29
7.3.11 Method UnSubscribeROIVideo . . . 30
7.3.11.1 unSubscribeRoiVideo - Payload Validation . . . 30
7.3.12 Method getCamStatus . . . 30
7.4 Software Deployment Methods . . . 31
7.4.1 Hardware Information Service Methods . . . 31
7.4.1.1 Method getId . . . 31
7.4.1.2 Method getHardwareIdentification . . . 31
7.4.2 Package Management Service . . . 32
7.4.2.1 Method CurrentStatus . . . 32
7.4.2.2 Method GetHistory . . . 32
7.4.2.3 Method GetId . . . 33
7.4.2.4 Method GetProgress . . . 33
7.4.2.5 Method GetSwClusterChangeInfo . . . 34
7.4.2.6 Method GetSwClusterInfo . . . 34
7.4.2.7 Method GetSwClusterManifestInfo . . . 35
7.4.2.8 Method GetSwPackages . . . 36
7.4.2.9 Method DeleteTransfer . . . 36
7.4.2.10 Method TransferStart . . . 37
7.4.2.11 Method TransferData . . . 37
7.4.2.12 Method TransferExit . . . 38
7.4.2.13 Method Cancel . . . 39
7.4.2.14 Method ProcessSwPackage . . . 40
7.4.2.15 Method RevertProcessedSwPackages . . . 41

7.4.2.16 Method Activate . . . 41
7.4.2.17 Method Rollback . . . 42
7.4.2.18 Method Finish . . . 42
7.4.3 Unsupported software deployment methods . . . 42
7.4.3.1 Method RegisterSoftwarePackage . . . 42
7.4.3.2 Method Resume . . . 42
7.4.3.3 Method Suspend . . . 43
8 Video streaming . . . 43
8.1 RTP . . . 43
8.2 RTP-Protocol . . . 44
8.3 Video stream/s . . . 45
8.4 Video Streaming Functions . . . 45
8.4.1 SOME/IP Subscription . . . 45
8.4.1.1 SubscribeROIVideo . . . 45
8.4.1.2 UnSubscribeROIVideo . . . 46
8.4.2 Variant Handling . . . 46
8.4.3 ROI Configuration . . . 46
8.4.4 Coordinate System Definition . . . 49
8.4.5 Flip / Mirror Functionality . . . 49
8.4.6 Aspect Ratio Conversion . . . 51
8.4.7 Zoom Functionality . . . 52
8.4.8 Pan Functionality . . . 52
9 GPTP . . . 53
9.1 gPTP Messages Handling . . . 53
9.1.1 Messages Filtering . . . 53
9.1.2 Messages Formatting . . . 53
9.2 Messages Error Handling . . . 55
9.3 Local Clocks Synchronisation . . . 56
9.4 Peer-to-peer (P2P) propagation delay algorithm . . . 57
9.5 Master/Slave Synchronization & Syntonization . . . 58
10 FSYNC . . . 59
10.1 Serdes Config . . . 60
10.2 Image Sensor Config . . . 60
10.3 Configuration and Initialization of secondary MAC . . . 60
10.4 Synchronization with gptp timebase . . . 60
10.5 Ethernet Control . . . 61
10.6 FSYNC general behavior . . . 61
11 Frame Tagging . . . 62
11.1 SEI Message in Video Stream . . . 62
11.2 User Data in SEI Payload . . . 63
12 External SPI Flash Memory . . . 66
12.1 Configuration of SPI-NOR controller driver . . . 66
13 NVM . . . 67
13.1 NVM - Production data block . . . 67
13.2 NVM - Software configuration block . . . 67
13.3 NVM - ImVer . . . 67
14 State Handling . . . 67
15 Camera Interface Setup . . . 68
15.1 Sensor Setup . . . 68
15.1.1 EEPROM Data Readout . . . 69
15.2 Sensor Functional Requirements . . . 69
15.3 SerDes Setup . . . 69
15.3.1 Power-Up . . . 70
15.3.2 Initialization - GMSL2 Link . . . 70
15.3.3 Configuration - MIPI CSI Lane Mapping . . . 71
15.3.4 Configuration - GPIO Forwarding . . . 71
15.3.5 Configuration - ADC . . . 72
15.3.6 Verification - MIPI Data Flow . . . 72
15.4 SerDes Functional Requirements . . . 72
15.5 VIN Setup . . . 74
15.5.1 VIN MIPI PHY Initialization . . . 74
15.5.2 VIN Configuration . . . 74
16 AE/AWB . . . 75
16.1 Auto Exposure . . . 75
16.2 Auto White Balance . . . 76
17 Encoder . . . 76
17.1 Static Encoder Config . . . 76
17.2 Dynamic Encoder Config . . . 79
18 Hardware Abstraction Layer (HAL) . . . 79
18.1 Microcontroller Interfaces . . . 79
19 Maintenance . . . 79
20 Non-testable items . . . 80
20.1 UART Format . . . 80
20.2 Liste an noch nicht testbaren SW-Requirements . . . 80
20.3 use case für den Einsatz mit dem Testinterface . . . 81
20.3.1 Manipulation der I2C SerDes-Daten . . . 81
20.3.1.1 Manipulation nach abgeschlossener Boot Phase . . . 81
20.3.1.2 Manipulation während / kurz nach der Boot Phase . . . 81
20.3.1.3 Bitweise Manipulation . . . 81
20.3.2 Auswerten der I2C PMIC-Daten . . . 81
20.3.3 Auswertung der gPTP-Daten . . . 81
20.3.4 Auswertung der MDIO-Daten zum ETH-PHY . . . 81
20.4 Definition von UART Zugriffen . . . 81
20.5 Ideensammlung für die Umsetzung . . . 81

21 Rejected items . . . 81
21.1 Reading from SPI Flash . . . 81
21.2 Writing to SPI Flash . . . 81
21.3 Erasing . . . 81
21.4 PMIC . . . 82
21.5 Heating . . . 82
21.5.1 Heating Modes . . . 82
21.6 Shutter . . . 83
21.6.1 Shutter modes . . . 83
21.7 SOME/IP . . . 84
21.7.1 General . . . 84
21.7.1.1 Data Structures . . . 84
21.7.2 Communication . . . 84
21.7.3 Service Discovery . . . 85
21.7.3.1 Startup Behaviour . . . 85
21.7.4 Camera service . . . 85
21.7.4.1 Startup behaviour . . . 85
21.7.4.2 Method getCamIdentification . . . 85
21.7.5 Method getCamStatus . . . 86
21.7.5.1 Method getDataSheet . . . 86
21.7.5.2 Method SubscribeROIVideo . . . 86
21.8 Video streaming . . . 87
21.8.1 Video stream/s . . . 87
21.8.2 Video Streaming Functions . . . 88
21.9 gPTP . . . 88
21.10 FSYNC . . . 89
21.10.1 Ethernet Control . . . 89
21.11 External SPI Flash Memor . . . 89
21.12 NVM . . . 91
21.12.1 NVM - Production data block . . . 91
21.12.2 NVM - Software configuration block . . . 92
22 Functional Safety Software Specifications . . . 93
22.1 Ambarella Chip SW Safety Requirements . . . 93
22.1.1 BIST TSRs . . . 93
22.1.2 Clock TSRs . . . 94
22.1.3 Power TSRs . . . 95
22.2 DRAM Safety Requirements . . . 95
22.3 Flash Safety Requirements . . . 96
22.4 Deserializer Safety Requirements . . . 96
22.4.1 Configuartion . . . 96
22.4.1.1 ERRB Signal . . . 96
22.4.1.2 VSYNC Signal . . . 97
22.4.1.3 I2C communication - primary channel . . . 97
22.4.1.4 Safety Mechanism Activation (ASIL B Scope) and Fault Identification . . . 98
22.4.1.4.1 SM3 - CRC for Control Channel . . . 98
22.4.1.4.2 SM4 - CRC for Forward Video Channel . . . 99
22.4.1.4.3 SM5 - Error Correction through Re-transmission on Control Channel . . . 99
22.4.1.4.4 SM6 - LOCK indicator of Forward Channel . . . 99
22.4.1.4.5 SM7 - ERRB indicator of Forward Channel . . . 99
22.4.1.4.6 SM15 - OV/UV comparators . . . 100
22.4.1.4.7 SM18 - Logic BIST . . . 100
22.4.1.4.8 SM19 - Memory BIST . . . 100
22.4.1.4.9 SM20 - End-to-end CRC during video transmission using CSI-2 forwarding . . . 100
22.4.1.4.10 SM21 - CRC on I2C and UART transactions . . . 100
22.4.1.4.11 SM22 - Message counter . . . 100
22.4.1.4.12 SM26 - Memory ECC . . . 100
22.4.1.4.13 SM27 - Configuration register CRC . . . 101
22.4.1.4.14 SM29 - CRC of non-volatile memory . . . 101
22.4.1.4.15 SM34 - DPLL lock detector . . . 101
22.4.2 Run-time behavior . . . 101
22.5 Ethernet PHY Safety Requirements . . . 102
22.5.1 Configuration . . . 102
22.5.2 Boot-Up . . . 103
22.5.3 Run Time . . . 103
22.5.4 Miscellaneous . . . 103
22.5.5 Requirements to Volvo . . . 103
22.5.6 Communication with PHY . . . 103
22.5.7 FuSa checks by PHY . . . 103
22.5.8 System reaction on FuSa error . . . 103
22.5.8.1 Communication with PHY . . . 103
22.6 PMIC Safety Requirements . . . 103
22.7 Safe State Requirements . . . 105
23 Cybersecurity Software Specifications . . . 105
23.1 Secure Communication (SOW2 - Vehicle ECU) . . . 105
23.1.1 Local Preparation . . . 105
23.1.1.1 PSK Loading . . . 105
23.1.1.2 Random Generation . . . 105
23.1.1.3 Ephemeral ECDHE Generation . . . 105
23.1.2 ClientHello Construction . . . 105
23.1.3 HelloRetryRequest . . . 106
23.1.4 ServerHello Validation . . . 106
23.1.5 Secret Derivation . . . 107
23.1.6 EncryptedExtensions . . . 107
23.1.7 Server Finished Verification . . . 107

23.1.8 Client Finished 107
23.1.9 Session Establishment 108
23.1.10 Application Data protection 108
23.1.10.1 Record Protection 108
23.1.10.2 Sequence Numbers 109
23.1.10.3 Anti-Replay 109
23.1.10.4 Record Size 109
23.1.10.5 Freshness Protection 109
23.1.11 DTLS Fragmentation 109
23.1.12 Error Handling and Security Logging 110
23.2 Secure Communication (SOW2 - SOW1) 110
23.2.1 Root Trust Initialization 110
23.2.2 Authentication Data Request 110
23.2.3 Device Authentication 111
23.2.4 Secure Session Establishment 111
23.2.4.1 Session Key Generation 111
23.2.4.2 Row Selection Generation 111
23.2.4.3 Session Key Protection 111
23.2.4.4 Key Sharing 112
23.2.5 Video Authentication 112
23.2.5.1 Authentication Tag Extraction 112
23.2.5.2 AES-CMAC Verification 112
23.2.5.3 Freshness Verification 113
23.2.6 Protected Control Commands 113
23.2.7 Failure Handling 113
23.3 Secure Debug 114
23.4 SPI Flash secure interface 116
23.5 DRAM secure interface 119
24 New SRS 120
24.1 Camera Module Management Feature 120
24.1.1 Scope 120
24.1.2 Functional Requirements 121
24.1.2.1 Configuration 121
24.1.2.1.1 Local De-serializer Configuration 121
24.1.2.1.2 Remote Serializer and camera components Configuration 121
24.1.2.2 Power-up Sequence 121
24.1.2.2.1 Power-UP Local De-Serializer 121
24.1.2.2.2 Power-UP remote Serializer (SOW1) 121
24.1.2.3 SerDes MIPI and MFPs Configuration 122
24.1.2.3.1 MIPI Lane Mapping and Polarity Configuration 122
24.1.2.3.2 Multi-Function Pins (MFPs) Forwarding Configuration 122
24.1.2.4 Serializer-Image Sensor Master Clock Configuration 123
24.1.2.5 Serializer-internal ADC configuration for temperature monitoring 123
24.1.2.6 Image Sensor Configuration 123
24.1.2.6.1 Image Sensor Initial Powerup and Configuration 123
24.1.2.6.2 Image Sensor Runtime Configuration Update 123
24.1.2.7 Main PMIC Initialization Configuration 123
24.1.3 Non-Functional Requirements 124
24.1.4 Interfaces 124
24.1.4.1 Hardware Interfaces 124
24.1.4.2 Input Interfaces 124
24.1.4.2.1 Output Interfaces 124
24.1.4.3 Software Interfaces 124
24.1.4.3.1 Input Interfaces 124
24.1.4.3.2 Output Interfaces 124
24.1.5 Configuration Parameters 124
24.2 Ethernet Communication Management Function 124
24.2.1 Ethernet Physical Layer Configuration Feature 124
24.2.1.1 Scope 124
24.2.1.2 Functional Requirements 124
24.2.1.2.1 PHY Configuration 124
24.2.1.2.2 MAC Cofiguration 125
24.2.1.3 Non-Functional Requirements 126
24.2.1.4 Interfaces 126
24.2.1.4.1 Software Interfaces 126
24.2.1.4.2 Hardware Interfaces 126
24.2.1.5 Ethernet UDP/IP Protocol Stack Handling Feature 127
24.2.1.5.1 Scope 127
24.2.1.5.2 Functional Requirements 127
24.2.1.5.3 Non-Functional Requirements 131
24.2.1.5.4 Interfaces 131
24.2.1.6 Ethernet RTP Streaming Handling Feature 132
24.2.1.6.1 Scope 132
24.2.1.6.2 Functional Requirements 132
24.2.1.6.3 RTP Protocol Serialization 133
24.2.1.6.4 SPS and PPS Payload Construction 134
24.2.1.6.5 SEI NAL Unit Framing & Header 135
24.2.1.6.6 Non-Functional Requirements 136
24.2.1.6.7 Interfaces 136
24.2.1.7 Ethernet Some-IP Protocol Management Feature 136
24.2.1.8 Scope 136
24.2.1.9 Functional Requirements 136
24.2.1.10 Non-Functional Requirements 136
24.2.1.11 Interfaces 136

24.2.1.11.1 Software Interfaces 136
24.2.1.11.2 Hardware Interfaces 136
25 Time Synchronization Feature 136
25.1 Scope 136
25.2 Functional Requirements 136
25.2.1 Network Clock Alignment (gPTP protocol) 136
25.2.1.1 gPTP Operation Role 136
25.2.1.2 Propagation Delay Calculation 137
25.2.1.3 MAC0 - Grand Master Clock Synchronization 137
25.2.1.4 MAC0 - PHY Clock Synchronization 137
25.2.1.5 gPTP Global Time Base Synchronization 138
25.2.2 Frame and Camera Capture Synchronization - Fsync 138
25.2.2.1 FSync Setup 138
25.2.2.2 FSync Base Rasterization and Clock Alignment 138
25.2.2.3 Dynamic FSync Phase Adjustment 138
25.2.2.4 FSync Verification and Fallback Management 139
25.2.2.5 FSync Variant Handling - Analog Camera Variant 139
25.3 Non-Functional Requirements 139
25.4 Interfaces 139
25.4.1.1 Software Interfaces 139
25.4.1.1.1 Input Interfaces 139
25.4.1.1.2 Output Interfaces 139
25.4.1.2 Hardware Interfaces 139
25.4.1.2.1 Input Interfaces 139
25.4.1.2.2 Output Interfaces 139
26 Video and Image Processing Function 139
26.1 Video Stream Tagging Feature 139
26.1.1 Scope 139
26.1.2 Functional Requirements 140
26.1.2.1 SEI NAL Unit structure in H.264 Stream 140
26.1.2.1.1 SEI NAL Unit Wrapper 140
26.1.2.1.2 SEI NAL Unit Trailing Byte 140
26.1.2.1.3 NAL Emulation Prevention 140
26.1.2.1.4 SEI Payload Structure and Placement 140
26.1.3 Non-Functional Requirements 141
26.1.4 Interfaces 141
26.1.4.1 Software Interfaces 141
26.1.4.1.1 Input Interfaces 141
26.1.4.1.2 Output Interfaces 142
26.1.4.2 Hardware Interfaces 142
26.1.4.2.1 Input Interfaces 142
26.1.4.2.2 Output Interfaces 142
26.2 Video Stream Management Feature 142
26.2.1 Scope 142
26.2.2 Functional Requirements 142
26.2.3 Non-Functional Requirements 142
26.2.4 Interfaces 142
26.2.4.1 Software Interfaces 142
26.2.4.1.1 Input Interfaces 142
26.2.4.1.2 Output Interfaces 142
26.2.4.2 Hardware Interfaces 142
26.2.4.2.1 Input Interfaces 142
26.2.4.2.2 Output Interfaces 142
26.2.5 Configuration Parameters 142
26.3 Image Processing Feature 142
26.3.1 Scope 142
26.3.2 Functional Requirements 142
26.3.2.1 Analog Camera Variant 142
26.3.2.2 Digital Camera Variant 142
26.3.3 Non-Functional Requirements 143
26.3.4 Interfaces 143
26.3.4.1 Software Interfaces 143
26.3.4.1.1 Input Interfaces 143
26.3.4.1.2 Output Interfaces 143
26.3.4.2 Hardware Interfaces 143
26.3.4.2.1 Input Interfaces 143
26.3.4.2.2 Output Interfaces 143
26.3.5 Configuration Parameters 143
26.4 Camera Calibration Management Feature 143
26.4.1 Scope 143
26.4.2 Functional Requirements 143
26.4.3 Non-Functional Requirements 143
26.4.4 Interfaces 143
26.4.4.1 Software Interfaces 143
26.4.4.1.1 Input Interfaces 143
26.4.4.1.2 Output Interfaces 144
26.4.4.2 Hardware Interfaces 144
26.4.4.2.1 Input Interfaces 144
26.4.4.2.2 Output Interfaces 144
26.4.5 Configuration Parameters 144
27 System Diagnostic Feature 144
27.1 Scope 144
27.2 Functional Requirements 144
27.3 Non-Functional Requirements 144

27.4 Interfaces . . . 144
27.4.1 Software Interfaces . . . 144
27.4.1.1 Input Interfaces . . . 144
27.4.1.2 Output Interfaces . . . 144
27.4.2 Hardware Interfaces . . . 144
27.4.2.1 Input Interfaces . . . 144
27.4.2.2 Output Interfaces . . . 144
27.5 Impact analysis on the operating environment . . . 144

1 General

1.1 General assumptions

Correctness of user configuration data

Where standard-compliant uC SW provides freedom of configuration to the user (i.e., MEKRA or the OEM), it is assumed that the user provides only correct (valid and verified) configuration data to the uC SW. Potentially safety-related responsibility of the uC SW regarding integrity checks remains unaffected by this assumption.

In case keywords are used which are not explained check at System allocation level at chapter MVC-92639 - Abbreviations and Definitions for an explanation.

1.2 Software build information

1.2.1 Software build information - build date

MVC-95850 - Software build information - build date

The application software shall use the build date based on ISO8601 in YYYY-MM-DDTHH:MM±HH:MM format (22 bytes) and provide this data in payload of camera service `getDataSheet` inside `deviceIdentification` field in sub-field `software timestamp`.

String	Byte length	Description/Example
YYYY-MM-DD	10	2025-09-23
T	1	Time separator
HH:MM	5	15:53
±HH:MM	6	Timezone offset to UTC: +02:00

Status	Ready for review
Verification Criteria	A client requests a valid SOME/IP getDataSheet response and checks build date of release with payload

1.2.2 Software build information - software version

MVC-95851 - Software build information - software version

The application software shall use the software version in format "Release_X", with X being the identifier of the applicable HW+SW version with a length of up to 34 bytes, and provide this data in payload of camera service getDatasheet inside deviceIdentification field in sub-field software version.

String	Byte length	Description/Example
Release	7	Information
_	1	Separator
X	variable up to 34	HW+SW version identifier, e.g. "MVCU01I_SW04C7".

Note: All bytes following the null-terminator shall be set to 0x00.

Status	Ready for review
Verification Criteria	A client sends a valid SOME/IP getDatasheet REQUEST. In the positive RESPONSE, the software version entry of deviceIdentification starts with the prefix "Release_" and the part after the prefix is identical to the HW+SW version identifier of the software release under test. All bytes following the null-terminator are 0x00.

2 Heating

2.1 General requirements

MVC-92876 - Heating control - Turn on lens heating

The application software shall set the level of SoC pin V16/GPIO1 to HIGH in order to activate the lens heating.

Status	Approved
Verification Criteria	Check if pin GPIO1 is set high when the request of switching on lens heating is sent

MVC-92875 - Heating control - Turn off lens heating

The application software shall set the level of SoC pin V16/GPIO1 to LOW in order to deactivate the lens heating.

Status	Approved
Verification Criteria	Check if pin GPIO1 is set low when the request of switching off lens heating is sent

MVC-100022 - Heating Control - Lens heating change frequency

The application software shall not change the lens heating more often than once every second.

Status	Ready for review
Verification Criteria	Change lens heating during Manual Mode twice within one second. Check Heating pin for a timeframe of one second between the changes.

MVC-100021 - Heating Control - Imager temperature reading frequency

While in software state OPERATIONAL the application software shall read the current imager temperature once every second.

Status	Ready for review
Verification Criteria	During Automatic Mode increase the sensor temperature to T_imager_auto_off. After heating turns off decrease the temperature to T_imager_auto_on in less than a second. Ensure that heating turns back on after a second.

MVC-111167 - Heating Control - Serializer temperature reading frequency

While in software state OPERATIONAL the application software shall read the current Serializer temperature (T_Serializer) once every second.

Status	Ready for review
Verification Criteria	Record the I2C traffic and check that the serializer temperature is read at least once per second.

MVC-92878 - Heating control - Measure temperature

The application software shall determine the current imager temperature value by reading register `atopif_tmc_cal_top` (0x602C) of device imager (Address: 0x20) via I2C access signal `serDesI2c`.

Register Name	Address	Field Name	Access	Field Position	Value	Reset Value	Description
atopif_tmc_cal_top	0x602C	atopif_tmc_cal_top	RO	[13:0]	0~8191	0x0000	tmc calibration top [13:13] Sign bit [12:4] Integer bits [3:0] Fraction bits

Detailed information about I2C connection can be found in MVC-94043 - HSI - Camera Interface

Note: The temperature should be measured at least once every 10 sec.

Status	Approved

Verification Criteria	check if the temperature register can be correctly read via I2C line

MVC-92908 - Heating control - Calculate temperature value (t_image)

The application software shall evaluate the temperature sensor atopif_tmc_cal_top value as follows:

Bits	*Name*	*Value*
[13]	SignBit	• 1: negative (-1) • 0: positive (+1)
[12:4]	IntegerBits	
[3:0]	FractionBits	

Temperature = SignBit * IntegerBits + (FractionBits * 1/16)

Note: The value 1/16 is based on a calibration value from the sensor.

Example:
atopif_tmc_cal_top = 0x2185 => 1 | 0 0001 1000 | 0101
IntegerBits = 0x18 => 24
FractionBits = 0x05 => 5

*Temperature = (-1) * 24 + (5 * 1/16)* = -23.6875

Status	Specified
Verification Criteria	Review code and check if the correct physical temperature can be calculated from the raw sensor value. When we implement getCamStatus there will be a test. Review-Test can be rejected then.

MVC-100210 - Heating Control - Measure Serializer Temperature

The application software shall determine the current Serializer temperature (`T_Serializer`) value by converting the Die temperature read via MVC-100207 - SerDes - ADC - Die Temperature readout as follows:

- `T_EST_OUT` [9 bit] = concatenating `T_EST_OUT_B1` and `T_EST_OUT_B0`
- `T_Serializer` (in °C) = (`T_EST_OUT` / 2) - 273

Status	Ready for review
Verification Criteria	Send getDataBlock SOME/IP REQUEST Message with deviceId = 0x0002 and blockId = 0x0000, convert returned blockContent and compare with expected T_Serializer data

MVC-92907 - Heating control - Error handling for invalid register values

If the imager temperature register **`atopif_tmc_cal_top`** can not be read or provides an invalid value, the application software shall switch to protection state.

Note:

- Valid calculated temperature values are in a range between -51°C and 512°C.
- the register will yield following invalid values:
  - 0x2000, if the temperature is under -51°C
  - 0x3000, if the temperature is above 512°C

Status	Rejected
Verification Criteria	Check if SW's heater control enters protection state, when imager temperature register cannot be read or it provides invalid value.

MVC-92925 - Heating control - Temperature threshold values

When the application software reads one of the following values from the imager temperature register, it shall interpret T_imager as follows:

- 0x2000: T_imager is below -51 °C
- 0x3000: T_imager is above 512 °C

These values shall not be converted using the regular temperature conversion formula.

Status	Ready for review
Verification Criteria	Stimulate the register values 0x2000 and 0x3000 separately and verify that the application software interprets them according to the specified temperature ranges without applying the regular temperature conversion formula. In particular, verify that 0x2000 is not interpreted as 0 °C and 0x3000 is not interpreted as -256 °C.

2.2 Heating configuration

MVC-92886 - Heating control - Load configuration at Startup

The uC SW shall read the following configuration data from the software config data section of SPI flash memory at NVM Readout phase at SW state PERIPHERAL_CONFIGURATION:

- `T_imager_auto_on`
- `T_imager_auto_off`
- `T_imager_protection_on`
- `T_imager_protection_off`

Please refer to MVC-95861 - NVM software configuration block for detailed config data layout in NVM.
and refer to MVC-96111 - State Handling - Transition INIT to PERIPHERAL_CONFIGURATION as state handling for NVM readout

Status	Rejected
Verification Criteria	Check if the heater is correctly switched on/off at ModeAuto when the specified T-imager_auto_on/off is reached. Check if the heater correctly switched on/off at Protection State when the specified T-imager_protection_on/off is reached

MVC-99878 - Heating Control - Temperature Values

When entering software state OPERATIONAL, the application software shall use the following values from NVM SW Configuration block for Automatic and Protection mode checks:

- `T_imager_auto_on`
- `T_imager_auto_off`
- `T_imager_protection_on`
- `T_imager_protection_off`

Status	Ready for review
Verification Criteria	During Automatic Mode, check that the heater is switchen on/off according to the values read from NVM flash. Check that the heater is switched on/off via changing from Normal Operation to Protection Mode and vice versa according to the values read from NVM flash.

MVC-92885 - Heating control - Startup

When entering software state OPERATIONAL, the application software shall perform the following sequence:

1. Enable automatic mode
2. Lens heating shall remain off until SW first time reads out the imager temperature.

Status	Specified
Verification Criteria	Check if SW's heater control is in automatic mode after startup when imager temperature is below T_imager_protection_on. Check if heater is off (GPIO1 = LOW) during startup before the first read of temperature sensor even when imager temperature is below T_imager_auto_on.

MVC-98193 - Heating control - Stop

When leaving software state OPERATIONAL, the application software shall disable automatic mode and turn off the lens heating.

Status	Ready for review
Verification Criteria	When leaving OPERATIONAL, there shall be no read access to the imager temperator and lens heating shall be deactivated until entering OPERATIONAL again

MVC-96464 - Heating control - Startup - Read value between T_imager_auto_off and T_imager_auto_on

When the application software initially reads an imager temperature between T_imager_auto_off and T_imager_auto_on, the application software shall turn off lens heating.

Status	Specified
Verification Criteria	Checked via testinterface

MVC-92965 - Heating control - Configure SERDES GPIO forwarding

The uC SW shall configure the Deserializer GPIO5 (MFP5) pin for "GPIO Forwarding" to Serializer GPIO5 (MFP3) pin, such that the heating control pins are routed to the lens heating.

Note: uC pin GPIO1 is connected to GPIO5 of deserializer

Status	Rejected
Verification Criteria	check if the camera heater can be switched on/off by setting uC GPIO1 to high/low

2.3 Heating modes

MVC-92877 - Heating control - Operations

The application software heater control shall support the following operations:

- Normal operation
- Protection state

Status	Approved
Verification Criteria	Check if SW's heater state will transit between normal and protection states when specific temperature thresholds are reached.

MVC-92880 - Heating control - Normal operation

While in normal operation, the application software shall support the following modes:

- Manual Mode
- Automatic Mode

Status	Approved
Verification Criteria	Check if manual mode and automatic mode can be entered correctly by the specific request, when SW's heater state is in normal operation.

MVC-92879 - Heating control - Protection state

While in protection state, the application software shall turn off the lens heating.

Status	Approved
Verification Criteria	check if lens heating is turned off when SW's heater state entered protection state.

MVC-92882 - Heating control - Normal operation to protection state

While in normal operation mode, the application software shall switch heating control to protection state if either:

- `T_imager >= T_imager_protection_on`
- `T_serializer > T_ser_protection_on`

Note: `T_imager_protection_on` is read from MVC-95861 - NVM software configuration block . `T_ser_protection_on` shall be hard-coded to 40°C.

Status	Ready for review
Verification Criteria	Enter manual heating on, ensure both thresholds for imager and serializer work (heating gets turned off once thresholds are reached, checked for both independently).

MVC-92881 - Heating control - Protection state to normal operation

While in protection state, the application software shall switch heating control to normal operation if both:

- `T_imager <= T_imager_protection_off`
- `T_serializer <= T_ser_protection_off`

Note: `T_imager_protection_off` is read from MVC-95861 - NVM software configuration block . `T_ser_protection_off` shall be hard-coded to 35°C.

Status	Ready for review
Verification Criteria	Enter manual heating on and enter protection state, ensure both thresholds for imager and serializer work (heating gets turned on once temperature drops below both thresholds).

MVC-99835 - Heating Control - Heating in Manual Mode Off

When Manual Mode Off is set, the application software shall turn off the lens heating.

Status	Ready for review
Verification Criteria	Send setCamControl Message over SOME/IP to change state to Manual Mode Off, check for positive RESPONSE and heater turned off.

MVC-99836 - Heating Control - Heating in Manual Mode On

When Manual Mode On is set, the application software shall turn on the lens heating.

Status	Ready for review
Verification Criteria	Send setCamControl Message over SOME/IP to change state to Manual Mode On, check for positive RESPONSE and heater turned on.

MVC-92884 - Heating ModeAuto - Turn on condition

While in automatic mode, when T_imager <= T_imager_auto_on, the application software shall turn on the lens heating.

Note: `T_imager_auto_on` is read from Config Data in NVM acc. to MVC-95861 - NVM software configuration block

Status	Specified
Verification Criteria	Check if heater is turned on if imager temperature dropped below the specified threshold, when SW's heater state is automatic mode

MVC-92883 - Heating ModeAuto - Turn off condition

While in automatic mode, when `T_imager >= T_imager_auto_off`, the application software shall turn off the lens heating

Note: `T_imager_auto_off` is read from Config Data in NVM acc. to MVC-95861 - NVM software configuration block

Status	Specified
Verification Criteria	Check if heater is turned off if imager temperature rises above the specified threshold, when SW's heater state is automatic mode

2.3.1 Heating Control - getCamControl

MVC-99887 - Heating Control - getCamControl

When the application software receives a valid **getCamControl** SOME/IP `REQUEST` message with **camControlIndex** = 0x09, it shall return the following values for the payload:

- **controlSupportedMode** = 0x07 (`StatRW_A_nO`)
- **controlMode**:
  - 0x02 (ModeManual) during Manual Mode
  - 0x04 (ModeAuto) during Automatic Mode
- **controlValue**:
  - 0x0000 if heating is turned off
  - 0x0001 if heating is turned on

Note: During mode change via **setCamControl**, the 'currently set' mode (the mode set via **setCamControl**) might differ from the 'currently used' mode (the mode the software is currently in) for up to 1 second. Due to this discrepancy, the mode sent by **getCam Control** will be the 'currently set' mode.

Status	Ready for review
Verification Criteria	Set specific Heating Mode with SOME/IP setCamControl Message, send SOME/IP getCamControl Message and check if the correct values are returned.

MVC-99886 - Heating Control - getCamControl - Protection Mode

While in Protection Mode, when the application software receives a valid **getCamControl** SOME/IP `REQUEST` message with **camControlIndex** = 0x09, it shall use the **controlMode** value from Normal Operation Mode with **controlValue** = 0x0000 (Heating Off) as return values for the payload.

Status	Ready for review
Verification Criteria	Set specific Heating Mode with SOME/IP setCamControl Message and enter Protection Mode afterwards, send SOME/IP getCamControl Message and check if the correct values (current Mode + Heating Off) are returned.

2.3.2 Heating Control - setCamControl

MVC-108666 - Heating Control - setCamControl - Payload Validation - Supported controlMode

When the application software receives a valid **setCamControl** SOME/IP `REQUEST` message with **camControlIndex** = 0x09, it shall send a SOME/IP `ERROR` message with **Return Code** `E_INVALID_CONTROL_MODE (0x36)` and **no payload** if the **controlMode** field is ***not*** one of the following valid values:

- 0x02 (ModeManual)
- 0x03 (ModeManualModified)
- 0x04 (ModeAuto)

Status	Ready for review
Verification Criteria	Send valid setCamControl REQUEST with invalid control mode, check response message

MVC-108667 - Heating Control - setCamControl - Payload Validation - Supported controlValue

When the application software receives a valid **setCamControl** SOME/IP `REQUEST` message with **camControlIndex** = 0x09 and **controlMode** = 0x03, it shall send a SOME/IP `ERROR` message with **Return Code** `E_INVALID_CONTROL_VALUE (0x37)` and **no payload** if the **controlValue** field is ***not*** one of the following valid values:

- 0x00 (Heating Off)
- 0x01 (Heating On)

Status	Ready for review
Verification Criteria	Send valid setCamControl REQUEST with controlMode = 0x03 and invalid control value, check response message

MVC-108669 - Heating Control - setCamControl - ModeManual

When the application software receives a valid **setCamControl** SOME/IP `REQUEST` message with **camControlIndex** = 0x09 and **controlMode** = 0x02, it shall switch to **Manual Mode** without turning the lens heating on or off.

Status	Ready for review
Verification Criteria	Send setCamControl REQUEST message with given payload, ensure correct mode change via getCamControl RESPONSE message

MVC-108671 - Heating Control - setCamControl - ModeManualModified - Heating Off

While in normal operation, when the application software receives a valid **setCamControl** SOME/IP `REQUEST` message with **camControlIndex** = 0x09, **controlMode** = 0x03 and **controlValue** = 0x00, it shall switch to **Manual Mode Off**.

Status	Ready for review
Verification Criteria	Send setCamControl REQUEST message with given payload, ensure correct mode change via getCamControl RESPONSE message

MVC-108670 - Heating Control - setCamControl - ModeManualModified - Heating On

While in normal operation, when the application software receives a valid **setCamControl** SOME/IP `REQUEST` message with **camControlIndex** = 0x09, **controlMode** = 0x03 and **controlValue** = 0x01, it shall switch to **Manual Mode On**.

Status	Ready for review
Verification Criteria	Send setCamControl REQUEST message with given payload, ensure correct mode change via getCamControl RESPONSE message

MVC-108668 - Heating Control - setCamControl - ModeAuto

While in normal operation, when the application software receives a valid **setCamControl** SOME/IP `REQUEST` message with **camControlIndex** = 0x09 and **controlMode** = 0x04, it shall switch to **Automatic Mode**.

Status	Ready for review
Verification Criteria	Send setCamControl REQUEST message with given payload, ensure correct mode change via getCamControl RESPONSE message

MVC-116337 - Heating Control - setCamControl - Protection Mode

While in Protection Mode, when the application software receives a valid **setCamControl** SOME/IP `REQUEST` message, it shall process it normally, so that the mode will be updated after Protection Mode has been exited.

Status	Ready for review
Verification Criteria	Send setCamControl REQUEST message while Protection Mode is enabled with values that change the Heating Mode, exit Protection Mode and ensure correct mode change via getCamControl RESPONSE message

2.3.3 Heating Control - getDataBlock

MVC-102679 - Heating Control - getDataBlock

When the application software receives a valid **getDataBlock** SOME/IP `REQUEST` message with **deviceId =** 0x0002 and **blockId =** 0x0000, it shall return the following values for the payload:

- **blockSize =** 0x000000002
- **blockData =**Serializer temperature `T_Serializer`

Status	Ready for review
Verification Criteria	Send getDataBlock SOME/IP REQUEST Message with deviceId = 0x0002 and blockId = 0x0000, expect SOME/IP RESPONSE Message with blockSize = 0x00000002 and blockData: 16 Bits (2 Bytes) of data

MVC-102680 - Heating Control - getDataBlock - Serializer temperature conversion

The application software shall convert the serializer temperature `T_Serializer` as described in either ISO 17215-3:2014 Chapter 6.6.25 or ISO 17215-3:2021 Chapter 7.6.25 to 16 Bit

Status	Ready for review
Verification Criteria	Send getDataBlock SOME/IP REQUEST Message with deviceId = 0x0002 and blockId = 0x0000, convert returned blockContent and compare with expected T_Serializer data

3 Shutter

3.1 General Requirements

MVC-92048 - Shutter control - Invalid input voltage

If the input voltage U_in is not in the range between Ub_min and Ub_max then the uC SW shall set the shutter to the configured default state without ensuring successful operation.

Status	Rejected
Verification Criteria	

MVC-92026 - Shutter control - Open the shutter

The application software shall set the level of the pin `C14 (GPIO52)` to `LOW`, in order to open the shutter.

Status	Ready for review
Verification Criteria	Check if the pin C14 (GPIO52) is set to LOW when open shutter request is sent. This can be performed by the command "gpio show 52".

MVC-92029 - Shutter control - Close the shutter

The application software shall set the level of the pin `C14 (GPIO52)` to `HIGH`, in order to close the shutter.

Status	Ready for review
Verification Criteria	Check if pin the pin C14 (GPIO52) is set to HIGH when close shutter request is sent. This can be performed by the command "gpio show 52".

MVC-92028 - Shutter control - Operation voltage

While the input voltage U_in is within the range Ub_min and Ub_max, the uC SW shall start the shutter functionality.

Status	Rejected
Verification Criteria	

3.2 Shutter configuration

MVC-92032 - Shutter control - Load configuration

When entering software state PERIPHERAL_CONFIGURATION, the software shall read from flash memory its configuration about shutter behavior according to parameter `shutterSupported`.
Note:
refer to MVC-95861 ~ NVM software configuration block for NVM configuration data layout.
refer to MVC-96111 - State Handling - Transition INIT to PERIPHERAL_CONFIGURATION for state machine handling.

Status	Rejected
Verification Criteria	Check if manipulation of flash entry leads to expected behaviour of shutter enable pin shutterEn

MVC-92030 - Shutter control - Support of the shutter control functionality

If the flag `shutterSupported` is set to `1`, the uC SW shall enable the shutter control functionality.
else, the uC SW shall disable the shutter control functionality.

Status	Rejected
Verification Criteria	Check if the manipulation of shutterSupported will lead to expected controllability of the shutter

MVC-92727 - Shutter control - Power supply

Where the flag `shutterSupported` is set to `1`, the application software shall set the level of the pin `C13/GPIO55` to LOW to enable the power supply of the external shutter module.

Note: for detailed information about the power supply pin, please refer to MVC-93509 - HSI - Shutter Interface

Status	Specified
Verification Criteria	Check if the manipulation of shutterSupported will lead to expected level of pin POWER_SHUT_EN

MVC-92034 - Shutter control - Startup functionality

Where the flag `shutterSupported` is set to 1 and when entering software state OPERATIONAL, the application software shall perform the following steps:

- Close the shutter.
- Enable the automatic mode.

Status	Ready for review
Verification Criteria	Check if shutter pin is high and if the control mode is automatic mode after SW startup

MVC-98289 - Shutter control - Startup functionality with unsupported shutter

Where the flag `shutterSupported` is set 0, the application software shall perform the following steps:

- Set the level of the pin `C13/GPIO55` to HIGH
- Set the level of the pin `C14/GPIO52` to HIGH

Status	Approved
Verification Criteria	Check if shutter power pin is high and shutter ctrl pin is high when in the NVM Software Configuration block shutterSupported is set to 0.

MVC-98194 - Shutter control - Stop

When leaving software state OPERATIONAL, the application software shall disable automatic mode.

Status	Ready for review
Verification Criteria	When leaving OPERATIONAL, observe pin C14 (GPIO52) shall be and remain HIGH until entering OPERATIONAL

MVC-92033 - Shutter control - Load operation modes

The uC SW shall read the operation control modes from the external SPI flash memory.

Status	Rejected
Verification Criteria	

MVC-92036 - Shutter control - Operation modes

The uC SW shall support the following operation modes:

- Automatic mode
- Manual mode

Status	Ready for review
Verification Criteria	Send SOME/IP messages and see if shutter reacts, ensure correct mode change via getCamControl RESPONSE message

MVC-92035 - Shutter control - Operation modes - SOME/IP Mapping

The uC SW shall map the operation modes to the SOME/IP control modes accordingly:

Operation mode	SOME/IP control mode
Manual mode	`ModeManual 0x02`
Manual mode	`ModeManualModified 0x03`
Automatic mode	`ModeAuto 0x04`

Status	Ready for review
Verification Criteria	Send SOME/IP messages and see if shutter reacts, ensure correct mode change via getCamControl RESPONSE message

3.3 Shutter modes

MVC-92027 - Shutter control - Default state

The uC SW shall set the default shutter control mode to automatic mode.

Status	Rejected
Verification Criteria	If the shutter is opened (powered off), then the shutter shall be closed after startup.

MVC-92042 - Shutter control - Operation mode - Automatic mode while streaming

Where `shutterSupported` is set to 1, the application software shall open the shutter,

- when any video stream is successfully subscribed and
- while in automatic mode.

Note: please refer to following requirements for successful subscribing video:

- MVC-95729 - SubscribeROIVideo - Positive Response - no camera stream/s running
- ~~MVC-96359~~ - SubscribeROIVideo - Positive Response - camera stream/s running - restart of stream

Status	Specified
Verification Criteria	Set the shutter control state to automatic mode and establish at least one video stream, then pin C14 (GPIO52) shall be LOW (shutter open)

MVC-92041 - Shutter control - Operation mode - Automatic mode while not streaming

Where `shutterSupported` is set to 1, the application software shall close the shutter,

- when both video streams are successfully unsubscribed and
- while in automatic mode.

Note: please refer to following requirements for successful unsubscribing video:

- MVC-96410 - UnSubscribeROIVideo - Positive response - camera stream/s running - stop of stream
- MVC-95808 - UnSubscribeROIVideo - Positive Response - no camera stream/s running

Status	Specified
Verification Criteria	Set the shutter control state to automatic mode and close all video stream, then pin C14 (GPIO52) shall be HIGH (shutter close). This can be performed by the command "gpio show 52".

3.3.1 Shutter Control - getCamControl

MVC-92047 - Shutter Control - getCamControl

When the application software receives a valid **getCamControl** SOME/IP `REQUEST` message with **camControlIndex** = 0x50, it shall return the following values for the payload:

- **controlSupportedMode** = 0x07 (`StatRW_A_nO`)
- **controlMode**:
  - 0x02 (ModeManual) during Manual Mode
  - 0x04 (ModeAuto) during Automatic Mode
- **controlValue**:
  - 0x0000 if shutter is closed
  - 0x0001 if shutter is open

Status	Specified
Verification Criteria	Send valid getCamControl REQUEST messages for all described cases and check each RESPONSE message

MVC-92730 - Shutter control - SOME/IP getCamControl message - Error response message

If the uC SW receives a SOME/IP getCamControl message with an `camControlIndex` that is out of range or not supported then the uC SW shall send an error response message with the error code `E_INVALID_CONTROL_INDEX`.

Status	Rejected
Verification Criteria	Send a SOME/IP getCamControl message with an camControlIndex that is out of range or not supported and check if an error response message with the error code E_INVALID_CONTROL_INDEX was returned.

3.3.2 Shutter Control - setCamControl

MVC-92731 - Shutter control - setCamControl - Payload Validation - Supported controlMode

When the application software receives a valid **setCamControl** SOME/IP `REQUEST` message with **camControlIndex** = 0x50, it shall send a SOME/IP `ERROR` message with **Return Code** `E_INVALID_CONTROL_MODE (0x36)` and **no payload** if the **controlM ode** field is ***not*** one of the following valid values:

- 0x02 (ModeManual)
- 0x03 (ModeManualModified)
- 0x04 (ModeAuto)

Status	Specified
Verification Criteria	Send valid setCamControl REQUEST with invalid control mode, check response message

MVC-92733 - Shutter control - setCamControl - Payload Validation - Supported controlValue

When the application software receives a valid **setCamControl** SOME/IP `REQUEST` message with **camControlIndex** = 0x50 and **controlMode** = 0x03, it shall send a SOME/IP `ERROR` message with **Return Code** `E_INVALID_CONTROL_VALUE (0x37)` and **no payload** if the **controlValue** field is ***not*** one of the following valid values:

- 0x00 (Shutter closed)
- 0x01 (Shutter open)

Status	Specified
Verification Criteria	Send valid setCamControl REQUEST with controlMode = 0x03 and invalid control value, check response message

MVC-92043 - Shutter Control - setCamControl - ModeManual

When the application software receives a valid **setCamControl** SOME/IP `REQUEST` message with **camControlIndex** = 0x50 and **controlMode** = 0x02, it shall switch to **Manual Mode** without changing the Shutter State.

Status	Specified
Verification Criteria	Send setCamControl REQUEST message with given payload, ensure correct mode change via getCamControl RESPONSE message and check if the shutter control pin remains in the same state as before.

MVC-92040 - Shutter Control - setCamControl - ModeManualModified - Closed Shutter

When the application software receives a valid **setCamControl** SOME/IP `REQUEST` message with **camControlIndex** = 0x50, **controlMode** = 0x03 and **controlValue** = 0x00, it shall switch to **Manual Mode** and close the shutter.

Status	Specified
Verification Criteria	Send setCamControl REQUEST message with given payload, ensure correct mode change via getCamControl RESPONSE message and check shutter control pin

MVC-92039 - Shutter Control - setCamControl - ModeManualModified - Open Shutter

When the application software receives a valid **setCamControl** SOME/IP `REQUEST` message with **camControlIndex** = 0x50, **controlMode** = 0x03 and **controlValue** = 0x01, it shall switch to **Manual Mode** and open the shutter.

Status	Specified
Verification Criteria	Send setCamControl REQUEST message with given payload, ensure correct mode change via getCamControl RESPONSE message and check shutter control pin

MVC-92038 - Shutter Control - setCamControl - ModeAuto

When the application software receives a valid **setCamControl** SOME/IP `REQUEST` message with **camControlIndex** = 0x50 and **controlMode** = 0x04, it shall switch to **Automatic Mode**.

Status	Specified
Verification Criteria	Send setCamControl REQUEST message with given payload, ensure correct mode change via getCamControl RESPONSE message

4 PMIC

MVC-97843 - PMIC I2C Bus Configuration

The application software shall configure the used I²C channel (see MVC-93518 - HSI - PMIC interface) to `I2C_SPEED_STANDARD`.

Status	Specified
Verification Criteria	Check I2C Trace that PMIC communication is performed in specified speed mode (see MVC-93518)

MVC-97842 - PMIC I2C Device Address

The application software shall use `PMIC_I2C_SLAVEID 0x06`.
Note:
0x03 corresponds to 0x06 in 8-bit shifted format
([7-bit address][R/W bit] -> 0x03 << 1 = 0x06)

Status	Specified
Verification Criteria	Check I2C Trace, if the PMIC is addressed with slave ID 0x06.

MVC-97845 - PMIC I2C Communication Channel

The application software shall use the `AMBA_I2C_CHANNEL4` to communicate.

Status	Specified
Verification Criteria	Check I2C Trace to see if the PMIC is communicating on the AMBA_I2C_CHANNEL4 bus (pins specified in MVC-93518).

MVC-97844 - PMIC Error Input Pin Configuration

The application software shall configure `GPIO20 (V14)` as output `(pmicErrorIn)`.

Note:

Due to the level shifter, the signal is inverted at the PMIC; "active" at the PMIC corresponds to LOW at the SoC pin.

In QA watchdog mode, this pin serves ERRIN (no periodic toggeling required).

Status	Specified
Verification Criteria	Check that GPIO20 (V14) is configured as output and that a LOW signal at SoC side corresponds to an active PMIC error input state.

MVC-96166 - PMIC Configuration

When entering software state INIT, the Application Software shall configure the watchdog functionality of the PMIC by setting register 0xB2 (WATCHDOG_CONFIG) to the the specified value (see MVC-95649 - PMIC- configuration of error input)

Status	Ready for review
Verification Criteria	Check I2C Trace

Note:

See p.97 MPQ70165FS-xxxx-AEC1 - Published Datasheet (MPS Rev#{3} 0.8) (002).pdf WATCHDOG_CONFIG (B2h)

At the moment the QA-watchdog shall stay disabled.

MVC-97847 - PMIC Write Register

The application software shall write data to a specified I^2C register of the PMIC by performing the following steps:

1. Prepare the I^2C transaction by specifying the PMIC Slave ID and the address of the register to write to.
2. Concatenate the register address with the data to be written into a transmission buffer.
3. Use the I^2C communication protocol (with the specified speed and channel) to send the write request, including the register address and the data, to the PMIC

Status	Specified
Verification Criteria	Check if write access to a known PMIC register is possible by observing the I2C trace during a specific SW state (e.g. WATCHDOG_CONFIG 0xB2 during INIT).

Note:

Register overview: see p.82 f. MPQ70165FS-xxxx-AEC1 - Published Datasheet (MPS Rev#{3} 0.8) (002).pdf REGISTER MAP

If an error occurs: see p.64 MPQ70165FS-xxxx-AEC1 - Published Datasheet (MPS Rev#{3} 0.8) (002).pdf Register Write Check

5 Ethernet

5.1 Ethernet Configuration

MVC-97820 - Ethernet MAC Address

The application software shall configure the Ethernet MAC layer to use the MAC address stored in the `macAddress` entry (from MVC-94698 - NVM Production Data Block Memory Map) for network communication over the PHY interface.

Status	Specified
Verification Criteria	Verify that the source MAC address of the transmitted SOME/IP packet matches the NVM parameter macAddress.

MVC-98192 - MAC Address Write

When entering the software state PERIPHERAL_CONFIGURATION, the application software shall write the MAC Address to the MAC at the Index mentioned in MVC-94136 - MAC configuration in the Registers **R16** and **R17**:

MAC Register	Content
MAC R16	MAC Address High
MAC R17	MAC Address Low

Status	Ready for review
Verification Criteria	Read MAC Registers R16 and R17 and check if the MAC Address from NVM is stored. Base 0xE000.E000 (MAC0), 0xE003.2000 (MAC1) offset 0x0040 (R16); 0x0044 (R17)

MVC-96408 - PHY RGMII-ID (RX) Configuration

When entering the software state PERIPHERAL_CONFIGURATION, the application software shall reconfigure the PHY via MDIO to operate in RGMII-ID (RX) mode by writing the following values to the specified MMD30 registers:

Register Address	Register Name	Value	Description
AC00h	PORT_INFRA_CONTROL	**4000h**	Enable configuration register access
AFC6h	MII_BASIC_CONFIG	**xxx7h**	Enable RGMII mode
AFCCh	RGMII_TXC_DELAY_CONFIG	**0012h**	Disable TX delay
AFCDh	RGMII_RXC_DELAY_CONFIG	**8012h**	Enable RX delay
AC00h	PORT_INFRA_CONTROL	**0000h**	Disable configuration register access

Note: In B-Sample hardware, the PHY is configured via pin strapping for RGMII-ID (TX and RX). This shall be corrected in the next hardware revision. Once corrected, this software requirement shall be removed.

Status	Specified
Verification Criteria	Using the AmbaShell command interface, read the corresponding MMD30 PHY registers via MDIO using: svc_net cl45read 30 [register_address]

MVC-96407 - PHY TC10 Enable Configuration

When entering the software state PERIPHERAL_CONFIGURATION, the application software shall configure the PHY via MDIO to enable TC10 functionality according to MVC-94058 - Phy configuration - TC10 by writing the following values to the specified MMD30 registers:

Register Address	Register Name	Value	Description
0040h	DEVICE_CONTROL	**2000h**	enable configuration of sleep control (no need to test)
0182h	WISE_CONFIG	**B000h**	enable sleep control
0040h	DEVICE_CONTROL	**0000h**	disable configuration of sleep control (no need to test)
0180h	WISE_CONTROL	**0000h**	allow the INH-pin to go LOW (disabling the power supplies for the SoC) if a successful Sleep handshake is established

Status	Specified
Verification Criteria	Using the AmbaShell command interface, read the corresponding MMD30 PHY registers via MDIO using: svc_net cl45read 30 [register_address].

MVC-96409 - PHY Identification and Link Speed Configuration

When entering the software state PERIPHERAL_CONFIGURATION, the application software shall read the PHY_ID_2 register via MDIO and set the MAC link speed according to MVC-94136 - MAC configuration

Status	Specified
Verification Criteria	Check MMD1 registers PMA_STATUS1 (1h): bit 7 = 0 and bit 2 = 1 → link up; PMA_CONTROL1 (0h): bit 13 = 0 and bit 6 = 1 → 1000 Mbps. Example using AmbaShell: svc_net cl45read 1 [register_address].

MVC-98184 - PHY Identification and Duplex Configuration

When entering the software state PERIPHERAL_CONFIGURATION, the application software shall set the MAC (at the Index mentioned in MVC-94136 - MAC configuration) duplex mode according to the PHY_ID_2 register read in MVC-96409 - PHY Identification and Link Speed Configuration :

PHY_ID_2	Duplex Mode
B013h (TJA1103/TJA1104)	Duplex Full
B030h (TJA1120/TJA1121)	Duplex Full

Status	Ready for review
Verification Criteria	Verify that the MAC uses Duplex Full Mode after exiting state PERIPHERAL_CONFIGURATION. Base 0xE000.E000 (MAC0), 0xE003.2000 (MAC1) offset 0x0000 (MAC_R0_MAC_config)

MVC-96189 - Ethernet - Monitoring of PHY Link Status

When the software enters the state PHYLINKCHECK, the application software shall monitor the PHY link status by polling the PHY_Status register (see MVC-92752 - HSI - get link status) at a fixed interval of 100 ms.

Status	Ready for review
Verification Criteria	disconnect and reconnect, then check MMD1 registers PMA_STATUS1 (0x1): bit 7 = 0 and bit 2 = 1 → link upExample using AmbaShell: svc_net cl45read 1 [register_address].

MVC-99329 - Ethernet - Communicate an invalid link status
When the software remains in the state PHYLINKCHECK for longer than 5 seconds, the application software shall communicate an invalid link status via the debug interface without stopping the polling of the link status.

Remark: This requirement is intended solely for debugging and research purposes and shall not be verified by the test team.

Status	Draft
Verification Criteria	

5.2 Remark / Note

This requirement is intended solely for debugging and research purposes and shall not be verified by the test team.

5.3 Client DHCP with selected FQDN features

Note: For the B-Sample, only standard DHCP is required (cf. ~~MVC-91225~~ - [AlloReqs] Ethernet Comm. - UDP/IP: Standard DHCP). Therfore, these subsections do not yet contain SW requirement workitems regarding: 5.3.2 - Host Name Option , 5.3.3 - Fully Qualified Domain Name (FQDN) Option , 5.3.4 - DHCP address renewal (FORCE RENEW) (Reminder: MVC-89193).

Objectives:

- Fast streaming ability after connection/startup of the Camera Module and the MEKRA Vision Controller (MVC).
- To keep negative impact of a changed IP address on a running stream as low as possible (i.e., at best no interruption or delay of a running stream).

Known constraints:

- MVC has no battery => powerless MVC cannot properly observe DHCP lease time until MVC is re-attached and timesynced (truck could be a different one).

Assumptions:

DHCP client 'chaddr' and (optional) 'client identifier'
The system level defines a value as 'chaddr' in the sense of IETF RFC 2131.
The system level may additionally define a value as 'client identifier' in the sense of IETF RFC 2131.
Note: Among others, section 4.2 of IETF RFC 2131 requires *"If the client supplies a 'client identifier', the client MUST use the same 'client identifier' in all subsequent messages, and the server MUST use that identifier to identify the client. If the client does not provide a 'client identifier' option, the server MUST use the contents of the 'chaddr' field to identify the client."*.

System interface for configuration of DHCP
The system level defines a configuration interface for the uC SW. This configuration interface allows the definition of the system's *host name* and *FQDN*. In addition, this configuration interface allows the activation/deactivation of each of the related DHCP options.

User responsibility for correctness of configuration
Where standard-compliant uC SW provides freedom of configuration to the user (i.e., MEKRA or the OEM), it is assumed that the user provides only correct (valid and verified) configuration data to the uC SW. Potential safety-related responsibility of the uC SW regarding integrity checks remains unaffected by this assumption.

Persistence of settings acknowledged by DHCP server
The system-level specifies: When the system receives an acknowledge of the DHCP server allowing the system to apply certain settings, then the system shall store the acknowledged settings in NVM. This includes the settings IPv4-address, subnet-mask, granted lease time. Until a setting is overwritten for the first time, its default value shall be: IPv4-address := 0.0.0.0, subnet-mask := 255.255.255.0, granted lease time := 28800 seconds (= 8 hours).
Note: The objective is, to keep the relevant settings available (even after reboot) for a potentential reuse proposal via DHCPREQUEST.

5.3.1 Dynamic Host Configuration Protocol (DHCP) Client

Note - DHCP component: DHCP was specified in the year 1997 (cf. IETF RFC 2131). Today several qualified DHCP stacks are available. Following a modular approach, we should rather try to integrate a qualified stack than developing our own from scratch.

Note - DCHP port numbers: *"DHCP uses UDP as its transport protocol. DHCP messages from a client to a server are sent to the 'DHCP server' port (67), and DHCP messages from a server to a client are sent to the 'DHCP client' port (68). A server with multiple network address (e.g., a multi-homed host) MAY use any of its network addresses in outgoing DHCP messages."*
according to section "4.1 Constructing and sending DHCP messages", IETF RFC 2131.

MVC-92839 - DHCP client
The uC SW shall implement the *Dynamic Host Configuration Protocol* (DHCP) defined in IETF RFC 2131, restricted to the client-part in an IPv4 network.
Note: MVC-90769 and MVC-89190 - The client part of DHCPv4 as specified by IETF RFC 2131 (Dynamic Host Configurat... .

Status	Draft
Verification Criteria	

MVC-92840 - DHCP timeouts, retransmission delays and number of retransmissions
The uC SW shall apply proper timeout handling, retransmission delays and number of retransmissions. This may include configuration data.

Note: See section "4.1 Constructing and sending DHCP messages", IETF RFC 2131. The chosen DHCP implementation needs to support this approach. The following requirements are meant as **proposal**:

PR1:
When the uC SW initiates an interaction sequence with the DHCP server, the uC SW shall apply the timeout (4 + pseudorand({-1;0;1})) seconds (i.e, 4 seconds plus a pseudo-randomly selected value out of {-1;0;1} seconds) for this interaction sequence.
Note: See section "4.1 Constructing and sending DHCP messages", IETF RFC 2131.

PR2:
In the following, a client-server interaction sequence is considered "complete" if the client has sent and received all messages belonging to the respecitive message sequence (e.g., from DHCPDISCOVER to DHCPACK res. DHCPNAK).
If a client-server interaction sequence had been initiated by the uC SW and wasn't completed within its respective timeout, then the uC SW shall ignore the messages of that interaction and either:
i) start retransmission of the interaction sequence with timeout (8 + pseudorand({-1;0;1})) seconds, if this is the first retransmission;
ii) start retransmission of the interaction sequence with timeout (16 + pseudorand({-1;0;1})) seconds, if this is the second retransmission;
iii) start retransmission of the interaction sequence with timeout (32 + pseudorand({-1;0;1})) seconds, if this is the third retransmission;
iv) raise an error as follows: <**TODO**: in accordance with error handling and reporting requirements>
Note: The above specifies a "retransmission strategy that incorporates a randomized exponential backoff algorithm to determine the delay between retransmissions". Latter is a MUST requirement according to section "4.1 Constructing and sending DHCP messages", IETF RFC 2131. However, in case the network speed is faster than 10 Mbit/s, the delays could be decreased respectively.

Status	Draft
Verification Criteria	

MVC-92866 - Request new DHCP binding
In case the persisted IP-address equals 0.0.0.0 when the system state is IPCONFIG, then the uC SW shall request a new binding from the DHCP server in accordance with subsection "3.1 Client-server interaction - allocating a network address", IETF RFC 2131.

Status	Rejected
Verification Criteria	

MVC-92867 - Request DHCP binding with reuse
In case the persisted IP-address doesn't equal 0.0.0.0 when the system state is IPCONFIG, then the uC SW shall request a reuse from the DHCP server, propsing the persisted settings in accordance with subsection "3.2 Client-server interaction - reusing a previously allocated network address", IETF RFC 2131.
Note: Indirectly, this defines also the behaviour of a system with a granted lease after reboot, because a reuse is requested regardless of a potentially remaining lease time.
Note: A FORCE RENEW as requested by MVC-89189 - DHCP address renewal (FORCE RENEW) as specified by IETF RFC 3203 but without DHCP could be implemented by a reset of the persisted IP-address to 0.0.0.0 followed by a transition to system state IPCONFIG.

Status	Rejected
Verification Criteria	

MVC-92841 - Try to prolong lease in a good time
While the system state is STANDBY or RUNNING: When the remaining lease time for its current DHCP binding is less than one hundredth (i.e., < 1%) of the lease time granted by the DHCP server, but at least if it gets below 70 seconds, the uC SW shall request a reuse for the granted lease time from the DHCP server.

Status	Rejected

Verification Criteria	

5.3.2 Host Name Option

5.3.3 Fully Qualified Domain Name (FQDN) Option

Note - no multiple options: Section 2 in IETF RFC 4720 refers to IETF RFC 3396 when it allows: *"Only one Client FQDN option MAY appear in a message, though it may be instantiated in a message as multiple options [9]. DHCP clients and servers supporting this option MUST implement DHCP option concatenation [9].".* Please, note that the uC SW is not intended to support "multiple options" in accordance with IETF RFC 3396, as the system is currently only required to support the *host name option* (cf. MVC-89188).

Note - sending *client FQDN option* forbids to send *host name option*: Regarding the *host name option* (section 3.1, IETF RFC 2132) and the *client FQDN option* (sections 2ff, IETF RFC 4702), please note *"The Host Name option [12], for example, contains an ASCII string representation of the client's host name. In general, a client does not need to send redundant data, and therefore clients that send the Client FQDN option in their messages MUST NOT also send the Host Name option."* under section 3.1, IETF RFC 4702.

Proposed for successor of B-Sample:

When uC SW sends a DHCP message containing the "Client FQDN option", it should set the two 1-octet fields RCODE1 and RCODE2 to 0.
Note: According to section "2.2. The RCODE Fields", IETF RFC 4702, both RCODE fields are depricated.
When uC SW receives a DHCP message containing the "Client FQDN option", it should ignore the two 1-octet fields RCODE1 and RCODE2.
Note: According to section "2.2. The RCODE Fields", IETF RFC 4702, both RCODE fields are depricated.

5.3.4 DHCP address renewal (FORCE RENEW)

5.4 Ethernet Config for gPTP Messages

MVC-105122 - gPTP Mac Destination Address

SOW2 SW shall only use the MAC broadcast destination address: (01:80:C2:00:00:0E) to send and receive GPTP messages.

Status	Draft
Verification Criteria	

MVC-105124 - gPTP Tx Messages Filtering

The uC SW shall be able to send following gPTP message:

Message Name	Message Type	Description
PDelay_Req	0x02	peer-to-peer delay request message, used to initiate P2P propagation delay and rate ratio evaluation.

Status	Draft
Verification Criteria	

MVC-105125 - gPTP Rx Messages Filtering

The uC SW shall be able to receive and parse following gPTP messages:

Message Name	Message Type	Description
Sync	0x00	Synchronization message originating from gPTP master, used to update local clock time and frequency.
Sync_Followup	0x08	Synchronization followup message originating from gPTP master, used to update local clock time and frequency.
PDelay_Resp	0x03	Peer-to-peer response message from the gPTP neighbor, used to calculate clock rate ratio and propagation delay to the neighbor.
PDelay_Resp_Followup	0x0A	Peer-to-peer response followup message from the gPTP neighbor, used to calculate clock rate ratio and propagation delay to the neighbor.

Status	Draft
Verification Criteria	

5.5 Ethernet MAC Requirements

MVC-102418 - Ethernet MAC Address

The Ethernet driver shall configure the Ethernet MAC0 with the provided MAC address during system initialization.

Status	Approved
Verification Criteria	Verify by test that the Ethernet MAC is programmed with the provided MAC address during system initialization.

MVC-102420 - PHY ID

The software shall identify the connected Ethernet PHY based on the values defined in the table by reading the PHY identification registers.

PHY ID	PHY Name
B013h	(TJA1103/TJA1104)
B030h	(TJA1120/TJA1121)

Status	Approved
Verification Criteria	Verify by test that the software reads the Ethernet PHY identification register via phySMI to identify the connected Ethernet PHY.

MVC-102415 - MAC Duplex Mode at Interface Bring-Up

The software shall configure the Ethernet MAC0 duplex mode to full duplex when a valid link is established.

Status	Approved
Verification Criteria	Verify that the Ethernet MAC0 interface operates in full-duplex mode after link establishment by confirming simultaneous bidirectional communication and absence of collision-related errors.

MVC-103020 - MAC Duplex Mode On Link Change

The software shall configure the Ethernet MAC0 duplex mode to full duplex upon link status change.

Status	Approved
Verification Criteria	Verify that the Ethernet MAC0 interface operates in full-duplex mode after a link status change by confirming simultaneous bidirectional communication and absence of collision-related errors.

MVC-102414 - MAC Speed at Interface Bring-Up for PHY TJA1103/TJA1104

The software shall configure the Ethernet MAC0 speed to 100 Mbps when a valid link is established in case of PHY TJA1103/TJA1104.

Status	Approved
Verification Criteria	Verify that the Ethernet MAC0 interface operates at 100 Mbps by confirming correct communication behavior and throughput consistent with 100 Mbps operation after link establishment.

MVC-103001 - MAC Speed on Link Change for PHY TJA1103/TJA1104

The software shall configure the Ethernet MAC0 speed to 100 Mbps upon link status change in case of PHY TJA1103/TJA1104.

Status	Approved
Verification Criteria	Verify that upon link status change the Ethernet MAC0 interface updates its operating speed to 100 Mbps in case of PHY TJA1103/TJA1104.

MVC-102417 - MAC Speed at Interface Bring-Up for PHY TJA1120/TJA1121

The software shall configure the Ethernet MAC0 speed to 1 Gbit/s when a valid link is established in case of PHY TJA1120/TJA1121.

Status	Approved
Verification Criteria	Verify that the Ethernet MAC0 interface operates at 1 Gbps by confirming correct communication behavior and throughput consistent with 1 Gbps operation after link establishment.

MVC-102416 - MAC Speed on Link Change for PHY TJA1120/TJA1121

The software shall configure the Ethernet MAC0 speed to 1 Gbit/s upon link status change in case of PHY TJA1120/TJA1121.

Status	Approved
Verification Criteria	Verify that upon link status change the Ethernet MAC0 interface updates its operating speed to 1 Gbps in case of PHY TJA1120/TJA1121.

MVC-102411 - Flow Control

The software shall disable the flow control.

Status	Approved
Verification Criteria	Verify by injecting pause frames and confirm that those frames are ignored and do not affect the software behavior.

MVC-102413 - PHY Identification and MAC Configuration

The software shall complete PHY identification and MAC configuration before enabling Ethernet frame transmission and reception.

Status	Approved
Verification Criteria	Verify by testing that no Ethernet data flow occurs before PHY identification and MAC configuration are completed.

MVC-108386 - Transmission Functionality

The software shall manage transmission of Ethernet frames over the configured Ethernet interface.

Status	Approved
Verification Criteria	Verify by generating transmission requests with different frame lengths and payload contents and monitor the Ethernet interface using a packet tracking tool (e.g., Wireshark).

MVC-108508 - Receive Functionality

The software shall handle the received Ethernet frames over the configured Ethernet interface.

Status	Approved
Verification Criteria	Verify by injecting Ethernet frames with different payload sizes and confirm successful reception and handling.

6 DHCP

MVC-96168 - DHCP Protocol

The application software shall implement the DHCP client functionality in accordance with RFC2131.

Status	Draft
Verification Criteria	Check ethernet trace whether a successful DHCP handshake happened

6.1 DHCP - Hostname

MVC-97819 - DHCP Protocol - Hostname option

The application software shall implement DHCP Host name option as specified by RFC2132 .

Status	Draft
Verification Criteria	Check DHCP packets from application software on ethernet trace and inspect corresponding hostname field option

MVC-96174 - DHCP - Hostname

While in state IPCONFIG the application software shall send DHCP messages with the hostname field set as follows:

device_type + "-" + serialnumber (e.g. "rearcam-00000001")

- device_type: From `someIpSdConfigurationOption` of NVM (MVC-95861 - NVM software configuration block) the value of device_type e.g. device_type=**fpcam**
- serialnumber: Entry `serialNumber` of NVM production data block (`MVC-95865 - NVM production data block`)

Status	Specified
Verification Criteria	Check in ethernet trace whether the field hostname in DHCP messages (DHCP Discover, DHCP Request) is set according to the description in requirement

MVC-96176 - Hostname - device_type length = 7 Bytes

When device_type length = 7 Byte, the DHCP hostname shall look as follows:

- device_type + "-" + serialnumber (e.g. "rearcam-00000001")

Status	Specified
Verification Criteria	Check in ethernet trace whether the field hostname in DHCP messages (DHCP Discover, DHCP Request) is set according to the description in requirement

MVC-96175 - Hostname - device_type length < 7 Bytes

When device_type length < 7 Byte the DHCP hostname shall look as follows:

- device_type (3 Bytes e.g. "cam") + "-" (1 Byte) + padding + serialnumber (8 Bytes e.g. 12345678) (e.g. "cam-000012345678" which is a total of 16 bytes)
- the padding shall consist of "0"s, so that the total length of device_type + "-" + padding is 8 Bytes long

Status	Specified
Verification Criteria	Check in ethernet trace whether the field hostname in DHCP messages (DHCP Discover, DHCP Request) is set according to the description in requirement

MVC-96177 - Hostname - device_type length >= 8 Bytes

When the devicetype >= 8 Byte the DHCP hostname shall look as follows:

- the first 8 Bytes of device_type + serialnumber (e.g. "backupca00000001")

Status	Specified
Verification Criteria	Check in ethernet trace whether the field hostname in DHCP messages (DHCP Discover, DHCP Request) is set according to the description in requirement

6.2 DHCP - IpAddress

MVC-96180 - DHCP - IpAddress setting

When receiving a DHCP ACK message following a successful handshake the application software shall set the ECUs IP address to the IP address provided by the DHCP server.

Status	Specified
Verification Criteria	Send DHCP ACK message and check response with expected IP address

MVC-96183 - DHCP - Lease time expiration

When the lease time provided by the DHCP server expires the application software shall set the ECUs IP address to 0.0.0.0.

Status	Specified
Verification Criteria	Check ethernet trace for DHCP Discover messages from application software with address 0.0.0.0

MVC-106131 - DHCP - Storing IpAddress

When a DHCP server assigns an IP address, the application software shall store the assigned IP address in NVM (see MVC-94612 - NVM fast DHCP Settings Memory Map for address).

Status	Draft
Verification Criteria	

7 SOME/IP

7.1 General

7.1.1 Network configuration

MVC-95600 - Network Configuration - Supported Protocols - Active State

During software state OPERATIONAL, the application software shall handle incoming **UDP** and **TCP** messages on port `cameraServicePort` read from the NVM.

Status	Draft
Verification Criteria	After a valid IP address has been acquired and Service Discovery starts sending OfferService messages, send valid SOME/IP messages to port cameraServicePort read from NVM and expect a SOME/IP RESPONSE or ERROR message to be returned.

MVC-116615 - Network Configuration - Supported Protocols - Inactive State

Outside of software state OPERATIONAL, if the application software receives an incoming UDP or TCP message on port cameraServicePort read from the NVM, it shall ignore it.

Status	Draft
Verification Criteria	After the application software looses a valid IP Address (e.g. the IP assigned via DHCP expires without renewal), send a valid SOME/IP REQUEST message to cameraServicePort. Expect no SOME/IP RESPONSE message to be received from cameraServicePort.

MVC-93136 - Network Configuration - Message Polling Rate

While in software state OPERATIONAL, the application software shall poll for incoming messages on the sockets on port `cameraServicePort` every 10ms.

Note: While SOME/IP messages only have to be responded to within 100ms, there is no lower limit to how fast they have to be responded to. Using a high polling rate gives ample time to process the message and send a response, even if the message was received while a previous message was still being processed.

Status	Draft
Verification Criteria	Send an invalid SOME/IP REQUEST message that causes an early exit in message validation (e.g. invalid Service ID) to ensure low computation time. Expect a SOME/IP ERROR message to be received within at most 10ms (+20% computational overhead tolerance) of sending the message.

MVC-95601 - Network byte order

The application software shall use the network byte order "big endian" for sending and receiving SOME/IP Messages.

Status	Specified
Verification Criteria	Ensure that the application software deserializes received SOME/IP messages in "big endian" byte order

MVC-95602 - IP Address

The application software shall use the ECUs IP address that is assigned to the camera for sending and receiving SOME/IP Messages.

Status	Specified
Verification Criteria	Ensure that the application software only uses the IP address assigned to it for SOME/IP network communication

MVC-95639 - Message frequency

When the application receives a Message on a dedicated socket for SOME/IP, it shall process the entire received Message as a single SOME/IP Message.

Status	Specified
Verification Criteria	The application software must not create more than one RESPONSE or ERROR message for every UDP message received on a SOME/IP socket

MVC-95608 - Responses

If the application software receives a SOME/IP Message on a Socket dedicated to it it shall always respond with either a `RESPONSE` or an `ERROR` Message.

Status	Rejected
Verification Criteria	Ensure that the application software always responds to an incoming UDP message on the SOME/IP socket with a SOME/IP RESPONSE or ERROR message, independent of whether the Message is valid or invalid SOME/IP or not a SOME/IP Message at all

MVC-95610 - Response time

The application software shall respond after the received SOME/IP Message has been fully processed.

Status	Rejected
Verification Criteria	Ensure that the application software does not send a RESPONSE message before processing finished

MVC-95609 - Response destination

The application software shall send its response to the IP Address and Port number that sent the received SOME/IP Message.

Status	Rejected
Verification Criteria	Ensure that the application software sends RESPONSE and ERROR messages to the sender of the original UDP message

MVC-95852 - Network Configuration - Message Response

The Application Software shall respond to every received SOME/IP message on the sockets on `cameraServicePort` with either a SOME/IP `RESPONSE` Message or a SOME/IP `ERROR` Message, sent to the IP and port that send the initial message.

Note: Due to the validations steps described in the sub-requirements of MVC-95604 - Validation sequence , this means that every message that is received on `cameraServicePort` will be treated as a SOME/IP message, falls within the valid size for the used protocol.

Status	Draft
Verification Criteria	Ensure that both valid and invalid SOME/IP Messages get responded to either via a RESPONSE or an ERROR message, ensure that non-SOME/IP messages (see Note) do not get a SOME/IP response. Ensure that the destination IP and port of the RESPONSE and ERROR messages are the same as the sender of the original messages.

MVC-95860 - Camera Service - SOME/IP Response - Source Port

The Application Software shall send the SOME/IP `RESPONSE` and `ERROR` messages over the same port that it received the initial SOME/IP message from.

Status	Specified
Verification Criteria	Ensure that the application software sends the SOME/IP RESPONSE and ERROR Messages on the correct Port

7.1.2 Data structures

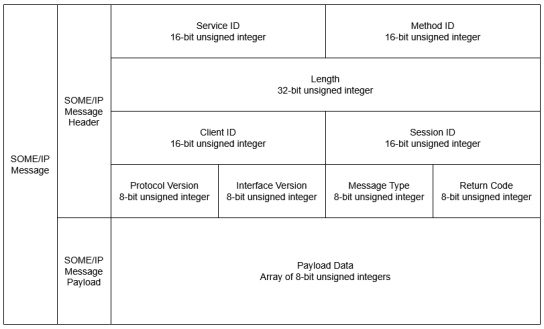
MVC-95683 - Padding

The application software shall not add any padding during serialization, unless it is explicitly specified.

Status	Specified
Verification Criteria	Ensure that SOME/IP Messages sent by the application software have no unwanted padding

MVC-95629 - Message layout

The SOME/IP Messages shall use the following layout for its attributes:



Status	Specified
Verification Criteria	Ensure that serialization and deserialization of SOME/IP Messages done by the application software uses the attribute layout

7.1.3 Message validation

MVC-96510 - SOME/IP Message Content - REQUEST

When the application software receives a SOME/IP Message of Type REQUEST it shall validate that its content matches MVC-94598 - SOME/IP - Header REQUEST .

Note: *The sizes of the fields are implicitly defined by the hex-values. All types are unsigned integers.*

Status	Ready for review
Verification Criteria	Ensure that received Messages get validated for their correct content

MVC-96512 - SOME/IP Message Content - RESPONSE

When the application software sends a SOME/IP Message of Type RESPONSE it shall have the content as specified in MVC-94597 - SOME/IP - Header RESPONSE .

Note: *The sizes of the fields are implicitly defined by the hex-values. All types are unsigned integers.*

Status	Ready for review
Verification Criteria	Ensure that RESPONSE Messages being sent contain the correct contents

MVC-96511 - SOME/IP Message Content - ERROR

When the application software sends a SOME/IP Message of Type ERROR it shall have the content as specified in MVC-94599 - SOME/IP - Header ERROR .

Note: *The sizes of the fields are implicitly defined by the hex-values. All types are unsigned integers.*

Status	Ready for review
Verification Criteria	Ensure that ERROR Messages being sent contain the correct contents

MVC-119333 - SOME/IP Message Content - NOTIFICATION

When the application software sends a SOME/IP Message of type `NOTIFICATION`, it shall have the contents as specified in MVC-118904 - SOME/IP - Header NOTIFICATION .

Note: *The sizes of the fields are implicitly deffined by the hex-values. All types are unsigned integers.*

Status	Draft
Verification Criteria	Ensure that NOTIFICATION Messages being sent contain the correct contents

MVC-95604 - Validation sequence

The application software shall validate received SOME/IP Messages in the following order:

1. erroneous length check of the received Message (MVC-95607 - Message size - minimum , MVC-95616 - Message size - maximum)
2. validation of the SOME/IP Messages `Length` attribute (MVC-95618 - Length attribute)
3. validation of the SOME/IP Messages `Service ID` attribute (MVC-95619 - Service ID)
4. validation of the SOME/IP `Protocol Version` attribute (MVC-95620 - Protocol Version)
5. validation of the SOME/IP `Interface Version` attribute (MVC-95621 - Interface Version)
6. validation of the SOME/IP `Message Type` attribute (MVC-95623 - Message Type)
7. validation of the SOME/IP `Return Code` attribute (MVC-95624 - Return Code)
8. validation of the SOME/IP `Method ID` attribute (MVC-95625 - Method ID)
9. length check of the received Messages SOME/IP `Payload` attribute (~~MVC-96157~~ - Method Length)

If no errors occur during validation, the application software shall consider the SOME/IP Message as "valid".

Status	Specified
Verification Criteria	Ensure that received Messages get validated in the correct order by sending Messages that would fail multiple checks and verify that the Return Code matches the earliest validation that should be performed

MVC-95607 - Message size - minimum

When the received Message is smaller than 16 Bytes, the application software shall ignore it.

Status	Specified
Verification Criteria	Ensure that the application software does not respond if a Message received on the SOME/IP Socket is smaller than 16 Bytes

MVC-95616 - Message size - maximum

When the received Message is larger than the maximum allowed size of 1472 Bytes, the application software shall ignore it.

Status	Specified
Verification Criteria	Ensure that the application software does not respond if a Message received on the SOME/IP Socket is larger than 1472 Bytes

MVC-95885 - Successful validation

If none of the validation checks fails, the application software shall process the requested SOME/IP method, which is determined by Service ID and Method ID.

Status	Draft
Verification Criteria	Ensure that the application software will call the requested camera service for valid SOME/IP Messages, testable with tests for requirements that describe SOME/IP RESPONSE messages for different methods

MVC-95884 - Successful processing

If the requested method was processed successfully, the application software shall respond with a SOME/IP `RESPONSE` Message with Return Code `E_OK`.

Status	Specified
Verification Criteria	Ensure that the application software returns a RESPONSE Message with Return Code E_OK after a valid SOME/IP REQUEST Message is called within a context that will lead to the camera service being processed without errors

MVC-95886 - Processing failure

If the requested method failed during its processing, the application software shall respond with a SOME/IP `ERROR` Message with the Return Code matching the processing error code.

Status	Specified
Verification Criteria	Ensure that the application software returns an ERROR Message with a correct Return Code after a valid SOME/IP REQUEST Message is called within a context that will lead to the camera service being processed with errors (e.g. an invalid ROI Index as payload)

MVC-95606 - Validation failure

If one of the validation checks fails, the application software shall abort the processing of the SOME/IP Message and send a SOME/IP `ERROR` Message with the corresponding Return Code.

Status	Specified
Verification Criteria	Ensure that if the application software receives an invalid SOME/IP Message it will respond with a SOME/IP ERROR Message with the Return Code relating to the first validation failure that is supposed to occur

MVC-95618 - Length attribute

The length attribute is calculated as follows: `Message Size in Bytes - 8 Bytes`
If the `Length` attribute of the received Message is unequal to that value, the application software shall send an `ERROR` Message with the Return Code `E_MALFORMED_MESSAGE`.

Status	Specified
Verification Criteria	Ensure that the application software returns a SOME/IP ERROR Message with Return Code E_MALFORMED_MESSAGE if a Message received on the SOME/IP Socket has an invalid Length attribute as described above

MVC-95619 - Service ID

If the `Service ID` attribute of the received Message is not one of the supported `Service ID`s as described in MVC-94537 - SOME/IP - Supported Services , the application software shall send an `ERROR` Message with the Return Code `E_UNKNOWN_SERVICE`.

Status	Draft
Verification Criteria	Ensure that the application software returns a SOME/IP ERROR Message with Return Code E_UNKNOWN_SERVICE if a Message received on the SOME/IP Socket has an unsupported Service ID, but is otherwise valid

MVC-119334 - Service ID - Wrong Protocol

If the `Service ID` attribute of the received Message is not supported for the protocol the message was received over as described in MVC-94537 - SOME/IP - Supported Services, the application software shall send an `ERROR` Message with the Return Code `E_UNKNOWN_SERVICE`.

Status	Draft
Verification Criteria	Ensure that the application software returns a SOME/IP ERROR Message with Return Code E_UNKNOWN_SERVICE if a Message is received on the wrong SOME/IP Socket for that Service, but is otherwise valid

MVC-95620 - Protocol Version

If the `Protocol Version` attribute of the received Message is not 0x01, the application software shall send an `ERROR` Message with the Return Code `E_WRONG_PROTOCOL_VERSION`.

Status	Approved
Verification Criteria	Ensure that the application software returns a SOME/IP ERROR Message with Return Code E_WRONG_PROTOCOL_VERSION if a Message received on the SOME/IP Socket has an unsupported Protocol Version, but is otherwise valid

MVC-95621 - Interface Version

If the `Interface Version` attribute of the received Message is not 0x01, the application software shall send an `ERROR` Message with the Return Code `E_WRONG_INTERFACE_VERSION`.

Status	Approved
Verification Criteria	Ensure that the application software returns a SOME/IP ERROR Message with Return Code E_WRONG_INTERFACE_VERSION if a Message received on the SOME/IP Socket has an unsupported Interface Version, but is otherwise valid

MVC-95623 - Message Type

If the `Message Type` attribute of the received Message is not 0x00 (`REQUEST`), the application software shall send an `ERROR` Message with the AUTOSAR specific Return Code `E_WRONG_MESSAGE_TYPE` (0x0A), instead of `E_MALFORMED_MESSAGE`.

Status	Specified
Verification Criteria	Ensure that the application software returns a SOME/IP ERROR Message with Return Code E_WRONG_MESSAGE_TYPE if a Message received on the SOME/IP Socket has an unsupported Message Type, but is otherwise valid

MVC-95624 - Return Code

If the `Return Code` attribute of the received Message is not 0x00 (`E_OK`), the application software shall send an `ERROR` Message with the Return Code `E_NOT_OK` to comply with AUTOSAR standard.

Status	Specified
Verification Criteria	Ensure that the application software returns a SOME/IP ERROR Message with Return Code E_NOT_OK if a Message received on the SOME/IP Socket has an unsupported Return Code, but is otherwise valid

MVC-95625 - Method ID

If the `Method ID` attribute of the received Message is not in the list of supported Method IDs as listed in MVC-94534 - SOME/IP - Supported Methods or MVC-117411 - SOME/IP - Supported Methods - Analog Variant , the application software shall send an `ERROR` Message with the Return Code `E_UNKNOWN_METHOD`.

Status	Draft
Verification Criteria	Ensure that the application software returns a SOME/IP ERROR Message with Return Code E_UNKNOWN_METHOD if a Message received on the SOME/IP Socket has an unsupported Method ID, but is otherwise valid

MVC-96157 - Method Length

If the length attribute of the received Messages payload does not fit to the expected length for the Messages `Method ID` attribute, the application software shall send an `ERROR` Message with the Return Code `E_MALFORMED_MESSAGE`.

Note: The payload length of each method is described in the requirements of the methods chapter.

Status	Rejected
Verification Criteria	Ensure that the application software returns a SOME/IP ERROR Message with Return Code E_MALFORMED_MESSAGE if a Message received on the SOME/IP Socket has a payload of invaid size for the given Method ID, but is otherwise valid

7.2 Service Discovery

We send OfferService messages to notify others that our service is available. This is the minimal implementation necessary so that other clients can use our SOME/IP service.

Assumptions:

ISO 17215-3 has requirements for Service Discovery in chapter 6.8.3. For those the following assumptions are made:

1. Only OfferService is implemented
2. We do not receive FindService messages
3. We don't implement setCameraMode(StartCameraService) and setCameraMode(StopCameraService)

We will not implement the full startup behaviour for B1 sample. Instead we will jump directly to startup state MAIN and only send OfferService messages with CYCLIC_OFFER_DELAY.

MVC-118165 - Service Discovery - Service Instances

Whenever the application software is supposed to send an OfferService message, it shall send an OfferService message for each supported Service Instance as described in MVC-94537 - SOME/IP - Supported Services.

Status	Draft
Verification Criteria	Check OfferService messages, ensure that they get sent for each supported Service Instance via field ServiceId

7.2.1 Network configuration

MVC-92629 - Multicast

Service Discovery shall send OfferService messages as multicast.

Status	Rejected
Verification Criteria	

MVC-95685 - SOME/IP-SD - OfferService - Source port

The application software shall send OfferService Notification messages for Service Discovery via a UDP socket to port `someIpSdSourcePort` read from the NVM.

Status	Draft
Verification Criteria	Ensure that the application software sends OfferService Notification messages via someIpSdSourcePort over UDP

MVC-95687 - SOME/IP-SD - OfferService - Destination IP

The application software shall use `someIpSdDestinationIP` read from the NVM as the destination address for OfferService Notification messages sent by the Service Discovery.

Status	Draft
Verification Criteria	Ensure that the application software has someIpSdDestinationIP as its destination IP address for OfferService Notification Messages

MVC-95688 - SOME/IP-SD - OfferService - Destination port

The application software shall use `someIpSdDestinationPort` read from the NVM as the destination port for OfferService Notification messages sent by the Service Discovery.

Status	Draft
Verification Criteria	Ensure that the application software has someIpSdDestinationPort as its destination Port for Service Discovery Messages

MVC-118925 - SOME/IP-SD - Eventgroups - Active State

During software state `OPERATIONAL`, the application software shall handle incoming **TCP** messages on port `someIpSdSourcePort` read from the NVM.

Status	Draft
Verification Criteria	

MVC-118924 - SOME/IP-SD - Eventgroups - Inactive State

Outside of software state `OPERATIONAL`, if the application software receives an incoming message on port `someIpSdSourcePort` read from the NVM, it shall ignore it.

Status	Draft
Verification Criteria	

MVC-118926 - SOME/IP-SD - Eventgroups - Response

When the application software handles an incoming TCP message on port someIpSdSourcePort, it shall use the senders IP address and Port number for sending response messages, when applicable.

Status	Draft
Verification Criteria	

MVC-93011 - UDP header for OfferService messages

Service Discovery shall send OfferService messages as UDP IPv4 frames according to RFC 768. The Checksum shall be calculated using the values provided in ~~MVC-93012~~ · UDP header for OfferService messages with the Source Address set to the current IPv4 address of the device.

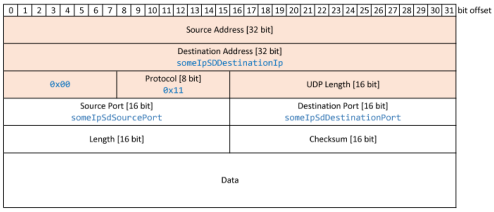


Figure 1 UDP header incl. IPv4 Pseudo header (in red)

***Note:** Source Address is the current IPv4 address. The pseudo header (fields in red) is just used for Checksum calculation and not part of the UDP header.*

***Note**: IPv4 UDP Framing might be better placed somewhere else. In the mean time it is here.*

Status	Rejected
Verification Criteria	

7.2.2 Startup behaviour

Startup behaviour is defined in ISO 17215-2 chapter 8.2.2

MVC-95855 - Service Discovery Startup

When entering software state OPERATIONAL, the application software shall start sending OfferService Notification Messages.

Status	Ready for review
Verification Criteria	Upon reception of the DHCP ACK (IPCONFIG → OPERATIONAL transition, see MVC-96172), verify via Ethernet trace that the application software starts sending OfferService Notification Messages.

MVC-96351 - Service Discovery Startup - Purpose

During startup the Service Discovery shall start sending OfferService Notification Messages.

Status	Rejected
Verification Criteria	During startup of the Service Discovery, ensure that the application software starts sending OfferService Notification Messages

MVC-95856 - Service Discovery Shutdown

When exiting software state OPERATIONAL, the application software shall stop sending OfferService Notification Messages.

Status	Ready for review
Verification Criteria	Complete the DHCP handshake and use the DHCP ACK as indication that the application software entered OPERATIONAL (see MVC-96172). Then let the assigned IP lease expire and verify via Ethernet trace that DHCP communication restarts and no further OfferService Notification Messages are sent (OPERATIONAL → IPCONFIG transition, see MVC-96178).

MVC-96352 - Service Discovery Shutdown - Purpose

During shutdown the Service Discovery shall stop sending OfferService Notification Messages.

Status	Rejected
Verification Criteria	During shutdown of the Service Discovery, ensure that the application software no longer sends OfferService Notification Messages

MVC-95794 - Service Discovery Startup - Initial Notification

When entering software state OPERATIONAL, the application software shall send its first OfferService Message within INITIAL_DELAY.

Status	Specified
Verification Criteria	Complete the DHCP handshake and measure via Ethernet trace from reception of the DHCP ACK to the first OfferService Message; verify compliance with INITIAL_DELAY in MVC-95828.

MVC-95795 - Service Discovery Startup - 1st Repetition

After the Service Discovery sends the initial Notification, the application software shall send the next OfferService Message after 1 * REPETITION_BASE_DELAY has passed.

Status	Specified
Verification Criteria	Read Ethernet traffic to ensure that the application software sends the OfferService Message after the expected delay

MVC-95810 - Service Discovery Startup - 2nd Repetition

After the Service Discovery sends the first repeated Notification, the application software shall send the next OfferService Message after 2 * REPETITION_BASE_DELAY has passed.

Status	Specified
Verification Criteria	Read Ethernet traffic to ensure that the application software sends the OfferService Message after the expected delay

MVC-95816 - Service Discovery Startup - 3rd Repetition

After the Service Discovery sends the second repeated Notification, the application software shall send the next OfferService Message after 4 * REPETITION_BASE_DELAY has passed.

Status	Specified
Verification Criteria	Read Ethernet traffic to ensure that the application software sends the OfferService Message after the expected delay

MVC-95796 - Service Discovery Startup - Cyclic Repetition

After the third repeated OfferService Notification, while in software state OPERATIONAL, the application software shall periodically send subsequent OfferService Messages within CYCLIC_OFFER_DELAY of the previous OfferService Message.

Status	Specified
Verification Criteria	Complete the DHCP handshake, wait until the three repeated OfferService Notifications have been sent, then measure consecutive OfferService intervals and verify compliance with CYCLIC_OFFER_DELAY in MVC-95828.

MVC-92675 - Startup behaviour diagram

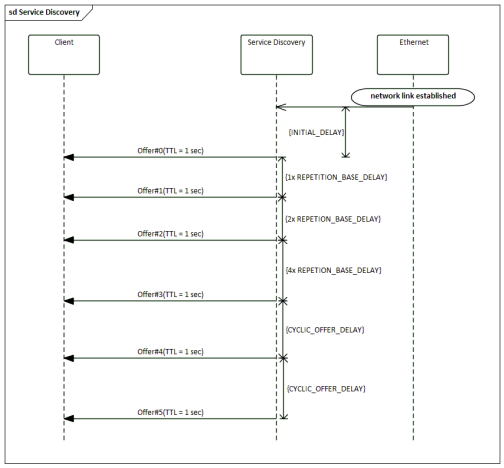


Figure 2 Service Discovery Startup behaviour

Note: Offer #0 to #5 refer to OfferService messsages.

Status	Specified
Verification Criteria	Ensure that the correct sequence is performed during startup

7.2.2.1 Delay Values

In the standard it is foreseen to define delays like follows:

NAMED_DELAY, NAMED_DELAY_MIN, NAMED_DELAY_MAX

These are the nominal, minimum and maximum delay of "NAMED_DELAY".

So each Delay value has a nominal value and an associated interval. When such a delay is implemented the concrete value to be used needs to be randomized.

MVC-95827 - Delay Values randomisation

Service Discovery shall randomly choose a concrete delay value from the minimum to maximum interval for the initial delay value.

Note: ISO17215-2:2014 sec. 8.2.1 mentions randomised delay for all Notifications, but this is currently not reflected in the System Allocation layer.

Status	Specified
Verification Criteria	Test the initial Notification message multiple times to ensure that the application randomizes the delay value

The standard is unclear when it comes to the special value REPETITION_BASE_DELAY (abbreviated as R in the following section Minimum and Maximum values are then R_min and R_max).
This value gets multiplied by a factor N to determine the nominal delay. It is then unclear how the resulting nominal delay should be randomized. The following idea is to randomize only one interval and leave the rest at nominal value. For example a factor of N = 3 leads to a concrete delay value of 2 times R + a random value chosen from the interval [R_min; R_max].

Note: The above resulted in a calculation with incorrect values after the first repetition. The new formula now simply calculates a random delay and multiplies it with N (which is now [0, MAX-1] instead of the previous [1, MAX].

MVC-95826 - Repetition Delay Value randomization

For sending the repeated notifications the application software shall calculate the delay value `D` as

$$D(n) = (x \cdot (R_{max} - R_{min}) + R_{min}) \cdot 2^n, x \in [0, 1]$$

where `n` is an integer in the range of `[0, REPETITION_MAX-1]` as described in MVC-92755 - Decreasing Speed of OfferService in Beginning - Repetition Phase, `x` is a random rational number used to randomize the delay and `R_max` and `R_min` are the maximum and minimum values of `REPETITION_BASE_DELAY` as described in MVC-95828 - Delay Values table .

Note: *The origin of this formula are explained in the paragraph above.*

Status	Specified
Verification Criteria	Ensure that the OfferService Message cyclic delay during REPETITION phase is within the calculateable range

MVC-95828 - Delay Values table

Service Discovery shall use the following table to calculate concrete delay values.

Delay Value	Nominal	Minimum	Maximum
INITIAL_DELAY	N/A	0 msec	20 msec
REPETITION_BASE_DELAY	5 msec	4 msec	6 msec
CYCLIC_OFFER_DELAY	100 msec	98 msec	102 ms

Status	Specified
Verification Criteria	Ensure that the application sends the OfferService Messages within the given delay ranges for each phase

7.2.3 OfferService Message

MVC-93006 - OfferService content

Service Discovery shall send OfferService messages with the content specified in MVC-92749 - Offerservice - content

Note: *The sizes of the fields are implicitly defined by the hex-values. All types are unsigned integers.*

Status	Specified
Verification Criteria	Ensure that the OfferService Message has the desired content

MVC-93005 - OfferService message compile time constant fields

OfferService content shall have all fields as compile time constants with the exception of the following fields:

- **SOME/IP message header:**
  - Length: calculated
  - Session ID: runtime value
- **OfferService Entry:**
  - Flags: runtime value
  - Service ID: multiple static values
  - Instance ID: runtime value
- **IPv4 endpoint option:**
  - IPv4 address: runtime value
  - L4 Proto: multiple static values
  - Port Number: Production Parameter
- **configuration option:**
  - Length: calculated
  - Zero-terminated Configuration String: Production Parameter

Status	Draft
Verification Criteria	Complete the DHCP handshake, capture OfferService Messages from the initial, repetition and main phases, and verify that all fields not listed as exceptions in the Description remain unchanged and match their specified values.

MVC-93007 - SOME/IP message header field: Length

The Length field shall be of type uint32 and shall contain the length in bytes starting from the 32 bit Request ID (which is composed of 16 bit Client ID and 16 bit Session ID) until the end of the message.

Status	Specified
Verification Criteria	Ensure that the Length attribute is correct for the OfferService Message

MVC-108247 - SOME/IP message header field: Session ID

The Session ID field shall be of type uint16 and shall start at 1, incrementing with every OfferService message send.

Status	Approved
Verification Criteria	After software boot, complete the DHCP handshake and verify via Ethernet trace that the first OfferService Message has Session ID 1 and each following OfferService Message increments it by 1.

MVC-108246 - SOME/IP message header field: Session ID Wrap Around

When the Session ID would wrap around due to integer overflow, the application software shall set the Session ID back to 1 and continue incrementing it with following OfferService message.

Status	Approved
Verification Criteria	Let the system run until the Session ID of the OfferService Messages wraps around. Verify via Ethernet trace the sequence 0xFFFE → 0xFFFF → 0x0001 → 0x0002.

MVC-108249 - SOME/IP SD message header field: Flags

The Flags field shall be of type uint8 and shall be set to 0b11000000 until the Session ID wraps around for the first time.

Status	Approved
Verification Criteria	After software boot, complete the DHCP handshake and verify via Ethernet trace that the Flags field remains 0b11000000 until the first Session ID wrap-around.

MVC-108248 - SOME/IP SD message header field: Flags - Wrap Around

When the Session ID of the SOME/IP message header wraps around as described in MVC-108246 - SOME/IP message header field: Session ID Wrap Around, the Flag field shall be set to 0b01000000 and shall remain at this value until the next software boot.

Status	Approved
Verification Criteria	Let the system run until the Session ID of the OfferService Messages wraps around. Verify via Ethernet trace that the Flags is set to 0b11000000 at Session ID 0xFFFF and to 0b01000000 from Session ID 0x0001 onward.

MVC-118164 - OfferService Entry field: Service ID

The application software shall set the Service ID field to the Service Instance that the current OfferService message is related to.

Status	Draft
Verification Criteria	Check OfferService messages, ensure that the ServiceId field has a supported Service ID value in each message

MVC-93009 - OfferService Entry field: Instance ID

The Instance ID field shall be of type uint16 and contain the value of the lowest Byte of the current IPv4 address.

Example: Last byte of IPv4 address 192.168.1.2 is 0x02, Instance ID is 0x0002.

Status	Specified
Verification Criteria	Ensure that the Instance ID attribute is correct for the OfferService Message

MVC-93008 - IPv4 endpoint option field: IPv4 address

The IPv4 address field shall be of type uint32 and shall contain the value of the IPv4 address that is assigned to the camera, as in MVC-95602 - IP Address .

Example: IPv4 address 192.168.1.2 is 0xC0A80102.

Status	Specified
Verification Criteria	Ensure that the IP address field has a correct value for the OfferService Message

MVC-118220 - IPv4 endpoint option field: L4 Proto

The application software shall set the L4-Proto field to the value representing the required protocol for the Service Instance of the current OfferService message.

Service Instance	L4-Proto
Camera Service (**0x433f**)	UDP (**0x11**)
Hardware Information Service (**0x05DB**)	TCP (**0x06**)
Package Management Service (**0x0586**)	TCP (**0x06**)

Status	Draft
Verification Criteria	Check OfferService messages, ensure that the L4 Proto field has the correct value (TCP: 0x06, UDP: 0x11) in each message

MVC-93002 - IPv4 endoint option field: Port Number

The Port Number field shall be of type uint16 and shall contain the value of the Software Configuration Parameter `cameraServicePort`.

Status	Specified
Verification Criteria	Ensure that the Port number field has a correct value for the OfferService Message

MVC-93004 - Configuration option field: Length

The Length shall be of type uint16 and shall contain the total number of bytes occupied by the Zero-terminated Configuration String (including any padding) + 1 for the Reserved field.

Note: The Length field excludes the length of the Length and the Type field from the configuration options.

Status	Specified
Verification Criteria	Ensure that the Length field has a correct value for the OfferService Message

MVC-93003 - Configuration option field: Zero-terminated Configuration String

The Zero-terminated Configuration String shall have the following layout:

Byte Length	Type	Value
1	Length	0x18
12	Key	"mac_address="
12	Value	MVC-96158 - Zero-terminated Configuration String: MAC Address
1	Length	0x12
10	Key	"serial_nb="
8	Value	MVC-96159 - Zero-terminated Configuration String: Serial Number
variable	Length-Key-Value trio	MVC-96161 - Zero-terminated Configuration String: SOME/IP SD Configuration Options

Status	Approved
Verification Criteria	Ensure that the configuration options sent by the OfferService Message contain the expected values when compared to the flash content

MVC-96158 - Zero-terminated Configuration String: MAC Address

The application software shall read `macAddress` from NVM flash and convert it from hexadecimal numbers to their ASCII equivalent (e.g. 0x98D3D2003019 = "98D3D2003019").

Status	Specified
Verification Criteria	Ensure that the value for macAddress in the OfferService Message is the converted version of the value stored in the NVM flash

MVC-96159 - Zero-terminated Configuration String: Serial Number

The application software shall read `serialNumber` from NVM flash and place it into the Configuration String as is.

Status	Specified
Verification Criteria	Ensure that the value for serialNumber in the OfferService Message is the same as the value stored in the NVM flash

MVC-96161 - Zero-terminated Configuration String: SOME/IP SD Configuration Options

The application software shall read `someIpSdConfigurationOption` from NVM flash and insert its content into the Configuration String until the first null-terminator byte (0x00) is reached.

Note: If the first byte is null it also counts as the first null-terminator byte, stopping the copying.

Status	Specified
Verification Criteria	Ensure that the value for someIpSdConfigurationOption in the OfferService Message is the same as the value stored in the NVM flash, up to the first "length field" byte that contains the value null (0x00), which marks the null-terminator

MVC-96160 - Zero-terminated Configuration String: Padding

The application software shall pad the Configuration String with null bytes (0x00) to the next multiple of 4 Bytes. If no padding is necessary it shall be omitted, as MVC-96161 - Zero-terminated Configuration String: SOME/IP SD Configuration Options alre ady ends with a null-terminator.

Status	Rejected
Verification Criteria	Ensure that the amount of padding in the OfferService Message is correct

7.2.4 Eventgroup Messages

MVC-118934 - Eventgroup Messages - Client

When the application software receives a SOME/IP-SD Eventgroup message it shall use the senders IP address as an identifier for different clients.

Status	Draft
Verification Criteria	This requirement is tested in conjunction with requirements that test different clients (e.g. SubscribeEvengroup and StopSubscribeEventgroup)

MVC-118412 - Eventgroup Messages - Content - Incoming

When the application software is receiving SOME/IP-SD Eventgroup messages their contents shall be as specified in MVC-117150 - SubscribeEventgroup/StopSubscribeEventgroup - content

***Note:** The sizes of the fields are implicitly defined by the hex-values. All types are unsigned integers.*

Status	Draft
Verification Criteria	Ensure that valid Eventgroup messages as described in MVC-117150 receive valid responses in return

MVC-119208 - Eventgroup Messages - Content - Outgoing

When the application software is sending SOME/IP-SD Eventgroup messages their contents shall be as specified in MVC-118432 - SubscribeEventgroupAck/SubscribeEventgroupNack - content

***Note:** The sizes of the fields are implicitly defined by the hex-values. All types are unsigned integers.*

Status	Draft
Verification Criteria	Send valid SubscribeEventgroup or StopSubscribeEventgroup message according to MVC-118412, ensure that the received SubscribeEventgroupAck or SubscribeEventgroupNack message has the desired content

MVC-118424 - Eventgroup Messages - SubscribeEventgroup

When the application software receives a SOME/IP-SD Eventgroup Message with Eventgroup Entry fields **Type** set to `0x06`, **ServiceId** set to `0x0586`, **TTL** set to `0xFFFFFF` and **EventgroupId** set to `0x0001`, it shall consider the message as a valid **Subscribe Eventgroup** message.

Status	Draft
Verification Criteria	Send a valid SubscribeEventgroup message as described, ensure that the software responds with an expected response (SubscribeEventgroupAck / SubscribeEventgroupNack)

MVC-118425 - Eventgroup Messages - StopSubscribeEventgroup

When the application software receives a SOME/IP-SD Eventgroup Message with Eventgroup Entry fields **Type** set to `0x06`, **ServiceId** set to `0x0586`, **TTL** set to `0x000000` and **EventgroupId** set to `0x0001`, it shall consider the message as a valid **StopSubs cribeEventgroup** message.

Status	Draft
Verification Criteria	Send a valid StopSubscribeEventgroup message as described, ensure that the software responds with an expected response (SubscribeEventgroupAck / SubscribeEventgroupNack)

MVC-118431 - Eventgroup Messages - SubscribeEventgroupAck

When the application software sends a SOME/IP-SD Eventgroup Message **SubscribeEventgroupAck** it shall have the following values in the **Eventgroup Entry**:

- **Type** == 0x07

ServiceId, **InstanceId**, **Major Version**, **Minor Version**, **EventgroupId** and **TTL** shall be set to the same value as the received **SubscribeEventgroup** or **StopSubscribeEventgroup** message that it is responding to.

Status	Draft
Verification Criteria	Send SOME/IP-SD Eventgroup request (e.g. SubscribeEventgroup while not being subscribed) to receive a SubscribeEventgroupAck response, check values of described fields

MVC-118430 - Eventgroup Messages - SubscribeEventgroupNack

When the application software sends a SOME/IP-SD Eventgroup Message **SubscribeEventgroupNack** it shall have the following values in the **Eventgroup Entry**:

- **Type** == 0x07
- **TTL** == 0x000000

ServiceId, **InstanceId**, **Major Version**, **Minor Version** and **EventgroupId** shall be set to the same value as the received **SubscribeEventgroup** or **StopSubscribeEventgroup** message that it is responding to.

Status	Draft
Verification Criteria	Send SOME/IP-SD Eventgroup request (e.g. StopSubscribeEventgroup while not being subscribed) to receive a SubscribeEventgroupNack response, check values of described fields

MVC-118414 - Eventgroup Messages - Unsupported Types

When the application software receives a SOME/IP-SD Eventgroup Message with Eventgroup Entry field **Type** set to a value other than `0x06`, it shall respond with a **SubscribeEventgroupNack** message.

Status	Draft
Verification Criteria	Send Eventgroup Message with Eventgroup Entry field Type set to a value other than 0x06, expect a Response Message of type SubscribeEventgroupNack

MVC-118415 - Eventgroup Messages - Unsupported Service IDs

When the application software receives a SOME/IP-SD Eventgroup Message with Eventgroup Entry field **ServiceId** set to a value other than `0x0586`, it shall respond with a **SubscribeEventgroupNack** message.

Status	Draft
Verification Criteria	Send Eventgroup Message with Eventgroup Entry field ServiceId set to a value other than 0x0586, expect a Response Message of type SubscribeEventgroupNack

MVC-118416 - Eventgroup Messages - Unsupported TTL

When the application software receives a SOME/IP-SD Eventgroup Message with Eventgroup Entry field **TTL** set to a value other than `0xFFFFFF` or `0x000000`, it shall respond with a **SubscribeEventgroupNack** message.

Status	Draft
Verification Criteria	Send Eventgroup Message with Eventgroup Entry field TTL set to a value other than 0xFFFFFF, expect a Response Message of type SubscribeEventgroupNack

MVC-118418 - Eventgroup Messages - Unsupported Eventgroup IDs

When the application software receives a SOME/IP-SD Eventgroup Message with Eventgroup Entry field **EventgroupId** set to a value other than `0x0001`, it shall respond with a **SubscribeEventgroupNack** message.

Status	Draft

Verification Criteria	Send Eventgroup Message with Eventgroup Entry field EventgroupId set to a value other than 0x0001, expect a Response Message of type SubscribeEventgroupNack

MVC-118858 - Eventgroup Messages - Client Registration

When the application software receives a valid **SubscribeEventgroup** message, it shall register the client for receiving notifications related to the Package Management Service and respond with a **SubscribeEventgroupAck** message.

Note: Since only a **TTL** of `0xFFFFFF` is supported, there is no need to handle the duration for which the client needs to stay registered.
Note: The clients identity will be handled via their IP address, as it is already done with the **getCamExclusive** Method.

Status	Draft
Verification Criteria	Send valid SubscribeEventgroup request, expect SubscribeEventgroupAck response. Force sending of currentStatus NOTIFICATION message, ensure that it gets send.

MVC-118859 - Eventgroup Messages - Client Registration - Re-Registration

When the application software receives a valid **SubscribeEventgroup** message by a client that is already registered for receiving notifications related to the Package Management Service, it shall leave the registered clients unchanged and respond with a **SubscribeEventgroupAck** message.

Note: Since only a **TTL** of `0xFFFFFF` is supported, there is no need to refresh the duration for which the client needs to stay registered.

Status	Draft
Verification Criteria	Send valid SubscribeEventgroup request while already being subscribed, expect SubscribeEventgroupAck response. Force sending of currentStatus NOTIFICATION message, ensure that it gets send.

MVC-118860 - Eventgroup Messages - Client Registration - Client Limits

When the application software receives a valid **SubscribeEventgroup** message while there are already 10 clients registered for receiving notifications related to the Package Management Service and the requesting client is not already registered, it shall reject the incoming message by not registering the client and responding with a **SubscribeEventgroupNack** message.

Status	Draft
Verification Criteria	Send SubscribeEventgroup request from client that is not yet registered while 10 other clients are already registered, expect SubscribeEventgroupNack response and no notification for that client on currentState NOTIFICATION message sendout

MVC-118862 - Eventgroup Messages - Client Unregistration - Valid Client

When the application software receives a valid **StopSubscribeEventgroup** message from a client that is registered for receiving notifications related to the Package Management Service, it shall remove the client from the registration and respond with a **SubscribeEventgroupAck** message.

Status	Draft
Verification Criteria	Send valid StopSubscribeEventgroup request with client that was previously successfully subscribed, expect a SubscribeEventgroupAck response and no more currentStatus NOTIFICATION messages on that client when one would be send

MVC-118861 - Eventgroup Messages - Client Unregistration - Invalid Client

When the application software receives a valid **StopSubscribeEventgroup** message from a client that is ***not*** registered for receiving notifications related to the Package Management Service, it shall not remove any clients from the registration and respond with a **SubscribeEventgroupNack** message.

Status	Draft
Verification Criteria	Send valid StopSubscribeEventgroup request with client that is not yet subscribed, expect SubscribeEventgroupNack response

7.3 Camera Service Methods

MVC-118228 - Camera Service Methods - Service ID

If the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Service ID** set to `0x433f`, it shall process the received message as a Camera Service Method.

Status	Draft
Verification Criteria	When sending SOME/IP REQUEST messages with ServiceId 0x433f, the behaviour of the application software must reflect the expected behaviour for Camera Service Methods. This shall be verified in conjunction with Requirements that yield a response message by application software.

7.3.1 Method getCamIdentification

MVC-108272 - getCamIdentification - Method Validation

When the applications software receives a valid SOME/IP `REQUEST` message with SOME/IP header fields **Method ID** set to `0x0010`, **Length** set to `8` and **no payload**, it shall accept the request as a valid **getCamIdentification** SOME/IP `REQUEST`.

Status	Approved
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-108262 - getCamIdentification - Method Length

When the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Method ID** set to `0x0010` and field **Length** set to a value other than 8, it shall respond with a SOME/IP `ERROR` message with **Return Code** set to `E_MALFORMED_MESSAGE`.

Status	Approved
Verification Criteria	Send getCamIdentification REQUEST mesage with wrong payload (correct length for given payload), expect ERROR response with return code E_MALFORMED_MESSAGE

MVC-93025 - getCamIdentification - Positive Response

When the application software receives a valid **getCamIdentification** SOME/IP `REQUEST` message, it shall send a SOME/IP `RESPONSE` message with SOME/IP header fields **Length** set to `23 (0x17)`, **Message Type** set to `RESPONSE (0x80)`, **Return Code** set to `E_OK (0x00)` and the following fields as **payload**:

ByteIndex	Output Parameter	BaseType	Description
0	snpn	uint8[5]	slave node part number (see MVC-95865 - NVM production data block)
5	serialNumberBCD	uint8[4]	serial number coded as BCD (as ASCII see MVC-95865 - NVM production data block): MVC-93026 - getCamIdentification - Conversion of serialNumber MVC-93027 - getCamIdenfication - Conversion of an invalid serialNumber
9	macAddress	uint8[6]	MAC address (see MVC-95865 - NVM production data block)

Status	Specified
Verification Criteria	Send a valid getCamIdentification SOME/IP REQUEST message, ensure that the SOME/IP RESPONSE message has the correct values

MVC-93026 - getCamIdentification - Conversion of serialNumber

The application software shall convert the 8-byte `serialNumber` read from NVM (coded as ASCII, see MVC-95865 - NVM production data block) to a 4-byte BCD coded `serialNumberBCD`.

Conversion:
Each byte in the `serialNumber` shall be converted to a nibble of the `serialNumberBCD` by subtracting `48` (ASCII '0').

Example:
`serialNumber: "1 2 3 4 5 6 7 8" -> serialNumberBCD: 0x12 34 56 78`

Status	Specified
Verification Criteria	After receiving a valid SOME/IP message with methodID getCamIdentification the application software responds with a valid response and payload shall contain the BCD-coded serial number. This BCD-coded serial number shall be compared to BCD-converted value from NVM for equality.

MVC-93027 - getCamIdenfication - Conversion of an invalid serialNumber

If at least one of the bytes of the 8-byte `serialNumber` contains a value greater than `57` (ASCII `'9'`) or less than `48` (ASCII `'0'`),
the application software shall set the `serialNumberBCD` to `0xFFFFFFFF`.

Status	Specified
Verification Criteria	After receiving a valid SOME/IP message with methodID getCamIdentification the application software responds with a valid response and payload shall contain the default serial number when serial number from NVM before BCD-conversion contains values greater than 57 or less than 48

7.3.2 Method getDataSheet

MVC-108388 - getDataSheet - Method Validation

When the application software receives a valid SOME/IP `REQUEST` message with SOME/IP header fields **Method ID** set to `0x0001`, **Length** set to 8 and **not payload**, it shall accept the request as a valid **getDataSheet** SOME/IP `REQUEST`.

Status	Approved
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-108387 - getDataSheet - Method Length

When the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Method ID** set to `0x0001` and field **Length** set to a value other than 8, it shall respond with a SOME/IP ERROR message with **Return Code** set to `E_MALFORMED_MESSAGE`.

Status	Approved
Verification Criteria	Send getDataSheet REQUEST mesage with wrong payload (correct length for given payload), expect ERROR response with return code E_MALFORMED_MESSAGE

MVC-95646 - getDatasheet - Positive Response

When the application software receives a valid getDatasheet request message it shall send a positive SOME/IP response message with return code `E_OK (0x00)` and the following SOME/IP message payload:

SOME/IP payload:

ByteIndex	Output Parameter	BaseType	Description
0	deviceIdentification	uint8[256]	MVC-95650 - getDatasheet - SOME/IP Payload Field - deviceIdentification
256	standardVersion	uint32	MVC-95654 - getDatasheet - SOME/IP Payload Field - standardVersion - value
260	imagerCharacteristic	sImagerCharacteristic	MVC-95659 - getDatasheet - SOME/IP Payload Field - imagerCharacteristic - value
305	intrinsicCamParam	sIntrinsicCamParam	MVC-95662 - getDatasheet - SOME/IP Payload Field - intrinsicCamParam - value
405	extrinsicCamParam	sExtrinsicCamParam	MVC-95665 - getDatasheet - SOME/IP Payload Field - extrinsicCamParam - value
501	defectPixelMap	sPixelMap	MVC-95673 - getDatasheet - SOME/IP Payload Field - defectPixelMap - value
505	numOfTemperatures	uint32	MVC-95676 - getDatasheet - SOME/IP Payload Field - numOfTemperatures - value
509	numOfRegionOfInterest	uint32	MVC-95679 - getDatasheet - SOME/IP Payload Field - numOfRegionOfInterest
513	maxLengthOfPersistentEntry	uint32	MVC-95682 - getDatasheet - SOME/IP Payload Field - maxLengthOfPersistentEntry - value

Note: The base types sImagerCharacteristic, sInstrinsicCamParam, sExtrinsicCamParam and sPixelMap are currently not used and therefore not described in detail.

Status	Specified
Verification Criteria	Application software sends a positive response when a valid SOME/IP message with method getDatasheet is send by a client

MVC-95650 - getDatasheet - SOME/IP Payload Field - deviceIdentification

The application software shall include the field `deviceIdentification` of type `uint8[256]` as part of the payload in the positive response message of the method `getDatasheet`.
It includes the following parameters from the Production Data and Software Configuration blocks and the current Software Version and Timestamp, which are hard-coded in the software:

Name	Byte Length	Location
macAddress	6	MVC-95865 - NVM production data block
batchNumber	10	MVC-95865 - NVM production data block
partNumber	12	MVC-95865 - NVM production data block
partRev	2	MVC-95865 - NVM production data block
serialNumber	8	MVC-95865 - NVM production data block
Timestamp	8	MVC-95865 - NVM production data block
epShort	4	MVC-95865 - NVM production data block
hwVer	4	MVC-95865 - NVM production data block
ParVer	4	MVC-95861 - NVM software configuration block
imVer	7	MVC-113471 - Readout ImVer
software timestamp	22	MVC-95850 - Software build information - build date
software version	variable up to 42 bytes	MVC-95851 - Software build information - software version

Status	Specified
Verification Criteria	Application software response contains in field deviceIdentification of payload identical values to NVM, software timestamp and software version

MVC-95653 - getDatasheet - SOME/IP Payload Field - deviceIdentification - Serialization

The application software shall separate each deviceIdentification entry by a '\n' separator.

Status	Specified
Verification Criteria	In response message each entry inside field deviceIdentification of payload is separated by a newline (0xA)

MVC-95652 - getDatasheet - SOME/IP Payload - deviceIdentification - Null-terminated

The application software shall add a null-terminator ('\0') at the end of `deviceIdentification` data to signal the end of it.

Status	Specified
Verification Criteria	Response payload of deviceIdentification ends with a null-terminating character

MVC-95654 - getDatasheet - SOME/IP Payload Field - standardVersion - value

The application software shall set the hardcoded value `0x00000001` for `standardVersion`.

Status	Specified
Verification Criteria	Response message payload at byte index 256 contains a 4 byte value which is 0x00000001

MVC-95659 - getDatasheet - SOME/IP Payload Field - imagerCharacteristic - value

The application software shall fill the data of `imagerCharacteristic` with zeros.

Status	Specified
Verification Criteria	Response message payload at byte index 260 up to byte index 304 is filled with zeros

MVC-95662 - getDatasheet - SOME/IP Payload Field - intrinsicCamParam - value

The application software shall fill the data of `intrinsicCamParam` with zeros.

Status	Specified
Verification Criteria	Response message payload at byte index 305 up to byte index 404 is filled with zeros

MVC-95665 - getDatasheet - SOME/IP Payload Field - extrinsicCamParam - value

The application software shall fill the data of `extrinsicCamParam` with zeros.

Status	Specified
Verification Criteria	Response message payload at byte index 405 up to byte index 500 is filled with zeros

MVC-95673 - getDatasheet - SOME/IP Payload Field - defectPixelMap - value

The application software shall fill the data of `defectPixelMap` with zeros.

Status	Specified
Verification Criteria	Response message payload at byte index 501 up to byte index 504 is filled with zeros

MVC-95676 - getDatasheet - SOME/IP Payload Field - numOfTemperatures - value

The application software shall set the hardcoded value `0x00000001` for `numOfTemperatures`.

Status	Specified
Verification Criteria	Response message payload at byte index 505 contains a 4 byte value which is 0x00000001

MVC-95679 - getDatasheet - SOME/IP Payload Field - numOfRegionOfInterest

The application software shall set the field `numOfRegionOfInterest` to the value of **numbersOfRegionsOfInterest** (see MVC-95861 - NVM software configuration block).

Status	Specified
Verification Criteria	Response message payload at byte index 509 contains a 4 byte value which is defined in NVM

MVC-95682 - getDatasheet - SOME/IP Payload Field - maxLengthOfPersistentEntry - value

The application software shall fill the data of `maxLengthOfPersistentEntry` with zeros.

Status	Specified
Verification Criteria	Response message payload at byte index 513 up to byte index 516 is filled with zeros

7.3.3 Method setCamID

MVC-108393 - setCamID - Method validation

When the application software receives a valid SOME/IP REQUEST message, it shall accept the request as valid setCamID SOME/IP REQUEST if SOME/IP header field **Method ID** is set to 0x0409, **Length** is set to 10 (0x08 header + 0x02 payload) and the following fields as **payload**:

Byte Index	Input Parameter	Base Type	Description
0	numCameras	uint8	Number of identified cameras in the vehicle via OfferService The application software shall ignore this value as there is no use-case to store and validate this information
1	cameraID	uint8	Camera ID to be output via SEI message or overlay for the HV application

Status	Approved
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-108394 - setCamID - Method Length

When the application software receives a SOME/IP REQUEST message with SOME/IP header field **Method ID** set to 0x0409 and field **Length** set to a value other than 10 (0x0A), it shall respond with a SOME/IP **ERROR message** with **Return Code** set to E_MALFORMED_MESSAGE.

Status	Approved
Verification Criteria	Send setCamID REQUEST message with wrong length, expect ERROR response with return code E_MALFORMED_MESSAGE.

MVC-108395 - setCamID - Evaluation of numCameras

When the application software receives a valid setCamID SOME/IP REQUEST message, it shall **ignore** the input parameter **numCameras**, as there is no use-case to store or validate this information.

Status	Rejected
Verification Criteria	Send valid setCamID REQUEST messages with different numCameras values; behavior and response shall be identical.

MVC-108438 - setCamID - Evaluation of cameraID - valid range

When the application software receives a valid setCamID SOME/IP REQUEST message with input parameter **cameraID** in the range 0x00 to 0x0F, it shall accept the value and use it for stream tagging (see MVC-105640 - User Data: Cam ID). The **default** value of the camera ID is 0x00.

Status	Specified
Verification Criteria	This response shall be verified in conjunction with Requirements that yield a response message by application software. The correct usage of the camera ID for stream tagging is verified in conjunction with the requirements of MVC-105640 - User Data: Cam ID

MVC-108440 - setCamID - Negative Response - invalid cameraID

If the application software receives a valid setCamID SOME/IP **REQUEST** message with input parameter **cameraID** outside the range defined in MVC-108438 - setCamID - Evaluation of cameraID - valid range, it shall respond with a SOME/IP **ERROR message** with Message **Type** 0x81, **Return Code** E_NOT_OK and **no payload**.

Status	Approved
Verification Criteria	Send setCamID REQUEST message with cameraID greater than 0x0F, expect ERROR response with return code E_NOT_OK; the camera ID at the output remains unchanged.

MVC-108441 - setCamID - Positive Response

When the application software receives a valid setCamID SOME/IP **REQUEST** message with a valid cameraID, it shall send a SOME/IP **RESPONSE message** with SOME/IP header fields **Length** set to 8 (0x08), **Message Type** set to RESPONSE (0x80), **Return Code** set to E_OK (0x00) and **no payload**.

Status	Approved
Verification Criteria	Send a valid setCamID REQUEST message, trace Ethernet and ensure that the SOME/IP RESPONSE message has Method ID 0x0409, Length 8, Message Type RESPONSE (0x80), Return Code E_OK and no payload.

7.3.4 Method setCamExclusive

MVC-99828 - setCamExclusive - Method validation

When the application software receives a valid SOME/IP `REQUEST` message, it shall accept the request as a valid **setCamExclusive** SOME/IP `REQUEST` if SOME/IP header fields **Method ID** is set to `0x0011`, **Length** is set to `12` and the payload has the following fields:

ByteIndex	Input Parameter	BaseType	Description
0	TTL	uint32	Time to live, after this time (in seconds) the lock expires.

Status	Approved
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-108564 - setCamExclusive - Method Length

When the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Method ID** set to `0x0011` and field **Length** set to a value other than 12, it shall respond with a SOME/IP `ERROR` message with **Return Code** set to `E_MALFORMED_MESSAGE`.

Status	Approved
Verification Criteria	Send setCamExclusive REQUEST message with payload of invalid length (length field in the header correct for the given payload), expect ERROR response with return code E_MALFORMED_MESSAGE

MVC-95701 - setCamExclusive - Not locked - Positive response

When the application software receives a valid setCamExclusive SOME/IP request while not locked, a response with message type `RESPONSE (0x80)`, return code `E_OK (0x00)` and **no payload** shall be sent.

Status	Approved
Verification Criteria	Send setCamExclusive request while camera is not locked and check SOME/IP response message.

MVC-99827 - setCamExclusive - Locked by same client - Positive response

When the application software receives a valid setCamExclusive SOME/IP request from the client that currently holds the lock, a response with message type `RESPONSE (0x80)`, return code `E_OK (0x00)` and **no payload** shall be sent.

Status	Approved
Verification Criteria	Obtain lock with a client via setCamExclusive request and send an additional setCamExclusive request from same client and check SOME/IP response message.

MVC-95715 - setCamExclusive - Locked by other client - Negative Response

While being locked by another client, when the application software receives a valid setCamExclusive SOME/IP request from a client that does not hold the lock, a response with message type `ERROR (0x81)`, return code `E_LOCKED_BY_FOREIGN_INSTANCE (0x20)` and **no payload** shall be sent.

Status	Specified
Verification Criteria	Obtain lock with a client via setCamExclusive request and send an additional setCamExclusive request from a different client and check SOME/IP response message.

MVC-99826 - setCamExclusive - Locking mechanism - Not locked

When the application software receives a valid setCamExclusive SOME/IP request while not locked, then it shall assign the lock to the requesting client for the duration based on the provided `TTL` in seconds.

Status	Specified
Verification Criteria	Send a valid setCamExclusive request and check lock via getCamStatus

MVC-95713 - setCamExclusive - SOME/IP Payload Field - TTL - default duration

When the application software receives a valid setCamExclusive SOME/IP request with a `TTL` as follows:
0x15180 ≤ `TTL` ≤ 0xFFFFFFFF,
then the application software shall set the lock duration to 10 seconds.

Status	Specified
Verification Criteria	Send a valid setCamExclusive request with TTL being =>0x15180 and check lock via getCamStatus

MVC-95712 - setCamExclusive SOME/IP Payload Field - TTL - lock duration

When the application software receives a valid setCamExclusive SOME/IP request with a TTL as follows:
0x0 ≤ `TTL` < 0x15180,
then the application software shall set the lock duration to `TTL` in seconds.

Status	Specified
Verification Criteria	Send a valid setCamExclusive request with TTL being <0x15180 and check lock via getCamStatus

MVC-95703 - setCamExclusive - Storing request IP address

While not being locked, when the application software receives a valid setCamExclusive SOME/IP request it shall store the IP address of the client until the lock duration expires.

Status	Specified
Verification Criteria	Send a valid setCamExclusive request and check ip address via getCamStatus

MVC-95714 - setCamExclusive - Locking mechanism - Locked by same client

When the application software receives a valid setCamExclusive SOME/IP request from the client that currently holds the lock, it shall update the lock duration based on provided `TTL` parameter.

Status	Specified
Verification Criteria	Send a valid setCamExclusive request after already locking with the same client and check lock via getCamStatus

MVC-99839 - setCamExclusive - Locking mechanism - Locked by different client

While being locked by another client, when the application software receives a valid setCamExclusive SOME/IP request from a client that does not hold the lock, it shall ignore the `TTL` value.

Status	Approved
Verification Criteria	Obtain lock with a client via setCamExclusive request and send an additional setCamExclusive request from a different client and check SOME/IP response message. Check lock duration based on initial provided TTL parameter.

7.3.5 Method eraseCamExclusive

MVC-99844 - eraseCamExclusive - Method Validation

When the application software receives a valid SOME/IP `REQUEST` message with SOME/IP header fields **Method ID** set to `0x0019`, **Length** set to `8` and **no payload**, the application software shall accept the request as a valid **eraseCamExclusive** SOME/IP `REQUEST`.

Status	Approved
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-108577 - eraseCamExclusive - Method Length

When the application software receives a SOME/IP `REQUEST` message with SOME/IP header fields **Method ID** set to `0x0019` and field **Length** set to a value other than 8, it shall respond with a SOME/IP `ERROR` message with **Return Code** set to `E_MALFORMED_MESSAGE`.

Status	Approved
Verification Criteria	Send eraseCamExclusive REQUEST mesage with wrong payload (correct length for given payload), expect ERROR response with return code E_MALFORMED_MESSAGE

MVC-99845 - eraseCamExclusive - Not locked - Negative response

When the application software receives a valid **eraseCamExclusive** SOME/IP `REQUEST` while not locked, a response with **Message Type** `ERROR (0x81)`, **Return Code** `E_NOT_LOCKED (0x22)` and **no payload** shall be sent.

Status	Approved
Verification Criteria	Send an eraseCamExclusive from a client and check SOME/IP response.

MVC-99846 - eraseCamExclusive - Locked by same client - Positive response

When the application software receives a valid **eraseCamExclusive** SOME/IP `REQUEST` from the client that currently holds the lock, a response with message type `RESPONSE (0x80)`, return code `E_OK (0x00)` and **no payload** shall be sent.

Status	Approved
Verification Criteria	Send a setCamExclusive from a client with any TTL value. Send an eraseCamExclusive from same client who and check SOME/IP response.

MVC-99855 - eraseCamExclusive - Locking mechanism - Locked by same client

When the application software receives a valid **eraseCamExclusive** SOME/IP `REQUEST` from the client that currently holds the lock, the application software shall release the lock.

Status	Approved
Verification Criteria	Send a setCamExclusive from a client with any TTL value. Send a eraseCamExclusive from same client. Send another eraseCamExclusive and check SOME/IP response.

MVC-99847 - eraseCamExclusive - Locked by different client - Negative response

When the application software receives a valid **eraseCamExclusive** SOME/IP `REQUEST` from from a client that does not hold the lock, a SOME/IP `ERROR` message with message type `ERROR (0x81)`, return code `E_LOCKED_BY_FOREIGN_INSTANCE (0x20)` and **no payload** shall be sent.

Status	Approved
Verification Criteria	Send a setCamExclusive from a client with any TTL value. Send an eraseCamExclusive from a different client and check SOME/IP response.

MVC-99856 - eraseCamExclusive - Locking mechanism - Locked by different client

When the application software receives a valid **eraseCamExclusive** SOME/IP `REQUEST` from a client that does not hold the lock, the application software shall not change the lock.

Status	Approved
Verification Criteria	Send a setCamExclusive from a client with any TTL value. Send an eraseCamExclusive from a different client and check SOME/IP response.

7.3.6 Method getCamControl

MVC-99869 - getCamControl - Method Validation

The application software shall accept the SOME/IP REQUEST as a valid getCamControl SOME/IP REQUEST when SOME/IP header fields **Method ID** set to `0x0203`, **Length** set to `9` and the payload has the following fields:

ByteIndex	Input Parameter	BaseType	Description
0	camControlIndex	uint8	The index to identify which camera functionality settings shall be returned.

Status	Approved
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-108649 - getCamControl - Method Length

When the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Method ID** set to `0x0203` and field **Length** set to a value other than 9, it shall respond with a SOME/IP `ERROR` message with **Return Code** set to `E_MALFOR`

MED_MESSAGE.

Status	✔ Approved
Verification Criteria	Send a getCamControl REQUEST message with SOME/IP header field Length set to a value other than 9; expect a SOME/IP ERROR response with Return Code E_MALFORMED_MESSAGE.

MVC-99870 - getCamControl - Invalid camControlIndex

When the application software receives a valid **getCamControl** SOME/IP `REQUEST` message with an unsupported **camControlIndex** according to MVC-92608 - setCamControl - Enumeration eControlIndex , it shall respond with a SOME/IP `ERROR` message with **Return Code** set to `E_INVALID_CONTROL_INDEX` and no payload.

Status	✔ Approved
Verification Criteria	Send a getCamControl REQUEST message with an unsupported camControlIndex; expect a SOME/IP ERROR response with Return Code E_INVALID_CONTROL_INDEX and no payload.

MVC-99871 - getCamControl - Positive response

When the application software receives a valid **getCamControl** SOME/IP `REQUEST` message with a supported **camControlIndex** according to MVC-92608 - setCamControl - Enumeration eControlIndex , it shall respond with a SOME/IP `RESPONSE` message with **Message Type** `RESPONSE (0x80)`, **Return Code** `E_OK (0x00)` and the following fields as payload:

ByteIndex	Output Parameter	BaseType	Description
0	supportedModes	uint8	Fixed value of StatRW_A_nO = 0x07
1	controlMode	uint8	returns current control mode for functionality
2	controlValue	uint16	returns current control value for functionality

Status	✔ Approved
Verification Criteria	Send a getCamControl REQUEST message with a supported camControlIndex; expect a SOME/IP RESPONSE with Return Code E_OK and payload fields supportedModes, controlMode, controlValue as specified.

7.3.7 Method setCamControl

MVC-99859 - setCamControl - Method Validation

When the application software receives a valid SOME/IP `REQUEST` message, it shall accept the request as a valid **setCamControl** SOME/IP `REQUEST` if SOME/IP header fields **Method ID** is set to `0x0201`, **Length** is set to `13` and the payload has the following fields:

ByteIndex	Input Parameter	BaseType	Description
0	camControlIndex	uint8	The index to identify which camera functionality shall be controlled.
1	supportedModes	uint8	Fixed value: StatRW_A_nO = 0x07
2	controlMode	uint8	The mode which can be changed based on provided parameter e.g. automatic or manual mode.
3	controlValue	uint16	The value to be set for corresponding camera functionality based on provided mode.

Status	✔ Approved
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-108682 - setCamControl - Method Length

When the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Method ID** set to `0x0201` and field **Length** set to a value other than `13`, it shall respond with a SOME/IP `ERROR` message with **Return Code** set to `E_MALFORMED_MESSAGE`.

Status	✔ Approved
Verification Criteria	Send a setCamControl REQUEST message with SOME/IP header field Length set to a value other than 13 (e.g. 4 or 12); expect an ERROR response with Return Code E_MALFORMED_MESSAGE.

MVC-108656 - setCamControl - Payload Validation

When the application software receives a valid **setCamControl** SOME/IP `REQUEST` from the client that currently holds the lock, it shall validate the payload parameters in the following order:

1. **camControlIndex**
2. **controlMode**
3. **controlValue**

Note: The **supportedModes** input parameter shall be ignored in accordance with MVC-92609 - setCamControl - Enumeration eControlSupportedModes .

Status	Specified
Verification Criteria	Check requirements that validate the payload parameters, ensure that the parameters get validated in the correct order

MVC-109030 - setCamControl - Payload Validation - Supported controlSupportedModes

When the application software receives a valid **setCamControl** SOME/IP `REQUEST` message with **camControlIndex** = 0x09 or **camControlIndex** = 0x50, it shall ignore the **controlSupportedModes** value.

Status	✔ Approved
Verification Criteria	Send setCamControl REQUEST messages with different controlSupportedModes, ensure that the value does not affect the response

MVC-99863 - setCamControl - Payload Validation - Supported camControlIndex

When evaluating the **camControlIndex** field of a valid **setCamControl** SOME/IP `REQUEST` message, the application software shall stop validation and send a SOME/IP `ERROR` message with **Return Code** `E_INVALID_CONTROL_INDEX (0x35)` and **no payload** if the **camControlIndex** field is invalid in accordance with MVC-92608 - setCamControl - Enumeration eControlIndex .

Status	✔ Approved
Verification Criteria	Lock camera via setCamExclusive, send a valid setCamControl with an invalid control index with same client and check SOME/IP reponse message.

MVC-99864 - setCamControl - Payload Validation - Supported controlMode

The application software shall evaluate the controlMode field of a valid setCamControl SOME/IP request and determine its validity by checking whether the value matches one of the allowed values defined in MVC-92599 - setCamControl - E_INVALID_CONTROL_MODE.

Status	✘ Rejected
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-99866 - setCamControl - Payload Validation - Supported controlValue

The application software shall evaluate the controlValue field of a valid setCamControl SOME/IP request and determine its validity by checking whether the value matches one of the allowed values defined in:

MVC-92570 - controlValue as input parameter - heating
MVC-92491 - controlValue as input parameter - shutter

Status	✘ Rejected
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-99860 - setCamControl - Negative Response - Camera not locked

When the application software receives a valid **setCamControl** SOME/IP `REQUEST` message while not locked, a SOME/IP `ERROR` message with **Message Type** `ERROR (0x81)`, **Return Code** `E_LOCK_EXPIRED (0x21)` and **no payload** shall be sent.

Status	✔ Approved
Verification Criteria	Send a valid setCamControl message without sending a setCamExclusive first and check SOME/IP response message.

MVC-99861 - setCamControl - Negative Response - Locked by different client

While being locked by another client, when the application software receives a valid **setCamControl** SOME/IP `REQUEST` message from a client that does not hold the lock, a SOME/IP `ERROR` message with **Message Type** `ERROR (0x81)`, **Return Code** `E_LOCKED_BY_FOREIGN_INSTANCE (0x20)` and **no payload** shall be sent. No CamControl parameters shall be modified.

Status	✔ Approved
Verification Criteria	Send a setCamExclusive with a client. Send a valid setCamControl with different client and check SOME/IP response message.

MVC-99868 - setCamControl - Positive Response

When the application software receives a valid **setCamControl** SOME/IP `REQUEST` message from a client that does hold the lock, a SOME/IP `RESPONSE` message with **Message Type** `RESPONSE (0x80)`, **Return Code** `E_OK (0x00)` and **no payload** shal



I be sent, if the validation of the payload parameters as described in MVC-108656 - setCamControl - Payload Validation succeeds.

Status	Approved
Verification Criteria	Send a setCamExclusive with a client. Send a valid setCamControl with valid parameters with same client and check SOME/IP reponse message.

MVC-99862 - setCamControl - Locked by client - Invalid camControlIndex - Negative response

When the application software receives a valid setCamControl SOME/IP request from the client that currently holds the lock, a response with message type `ERROR (0x81)`, return code `E_INVALID_CONTROL_INDEX(0x35)` and **no payload** shall be sent if the camControlIndex is invalid.

Status	Rejected
Verification Criteria	

MVC-99865 - setCamControl - Locked by client - Invalid controlMode - Negative response

When the application software receives a valid setCamControl SOME/IP request from the client that currently holds the lock, a response with message type `ERROR (0x81)`, return code `E_INVALID_CONTROL_MODE(0x36)` and **no payload** shall be sent if the controlMode is invalid.

Status	Rejected
Verification Criteria	Send a setCamExclusive with a client. Send a valid setCamControl with an invalid control mode with same client and check SOME/IP reponse message.

MVC-99867 - setCamControl - Locked by client - Invalid controlValue - Negative response

When the application software receives a valid setCamControl SOME/IP request from the client that currently holds the lock, a response with message type `ERROR (0x81)`, return code `E_INVALID_CONTROL_VALUE (0x37)` and **no payload** shall be sent if the controlValue is invalid.

Status	Rejected
Verification Criteria	Send a setCamExclusive with a client. Send a valid setCamControl with an invalid control value with same client and check SOME/IP reponse message.

7.3.8 Method getDataBlock

MVC-102665 - getDataBlock - Method Validation

When the application software receives a valid SOME/IP `REQUEST` message, it shall accept the request as a valid **getDataBlock** SOME/IP `REQUEST` if SOME/IP header field **Method ID** is set to `0x0251`, **Length** is set to 12 and the following fields as payload:

Byte Index	Input Parameter	Base Type	Description
0	deviceId	uint16	Device target from which the data block needs to be retreived from
2	blockId	uint16	Block Identifier

Status	Ready for review
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-109051 - getDataBlock - Method Length

When the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Method ID** set to `0x0251` and field **Length** set to a value other than `12`, it shall respond with a SOME/IP `ERROR` message with **Return Code** set to `E_MALFORMED_MESSAGE`.

Status	Specified
Verification Criteria	Send getDataBlock REQUEST mesage with wrong payload (correct length for given payload), expect ERROR response with return code E_MALFORMED_MESSAGE

MVC-102667 - getDataBlock - Invalid Block Definition

When the application software receives a valid **getDataBlock** SOME/IP `REQUEST` message, it shall respond with a SOME/IP `ERROR` message with **Message Type** 0x81, **Return Code** 0x50 (`E_INVALID_BLOCK_DEFINITION`) and **no payload** if the block definition is not part of the supported blocks according to MVC-102146 - getDatablock- supported blocks in devices .

Status	Ready for review
Verification Criteria	Send getDataBlock message with invalid block definitions, receive error message and check for return code E_INVALID_BLOCK_DEFINITION`.

MVC-102668 - getDataBlock - Method validation - Valid block definition

When the application software receives a valid **getDataBlock** SOME/IP `REQUEST` message, it shall respond with a SOME/IP `RESPONSE` message with **Message Type** `0x80`, **Return Code** `0x00 (E_OK)` and the following fields as payload if the block definition is valid as described in MVC-102146 - getDatablock- supported blocks in devices :

Byte Index	Output Parameter	Base Type	Description
0	blockSize	uint32	Size of blockData in Bytes
4	blockData	uint8 Array	Block data

Status	Ready for review
Verification Criteria	Send getDataBlock message with valid block definitions, receive response message and check for expected output parameters and values

7.3.9 Method setDataBlock

MVC-102708 - setDataBlock - Method Validation

When the application software receives a valid SOME/IP `REQUEST` message, it shall accept the request as a valid **setDataBlock** SOME/IP `REQUEST` if SOME/IP header field **Method ID** is set to `0x0250`, **Length** is set to 16 + value of **blockSize** and the following fields as payload:

Byte Index	Input Parameter	Base Type	Description
0	deviceId	uint16	Deivce target to which the data block gets written to
2	blockId	uint16	Block identifier
4	blockSize	uint32	Block size in Bytes
8	blockContent	uint8 Array	Block data

Status	Approved
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-109052 - setDataBlock - Method Length

When the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Method ID** set to `0x0250` and field **Length** set to a value other than 16 + value of **blockSize** according to MVC-102708 - setDataBlock - Method Validation , it shall respond with a SOME/IP `ERROR` message with **Return Code** set to `E_MALFORMED_MESSAGE`.

Note: If the given payload does not include the **blockSize** element the method length will always be mismatched as described in MVC-92418 - SOME/IP - Method Length Mismatch . This means that the minimum viable **Length** is 16 with a **blockSize** of 0 and **blockContent** holding 0 bytes of data.

Status	Approved
Verification Criteria	Send setDataBlock REQUEST message with wrong payload (correct length for given payload) e.g. add an additional Byte to blockContent that is not accounted for in blockSize, expect ERROR response with return code E_MALFORMED_MESSAGE

MVC-102731 - setDataBlock - Invalid Block Definition

When the application software receives a valid **setDataBlock** SOME/IP `REQUEST` message, it shall respond with a SOME/IP `ERROR` message with **Message Type** `0x81`, **Return Code** 0x50 (`E_INVALID_BLOCK_DEFINITION`) and **no payload** if the block definition is not part of the supported blocks described in MVC-102134 - setDatablock- supported blocks in devices .

Status	Approved
Verification Criteria	Send setDataBlock message with invalid block definitions, receive error message and check for return code E_INVALID_BLOCK_DEFINITION`.

MVC-102732 - setDataBlock - Method validation - Valid block definition

When the application software receives a valid **setDataBlock** SOME/IP `REQUEST` message, it shall respond with a SOME/IP `RESPONSE` message with **Message Type** 0x80, **Return Code** 0x00 (`E_OK`) and **no payload** if the block definition is valid as described in MVC-102731 - setDataBlock - Invalid Block Definition.

Status	Approved

Verification Criteria	Send setDataBlock message with valid block definitions, send getDataBlock message and check response for expected output parameters and values

7.3.10 Method subscribeRoiVideo

MVC-109182 - subscribeRoiVideo - Method Validation

When the application software receives a valid SOME/IP `REQUEST` message, it shall accept the request as a valid **subscribeRoiVideo** SOME/IP `REQUEST` if SOME/IP header fields **Method ID** is set to `0x0131`, **Length** is set to 12 and the payload has the following fields:

Byte Index	Input Parameter	Base Type	Description
0	roiIndex	uint32	The Index for the requested Region of Interest (ROI)

Status	Ready for review
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-109183 - subscribeRoiVideo - Method Length

When the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Method ID** set to `0x0131` and **Length** field set to a value other than 12, it shall respond with a SOME/IP `ERROR` message with **Return Code** set to `E_MALFORMED_MESSAGE`.

Status	Ready for review
Verification Criteria	Send subscribeRoiVideo REQUEST message with wrong payload (correct length for given payload), expect ERROR response with return code E_MALFORMED_MESSAGE

MVC-95730 - subscribeRoiVideo - Invalid roiIndex

When the application software receives a valid **subscribeRoiVideo** SOME/IP `REQUEST` message with an invalid **`roiIndex`** then it shall send a SOME/IP `ERROR` message with return code `E_INVALID_ROI_INDEX (0x30)`.

Status	Ready for review
Verification Criteria	Application software responds with a negative response if an invalid roiIndex is received

7.3.10.1 subscribeRoiVideo - Payload Validation

MVC-95735 - subscribeRoiVideo - Payload Validation - roiIndex

When the application software receives a valid **subscribeRoiVideo** SOME/IP `REQUEST` message, it shall validate the following fields in the **roiIndex** payload parameter field:

Bit Index	Input Parameter	Bit Range	Description
0	ROI Presets	29 - 0	MVC-95750- SubscribeROIVideo - SOME/IP Payload Field - roiIndex - evaluation - Input parameter - ROIPresets
30	Mirror	30	MVC-95759- SubscribeROIVideo - SOME/IP Payload Field - roiIndex - evaluation - Input parameter - Mirror
31	Flip	31	MVC-95762- SubscribeROIVideo - SOME/IP Payload Field - roiIndex - evaluation - Input parameter - Flip

When all input parameters are in range the `roiIndex` is evaluated as valid otherwise it is considered invalid.

Status	Ready for review
Verification Criteria	Application software responds with a positive response when each input parameter is in valid range otherwise roiIndex is considered invalid and a negative response is send

MVC-95737 - subscribeRoiVideo - Payload Validation - roiIndex - Read Order

The application software shall interpret the parameter **roiIndex** in MSB (most significant bit) order.

Status	Ready for review
Verification Criteria	Verify input parameters order of roiIndex evulation based on table provided in parent requirement

MVC-95750 - subscribeRoiVideo - Payload Validation - roiIndex - ROI Presets

The application software shall support the following binary values for **ROI Presets**:

- ROI 0 : `0b0000`
- ROI 1 : `0b0001`
- ROI 2 : `0b0010`
- ROI 3 : `0b0011`
- ROI 4 : `0b0100`
- ROI 5 : `0b0101`
- ROI 9 : `0b1001`
- ROI 10: `0b1010`

Note:

Currently, it is not possible to add a link to the relevant system requirements, so the allocation is listed here:

MVC-100053 - subscribeRoiVideo - E_OK

Status	Draft
Verification Criteria	Check if supported binary values are considered a valid input parameter which yields a valid roiIndex if all other input parameters are valid.

MVC-95759 - subscribeRoiVideo - Payload Validation - roiIndex - Mirror

The application software shall support the following binary values:

- 0b0 = image mirrored
- 0b1 = image not mirrored

Note:

Currently, it is not possible to add a link to the relevant system requirements, so the allocation is listed here:

MVC-100053 - subscribeRoiVideo - E_OK

Status	Ready for review
Verification Criteria	Check if supported binary values are considered a valid input parameter which yields a valid roiIndex if all other input parameters are valid.

MVC-95762 - subscribeRoiVideo - Payload Validation - roiIndex - Flip

The application software shall support the following binary values:

- 0b0 = image flipped
- 0b1 = image not flipped

Note:

Currently, it is not possible to add a link to the relevant system requirements, so the allocation is listed here:

MVC-100053 - subscribeRoiVideo - E_OK

Status	Ready for review
Verification Criteria	Check if supported binary values are considered a valid input parameter which yields a valid roiIndex if all other input parameters are valid.

MVC-95761 - SubscribeROIVideo - SOME/IP Payload Field - roiIndex - evaluation - Input parameter - Reserved

The application software shall treat the following `roiIndex:ROIPreset` values as reserved:

- 0b0110
- 0b0111
- 0b1000

Requests containing these values shall be evaluated as invalid.

Note:

Currently, it is not possible to add a link to the relevant system requirements, so the allocation is listed here:

MVC-100053 - subscribeRoiVideo - E_OK

Status	Rejected
Verification Criteria	Check if supported binary values are considered a valid input parameter which yields a valid roiIndex if all other input parameters are valid.

7.3.11 Method UnSubscribeROIVideo

MVC-109187 - unsubscribeRoiVideo - Method Validation

When the application software receives a valid SOME/IP `REQUEST` message, it shall accept the request as a valid **unsubscribeRoiVideo** SOME/IP `REQUEST` if SOME/IP header fields **Method ID** is set to `0x0132`, **Length** is set to 12 and the payload has the following fields:

Byte Index	Input Parameter	Base Type	Description
0	roiIndex	uint32	The Index for the requested Region of Interest (ROI)

Status	Ready for review
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-109186 - unsubscribeRoiVideo - Method Length

When the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Method ID** set to `0x0132` and **Length** field set to a value other than 12, it shall respond with a SOME/IP `ERROR` message with **Return Code** set to `E_MALFORMED_MESSAGE`.

Status	Ready for review
Verification Criteria	Send unSubscribeRoiVideo REQUEST message with wrong payload (correct length for given payload), expect ERROR response with return code E_MALFORMED_MESSAGE

MVC-95813 - unSubscribeROIVideo - Invalid roiIndex

When the application software receives a valid **unSubscribeROIVideo** SOME/IP `REQUEST` message with an invalid **roiIndex** then it shall send a SOME/IP `ERROR` message with return code `E_INVALID_ROI_INDEX (0x30)`.

Status	Ready for review
Verification Criteria	Application software responds with a SOME/IP ERROR response if an invalid roiIndex is received (none is invalid when only 1 stream is running)

7.3.11.1 unSubscribeRoiVideo - Payload Validation

MVC-95811 - unSubscribeRoiVideo - Payload Validation - roiIndex

The application software shall interpret the provided field **roiIndex** of a valid **unSubscribeROIVideo** SOME/IP `REQUEST` message as follows:

Bit Index	Input Parameter	Bit Range	Description
0	ROIPreset	29 - 0	see MVC-95750 - subscribeRoiVideo - Payload Validation - roiIndex - ROI Presets for valid indices
30	Mirror	30	Ignored for `unSubscribeROIVideo`
31	Flip	31	Ignored for `unSubscribeROIVideo`

When all input parameters are in range the **roiIndex** is evaluated as valid otherwise it is considered invalid.

Status	Ready for review
Verification Criteria	Check if supported binary values are considered a valid input parameter which yields a valid roiIndex if all other input parameters are valid.

MVC-95814 - unSubscribeROIVideo - SOME/IP Payload Field - roiIndex - Read Order

The application software shall interpret the parameter **roiIndex** in MSB (most significant bit) order.

Status	Ready for review
Verification Criteria	Verify input parameters order of roiIndex evulation based on table provided in parent requirement

MVC-95815 - UnSubscribeROIVideo - SOME/IP Payload Field - roiIndex - evaluation - Input parameter -Stream Selection

The application software shall support the following binary values:

- 0b00 = HV ISP + IP Config 1
- 0b01 = HV ISP + IP Config 2

All other binary values are considered invalid.

Status	Rejected
Verification Criteria	Check if supported binary values are considered a valid input parameter which yields a valid roiIndex if all other input parameters are valid.

7.3.12 Method getCamStatus

MVC-108496 - getCamStatus - Method Validation

When the application software receives a valid SOME/IP `REQUEST` message with SOME/IP header fields **Method ID** set to `0x0002`, **Length** set to `8` and **no payload**, it shall accept the request as a valid **getCamStatus** SOME/IP `REQUEST`.

Status	Approved
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-108497 - getCamStatus - Method Length

When the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Method ID** set to `0x0002` and field **Length** set to a value other than `8 (0x08)`, it shall respond with a SOME/IP `ERROR` message with **Return Code** set to `E_MALFORMED_MESSAGE`.

Status	Approved
Verification Criteria	Send a getCamStatus REQUEST message with SOME/IP header field Length set to a value >8 (add bytes to the payload according to the length value, so the SOME/IP Message has a valid length); expect an ERROR response with Return Code E_MALFORMED_MESSAGE.

MVC-108498 - getCamStatus - Positive Response

When the application software receives a valid **getCamStatus** SOME/IP `REQUEST` message, it shall send a SOME/IP `RESPONSE` message with SOME/IP header fields **Length** set to `15 (0x0F)`, **Message Type** set to `RESPONSE (0x80)`, **Return Code** set to `E_OK (0x00)` and the following fields as **payload**:

ByteIndex	Output Parameter	BaseType	Description
0	lockedBy	uint32	User of the camera (e.g. IP address) is set/unset by specific methods.
4	powerStatus	uint8	External Error Code with the highest priority based on the definition in MVC-102677 - external error codes - priority ***NOTE:** While External Error Codes are not supported this parameter will be filled with zeros.*
5	Temperature	uint16	Actual temperature of the camera imager temperature. Format: 16bit temperature (see ISO17215-3 6.6.25 Temperature)

Status	Approved
Verification Criteria	Send a valid getCamStatus SOME/IP REQUEST message, ensure that the SOME/IP RESPONSE message has the correct values

MVC-108628 - getCamStatus - Serialisation of lockedBy

The application software shall serialise the lockedBy field with Big-Endian (MSB).

Example:
"192.168.10.1 -> 0xc0a80a01"

Status	Approved
Verification Criteria	After receiving a valid SOME/IP message with methodID getCamStatus the application software responds with a valid response and payload shall contain the lockedBy field. This filed shall be compored to Ip of exlusive ip and checked for difference.

MVC-108629 - getCamStatus - powerStatus

The application software shall set the powerStatus to 0x00, as it is currently not used.

Status	Approved
Verification Criteria	After receiving a valid SOME/IP message with methodID getCamStatus the application software responds with a valid response and payload shall contain the power status set to 0x00.

7.4 Software Deployment Methods

7.4.1 Hardware Information Service Methods

MVC-118236 - Hardware Information Service Methods - Service ID

If the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Service ID** set to `0x05DB`, it shall process the received message as a Hardware Information Service Method.

Status	Draft
Verification Criteria	When sending SOME/IP REQUEST messages with ServiceId 0x05DB, the behaviour of the application software must reflect the expected behaviour for Hardware Information Service Methods. This shall be verified in conjunction with Requirements that yield a response message by application software.

7.4.1.1 Method getId

MVC-113886 - getId - Method Validation

When the application software receives a valid SOME/IP `REQUEST` message with SOME/IP header fields **Method ID** set to `0x0066`, **Length** set to `8` and **no payload**, it shall accept the request as a valid **getId** SOME/IP `REQUEST`.

Status	Draft
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-113885 - getId - Method Length

When the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Method ID** set to `0x0066` and field **Length** set to a value other than `8 (0x08)`, it shall respond with a SOME/IP `ERROR` message with **Return Code** set to `E_MALFORMED_MESSAGE`.

Status	Draft
Verification Criteria	Send a getId REQUEST message with SOME/IP header field Length set to a value other than 8 (e.g. 4 or 12); expect an ERROR response with Return Code E_MALFORMED_MESSAGE.

MVC-113887 - getId - Positive Response

When the application software receives a valid **getId** SOME/IP `REQUEST` message, it shall send a SOME/IP `RESPONSE` message with SOME/IP header fields **Length** set to `50(0x32)`, **Message Type** set to `RESPONSE (0x80)`, **Return Code** set to `E_OK (0x00)` and the following fields as **payload**:

ByteIndex	Output Parameter	BaseType	Description
0	?	uint8	instance identifier of Type `IdentifierType??`

Status	Draft
Verification Criteria	

MVC-114934 - getId - Negative Response - Internal error

When the application software receives a valid **getId** SOME/IP request, a response with message type ERROR `(0x81)`, return code `internalError (0x02)` and **no payload** shall be sent if the application software encountered an internal error.

Status	Draft
Verification Criteria	

7.4.1.2 Method getHardwareIdentification

MVC-113889 - getHardwareIdentification - Method Validation

When the application software receives a valid SOME/IP `REQUEST` message with SOME/IP header fields **Method ID** set to `0x0065`, **Length** set to `8` and **no payload**, it shall accept the request as a valid **getHardwareIdentification** SOME/IP `REQUEST`.

Status	Draft
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-113892 - getHardwareIdentification - Method Length

When the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Method ID** set to `0x0065` and field **Length** set to a value other than `8 (0x08)`, it shall respond with a SOME/IP `ERROR` message with **Return Code** set to `E_MALFORMED_MESSAGE`.

Status	Draft
Verification Criteria	Send a getHardwareIdentification REQUEST message with SOME/IP header field Length set to a value other than 8 (e.g. 4 or 12); expect an ERROR response with Return Code E_MALFORMED_MESSAGE.

MVC-113891 - getHardwareIdentification - Positive Response

When the application software receives a valid **getHardwareIdentification** SOME/IP `REQUEST` message, it shall send a SOME/IP `RESPONSE` message with SOME/IP header fields **Length** set to `28 (0x1C)`, **Message Type** set to `RESPONSE (0x80)`, **Return Code** set to `E_OK (0x00)` and the following fields as **payload**:

ByteIndex	Output Parameter	BaseType	Description
0	module_type	uint8	MVC-113894 - getHardwareIdentification - SOME/IP Payload Field - module_type
1	part_number	uint8	MVC-113893 - getHardwareIdentification - SOME/IP Payload Field - part_number
9	revision	uint8	MVC-113896 - getHardwareIdentification - SOME/IP Payload Field - revision
12	serial_number	uint8	MVC-113895 - getHardwareIdentification - SOME/IP Payload Field - serial_number

Status	Draft
Verification Criteria	

MVC-113894 - getHardwareIdentification - SOME/IP Payload Field - module_type

When the application software receives a valid **getHardwareIdentification** SOME/IP `REQUEST` message, it shall set the field `module_type` of the corresponding SOME/IP `RESPONSE` message to the value based on hardware component type with the following valid values:

Range/Symbol	Limit	Description
Hardware	0x00	Main hardware
Sub-Hardware	0x01	Sub-hardware

Status	Draft
Verification Criteria	

MVC-113893 - getHardwareIdentification - SOME/IP Payload Field - part_number

When the application software receives a valid **getHardwareIdentification** SOME/IP `REQUEST` message, it shall set the field `part_number` of the corresponding SOME/IP `RESPONSE` message to the value of the NVM production-data field `partNumber`.

Status	Draft
Verification Criteria	

MVC-113896 - getHardwareIdentification - SOME/IP Payload Field - revision

When the application software receives a valid **getHardwareIdentification** SOME/IP `REQUEST` message, it shall set the field `revision` of the corresponding SOME/IP `RESPONSE` message to the value of the NVM production-data field `snpn??`.

Status	Draft
Verification Criteria	

MVC-113895 - getHardwareIdentification - SOME/IP Payload Field - serial_number

When the application software receives a valid **getHardwareIdentification** SOME/IP `REQUEST` message, it shall set the field `serial_number` of the corresponding SOME/IP `RESPONSE` message to the value of the NVM production-data field `serialNumber`.

Status	Draft
Verification Criteria	

MVC-114933 - getHardwareIdentification - Negative Response - Internal error

When the application software receives a valid **getHardwareIdentification** SOME/IP request, a response with message type ERROR `(0x81)`, return code `internalError (0x02)` and **no payload** shall be sent if the application software encountered an internal error.

Status	Draft
Verification Criteria	

7.4.2 Package Management Service

MVC-118237 - Package Management Service Methods - Service ID

If the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Service ID** set to `0x0586`, it shall process the received message as a Package Management Service Method.

Status	Draft
Verification Criteria	When sending SOME/IP REQUEST messages with ServiceId 0x0586, the behaviour of the application software must reflect the expected behaviour for Package Management Service Methods. This shall be verified in conjunction with Requirements that yield a response message by application software.

7.4.2.1 Method CurrentStatus

TODO: Add requirements in SD to send a notification as soon as current status was changed

MVC-118901 - CurrentStatus - Notification

When the UCM changes its current Status, the application software shall send a SOME/IP NOTIFICATION message with SOME/IP header field **EventId** set to `2` (`0x0002`), and all other fields as specified in MVC-113908 - CurrentStatus - Positive Response.

Status	Draft
Verification Criteria	

MVC-113906 - CurrentStatus - Method Validation

When the application software receives a valid SOME/IP `REQUEST` message with SOME/IP header fields **Method ID** set to `0x0001`, **Length** set to `8` and **no payload**, it shall accept the request as a valid **CurrentStatus** SOME/IP `REQUEST`.

Status	Draft
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-113909 - CurrentStatus - Method Length

When the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Method ID** set to `0x0001` and field **Length** set to a value other than `8 (0x08)`, it shall respond with a SOME/IP `ERROR` message with **Return Code** set to `E_MALFORMED_MESSAGE`.

Status	Draft
Verification Criteria	Send a CurrentStatus REQUEST message with SOME/IP header field Length set to a value other than 8 (e.g. 4 or 12); expect an ERROR response with Return Code E_MALFORMED_MESSAGE.

MVC-113908 - CurrentStatus - Positive Response

When the application software receives a valid **CurrentStatus** SOME/IP `REQUEST` message, it shall send a SOME/IP `RESPONSE` message with SOME/IP header fields **Length** set to `10 (0x0A)`, **Message Type** set to `RESPONSE (0x80)`, **Return Code** set to `E_OK (0x00)` and the following fields as **payload**:

ByteIndex	Output Parameter	BaseType	Description
0	updateState	uint8	MVC-113911 - CurrentStatus - SOME/IP Payload Field - updateState
1	runningState	uint8	MVC-113910 - CurrentStatus - SOME/IP Payload Field - runningState

Status	Draft
Verification Criteria	

MVC-113911 - CurrentStatus - SOME/IP Payload Field - updateState

When the application software receives a valid **CurrentStatus** SOME/IP `REQUEST` message, it shall set the field `updateState` of the corresponding SOME/IP `RESPONSE` message to the value of the current state of the UCM.

Status	Draft
Verification Criteria	

MVC-113910 - CurrentStatus - SOME/IP Payload Field - runningState

When the application software receives a valid **CurrentStatus** SOME/IP `REQUEST` message, it shall set the field `runningState` of the corresponding SOME/IP `RESPONSE` message to the hard-coded value `kRunning` (0x00).

Status	Draft
Verification Criteria	

7.4.2.2 Method GetHistory

MVC-113916 - GetHistory - Method Validation

The application software shall accept the SOME/IP REQUEST as a valid **GetHistory** SOME/IP REQUEST when SOME/IP header fields **Method ID** set to `0x0007`, **Length** set to `24 (0x18)` and the payload has the following fields:

ByteIndex	Input Parameter	BaseType	Description
0	timestampGE	uint64	represents the earliest timestamp (inclusive)
9	timestampLT	uint64	represent the latest timestamp (exlusive)

Status	Draft
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-113919 - GetHistory - Method Length

When the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Method ID** set to `0x0007` and field **Length** set to a value other than 24, it shall respond with a SOME/IP `ERROR` message with **Return Code** set to `E_MALFORMED_MESSAGE`.

Status	Draft
Verification Criteria	Send a GetHistory REQUEST message with SOME/IP header field Length set to a value other than 24 (e.g. 4 or 12); expect an ERROR response with Return Code E_MALFORMED_MESSAGE.

MVC-113918 - GetHistory - Positive Response

When the application software receives a valid **GetHistory** SOME/IP `REQUEST` message, it shall send a SOME/IP `RESPONSE` message with SOME/IP header fields **Length** set to `64 (0x40)`, **Message Type** set to `RESPONSE (0x80)`, **Return Code** set to `E_OK (0x00)` and the following fields as **payload**:

ByteIndex	Output Parameter	BaseType	Description
0	time	uint64	MVC-113921 - GetHistory - SOME/IP Payload Field - time
8	swClusterName	SwClusterNameType	MVC-113920 - GetHistory - SOME/IP Payload Field - swClusterName
??	version	StrongRevisionLabelString	MVC-113923 - GetHistory - SOME/IP Payload Field - version
54	action	uint8	MVC-113922 - GetHistory - SOME/IP Payload Field - action
55	resolution	uint8	MVC-113914 - GetHistory - SOME/IP Payload Field - resolution
56	failureError	uint64	MVC-113915 - GetHistory - SOME/IP Payload Field - failureError

Status	Draft
Verification Criteria	

MVC-113921 - GetHistory - SOME/IP Payload Field - time

When the application software receives a valid **GetHistory** SOME/IP `REQUEST` message, it shall set the field `time` of the corresponding SOME/IP `RESPONSE` message to the value of **TBD**.

Status	Draft
Verification Criteria	

MVC-113920 - GetHistory - SOME/IP Payload Field - swClusterName

When the application software receives a valid **GetHistory** SOME/IP `REQUEST` message, it shall set the field `swClusterName` of the corresponding SOME/IP `RESPONSE` message to the value of **TBD**.

Status	Draft
Verification Criteria	

MVC-113923 - GetHistory - SOME/IP Payload Field - version

When the application software receives a valid **GetHistory** SOME/IP `REQUEST` message, it shall set the field `version` of the corresponding SOME/IP `RESPONSE` message to the value of **TBD**.

Status	Draft
Verification Criteria	

MVC-113922 - GetHistory - SOME/IP Payload Field - action

When the application software receives a valid **GetHistory** SOME/IP `REQUEST` message, it shall set the field `action` of the corresponding SOME/IP `RESPONSE` message to the hard-coded value `kUpdate` (0x00).

Status	Draft
Verification Criteria	

MVC-113914 - GetHistory - SOME/IP Payload Field - resolution

When the application software receives a valid **GetHistory** SOME/IP `REQUEST` message, it shall set the field `resolution` of the corresponding SOME/IP `RESPONSE` message to the value of current action of the UCM.

Status	Draft
Verification Criteria	

MVC-113915 - GetHistory - SOME/IP Payload Field - failureError

When the application software receives a valid **GetHistory** SOME/IP `REQUEST` message, it shall set the field `failureError` of the corresponding SOME/IP `RESPONSE` message to the value of **TBD**.

Status	Draft
Verification Criteria	

7.4.2.3 Method GetId

MVC-113924 - GetId - Method Validation

When the application software receives a valid SOME/IP `REQUEST` message with SOME/IP header fields **Method ID** set to `0x0008`, **Length** set to `8` and **no payload**, it shall accept the request as a valid **GetId** SOME/IP `REQUEST`.

Status	Draft
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-113927 - GetId - Method Length

When the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Method ID** set to `0x0008` and field **Length** set to a value other than `8 (0x08)`, it shall respond with a SOME/IP `ERROR` message with **Return Code** set to `E_MALFORMED_MESSAGE`.

Status	Draft
Verification Criteria	Send a GetId REQUEST message with SOME/IP header field Length set to a value other than 8 (e.g. 4 or 12); expect an ERROR response with Return Code E_MALFORMED_MESSAGE.

MVC-113926 - GetId - Positive Response

When the application software receives a valid **GetId** SOME/IP `REQUEST` message, it shall send a SOME/IP `RESPONSE` message with SOME/IP header fields **Length** set to `32(0x20)`, **Message Type** set to `RESPONSE (0x80)`, **Return Code** set to `E_OK (0x00)` and the following fields as **payload**:

ByteIndex	Output Parameter	BaseType	Description
0	id	ASCII	MVC-113929 - GetId - SOME/IP Payload Field - id
??	version	ASCII	MVC-113928 - GetId - SOME/IP Payload Field - version

Status	Draft
Verification Criteria	

MVC-113929 - GetId - SOME/IP Payload Field - id

When the application software receives a valid **GetId** SOME/IP `REQUEST` message, it shall set the field `id` of the corresponding SOME/IP `RESPONSE` message to the value **TBD**.

Status	Draft
Verification Criteria	

MVC-113928 - GetId - SOME/IP Payload Field - version

When the application software receives a valid **GetId** SOME/IP `REQUEST` message, it shall set the field `version` of the corresponding SOME/IP `RESPONSE` message to the value **TBD**.

Status	Draft
Verification Criteria	

7.4.2.4 Method GetProgress

MVC-113938 - GetProgress - Method Validation

When the application software receives a valid SOME/IP `REQUEST` message with SOME/IP header fields **Method ID** set to `0x0014`, **Length** set to `8` and **no payload**, it shall accept the request as a valid **GetProgress** SOME/IP `REQUEST`.

Status	Draft
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-113941 - GetProgress - Method Length

When the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Method ID** set to `0x0014` and field **Length** set to a value other than `8 (0x08)`, it shall respond with a SOME/IP `ERROR` message with **Return Code** set to `E_MALFORMED_MESSAGE`.

Status	Draft
Verification Criteria	Send a GetProgress REQUEST message with SOME/IP header field Length set to a value other than 8 (e.g. 4 or 12); expect an ERROR response with Return Code E_MALFORMED_MESSAGE.

MVC-113940 - GetProgress - Positive Response

When the application software receives a valid **GetProgress** SOME/IP `REQUEST` message, it shall send a SOME/IP `RESPONSE` message with SOME/IP header fields **Length** set to `14(0x0E)`, **Message Type** set to `RESPONSE (0x80)`, **Return Code** set to `E_OK (0x00)` and the following fields as **payload**:

ByteIndex	Output Parameter	BaseType	Description
0	currentStatus	uint8	MVC-113943 - GetProgress - SOME/IP Payload Field - currentStatus
1	progress	uint8	MVC-113942 - GetProgress - SOME/IP Payload Field - progress
2	estimatedDuration	uint32	MVC-113944 - GetProgress - SOME/IP Payload Field - estimatedDuration

Status	Draft
Verification Criteria	

MVC-113943 - GetProgress - SOME/IP Payload Field - currentStatus

When the application software receives a valid **GetProgress** SOME/IP `REQUEST` message, it shall set the field `currentStatus` of the corresponding SOME/IP `RESPONSE` message to the value of the current status of the UCM.

Status	Draft
Verification Criteria	

MVC-113942 - GetProgress - SOME/IP Payload Field - progress

When the application software receives a valid **GetProgress** SOME/IP `REQUEST` message, it shall set the field `progress` of the corresponding SOME/IP `RESPONSE` message to the value of the progress of the software download from the UCM.

Status	Draft
Verification Criteria	

MVC-113944 - GetProgress - SOME/IP Payload Field - estimatedDuration

When the application software receives a valid **GetProgress** SOME/IP `REQUEST` message, it shall set the field `estimatedDuration` of the corresponding SOME/IP `RESPONSE` message to the estimated duration in seconds until the software download is complete. This is reported by the UCM .

Status	Draft
Verification Criteria	

7.4.2.5 Method GetSwClusterChangeInfo

MVC-113945 - GetSwClusterChangeInfo - Method Validation

When the application software receives a valid SOME/IP `REQUEST` message with SOME/IP header fields **Method ID** set to `0x0009`, **Length** set to `8` and **no payload**, it shall accept the request as a valid **GetSwClusterChangeInfo** SOME/IP `REQUEST`.

Status	Draft
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-113948 - GetSwClusterChangeInfo - Method Length

When the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Method ID** set to `0x0009` and field **Length** set to a value other than `8 (0x08)`, it shall respond with a SOME/IP `ERROR` message with **Return Code** set to `E_MALFORMED_MESSAGE`.

Status	Draft
Verification Criteria	Send a GetSwClusterChangeInfo REQUEST message with SOME/IP header field Length set to a value other than 8 (e.g. 4 or 12); expect an ERROR response with Return Code E_MALFORMED_MESSAGE.

MVC-113947 - GetSwClusterChangeInfo - Positive Response

When the application software receives a valid **GetSwClusterChangeInfo** SOME/IP `REQUEST` message, it shall send a SOME/IP `RESPONSE` message with SOME/IP header fields **Length** set to `32 (0x20)`, **Message Type** set to `RESPONSE (0x80)`, **Return Code** set to `E_OK (0x00)` and the following fields as **payload**:

ByteIndex	Output Parameter	BaseType	Description
0	name	ASCII	MVC-113950 - GetSwClusterChangeInfo - SOME/IP Payload Field - name
??	version	ASCII	MVC-113949 - GetSwClusterChangeInfo - SOME/IP Payload Field - version
??	state	ASCII	MVC-113952 - GetSwClusterChangeInfo - SOME/IP Payload Field - state
??	size	uint64	MVC-113951 - GetSwClusterChangeInfo - SOME/IP Payload Field - size

Status	Draft
Verification Criteria	

MVC-113950 - GetSwClusterChangeInfo - SOME/IP Payload Field - name

When the application software receives a valid **GetSwClusterChangeInfo** SOME/IP `REQUEST` message, it shall set the field `name` of the corresponding SOME/IP `RESPONSE` message to the value of **TBD**.

Status	Draft
Verification Criteria	

MVC-113949 - GetSwClusterChangeInfo - SOME/IP Payload Field - version

When the application software receives a valid **GetSwClusterChangeInfo** SOME/IP `REQUEST` message, it shall set the field `version` of the corresponding SOME/IP `RESPONSE` message to the value of **TBD**.

Status	Draft
Verification Criteria	

MVC-113952 - GetSwClusterChangeInfo - SOME/IP Payload Field - state

When the application software receives a valid **GetSwClusterChangeInfo** SOME/IP `REQUEST` message, it shall set the field `state` of the corresponding SOME/IP `RESPONSE` message to the value of **TBD**.

Status	Draft
Verification Criteria	

MVC-113951 - GetSwClusterChangeInfo - SOME/IP Payload Field - size

When the application software receives a valid **GetSwClusterChangeInfo** SOME/IP `REQUEST` message, it shall set the field `state` of the corresponding SOME/IP `RESPONSE` message to the value of **TBD**.

Status	Draft
Verification Criteria	

7.4.2.6 Method GetSwClusterInfo

MVC-113958 - GetSwClusterInfo - Method Validation

When the application software receives a valid SOME/IP `REQUEST` message with SOME/IP header fields **Method ID** set to `0x000B`, **Length** set to `8` and **no payload**, it shall accept the request as a valid **GetSwClusterInfo** SOME/IP `REQUEST`.

Status	Draft
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-113957 - GetSwClusterInfo - Method Length

When the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Method ID** set to `0x000B` and field **Length** set to a value other than `8 (0x08)`, it shall respond with a SOME/IP `ERROR` message with **Return Code** set to `E_MALFORMED_MESSAGE`.

Status	Draft
Verification Criteria	Send a GetSwClusterInfo REQUEST message with SOME/IP header field Length set to a value other than 8 (e.g. 4 or 12); expect an ERROR response with Return Code E_MALFORMED_MESSAGE.

MVC-113960 - GetSwClusterInfo - Positive Response

When the application software receives a valid **GetSwClusterInfo** SOME/IP `REQUEST` message, it shall send a SOME/IP `RESPONSE` message with SOME/IP header fields **Length** set to `32 (0x20)`, **Message Type** set to `RESPONSE (0x80)`, **Return Code** set to `E_OK (0x00)` and the following fields as **payload**:

ByteIndex	Output Parameter	BaseType	Description
0	name	ASCII	MVC-113959 - GetSwClusterChangeInfo - SOME/IP Payload Field - name
??	version	ASCII	MVC-113962 - GetSwClusterChangeInfo - SOME/IP Payload Field - version
??	state	ASCII	MVC-113961 - GetSwClusterChangeInfo - SOME/IP Payload Field - state
??	size	uint64	MVC-113954 - GetSwClusterChangeInfo - SOME/IP Payload Field - size

Status	Draft
Verification Criteria	

MVC-113959 - GetSwClusterChangeInfo - SOME/IP Payload Field - name

When the application software receives a valid **GetSwClusterChangeInfo** SOME/IP `REQUEST` message, it shall set the field `name` of the corresponding SOME/IP `RESPONSE` message to the value of **TBD**.

Status	Draft

Verification Criteria	

MVC-113962 - GetSwClusterChangeInfo - SOME/IP Payload Field - version

When the application software receives a valid **GetSwClusterChangeInfo** SOME/IP `REQUEST` message, it shall set the field `version` of the corresponding SOME/IP `RESPONSE` message to the value of **TBD**.

Status	Draft
Verification Criteria	

MVC-113961 - GetSwClusterChangeInfo - SOME/IP Payload Field - state

When the application software receives a valid **GetSwClusterChangeInfo** SOME/IP `REQUEST` message, it shall set the field `state` of the corresponding SOME/IP `RESPONSE` message to the value of **TBD**.

Status	Draft
Verification Criteria	

MVC-113954 - GetSwClusterChangeInfo - SOME/IP Payload Field - size

When the application software receives a valid **GetSwClusterChangeInfo** SOME/IP `REQUEST` message, it shall set the field `state` of the corresponding SOME/IP `RESPONSE` message to the value of **TBD**.

Status	Draft
Verification Criteria	

7.4.2.7 Method GetSwClusterManifestInfo

MVC-113965 - GetSwClusterManifestInfo - Method Validation

The application software shall accept the SOME/IP REQUEST as a valid **GetSwClusterManifestInfo** SOME/IP `REQUEST` when SOME/IP header fields **Method ID** set to `0x000A`, **Length** set to `32 (0x20)` and the payload has the following fields:

ByteIndex	Input Parameter	BaseType	Description
0	??	SwClusterTypeName?	??

Status	Draft
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-113964 - GetSwClusterManifestInfo - Method Length

When the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Method ID** set to `0x000A` and field **Length** set to a value other than 32 (0x20), it shall respond with a SOME/IP `ERROR` message with **Return Code** set to `E_MALFORMED_MESSAGE`.

Status	Draft
Verification Criteria	Send a GetSwClusterManifestInfo REQUEST message with SOME/IP header field Length set to a value other than 32 (e.g. 4 or 12); expect an ERROR response with Return Code E_MALFORMED_MESSAGE.

MVC-113972 - GetSwClusterManifestInfo - ApplicationError

When the application software receives a valid **GetSwClusterManifestInfo** SOME/IP `REQUEST` message, it shall respond with a SOME/IP `ERROR` message with **Message Type** `0x81`, **Return Code** 0x25 (`kSoftwareClusterMissing`) and **no payload** if the software cluster in the input parameter is not in state `kPresent`.

Status	Draft
Verification Criteria	

MVC-113967 - GetSwClusterManifestInfo - Positive Response

When the application software receives a valid **GetSwClusterManifestInfo** SOME/IP `REQUEST` message, it shall send a SOME/IP `RESPONSE` message with SOME/IP header fields **Length** set to `32(0x20)`, **Message Type** set to `RESPONSE (0x80)`, **Return Code** set to `E_OK (0x00)` and the following fields as **payload**:

ByteIndex	Output Parameter	BaseType	Description
0	swClusterName	ASCII	
??	version	ASCII	
??	typeApproval		unused
??	license		unused
??	releaseNotes	ASCII	
??	dependencyModel		

Status	Draft
Verification Criteria	

MVC-113966 - GetSwClusterManifestInfo - SOME/IP Payload Field - swClusterName

When the application software receives a valid **GetSwClusterManifestInfo** SOME/IP `REQUEST` message, it shall set the field `swClusterName` of the corresponding SOME/IP `RESPONSE` message to the value of **TBD**.

Status	Draft
Verification Criteria	

MVC-113969 - GetSwClusterManifestInfo - SOME/IP Payload Field - version

When the application software receives a valid **GetSwClusterManifestInfo** SOME/IP `REQUEST` message, it shall set the field `version` of the corresponding SOME/IP `RESPONSE` message to the value of **TBD**.

Status	Draft
Verification Criteria	

MVC-113968 - GetSwClusterManifestInfo - SOME/IP Payload Field - typeApproval

When the application software receives a valid **GetSwClusterManifestInfo** SOME/IP `REQUEST` message, it shall set the field `typeApproval` of the corresponding SOME/IP `RESPONSE` message to the value of **TBD**.

Status	Draft
Verification Criteria	

MVC-113971 - GetSwClusterManifestInfo - SOME/IP Payload Field - license

When the application software receives a valid **GetSwClusterManifestInfo** SOME/IP `REQUEST` message, it shall set the field `license` of the corresponding SOME/IP `RESPONSE` message to the value of **TBD**.

Status	Draft
Verification Criteria	

MVC-113970 - GetSwClusterManifestInfo - SOME/IP Payload Field - releaseNotes

When the application software receives a valid **GetSwClusterManifestInfo** SOME/IP `REQUEST` message, it shall set the field `releaseNotes` of the corresponding SOME/IP `RESPONSE` message to the value of **TBD**.

Status	Draft
Verification Criteria	

MVC-113963 - GetSwClusterManifestInfo - SOME/IP Payload Field - dependencyModel

When the application software receives a valid **GetSwClusterManifestInfo** SOME/IP `REQUEST` message, it shall set the field `dependencyModel` of the corresponding SOME/IP `RESPONSE` message to the value of **TBD**.

Status	Draft
Verification Criteria	

MVC-114980 - GetSwClusterManifestInfo - Negative Response - Software cluster missing

When the application software receives a valid **ProcessSwPackage** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kSoftwareClusterMissing (0x25)` and **no payload** shall be sent if the UCM identified a missing software cluster.

Status	Draft
Verification Criteria	

7.4.2.8 Method GetSwPackages

MVC-113975 - GetSwPackages - Method Validation

When the application software receives a valid SOME/IP `REQUEST` message with SOME/IP header fields **Method ID** set to `0x000C`, **Length** set to `8` and **no payload**, it shall accept the request as a valid **GetSwPackages** SOME/IP `REQUEST`.

Status	Draft
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-113974 - GetSwPackages - Method Length

When the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Method ID** set to `0x000C` and field **Length** set to a value other than `8 (0x08)`, it shall respond with a SOME/IP `ERROR` message with **Return Code** set to `E_MALFORMED_MESSAGE`.

Status	Draft
Verification Criteria	Send a GetSwPackages REQUEST message with SOME/IP header field Length set to a value other than 8 (e.g. 4 or 12); expect an ERROR response with Return Code E_MALFORMED_MESSAGE.

MVC-113976 - GetSwPackages - Positive Response

When the application software receives a valid **GetSwPackages** SOME/IP `REQUEST` message, it shall send a SOME/IP `RESPONSE` message with SOME/IP header fields **Length** set to `32(0x20)`, **Message Type** set to `RESPONSE (0x80)`, **Return Code** set to `E_OK (0x00)` and the following fields as **payload**:

ByteIndex	Output Parameter	BaseType	Description
0	swClusterName	ASCII	MVC-113978 - GetSwPackages - SOME/IP Payload Field - swClusterName
??	swPackageName	ASCII	MVC-113977 - GetSwPackages - SOME/IP Payload Field - swPackageName
??	version	ASCII	MVC-113980 - GetSwPackages - SOME/IP Payload Field - version
??	transferId		MVC-113979 - GetSwPackages - SOME/IP Payload Field - transferId
??	consecutiveBytesReceived	uint64	MVC-113982 - GetSwPackages - SOME/IP Payload Field - consecutiveBytesReceived
??	consecutiveBlocksReceived	uint64	MVC-113981 - GetSwPackages - SOME/IP Payload Field - consecutiveBlocksReceived
??	state	uint8	MVC-113983 - GetSwPackages - SOME/IP Payload Field - state

Status	Draft
Verification Criteria	

MVC-113978 - GetSwPackages - SOME/IP Payload Field - swClusterName

When the application software receives a valid **GetSwPackages** SOME/IP `REQUEST` message, it shall set the field `swClusterName` of the corresponding SOME/IP `RESPONSE` message to the value of **TBD**.

Status	Draft
Verification Criteria	

MVC-113977 - GetSwPackages - SOME/IP Payload Field - swPackageName

When the application software receives a valid **GetSwPackages** SOME/IP `REQUEST` message, it shall set the field `swPackageName` of the corresponding SOME/IP `RESPONSE` message to the value of **TBD**.

Status	Draft
Verification Criteria	

MVC-113980 - GetSwPackages - SOME/IP Payload Field - version

When the application software receives a valid **GetSwPackages** SOME/IP `REQUEST` message, it shall set the field `version` of the corresponding SOME/IP `RESPONSE` message to the value of **TBD**.

Status	Draft
Verification Criteria	

MVC-113979 - GetSwPackages - SOME/IP Payload Field - transferId

When the application software receives a valid **GetSwPackages** SOME/IP `REQUEST` message, it shall set the field `transferId` of the corresponding SOME/IP `RESPONSE` message to the value of **TBD**.

Status	Draft
Verification Criteria	

MVC-113982 - GetSwPackages - SOME/IP Payload Field - consecutiveBytesReceived

When the application software receives a valid **GetSwPackages** SOME/IP `REQUEST` message, it shall set the field `consecutiveBytesReceived` of the corresponding SOME/IP `RESPONSE` message to the value of **TBD**.

Status	Draft
Verification Criteria	

MVC-113981 - GetSwPackages - SOME/IP Payload Field - consecutiveBlocksReceived

When the application software receives a valid **GetSwPackages** SOME/IP `REQUEST` message, it shall set the field `consecutiveBlocksReceived` the corresponding SOME/IP `RESPONSE` message to the value of **TBD**.

Status	Draft
Verification Criteria	

MVC-113983 - GetSwPackages - SOME/IP Payload Field - state

When the application software receives a valid **GetSwPackages** SOME/IP `REQUEST` message, it shall set the field `state` the corresponding SOME/IP `RESPONSE` message to the value of **TBD**.

Status	Draft
Verification Criteria	

MVC-113984 - GetSwPackages - SOME/IP Payload Field - state - kTransferring

When the application software receives a valid **GetSwPackages** SOME/IP `REQUEST` message, it shall set all fields except `transferId`, `consecutiveBytesReceived` and `consecutiveBlocksReceived` in the corresponding SOME/IP `RESPONSE` me ssage to zero if the current state of the corresponding software package reported by the UCM is in state `kTransferring` (0x00).

Status	Draft
Verification Criteria	

7.4.2.9 Method DeleteTransfer

MVC-113986 - DeleteTransfer - Method Validation

The application software shall accept the SOME/IP REQUEST as a valid **DeleteTransfer** SOME/IP `REQUEST` when SOME/IP header fields **Method ID** set to `0x0005`, **Length** set to `24 (0x18)` and the payload has the following fields:

ByteIndex	Input Parameter	BaseType	Description
0	TransferIdType	uint8	Array of 16 elements of type uint8

Status	Draft
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-113989 - DeleteTransfer - Method Length

When the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Method ID** set to `0x0005` and field **Length** set to a value other than `24 (0x18)`, it shall respond with a SOME/IP `ERROR` message with **Return Code** set to `E_MALFORMED_MESSAGE`.

Status	Draft

Verification Criteria	Send a DeleteTransfer REQUEST message with SOME/IP header field Length set to a value other than 24 (e.g. 4 or 12); expect an ERROR response with Return Code E_MALFORMED_MESSAGE.

MVC-113988 - DeleteTransfer - Positive Response

When the application software receives a valid **DeleteTransfer** SOME/IP request, a response with message type `RESPONSE (0x80)`, return code `E_OK (0x00)` and **no payload** shall be sent if a valid transferId was provided and the operation was permitted.

Status	Draft
Verification Criteria	

MVC-113990 - DeleteTransfer - Negative Response - Invalid transferId

When the application software receives a valid **DeleteTransfer** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kTransferIdInvalid (0x04)` and **no payload** shall be sent if the UCM identifies the provided transferId as invalid.

Status	Draft
Verification Criteria	

MVC-113991 - DeleteTransfer - Negative Response - Operation not permitted

When the application software receives a valid **DeleteTransfer** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kOperationNotPermitted (0x05)` and **no payload** shall be sent if the UCM identifies the request as not permitted.

Status	Draft
Verification Criteria	

7.4.2.10 Method TransferStart

MVC-113995 - TransferStart - Method Validation

The application software shall accept the SOME/IP REQUEST as a valid **TransferStart** SOME/IP `REQUEST` when SOME/IP header fields **Method ID** set to `0x0013`, **Length** set to `16 (0x10)` and the payload has the following fields:

ByteIndex	Input Parameter	BaseType	Description
0	size	uint8	Size (in bytes) of the Software Package to be transferred

Status	Draft
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-113994 - TransferStart - Method Length

When the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Method ID** set to `0x0013` and field **Length** set to a value other than `16 (0x10)`, it shall respond with a SOME/IP `ERROR` message with **Return Code** set to `E_MALFORMED_MESSAGE`.

Status	Draft
Verification Criteria	Send a TransferStart REQUEST message with SOME/IP header field Length set to a value other than 16 (e.g. 4 or 12); expect an ERROR response with Return Code E_MALFORMED_MESSAGE.

MVC-113996 - TransferStart - Positive Response

When the application software receives a valid **TransferStart** SOME/IP `REQUEST` message, it shall send a SOME/IP `RESPONSE` message with SOME/IP header fields **Length** set to `28 (0x1C)`, **Message Type** set to `RESPONSE (0x80)`, **Return Code** set to `E_OK (0x00)` and the following fields as **payload**:

ByteIndex	Output Parameter	BaseType	Description
0	TransferIdType	uint8	Array of 16 elements of type uint8
16	blockSize	uint32	blockSize of TransferData

Status	Draft
Verification Criteria	

MVC-113997 - TransferStart - Negative Response - Insufficient memory

When the application software receives a valid **TransferStart** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kMemoryInsufficient (0x01)` and **no payload** shall be sent if the UCM identifies insufficient memory to perform a transfer operation.

Status	Draft
Verification Criteria	

7.4.2.11 Method TransferData

MVC-113998 - TransferData - Method Validation

The application software shall accept the SOME/IP REQUEST as a valid **TransferData** SOME/IP `REQUEST` when SOME/IP header fields **Method ID** set to `0x0011`, **Length** set to `32 (0x20)` and the payload has the following fields:

ByteIndex	Input Parameter	BaseType	Description
0	TransferIdType	uint8	Array of 16 elements of type uint8
16	TransferData	uint8	Data of software package
??	blockCounter	uint8	Block counter of current transfer

Status	Draft
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-114000 - TransferData - Method Length

When the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Method ID** set to `0x0011` and field **Length** set to a value other than `32 (0x20)`, it shall respond with a SOME/IP `ERROR` message with **Return Code** set to `E_MALFORMED_MESSAGE`.

Status	Draft
Verification Criteria	Send a TransferData REQUEST message with SOME/IP header field Length set to a value other than 32 (e.g. 4 or 12); expect an ERROR response with Return Code E_MALFORMED_MESSAGE.

MVC-114001 - TransferData - Positive Response

When the application software receives a valid **TransferData** SOME/IP request, a response with message type `RESPONSE (0x80)`, return code `E_OK (0x00)` and **no payload** shall be sent if the UCM allows the data transfer.

Status	Draft
Verification Criteria	

MVC-114003 - TransferData - Negative Response - Operation not permitted

When the application software receives a valid **TransferData** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kOperationNotPermitted (0x05)` and **no payload** shall be sent if the UCM identifies the request as not permitted.

Status	Draft
Verification Criteria	

MVC-114002 - TransferData - Negative Response - Invalid transferId

When the application software receives a valid **TransferData** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kTransferIdInvalid (0x04)` and **no payload** shall be sent if the UCM identifies the provided transferId as invalid.

Status	Draft
Verification Criteria	

MVC-114005 - TransferData - Negative Response - Incorrect blockCounter

When the application software receives a valid **TransferData** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kBlockIncorrect (0x02)` and **no payload** shall be sent if the UCM identifies the provided blockCounter as invalid.

Status	Draft
Verification Criteria	

MVC-114004 - TransferData - Negative Response - Incorrect blockSize

When the application software receives a valid **TransferData** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kBlockSizeIncorrect (0x1E)` and **no payload** shall be sent if the UCM identifies the provided blockSize as invalid.

Status	Draft
Verification Criteria	

MVC-114007 - TransferData - Negative Response - Incorrect size of SoftwarePackage

When the application software receives a valid **TransferData** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kSizeIncorrect (0x03)` and **no payload** shall be sent if the UCM identifies the provided size of the software Package exceeds the provided size in TransferStart.

Status	Draft
Verification Criteria	

MVC-114006 - TransferData - Negative Response - Incorrect size of SoftwarePackage

When the application software receives a valid **TransferData** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kMemoryInsufficient (0x01)` and **no payload** shall be sent if the UCM identifies insufficient memory to perform a transfer operation.

Status	Draft
Verification Criteria	

MVC-114009 - TransferData - Negative Response - Failed transfer

When the application software receives a valid **TransferData** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kTransferFailed (0x26)` and **no payload** shall be sent if the UCM cannot persist transferred block.

Status	Draft
Verification Criteria	

MVC-114008 - TransferData - Negative Response - Inconsistent block

When the application software receives a valid **TransferData** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kBlockInconsistent (0x19)` and **no payload** shall be sent if the UCM reports an inconsistency for transferred block fails.

Status	Draft
Verification Criteria	

MVC-114011 - TransferData - Negative Response - SoftwarePackage format not supported

When the application software receives a valid **TransferData** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kPackageFormatUnsupported (0x28)` and **no payload** shall be sent if the UCM identifies software package archiving format is not supported for current transfer.

Status	Draft
Verification Criteria	

MVC-114010 - TransferData - Negative Response - Authentication fail

When the application software receives a valid **TransferData** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kAuthenticationFailed (0x08)` and **no payload** shall be sent if the UCM fails to authentificate package.

Status	Draft
Verification Criteria	

MVC-114013 - TransferData - Negative Response - Package manifest invalid

When the application software receives a valid **TransferData** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kPackageManifestInvalid (0x0D)` and **no payload** shall be sent if the UCM fails to read package manifest.

Status	Draft
Verification Criteria	

MVC-114012 - TransferData - Negative Response - SoftwarePackage version incompatible

When the application software receives a valid **TransferData** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kPackageVersionIncompatible (0x18)` and **no payload** shall be sent if the UCM identifies a wrong software package version.

Status	Rejected
Verification Criteria	

MVC-114014 - TransferData - Negative Response - SoftwarePackage inconsistent

When the application software receives a valid **TransferData** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kPackageInconsistent (0x07)` and **no payload** shall be sent if the UCM identifies a software package integrity failure.

Status	Draft
Verification Criteria	

MVC-114016 - TransferData - Negative Response - SwCluster removal denied

When the application software receives a valid **TransferData** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kSwclRemovalDenied (0x27)` and **no payload** shall be sent if the UCM identifies that a software cluster package cannot be removed.

Status	Rejected
Verification Criteria	

MVC-114015 - TransferData - Negative Response - SoftwarePackage version outdated

When the application software receives a valid **TransferData** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kOldVersion (0x09)` and **no payload** shall be sent if the UCM identifies that a software package version is outdated.

Status	Draft
Verification Criteria	

7.4.2.12 Method TransferExit

MVC-114019 - TransferExit - Method Validation

The application software shall accept the SOME/IP REQUEST as a valid **TransferExit** SOME/IP `REQUEST` when SOME/IP header fields **Method ID** set to `0x0012`, **Length** set to `16 (0x10)` and the payload has the following fields:

ByteIndex	Input Parameter	BaseType	Description
0	TransferIdType	uint8	Array of 16 elements of type uint8 ?? TBD: Length is not correct??

Status	Draft
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-114018 - TransferExit - Method Length

When the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Method ID** set to `0x0012` and field **Length** set to a value other than `16 (0x10)`, it shall respond with a SOME/IP `ERROR` message with **Return Code** set to `E_MALFORMED_MESSAGE`.

Status	Draft

Verification Criteria	Send a TransferExit REQUEST message with SOME/IP header field Length set to a value other than 16 (e.g. 4 or 12); expect an ERROR response with Return Code E_MALFORMED_MESSAGE.

MVC-114020 - TransferExit - Positive Response

When the application software receives a valid **TransferExit** SOME/IP request, a response with message type `RESPONSE (0x80)`, return code `E_OK (0x00)` and **no payload** shall be sent if the UCM allows the operation.

Status	Draft
Verification Criteria	

MVC-114024 - TransferExit - Negative Response - Operation not permitted

When the application software receives a valid **TransferExit** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kOperationNotPermitted (0x05)` and **no payload** shall be sent if the UCM identifies the request as not permitted.

Status	Draft
Verification Criteria	

MVC-114023 - TransferExit - Negative Response - Invalid transferId

When the application software receives a valid **TransferExit** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kTransferIdInvalid (0x04)` and **no payload** shall be sent if the UCM identifies the provided transferId as invalid.

Status	Draft
Verification Criteria	

MVC-114025 - TransferExit - Negative Response - Data insufficient

When the application software receives a valid **TransferExit** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kDataInsufficient (0x06)` and **no payload** shall be sent if the UCM identifies an inconsistency between sizes of transfer operations.

Status	Draft
Verification Criteria	

MVC-114027 - TransferExit - Negative Response - SoftwarePackage format not supported

When the application software receives a valid **TransferExit** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kPackageFormatUnsupported (0x28)` and **no payload** shall be sent if the UCM identifies software package archiving format is not supported for current transfer.

Status	Draft
Verification Criteria	

MVC-114026 - TransferExit - Negative Response - Authentication fail

When the application software receives a valid **TransferExit** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kAuthenticationFailed (0x08)` and **no payload** shall be sent if the UCM fails to authentificate package.

Status	Draft
Verification Criteria	

MVC-114028 - TransferExit - Negative Response - Software package inconsistent

When the application software receives a valid **TransferExit** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kPackageInconsistent (0x07)` and **no payload** shall be sent if the UCM identifies a software package integrity failure.

Status	Draft
Verification Criteria	

MVC-114030 - TransferExit - Negative Response - Software package version incompatible

When the application software receives a valid **TransferExit** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kPackageVersionIncompatible (0x18)` and **no payload** shall be sent if the UCM identifies a wrong software package version.

Status	Rejected
Verification Criteria	

MVC-114029 - TransferExit - Negative Response - Package manifest invalid

When the application software receives a valid **TransferExit** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kPackageManifestInvalid (0x0D)` and **no payload** shall be sent if the UCM fails to read package manifest.

Status	Draft
Verification Criteria	

MVC-114032 - TransferExit - Negative Response - SwCluster removal denied

When the application software receives a valid **TransferExit** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kSwclRemovalDenied (0x27)` and **no payload** shall be sent if the UCM identifies that a software cluster package cannot be removed.

Status	Rejected
Verification Criteria	

MVC-114031 - TransferData - Negative Response - Software package version outdated

When the application software receives a valid **TransferData** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kOldVersion (0x09)` and **no payload** shall be sent if the UCM identifies that a software package version is outdated.

Status	Draft
Verification Criteria	

7.4.2.13 Method Cancel

MVC-114689 - Cancel - Method Validation

The application software shall accept the SOME/IP REQUEST as a valid **Cancel** SOME/IP `REQUEST` when SOME/IP header fields **Method ID** set to `0x0004`, **Length** set to `16 (0x10)` and the payload has the following fields:

ByteIndex	Input Parameter	BaseType	Description
0	TransferIdType	uint8	Array of 16 elements of type uint8 ?? TBD: Length is not correct??

Status	Draft
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-114688 - Cancel - Method Length

When the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Method ID** set to `0x0004` and field **Length** set to a value other than `16 (0x10)`, it shall respond with a SOME/IP `ERROR` message with **Return Code** set to `E_MALFORMED_MESSAGE`.

Status	Draft
Verification Criteria	Send a Cancel REQUEST message with SOME/IP header field Length set to a value other than 16 (e.g. 4 or 12); expect an ERROR response with Return Code E_MALFORMED_MESSAGE.

MVC-114691 - Cancel - Positive Response

When the application software receives a valid **Cancel** SOME/IP request, a response with message type `RESPONSE (0x80)`, return code `E_OK (0x00)` and **no payload** shall be sent if the UCM allows the operation.

Status	Draft
Verification Criteria	

MVC-114690 - Cancel - Negative Response - Invalid transferId

When the application software receives a valid **Cancel** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kTransferIdInvalid (0x04)` and **no payload** shall be sent if the UCM identifies the provided transferId as invalid.

Status	Draft
Verification Criteria	

MVC-114692 - Cancel - Negative Response - Operation not permitted

When the application software receives a valid **Cancel** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kOperationNotPermitted (0x05)` and **no payload** shall be sent if the UCM identifies the request as not permitted.

Status	Draft
Verification Criteria	

7.4.2.14 Method ProcessSwPackage

MVC-114698 - ProcessSwPackage - Method Validation

The application software shall accept the SOME/IP REQUEST as a valid **ProcessSwPackage** SOME/IP `REQUEST` when SOME/IP header fields **Method ID** set to `0x000E`, **Length** set to `16 (0x10)` and the payload has the following fields:

ByteIndex	Input Parameter	BaseType	Description
0	TransferIdType	uint8	Array of 16 elements of type uint8 ?? TBD: Length is not correct??

Status	Draft
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-114697 - ProcessSwPackage - Method Length

When the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Method ID** set to `0x000E` and field **Length** set to a value other than `16(0x10)`, it shall respond with a SOME/IP `ERROR` message with **Return Code** set to `E_MALFORMED_MESSAGE`.

Status	Draft
Verification Criteria	Send a ProcessSwPackage REQUEST message with SOME/IP header field Length set to a value other than 16 (e.g. 4 or 12); expect an ERROR response with Return Code E_MALFORMED_MESSAGE.

MVC-114700 - ProcessSwPackage - Positive Response

When the application software receives a valid **ProcessSwPackage** SOME/IP request, a response with message type `RESPONSE (0x80)`, return code `E_OK (0x00)` and **no payload** shall be sent if the UCM allows the operation.

Status	Draft
Verification Criteria	

MVC-114699 - ProcessSwPackage - Negative Response - Operation not permitted

When the application software receives a valid **ProcessSwPackage** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kOperationNotPermitted (0x05)` and **no payload** shall be sent if the UCM identifies the request as not permitted.

Status	Draft
Verification Criteria	

MVC-114702 - ProcessSwPackage - Negative Response - Service busy

When the application software receives a valid **ProcessSwPackage** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kServiceBusy (0x0C)` and **no payload** shall be sent if the UCM identifies that another processing is ongoing.

Status	Draft
Verification Criteria	

MVC-114701 - ProcessSwPackage - Negative Response - Invalid transferId

When the application software receives a valid **ProcessSwPackage** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kTransferIdInvalid (0x04)` and **no payload** shall be sent if the UCM identifies the provided transferId as invalid.

Status	Draft
Verification Criteria	

MVC-114704 - ProcessSwPackage - Negative Response - Authentication fail

When the application software receives a valid **TransferExit** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kAuthenticationFailed (0x08)` and **no payload** shall be sent if the UCM fails to authentificate package.

Status	Draft
Verification Criteria	

MVC-114703 - ProcessSwPackage - Negative Response - SoftwarePackage version incompatible

When the application software receives a valid **ProcessSwPackage** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kPackageVersionIncompatible (0x18)` and **no payload** shall be sent if the UCM identifies a wrong software package version.

Status	Rejected
Verification Criteria	

MVC-114696 - ProcessSwPackage - Negative Response - Package manifest invalid

When the application software receives a valid **ProcessSwPackage** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kPackageManifestInvalid (0x0D)` and **no payload** shall be sent if the UCM fails to read package manifest.

Status	Draft
Verification Criteria	

MVC-114707 - ProcessSwPackage - Negative Response - SoftwareCluster missing

When the application software receives a valid **ProcessSwPackage** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kSoftwareClusterMissing (0x25)` and **no payload** shall be sent if the UCM identified a missing software cluster.

Status	Draft
Verification Criteria	

MVC-114706 - ProcessSwPackage - Negative Response - Delta incompatible

When the application software receives a valid **ProcessSwPackage** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kSoftwareClusterMissing (0x25)` and **no payload** shall be sent if the UCM identified a delta dependency check error.

Status	Draft
Verification Criteria	

MVC-114708 - ProcessSwPackage - Negative Response - Insufficient memory

When the application software receives a valid **ProcessSwPackage** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kMemoryInsufficient (0x01)` and **no payload** shall be sent if the UCM identifies insufficient memory to perform a processing operation.

Status	Draft
Verification Criteria	

MVC-114705 - ProcessSwPackage - Negative Response - Checksum

When the application software receives a valid **ProcessSwPackage** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kChecksumDescriptionInvalid (0x23)` and **no payload** shall be sent if the UCM fails to recognises the checksum attribute.

Status	Draft
Verification Criteria	

MVC-114709 - ProcessSwPackage - Negative Response - Software package integrity

When the application software receives a valid **ProcessSwPackage** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kProcessedSoftwarePackageInconsistent (0x17)` and **no payload** shall be sent if the UCM identifies a software package integrity failure..

Status	Draft
Verification Criteria	

MVC-114711 - ProcessSwPackage - Negative Response - SwCluster removal denied

When the application software receives a valid **ProcessSwPackage** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kSwclRemovalDenied (0x27)` and **no payload** shall be sent if the UCM identifies that a software cluster package cannot be removed.

Status	Rejected
Verification Criteria	

MVC-114710 - ProcessSwPackage - Negative Response - SoftwarePackage version outdated

When the application software receives a valid **ProcessSwPackage** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kOldVersion (0x09)` and **no payload** shall be sent if the UCM identifies that a software package version is outdated.

Status	Draft
Verification Criteria	

MVC-114712 - ProcessSwPackage - Negative Response - Processing of software package canceled

When the application software receives a valid **ProcessSwPackage** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kProcessSwPackageCanceled (0x16)` and **no payload** shall be sent if the UCM identifies a cancel request while processing a software package.

Status	Draft
Verification Criteria	

7.4.2.15 Method RevertProcessedSwPackages

MVC-114715 - RevertProcessedSwPackages - Method Validation

When the application software receives a valid SOME/IP `REQUEST` message with SOME/IP header fields **Method ID** set to `0x000F`, **Length** set to `8` and **no payload**, it shall accept the request as a valid **RevertProcessedSwPackages** SOME/IP `REQUEST`.

Status	Draft
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-114714 - RevertProcessedSwPackages - Method Length

When the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Method ID** set to `0x000F` and field **Length** set to a value other than `8 (0x08)`, it shall respond with a SOME/IP `ERROR` message with **Return Code** set to `E_MALFORMED_MESSAGE`.

Status	Draft
Verification Criteria	Send a RevertProcessedSwPackages REQUEST message with SOME/IP header field Length set to a value other than 8 (e.g. 4 or 12); expect an ERROR response with Return Code E_MALFORMED_MESSAGE.

MVC-114717 - RevertProcessedSwPackages - Positive Response

When the application software receives a valid **RevertProcessedSwPackages** SOME/IP request, a response with message type `RESPONSE (0x80)`, return code `E_OK (0x00)` and **no payload** shall be sent if the UCM allows the operation.

Status	Draft
Verification Criteria	

MVC-114716 - RevertProcessedSwPackages - Negative Response - Operation not permitted

When the application software receives a valid **RevertProcessedSwPackages** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kOperationNotPermitted (0x05)` and **no payload** shall be sent if the UCM identifies the request as not permitted.

Status	Draft
Verification Criteria	

MVC-114718 - RevertProcessedSwPackages - Negative Response - No revert

When the application software receives a valid **RevertProcessedSwPackages** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kNotAbleToRevertPackages (0x0F)` and **no payload** shall be sent if the UCM is unable to revert a processing of a software package.

Status	Draft
Verification Criteria	

7.4.2.16 Method Activate

MVC-114868 - Activate - Method Validation

When the application software receives a valid SOME/IP `REQUEST` message with SOME/IP header fields **Method ID** set to `0x0003`, **Length** set to `8` and **no payload**, it shall accept the request as a valid **Activate** SOME/IP `REQUEST`.

Status	Draft
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-114867 - Activate - Method Length

When the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Method ID** set to `0x0003` and field **Length** set to a value other than `8 (0x08)`, it shall respond with a SOME/IP `ERROR` message with **Return Code** set to `E_MALFORMED_MESSAGE`.

Status	Draft
Verification Criteria	Send a Activate REQUEST message with SOME/IP header field Length set to a value other than 8 (e.g. 4 or 12); expect an ERROR response with Return Code E_MALFORMED_MESSAGE.

MVC-114876 - Activate - Positive Response

When the application software receives a valid **Activate** SOME/IP request, a response with message type `RESPONSE (0x80)`, return code `E_OK (0x00)` and **no payload** shall be sent if the UCM allows the operation.

Status	Draft
Verification Criteria	

MVC-114892 - Activate - Negative Response - Operation not permitted

When the application software receives a valid **Activate** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kOperationNotPermitted (0x05)` and **no payload** shall be sent if the UCM identifies the request as not permitted.

Status	Draft
Verification Criteria	

MVC-114891 - Activate - Negative Response - Dependency missing

When the application software receives a valid **Activate** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kDependencyMissing (0x15)` and **no payload** shall be sent if the UCM identifies a missing dependency.

Status	Rejected
Verification Criteria	

MVC-114894 - Activate - Negative Response - Persistency failed

When the application software receives a valid **Activate** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kPersistencyAllocationFailed (0x29)` and **no payload** shall be sent if the UCM failed to persistent data.

Status	Draft
Verification Criteria	

MVC-114893 - Activate - Negative Response - Session update rejection

When the application software receives a valid **Activate** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kUpdateSessionRejected (0x21)` and **no payload** shall be sent if the UCM rejected the start of an update.

Status	Draft
Verification Criteria	

MVC-114896 - Activate - Negative Response - Prepare update failed

When the application software receives a valid **Activate** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kPrepareUpdateFailed (0x13)` and **no payload** shall be sent if the UCM encountered an error in the update preparation step.

Status	Draft
Verification Criteria	

MVC-114895 - Activate - Negative Response - Verification failed

When the application software receives a valid **Activate** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kVerificationFailed (0x24)` and **no payload** shall be sent if the UCM failed to verify the activation.

Status	Draft
Verification Criteria	

7.4.2.17 Method Rollback

MVC-114901 - Rollback - Method Validation

When the application software receives a valid SOME/IP `REQUEST` message with SOME/IP header fields **Method ID** set to `0x0010`, **Length** set to `8` and **no payload**, it shall accept the request as a valid **Rollback** SOME/IP `REQUEST`.

Status	Draft
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-114900 - Rollback - Method Length

When the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Method ID** set to `0x0010` and field **Length** set to a value other than `8 (0x08)`, it shall respond with a SOME/IP `ERROR` message with **Return Code** set to `E_MALFORMED_MESSAGE`.

Status	Draft
Verification Criteria	Send a Rollback REQUEST message with SOME/IP header field Length set to a value other than 8 (e.g. 4 or 12); expect an ERROR response with Return Code E_MALFORMED_MESSAGE.

MVC-114903 - Rollback - Positive Response

When the application software receives a valid **Rollback** SOME/IP request, a response with message type `RESPONSE (0x80)`, return code `E_OK (0x00)` and **no payload** shall be sent if the UCM allows the operation.

Status	Draft
Verification Criteria	

MVC-114902 - Rollback - Negative Response - Operation not permitted

When the application software receives a valid **Rollback** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kOperationNotPermitted (0x05)` and **no payload** shall be sent if the UCM identifies the request as not permitted.

Status	Draft
Verification Criteria	

7.4.2.18 Method Finish

MVC-114905 - Finish - Method Validation

When the application software receives a valid SOME/IP `REQUEST` message with SOME/IP header fields **Method ID** set to `0x0006`, **Length** set to `8` and **no payload**, it shall accept the request as a valid **Finish** SOME/IP `REQUEST`.

Status	Draft
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-114904 - Finish - Method Length

When the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Method ID** set to `0x0006` and field **Length** set to a value other than `8 (0x08)`, it shall respond with a SOME/IP `ERROR` message with **Return Code** set to `E_MALFORMED_MESSAGE`.

Status	Draft
Verification Criteria	Send a Finish REQUEST message with SOME/IP header field Length set to a value other than 8 (e.g. 4 or 12); expect an ERROR response with Return Code E_MALFORMED_MESSAGE.

MVC-114907 - Finish - Positive Response

When the application software receives a valid **Finish** SOME/IP request, a response with message type `RESPONSE (0x80)`, return code `E_OK (0x00)` and **no payload** shall be sent if the UCM allows the operation.

Status	Draft
Verification Criteria	

MVC-114906 - Finish - Negative Response - Operation not permitted

When the application software receives a valid **Finish** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kOperationNotPermitted (0x05)` and **no payload** shall be sent if the UCM identifies the request as not permitted.

Status	Draft
Verification Criteria	

7.4.3 Unsupported software deployment methods

7.4.3.1 Method RegisterSoftwarePackage

MVC-114952 - RegisterSoftwarePackage - Method Validation

When the application software receives a valid SOME/IP `REQUEST` message with SOME/IP header fields **Method ID** set to `0x0015`, **Length** set to `8` and **no payload**, it shall accept the request as a valid **RegisterSoftwarePackage** SOME/IP `REQUEST`.

Status	Draft
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-114951 - RegisterSoftwarePackage - Method Length

When the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Method ID** set to `0x0015` and field **Length** set to a value other than `8 (0x08)`, it shall respond with a SOME/IP `ERROR` message with **Return Code** set to `E_MALFORMED_MESSAGE`.

Status	Draft
Verification Criteria	Send a RegisterSoftwarePackage REQUEST message with SOME/IP header field Length set to a value other than 8 (e.g. 4 or 12); expect an ERROR response with Return Code E_MALFORMED_MESSAGE.

MVC-114954 - RegisterSoftwarePackage - Negative Response - Method not supported

When the application software receives a valid **RegisterSoftwarePackage** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kInvalidUri (0x2B)` and **no payload** shall be sent.

Status	Draft
Verification Criteria	

7.4.3.2 Method Resume

MVC-114953 - Resume - Method Validation

When the application software receives a valid SOME/IP `REQUEST` message with SOME/IP header fields **Method ID** set to `0x0016`, **Length** set to `8` and **no payload**, it shall accept the request as a valid **Resume** SOME/IP `REQUEST`.

Status	Draft
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-114946 - Resume - Method Length

When the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Method ID** set to `0x0016` and field **Length** set to a value other than `8 (0x08)`, it shall respond with a SOME/IP `ERROR` message with **Return Code** set to `E_`

MALFORMED_MESSAGE.

Status	Draft
Verification Criteria	Send a Resume REQUEST message with SOME/IP header field Length set to a value other than 8 (e.g. 4 or 12); expect an ERROR response with Return Code E_MALFORMED_MESSAGE.

MVC-114956 - Resume - Negative Response - Method not supported

When the application software receives a valid **Resume** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kOperationNotPermitted (0x05)` and **no payload** shall be sent.

Status	Draft
Verification Criteria	

7.4.3.3 Method Suspend

MVC-114955 - Suspend - Method Validation

When the application software receives a valid SOME/IP `REQUEST` message with SOME/IP header fields **Method ID** set to `0x0017`, **Length** set to `8` and **no payload**, it shall accept the request as a valid **Suspend** SOME/IP `REQUEST`.

Status	Draft
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-114958 - Suspend - Method Length

When the application software receives a SOME/IP `REQUEST` message with SOME/IP header field **Method ID** set to `0x0017` and field **Length** set to a value other than `8 (0x08)`, it shall respond with a SOME/IP `ERROR` message with **Return Code** set to `E_MALFORMED_MESSAGE`.

Status	Draft
Verification Criteria	Send a Suspend REQUEST message with SOME/IP header field Length set to a value other than 8 (e.g. 4 or 12); expect an ERROR response with Return Code E_MALFORMED_MESSAGE.

MVC-114957 - Suspend - Negative Response - Method not supported

When the application software receives a valid **Suspend** SOME/IP request, a response with message type ERROR `(0x81)`, return code `kOperationNotPermitted (0x05)` and **no payload** shall be sent.

Status	Draft
Verification Criteria	

8 Video streaming

8.1 RTP

MVC-95979 - RTP - UDP Socket Stream1

The application software shall transmit the video stream "Stream 1" using a UDP socket with `rtpSourcePort1` (from NVM see MVC-95861 - NVM software configuration block) as UDP source port.

Status	Draft
Verification Criteria	Check if the correct UDP source port is used for the transmission of stream 1 according to the value in NVM.

MVC-95978 - RTP - UDP Socket Stream 2

The application software shall transmit the video stream "Stream 2" using a UDP socket with `rtpSourcePort2` (from NVM see MVC-95861 - NVM software configuration block) as UDP source port.

Status	Draft
Verification Criteria	Check if the correct UDP source port is used for the transmission of stream 2 according to the value in NVM.

MVC-95981 - RTP - Source IP

The application software shall use the current IP address of the ECU as the source IP address for the transmission of the RTP video streams.

Status	Specified
Verification Criteria	Check if the correct ipaddress is used as source ip for the transmission of stream 1 and stream 2 according to the ip configuration of the ECU.

MVC-95980 - RTP - Destination IP/Port 1 Configured

The application software shall send the "Stream 1" to the IP address configured in `rtpDestinationIp1 (rtpDestinationIp1 != 0.0.0.0)` and the destination port which is the sum of `rtpDestinationPort1` (from NVM see MVC-95861 - NVM software configuration block) and the last octet of the current IP address of the ECU.

Status	Draft
Verification Criteria	Check if the correct destination ip address and UDP destination port is used for the transmission of stream 1 according to the values in NVM.

MVC-95983 - RTP - Destination IP 1 Unconfigured

If `rtpDestinationIp1` is equal to 0.0.0.0, the application software shall send "Stream 1" to the IP address of the client that requested the video stream.

Status	Specified
Verification Criteria	Check if the correct destination ip address and UDP destination port is used for the transmission of stream 1 according to the values in NVM.

MVC-95982 - RTP - Destination IP/Port 2 Configured

The application software shall send the "Stream 2" to the IP address configured in `rtpDestinationIp2 (rtpDestinationIp2 != 0.0.0.0)` and the destination port which is the sum of `rtpDestinationPort2` (from NVM see MVC-95861 - NVM software configuration block) and the last octet of the current IP address of the ECU.

Status	Draft
Verification Criteria	Check if the correct destination ip address and UDP destination port is used for the transmission of stream 2 according to the values in NVM.

MVC-95975 - RTP - Destination IP 2 Unconfigured

If the `rtpDestinationIp2` is equal to 0.0.0.0, the application software shall send "Stream 2" shall send to the IP address of the client that requested the video stream.

Status	Specified
Verification Criteria	Check if the correct destination ip address and UDP destination port is used for the transmission of stream 2 according to the values in NVM.

MVC-96064 - RTP - UDP header

The application software shall transmit RTP packets using the UDP header shown below.

UDP Header:

Byte Offset	Byte 0	Byte 1	Byte 2	Byte 3
0	Source port		Destination port	
4	Length		Checksum	
8	Payload data			

- For source port see MVC-95979 and MVC-95978
- For destination port see MVC-95980 and MVC-95982
- For length and checksum see [RFC768]
- For payload data see MVC-96057 - RTP packet structure

Status	Draft
Verification Criteria	Analyze any RTP packet and check whether corresponding UDP fields are available and set according to values from NVM and RFC768

8.2 RTP-Protocol

MVC-96054 - RTP packet byte-order

The application software shall serialize an RTP packet using network byte order which is big-endian.

Status	Specified
Verification Criteria	Inspect at least one RTP packet produced by the Application Software and confirm that all multi-byte RTP header fields (such as sequence number, timestamp, and SSRC) are serialized in network byte order (big-endian). This can be performed by: Capturing a raw RTP packet generated by the application. Parsing the header fields byte-by-byte. Verifying that fields are encoded most significant byte first, in accordance with the RTP specification and general network protocol standards. For each field, compare the byte pattern to the expected big-endian format using test values with known little- and big-endian representations.

MVC-96057 - RTP packet structure

The application software shall use the following fields for serializing RTP packets (see RFC3550, section 5.1: https://datatracker.ietf.org/doc/html/rfc3550#section-5.1)

RTP field name	Size (in bits)	Value (hex representation)	Note
Version	2	0x2	
Padding	1	0x0	
Extension	1	0x0	
CSRC count	4	0x0	
Marker	1	MVC-96060 - RTP packet field - marker bit	
Payload type	7	0x60	=96(dec)
Sequence number	16	MVC-96058 - RTP packet field - sequence number	
Timestamp	32	MVC-96059 - RTP packet field - timestamp	
SSRC	32	MVC-96056 - RTP packet field - SSRC	
Payload	variable, depending on encoder settings	MVC-96062 - RTP packet field - payload - IDR NAL unit	

Status	Specified
Verification Criteria	inspect RTP packets and check for the structure and fixed values. variable parts of the rtp packet are specified in separate requirements and should be verified there

MVC-96056 - RTP packet field - SSRC

The application software shall generate a globally unique SSRC value for each requested video stream.

Note: Globally means unique in the whole network. Each stream must be sent with a different SSRC value.
For more background information, see section 5.1 "SSRC" in RFC 3550. RFC 3550 also requests the SSRC value to be chosen randomly (SHOULD).
As the scope of RTP transmission is limited (e.g. within a vehicle network), using a method such as including the MAC address of the camera in the SSRC value may be sufficient to ensure uniqueness.

Status	Specified
Verification Criteria	check for SSRC value of rtp packets: - value must be different for each stream - value must be also different accross different DUTs

MVC-96058 - RTP packet field - sequence number

The application software shall increment the RTP sequence number by one for each packet within the same video stream, starting at a random number when the stream is initiated.

Status	Specified
Verification Criteria	check that for each video stream: sequence number is incremented by one for each RTP packet

MVC-96059 - RTP packet field - timestamp

The application software shall set the RTP timestamp field of each RTP packet based on the sampling time of the corresponding video frame, using a 90 kHz clock.

Note:
The timestamp is a relative value, meaning it cannot be related to wall-clock time. It is only meaningful in relation to the timestamps of other video frames within the same RTP stream.
The initial timestamp value does not matter.
The timestamp shall remain the same for all RTP packets that belong to the same video frame, and shall be incremented once per video frame.
For example, a stream with 30 fps results in a timestamp increment of 3000 per frame (90,000 / 30).
Refer to RFC 6184, section 5.1: Timestamp

Status	Ready for review
Verification Criteria	check that: - increment correlates with framerate - each video frame (set of SPS/PPS/IDR NAL units) has same timesamp value - check that increment of timestamp matches increment of time in real world (e.g. by filming a precise clock and align RTP output frames to this precise clock. Precise clock means with a tolerance of +/- 0.1s.)

MVC-96060 - RTP packet field - marker bit

The application software shall set the marker bit in the RTP packet field in each last RTP packet of a fragmented IDR NAL-Unit.

For all other RTP packets the marker bit shall be set to 0.

Context: see MVC-96062 - RTP packet field - payload - IDR NAL unit

Note:
This behavior applies specifically to the handling of IDR NAL units that are fragmented across multiple RTP packets. The marker bit indicates the end of an access unit.
For background information, see section 5.1 "Marker Bit" in RFC 6184

Status	Specified
Verification Criteria	check that only the last RTP packet of a fragmented IDR NAL-Unit has the marker bit set to 1.

MVC-96062 - RTP packet field - payload - IDR NAL unit

The application software shall split an IDR NAL unit into several RTP packets (resp. RTP payloads) as follows:
Take chunk of IDR NAL unit of maximum RTP payload size minus 2 byte and add a FU-A indicator and FU-A header (one byte each).

See rfc6184:5.8, Figure 14. "RTP payload format for FU-A" for reference.

Repeat with next chunk of IDR NAL unit until all data has been packed and sent in RTP packets. The last chunk may be smaller than the maximum RTP payload size.

The maximum RTP payload size is calculated the following way (assuming a Ethernet MTU of 1500 byte):
Ethernet-MTU(1500) - IP-Header-Size(20) - UDP-Header-Size(8) - RTP-Header-Size(12)
=> 1460

Note: the size of the IDR NAL unit varies depending on the encoder settings, e.g. the encoding bitrate.

Status	Specified
Verification Criteria	For each group of RTP packets corresponding to an encoded video frame (the encoded video frame identified by two consecutive SPS NAL units or the marker bit), verify in order: The first RTP packet carries a NAL unit with type 7 (SPS), as found by checking the lower 5 bits of the first payload byte per RFC 6184/section 1.3 The second RTP packet carries a NAL unit with type 8 (PPS) Following are RTP packets carrying a fragmented IDR NAL unit (type 5).

MVC-96063 - RTP packet sequence for each running stream

The application software shall send RTP packets in the following sequence for each running stream (identified by SSRC field):

1. RTP packet with SPS NAL unit in RTP payload (unit type 7, see rfc6184#section-1.3 resp. [1]), depending on encoder settings
2. RTP packet with PPS NAL unit RTP payload (unit type 8, see rfc6184#section-1.3 resp. [1]), depending on encoder settings

3. RTP packets with IDR NAL unit RTP payload (unit type 5, see rfc6184#section-1.3)(see MVC-96062 - RTP packet field - payload - IDR NAL unit). Note that the IDR NAL unit is split in several RTP packets as it is too large to fit in one RTP Packet payload.

After sending a IDR NAL unit (step 3) the seqence starts again with a SPS NAL unit (step 1).

Note: one encoded video frame results in a sequence of RTP packets as described above (steps 1. to 3.)

[1]: ITU-T Recommendation H.264, "Advanced video coding for generic audiovisual services", March 2010.

Status	Specified
Verification Criteria	capture RTP packets, filter out NAL units different from IDR NAL unit. For each IDR NAL unit (identified by e.g identical timestamp) check the following: FU-A Fragmentation Structure Use trusted H.264 decoder and use received data as input to this decoder. check that h.264 decoder does not report errors when decoding.

8.3 Video stream/s

MVC-95769 - SubscribeROIVideo - Start of stream

The application software shall start a video stream within 100 ms to the destination IP based on configuration when a valid SubscribeROIVideo SOME/IP request (MVC-95729 - SubscribeROIVideo - Positive Response - no camera stream/s running) is received.

Possible configurations are:
MVC-95980 - RTP - Destination IP/Port 1 Configured
MVC-95983 - RTP - Destination IP 1 Unconfigured
MVC-95982 - RTP - Destination IP/Port 2 Configured
MVC-95975 - RTP - Destination IP 2 Unconfigured

Status	Draft
Verification Criteria	After sending a SOME/IP subscribe roi message check timestamp and compare to timestamp of first received RTP packet

MVC-95773 - SubscribeROIVideo - Dual-Stream Support

The application software shall support sending up to two different video streams to different clients based on its configuration.

Possible configurations are:
MVC-95980 - RTP - Destination IP/Port 1 Configured
MVC-95983 - RTP - Destination IP 1 Unconfigured
MVC-95982 - RTP - Destination IP/Port 2 Configured
MVC-95975 - RTP - Destination IP 2 Unconfigured

Status	Ready for review
Verification Criteria	Two clients send a valid subscribe roi message and rtp packets to both clients are receiving rtp packets

MVC-100833 - Parallel Video Streams

The application software shall support up to two active video streams at the same time.
Note: Implementation status -> implemented
Currently, it is not possible to add a link to the relevant system requirements, so the allocation is listed here:

- *MVC-100028 - subscribeRoiVideo - Dual Stream*

Status	Draft
Verification Criteria	Start one stream with ROI preset from {0, 1, 2, 3, 4, 5, 9} and one stream with ROI preset 10. Verify that both streams are active at the same time.

MVC-100832 - Allowed Combination of ROI Presets

If two video streams are active at the same time, one active stream shall use a ROI preset from the set {0, 1, 2, 3, 4, 5, 9} and the other active stream shall use ROI preset 10.
Note: Implementation status -> implemented
Currently, it is not possible to add a link to the relevant system requirements, so the allocation is listed here:

- *MVC-100013 - Supported ROIIndexes*
- *MVC-100010 - Overview - Implemented ROIs*
- *MVC-100035 - subscribeRoiVideo - Reception*

Status	Draft
Verification Criteria	Start two streams in parallel. Verify that the valid two-stream combination consists of one stream using ROI preset from {0, 1, 2, 3, 4, 5, 9} and one stream using ROI preset 10.

MVC-100855 - Stream Allocation

The application software shall allocate ROI presets 0, 1, 2, 3, 4, 5 and 9 to video stream 1 and ROI preset 10 to video stream 2.
Note: Implementation status -> implemented
Currently, it is not possible to add a link to the relevant system requirements, so the allocation is listed here:

- *MVC-100013 - Supported ROIIndexes*
- *MVC-100010 - Overview - Implemented ROIs*
- *MVC-100023 - Configuration and resource allocation of supported ROI*

Status	Draft
Verification Criteria	Send a SubscribeROIVideo request for the ROI preset 0-5 and 9 and execute the AmbaShell commands"svc_dsp dump_liv" and "svc_rec dump". Verify that the active configuration is shown on strm1 / strm_1. Send a SubscribeROIVideo request for ROI preset 10 and execute the AmbaShell commands "svc_dsp dump_liv" and "svc_rec dump". Verify that the active configuration is shown on strm2 / strm_2.

8.4 Video Streaming Functions

8.4.1 SOME/IP Subscription

MVC-109190 - Subscription stream mapping

The application software shall map **valid** roi indices (see MVC-95750 - subscribeRoiVideo - Payload Validation - roiIndex - ROI Presets) as follows:

ROI Presets	Stream number
0-5, 9 (HV)	1
10 (CV)	2

Status	Draft
Verification Criteria	Ensure that the ROI Presets start/stop the correct stream via related Tests

8.4.1.1 SubscribeROIVideo

MVC-95729 - SubscribeROIVideo - Positive Response - no camera stream/s running

While no video stream is running, when the application software receives a valid SubscribeROIVideo request message with a **valid** `roiIndex`, it shall subscribe the requesting `client` to the requested video stream, start the video stream, and send a positive SOME/IP response message with return code `E_OK (0x00)`.

Status	Ready for review
Verification Criteria	Send subscribeRoiVideo REQUEST with valid roiIndex, ensure SOME/IP RESPONSE with E_OK and start of video stream

MVC-96405 - SubscribeROIVideo - Positive response - camera stream/s running - start of stream

While any video stream is running, when the application software receives a valid SubscribeROIVideo request message with a **valid** `roiIndex` that does not correspond to any currently running stream, it shall subscribe the requesting `client` to the requested video stream, start the video stream, and send a positive SOME/IP response message with return code `E_OK (0x00)`.

Status	Ready for review
Verification Criteria	Client 1 subscribes to stream 1 and Client 2 subscribes to stream 2. Check for positive responses for both clients and RTP packet streams to Client 1 and Client 2 on ethernet.

MVC-109192 - SubscribeROIVideo - Positive response - camera stream/s running - restart of stream

While any video stream is running, when the application software receives a valid SubscribeROIVideo request message with a **valid** `roiIndex` and `client` that matches a running stream, it shall restart the requested video stream and send a positive SOME/IP response message with return code `E_OK (0x00)`.

Status	Ready for review
Verification Criteria	Send two subscribeRoiVideo REQUEST with different valid roiIndex for Stream 1, ensure SOME/IP RESPONSE with E_OK for both and start of video stream followed by a restart with different ROI Preset

MVC-96406 - SubscribeROIVideo - Negative response - camera stream/s running

While any video stream is running, when the application software receives a valid SubscribeROIVideo request message with a **valid** `roiIndex`, if `roiIndex` matches a currently running video stream but `client` does not then the application software shall send a negative SOME/IP response message with return code `E_NOT_OK (0x01)`.

Status	Specified
Verification Criteria	A client subscribes to any video stream and another client tries to subscribe to the same stream.

8.4.1.2 UnSubscribeROIVideo

MVC-95808 - UnSubscribeROIVideo - Positive Response - no camera stream/s running

While no video stream is running, when the application software receives a valid **unSubscribeROIVideo** SOME/IP `REQUEST` message with a valid **roiIndex**, it shall send a positive SOME/IP `RESPONSE` message with return code `E_OK (0x00)`.

Status	Ready for review
Verification Criteria	Send unSubscribeRoiVideo REQUEST with valid roiIndex while no stream is running, ensure SOME/IP RESPONSE with E_OK

MVC-96410 - UnSubscribeROIVideo - Positive response - camera stream/s running - stop of stream

While any video stream is running, when the application software receives a valid UnSubscribeROIVideo request message with a **valid** `roiIndex`, it shall stop the requested video stream if `roiIndex` and `client` in the request match a currently running video stream. Then the application software shall send a positive SOME/IP response message with return code `E_OK (0x00)`.

Note: The same video stream may be mapped to multiple ROI Presets.

Status	Ready for review
Verification Criteria	A client sends a subscribe message to receive a video stream. Afterwards, the same client sends an unsubscribe message with the same index to stop video stream. Check ethernet trace if any RTP packets are send to the client.

MVC-96411 - UnSubscribeROIVideo - Positive response - camera stream/s running - stop of inactive stream

While any video stream is running, when the application software receives a valid UnSubscribeROIVideo request message with a **valid** `roiIndex`, that does not correspond to any currently running stream, it shall send a positive SOME/IP response message with return code `E_OK (0x00)`.

Status	Draft
Verification Criteria	Client sends a subscribe message to get video stream 1 and sends an unsubscribe message with index to unsubscribe video stream 2. Ensure SOME/IP RESPONSE with E_OK and Stream 1 still running

MVC-96412 - UnSubscribeROIVideo - Negative response - camera stream/s running - stop by different client

While any video stream is running, when the application software receives a valid UnSubscribeROIVideo request message with a **valid** `roiIndex`, if `roiIndex` matches a currently running video stream but `client` does not then the application software shall send a negative SOME/IP response message with return code `E_NOT_OK (0x01)`.

Status	Specified
Verification Criteria	ClientID1 subscribes to video stream and ClientID2 sends an unsubscribe message with index of the same video stream as ClientID1.

MVC-109084 - UnSubscribeROIVideo -Positive Respone - One stream running

While only one video stream is running, when the application software receives a valid **unSubscribeROIVideo** SOME/IP `REQUEST` message from the same `client` who previously subscribed, then it shall consider any **`roiIndex`** as valid, stop the corresponding video stream and send a SOME/IP `RESPONSE` message with return code `E_OK (0x00)`.

Note: System Allocation Requirements are outdated, no satisfy link

Status	Draft
Verification Criteria	Attempt unsubscribing of Stream while only 1 is running with correct valid ROI, valid ROI for other Stream and invalid ROI Values. Ensure SOME/IP RESPONSE with E_OK and stopping of Stream in all cases

8.4.2 Variant Handling

MVC-100862 - Variant Determination Source

The application software shall determine the camera variant from someIpSdConfigurationOption field device_type.
Note: Implementation status -> implemented as current work around
According to MVC-100050, this shall later be replaced by dedicated variable in the MVC-95861 - NVM software configuration block.
Currently, it is not possible to add a link to the relevant system requirements, so the allocation is listed here:

- *MVC-100050 - Variant determination for Orientation and ROI Settings*

Status	Draft
Verification Criteria	Using UART serial monitor, boot the device with configured someIpSdConfigurationOption enty containing device_type. Verify in the runtime log output that the content of someIpSdConfigurationOption is printed and contains the configured device_type value. After sending a valid SubscribeROIVideo request, verify in the runtime log output that the request is processed with a determined deviceID and that the ROI configuration is selected accordingly (deviceID: rcam_2 = 0, acam_2 = 1, acam_2s = 2).

MVC-100863 - Supported Variant Values

The application software shall support the values rcam_2, acam_2 and acam_2s for variant determination.
Note: Implementation status -> implemented
Currently, it is not possible to add a link to the relevant system requirements, so the allocation is listed here:

- *MVC-100050 - Variant determination for Orientation and ROI Settings*

Status	Draft
Verification Criteria	Using UART serial monitor, repeat the startup and subscription test with device_type = rcam_2, device_type = acam_2 and device_type = acam_2s in someIpSdConfigurationOption.

MVC-100856 - Variant-based ROI Configuration Selection

The application software shall select the ROI preset configuration based on the determined camera variant.
Note: Implementation status -> implemented
Currently, it is not possible to add a link to the relevant system requirements, so the allocation is listed here:

- *MVC-100050 - Variant determination for Orientation and ROI Settings*
- *MVC-100041 - Config: ROI 0*
- *MVC-100040 - Config: ROI 1*
- *MVC-100043 - Config: ROI 2*
- *MVC-100042 - Config: ROI 3*
- *MVC-100045 - Config: ROI 4*
- *MVC-100044 - Config: ROI 5*

Status	Draft
Verification Criteria	Configure the camera once as rcam_2 or acam_2 and once as acam_2s. For the same ROI preset from 0-5, execute the AmbaShell command "svc_dsp dump_liv" after sending SubscribeROIVideo requests. Verify that the applied ROI geometry for acam_2s differs from the geometry used for rcam_2/acam_2 according to the reqirement tables.

☑ *MVC-100825 - Assumption: Default value if no variant is detected*

8.4.3 ROI Configuration

*Does the abbreviation GOP stand for "**G**roup **O**f **P**ictures"?*

MVC-100837 - ROI Preset 0

The application software shall configure ROI preset 0 based on the determined camera variant as follows:

Presets	rcam_2, acam_2	acam_2s
ROI Assignment	0	0
Stream Assignment	video stream 1	video stream 1

Bitrate [MBit/s]	20	20
Liveview Dimensions	1280 x 720	1280 x 720
Liveview ROI Geometry	ROI offset: 0 x 454 ROI dimensions: 1920 x 1080	ROI offset: 220 x 319 ROI dimensions: 1478 x 896
Encoder Dimensions	1280 x 720	1280 x 720
Encoder Frame Rate Timescale	30000	30000
Encoder Cropping	Encoder GOP configuration N: 1 Encoder cropping flag: 0 Encoder cropping left offset: 0 Encoder cropping right offset: 0 Encoder cropping top offset: 0 Encoder cropping bottom offset: 0	Encoder GOP configuration N: 1 Encoder cropping flag: 0 Encoder cropping left offset: 0 Encoder cropping right offset: 0 Encoder cropping top offset: 0 Encoder cropping bottom offset: 0

Note: Implementation status -> Implemented
Currently, it is not possible to add a link to the relevant system requirements, so the allocation is listed here:

- *MVC-100041 - Config: ROI 0*
- *MVC-100581 - Configuration and resource allocation of supported ROIs*
- *MVC-100032 - ROI0-5 definitions cropwindow*

Status	Draft
Verification Criteria	Configure the corresponding ROI preset and camera variant. Execute the AmbaShell command interface with: "svc_dsp dump_liv" and "svc_rec dump". Verify that stream assignment, bitrate, liveview dimensions, ROI geometry, encoder dimensions and frame rate timescale match the values defined in the table. For encoder cropping verify in the output image that no visible border artefacts occur.

MVC-100836 - ROI Preset 1

The application software shall configure ROI preset 1 based on the determined camera variant as follows:

Presets	rcam_2, acam_2	acam_2s
ROI Assignment	1	1
Stream Assignment	video stream 1	video stream 1
Bitrate [MBit/s]	14	14
Liveview Dimensions	952 x 536	952 x 536
Liveview ROI Geometry	ROI offset: 0 x 454 ROI dimensions: 1920 x 1080	ROI offset: 220 x 319 ROI dimensions: 1478 x 896
Encoder Dimensions	952 x 536	952 x 536
Encoder Frame Rate Timescale	30000	30000
Encoder Cropping	Encoder GOP configuration N: 1 Encoder cropping flag: 1 Encoder cropping left offset: 0 Encoder cropping right offset: 4 Encoder cropping top offset: 0 Encoder cropping bottom offset: 4	Encoder GOP configuration N: 1 Encoder cropping flag: 1 Encoder cropping left offset: 0 Encoder cropping right offset: 4 Encoder cropping top offset: 0 Encoder cropping bottom offset: 4

Note: configuration of encoder cropping because display resolution is not divisible by 16 see: MEKRALANGPOC-76
Implementation status -> Implemented
Specified in MVC-100581: ROIs 0–5 and 9 are assigned to ISP 1, and ROI 10 is assigned to ISP 0.
Currently, it is not possible to add a link to the relevant system requirements, so the allocation is listed here:

- *MVC-100040 - Config: ROI 1*
- *MVC-100581 - Configuration and resource allocation of supported ROIs*
- *MVC-100032 - ROI0-5 definitions cropwindow*

Status	Draft
Verification Criteria	Configure the corresponding ROI preset and camera variant. Execute the AmbaShell command interface with: "svc_dsp dump_liv" and "svc_rec dump". Verify that stream assignment, bitrate, liveview dimensions, ROI geometry, encoder dimensions and frame rate timescale match the values defined in the table. For encoder cropping verify in the output image that no visible border artefacts occur.

MVC-100839 - ROI Preset 2

The application software shall configure ROI preset 2 based on the determined camera variant as follows:

Presets	rcam_2, acam_2	acam_2s
ROI Assignment	2	2
Stream Assignment	video stream 1	video stream 1
Bitrate [MBit/s]	9	9
Liveview Dimensions	640 x 360	640 x 360
Liveview ROI Geometry	ROI offset: 0 x 454 ROI dimensions: 1920 x 1080	ROI offset: 220 x 319 ROI dimensions: 1478 x 896
Encoder Dimensions	640 x 360	640 x 360
Encoder Frame Rate Timescale	30000	30000
Encoder Cropping	Encoder GOP configuration N: 1 Encoder cropping flag: 1 Encoder cropping left offset: 0 Encoder cropping right offset: 0 Encoder cropping top offset: 0 Encoder cropping bottom offset: 4	Encoder GOP configuration N: 1 Encoder cropping flag: 1 Encoder cropping left offset: 0 Encoder cropping right offset: 0 Encoder cropping top offset: 0 Encoder cropping bottom offset: 4

Note: configuration of encoder cropping because display resolution is not divisible by 16 see: MEKRALANGPOC-76
Implementation status -> Implemented
Currently, it is not possible to add a link to the relevant system requirements, so the allocation is listed here:

- *MVC-100043 - Config: ROI 2*
- *MVC-100581 - Configuration and resource allocation of supported ROIs*
- *MVC-100032 - ROI0-5 definitions cropwindow*

Status	Draft
Verification Criteria	Configure the corresponding ROI preset and camera variant. Execute the AmbaShell command interface with: "svc_dsp dump_liv" and "svc_rec dump". Verify that stream assignment, bitrate, liveview dimensions, ROI geometry, encoder dimensions and frame rate timescale match the values defined in the table. For encoder cropping verify in the output image that no visible border artefacts occur.

MVC-100838 - ROI Preset 3

The application software shall configure ROI preset 3 based on the determined camera variant as follows:

Presets	rcam_2, acam_2	acam_2s
ROI Assignment	3	3
Stream Assignment	video stream 1	video stream 1
Bitrate [MBit/s]	20	20
Liveview Dimensions	1280 x 720	1280 x 720
Liveview ROI Geometry	ROI offset: 0 x 454 ROI dimensions: 1920 x 1080	ROI offset: 515 x 539 ROI dimensions: 889 x 457
Encoder Dimensions	1280 x 720	1280 x 720
Encoder Frame Rate Timescale	30000	30000

Encoder Cropping	Encoder GOP configuration N: 1 Encoder cropping flag: 0 Encoder cropping left offset: 0 Encoder cropping right offset: 0 Encoder cropping top offset: 0 Encoder cropping bottom offset: 0	Encoder GOP configuration N: 1 Encoder cropping flag: 0 Encoder cropping left offset: 0 Encoder cropping right offset: 0 Encoder cropping top offset: 0 Encoder cropping bottom offset: 0

Note: Implementation status -> Implemented
Currently, it is not possible to add a link to the relevant system requirements, so the allocation is listed here:

- *MVC-100042 - Config: ROI 3*
- *MVC-100581 - Configuration and resource allocation of supported ROIs*
- *MVC-100032 - ROI0-5 definitions cropwindow*

Status	Draft
Verification Criteria	Configure the corresponding ROI preset and camera variant. Execute the AmbaShell command interface with: "svc_dsp dump_liv" and "svc_rec dump". Verify that stream assignment, bitrate, liveview dimensions, ROI geometry, encoder dimensions and frame rate timescale match the values defined in the table. For encoder cropping verify in the output image that no visible border artefacts occur.

MVC-100841 - ROI Preset 4

The application software shall configure ROI preset 4 based on the determined camera variant as follows:

Presets	rcam_2, acam_2	acam_2s
ROI Assignment	4	4
Stream Assignment	video stream 1	video stream 1
Bitrate [MBit/s]	14	14
Liveview Dimensions	952 x 536	952 x 536
Liveview ROI Geometry	ROI offset: 0 x 454 ROI dimensions: 1920 x 1080	ROI offset: 515 x 539 ROI dimensions: 889 x 457
Encoder Dimensions	952 x 536	952 x 536
Encoder Frame Rate Timescale	30000	30000
Encoder Cropping	Encoder GOP configuration N: 1 Encoder cropping flag: 1 Encoder cropping left offset: 0 Encoder cropping right offset: 4 Encoder cropping top offset: 0 Encoder cropping bottom offset: 4	Encoder GOP configuration N: 1 Encoder cropping flag: 1 Encoder cropping left offset: 0 Encoder cropping right offset: 4 Encoder cropping top offset: 0 Encoder cropping bottom offset: 4

Note: configuration of encoder cropping because display resolution is not divisible by 16 see: MEKRALANGPOC-76
Implementation status -> Implemented
Currently, it is not possible to add a link to the relevant system requirements, so the allocation is listed here:

- *MVC-100045 - Config: ROI 4*
- *MVC-100581 - Configuration and resource allocation of supported ROIs*
- *MVC-100032 - ROI0-5 definitions cropwindow*

Status	Draft
Verification Criteria	Configure the corresponding ROI preset and camera variant. Execute the AmbaShell command interface with: "svc_dsp dump_liv" and "svc_rec dump". Verify that stream assignment, bitrate, liveview dimensions, ROI geometry, encoder dimensions and frame rate timescale match the values defined in the table. For encoder cropping verify in the output image that no visible border artefacts occur.

MVC-100840 - ROI Preset 5

The application software shall configure ROI preset 5 based on the determined camera variant as follows:

Presets	rcam_2, acam_2	acam_2s
ROI Assignment	5	5
Stream Assignment	video stream 1	video stream 1
Bitrate [MBit/s]	9	9
Liveview Dimensions	640 x 360	640 x 360
Liveview ROI Geometry	ROI offset: 0 x 454 ROI dimensions: 1920 x 1080	ROI offset: 515 x 539 ROI dimensions: 889 x 457
Encoder Dimensions	640 x 360	640 x 360
Encoder Frame Rate Timescale	30000	30000
Encoder Cropping	Encoder GOP configuration N: 1 Encoder cropping flag: 1 Encoder cropping left offset: 0 Encoder cropping right offset: 0 Encoder cropping top offset: 0 Encoder cropping bottom offset: 4	Encoder GOP configuration N: 1 Encoder cropping flag: 1 Encoder cropping left offset: 0 Encoder cropping right offset: 0 Encoder cropping top offset: 0 Encoder cropping bottom offset: 4

Note: configuration of encoder cropping because display resolution is not divisible by 16 see: MEKRALANGPOC-76
Implementation status -> Implemented
Currently, it is not possible to add a link to the relevant system requirements, so the allocation is listed here:

- *MVC-100044 - Config: ROI 5*
- *MVC-100581 - Configuration and resource allocation of supported ROIs*
- *MVC-100032 - ROI0-5 definitions cropwindow*

Status	Draft
Verification Criteria	Configure the corresponding ROI preset and camera variant. Execute the AmbaShell command interface with: "svc_dsp dump_liv" and "svc_rec dump". Verify that stream assignment, bitrate, liveview dimensions, ROI geometry, encoder dimensions and frame rate timescale match the values defined in the table. For encoder cropping verify in the output image that no visible border artefacts occur.

MVC-100843 - ROI Preset 9

The application software shall configure ROI preset 9 as follows:

Presets	not variant based (same presets for all device types)
ROI Assignment	9
Stream Assignment	video stream 1
Bitrate [MBit/s]	50
Liveview Dimensions	1920 x 1536
Liveview ROI Geometry	ROI offset: 0 x 0 ROI dimensions: 1920 x 1536
Encoder Dimensions	1920 x 1536
Encoder Frame Rate Timescale	30000
Encoder Cropping	Encoder GOP configuration N: 1 Encoder cropping flag: 0 Encoder cropping left offset: 0 Encoder cropping right offset: 0 Encoder cropping top offset: 0 Encoder cropping bottom offset: 0

Note: Implementation status -> Implemented
Currently, it is not possible to add a link to the relevant system requirements, so the allocation is listed here:

- *MVC-100581 - Configuration and resource allocation of supported ROIs*
- *MVC-100318 - Config: ROI 9*

Status	Draft

Verification Criteria	Configure the corresponding ROI preset and camera variant. Execute the AmbaShell command interface with: "svc_dsp dump_liv" and "svc_rec dump". Verify that stream assignment, bitrate, liveview dimensions, ROI geometry, encoder dimensions and frame rate timescale match the values defined in the table. For encoder cropping verify in the output image that no visible border artefacts occur.

MVC-100842 - ROI Preset 10

The application software shall configure ROI preset 10 as follows:

Presets	not variant based (same presets for all device types)
ROI Assignment	10
Stream Assignment	video stream 2
Bitrate [MBit/s]	50
Liveview Dimensions	1024 x 864
Liveview ROI Geometry	ROI offset: 0 x 0 ROI dimensions: 1920 x 1536
Encoder Dimensions	1024 x 864
Encoder Frame Rate Timescale	10000
Encoder Cropping	Encoder GOP configuration N: 1 Encoder cropping flag: 0 Encoder cropping left offset: 0 Encoder cropping right offset: 0 Encoder cropping top offset: 0 Encoder cropping bottom offset: 0

Note: Implementation status -> Implemented
Currently, it is not possible to add a link to the relevant system requirements, so the allocation is listed here:

- *MVC-100581 - Configuration and resource allocation of supported ROIs*
- *MVC-100315 - Config: ROI 10*

Status	Draft
Verification Criteria	Configure the corresponding ROI preset and camera variant. Execute the AmbaShell command interface with: "svc_dsp dump_liv" and "svc_rec dump". Verify that stream assignment, bitrate, liveview dimensions, ROI geometry, encoder dimensions and frame rate timescale match the values defined in the table. For encoder cropping verify in the output image that no visible border artefacts occur.

8.4.4 Coordinate System Definition

MVC-96506 - Coordinate System Definition - Interpretation of the Sensor Coordinate System

The application software shall interpret the sensor data based on a coordinate system with origin (0,0) at the top-left corner of the sensor area, where the x-axis extends to the right and the y-axis extends downward.

Status	Specified
Verification Criteria	Check that (0,0) is top-left; X increases to the right, Y increases downward.

MVC-96503 - Coordinate System Definition - Coordinate Consistency Across Frames

The application software shall maintain consistent coordinate references across input and output frames to ensure alignment of ROI definitions and image transformation operations.

Status	Specified
Verification Criteria	Check that (0,0) is top-left; X increases to the right, Y increases downward.

8.4.5 Flip / Mirror Functionality

- Streams with latency focus correspond to Human Vision streams and currently include ROI 0–5 and ROI 9 running on ISP0.
- Streams without latency focus correspond to Computer Vision stream and currently include ROI 10 only running on ISP1.

Note: Specified in MVC-100581

- *Flip refers to transformation on the x-axis.*
- *Mirror refers to transformation on the y-axis.*

This section provides an overview of how effective flip and mirror settings are determined depending on the stream type and is applied through SW.

- Streams with Latency Focus are referred to as HV Stream.
- Streams without Latency Focus are referred to as CV Stream.

For the definition of latency-based stream categorization, refer to: MVC-100851 - Stream Categorization Based on Latency Requirements



Flip and mirror modes are triggered on DSP level using the AmbaDSP_LiveviewYuvStreamSync/AmbaDSP_LiveviewUpdateConfig function.
For internal development or test purpose, the SvcLiveview_UpdateStream may be used for evaluation.
In the SvcLiveview_UpdateStream the AmbaDSP_LiveviewUpdateConfig uses AMBA_DSP_LIVEVIEW_STREAM_CFG_, here the pChanCfg points to AMBA_DSP_LIVEVIEW_CHANNEL_WINDOW_s which contains DSP_ROTATE_FLIP.

See:
Jira Ticket 98,
4.2.1.2 AmbaDSP_LiveviewConfig > AMBA_DSP_LIVEVIEW_STREAM_CFG_s
4.2.1.3 AmbaDSP_LiveviewConfig > AMBA_DSP_LIVEVIEW_CHANNEL_WINDOW_s
4.2.1.5 DSP_ROTATE_FLIP
2.8 View Ctrl Data Structure — StillCfg
and 3.7 Request Stage — Pan / Tilt / Zoom / Set the ROI by the DSP VOUT API/Figure 38. Example of Setting the ROI by the DSP VOUT API.

MVC-96492 - Flip / Mirror Functionality - Use of Flip/Mirror API Interface

The application software shall use the DSP driver components and functions in order to flip/mirror the image.
Note:
- Flip refers to transformation on the x-axis
- Mirror refers to transformation on the y-axis
further information can be found in 8.4.5-4 - DSP Flip/mirror Development Reference .

Status	Specified
Verification Criteria	Check if DSP driver API is used to activate flip and mirror modes.

MVC-96494 - Flip / Mirror Functionality - Activation of Flip/Mirror Modes via SOME/IP

The application software shall activate horizontal (x-axis) and vertical (y-axis) image flipping modes, when a SubscribeROIVideo message is received based on the input parameters from Mirror and Flip.

Status	Specified
Verification Criteria	Check if Bit 31 and Bit 30 of the ROI Index are correctly interpreted and applied as horizontal and vertical flip settings.

MVC-96493 - Flip / Mirror Functionality - Consistency of Coordinate-Based Functions after Flip/Mirror

The application software shall ensure that coordinate-based functions (e.g. ROI, panning, zooming) remain logically consistent after applying flip/mirror operations, i.e. no unintended shift or distortion of the coordinates shall occur.

Status	Specified
Verification Criteria	Check frames before and after applying flip/mirror and compare them to ensure no unintended shift or distortion of coordinates occurs.

MVC-100845 - Sensor Initial Flip/Mirror Configuration on Startup

When entering the software state PERIPHERAL_CONFIGURATION, the application software shall configure the initial sensor flip and mirror state such that:

- flip = off, mirror = off.

Note: Implementation status -> implemented
Currently, it is not possible to add a link to the relevant system requirements, so the allocation is listed here:

- *MVC-100632 - Default Sensor Readout setting*

Status	Draft
Verification Criteria	Read the sensor registers after system startup and before any stream subscription, and verify that the configured flip and mirror states match the expected values for each hardware variant.

MVC-100844 - Effective Flip/Mirror Determination

When a flip or mirror operation is requested, the application software shall determine the effective flip and mirror state by applying an XOR operation between:

- the requested flip/mirror state, and
- the sensor default physical orientation of the selected hardware variant.

Whereas sensor default physical orientation of the different hardware variant

- rcam_2, acam_2: default physical orientation is normal; for calculation: flip = off, mirror = off.
- acam_2s: default physical orientation is rotated (flipped and mirrored); for calculation: flip = on, mirror = on.

Note:
See MVC-100852 - Effective Flip/Mirror Determination per Stream Type - Concept Overview

Implementation status -> Implemented

Currently, it is not possible to add a link to the relevant system requirements, so the allocation is listed here:

- *MVC-100632 - Default Sensor Readout setting*
- *MVC-100014 - subscribeRoiVideo: Mirror/Flip Sensor- rcam_2 & acam_2*
- *~~MVC-100590~~ - Default Sensor setting - acam_2s*
- *MVC-100029 - subscribeRoiVideo: Mirror/Flip Sensor - acam_2s*

Status	Draft
Verification Criteria	Verify for hardware variants with different default physical orientations that the calculated effective flip/mirror state matches the XOR of requested state and sensor default physical orientation. For identical requested states, the resulting output orientation shall be identical across all supported hardware variants.

MVC-100847 - ISP Streams with Latency Focus

While a stream is configured with latency focus, the application software shall perform requested flip and mirror operations through sensor register configuration.

Sensor register to be used according to Application note "S5K3B6SA_Application Note_ALL_REV 0.03":

Index	Reset Value	Bits	register name	Description
0x325C	0x0000	[0]	tgr_h_desc5_orient	Enable horizontal mirror for Active
0x32EC	0x0000	[0]	tgr_v_desc8_orient	Enable vertical flip

Note:
Flip and mirror operations for streams with latency focus are not performed in the ISP processing pipeline.
See MVC-100851 - Stream Categorization Based on Latency Requirements.

Implementation status -> Implemented

Currently, it is not possible to add a link to the relevant system requirements, so the allocation is listed here:

- *MVC-100014 - subscribeRoiVideo: Mirror/Flip Sensor- rcam_2 & acam_2*
- *MVC-100029 - subscribeRoiVideo: Mirror/Flip Sensor - acam_2s*

Status	Draft
Verification Criteria	Verify that, for a stream configured with latency focus, a requested flip or mirror operation causes the corresponding sensor registers to be updated. Verify that the resulting output orientation matches the requested orientation for all possible flip and mirror combinations.

MVC-100846 - Orientation Preservation of Non-Latency-Focus Streams

When the sensor flip or mirror configuration is changed to fulfill the requested orientation of a stream with latency focus, the application software shall update each running stream without latency focus such that its requested output orientation is preserved.

Note:
A change of the sensor flip or mirror configuration affects all streams sourced from the sensor. Therefore, streams without latency focus may require compensating ISP processing to keep their requested output orientation unchanged.

See MVC-100851 - Stream Categorization Based on Latency Requirements.

Implementation status -> Implemented

Currently, it is not possible to add a link to the relevant system requirements, so the allocation is listed here:

- *MVC-100232 - subscribeRoiVideo: Mirror/Flip ISP - Sensor Reconfiguration (rcam_2 & acam_2)*
- *MVC-100642 - subscribeRoiVideo: Mirror/Flip ISP - Sensor Reconfiguration (acam_2s)*

Status	Draft
Verification Criteria	Start one stream with latency focus and one stream without latency focus with both streams requested in normal orientation. Then request a mirror or flip operation for the stream with latency focus. Verify that the sensor configuration changes accordingly, that the latency-focus stream shows the newly requested orientation, and that the stream without latency focus still shows its originally requested normal orientation.

MVC-100849 - ISP Streams without Latency Focus

While a stream is configured without latency focus, the application software shall perform requested flip and mirror operations in the ISP processing pipeline.

Note:
Flip and mirror operations for streams without latency focus are not performed through sensor register configuration. Further information can be found in 8.4.5-4 - DSP Flip/mirror Development Reference .

See MVC-100851 - Stream Categorization Based on Latency Requirements.

Implementation status -> implemented

Currently, it is not possible to add a link to the relevant system requirements, so the allocation is listed here:

- *MVC-100639 - SubscribeRoiVideo - Mirror/Flip ISP - rcam_2 & acam_2: normal*
- *MVC-100638 - SubscribeRoiVideo - Mirror/Flip ISP - rcam_2 & acam_2: mirrored*
- *MVC-100641 - SubscribeRoiVideo - Mirror/Flip ISP- rcam_2 & acam_2: flipped*
- *MVC-100640 - SubscribeRoiVideo - Mirror/Flip ISP - rcam_2 & acam_2: rotated*

- *MVC-100636 - SubscribeRoiVideo - Mirror/Flip ISP - acam_2s: normal*
- *MVC-100635 - SubscribeRoiVideo - Mirror/Flip ISP - acam_2s: mirrored*
- *MVC-100606 - SubscribeRoiVideo - Mirror/Flip ISP - acam_2s: flipped*
- *MVC-100637 - SubscribeRoiVideo - Mirror/Flip ISP - acam_2s: rotated*

Status	Draft
Verification Criteria	Verify that, for a stream configured without latency focus, a requested flip or mirror operation is executed in the ISP processing pipeline and does not directly change the sensor registers for that stream request.

MVC-100848 - ISP Processing: Effective State Calculation

When flip or mirror processing is performed in the ISP, the application software shall calculate the effective ISP flip and mirror state by applying an XOR operation between:

- the requested output orientation of the stream,
- the current sensor flip and mirror state, and
- the sensor default physical orientation of the selected hardware variant.

Whereas sensor default physical orientation of the different hardware variant

- rcam_2, acam_2: default physical orientation is normal; for calculation: flip = off, mirror = off.
- acam_2s: default physical orientation is rotated (flipped and mirrored); for calculation: flip = on, mirror = on.

Note:
See MVC-100852 - Effective Flip/Mirror Determination per Stream Type - Concept Overview

Implementation status -> Implemented

Currently, it is not possible to add a link to the relevant system requirements, so the allocation is listed here:

- *MVC-100639 - SubscribeRoiVideo - Mirror/Flip ISP - rcam_2 & acam_2: normal*
- *MVC-100638 - SubscribeRoiVideo - Mirror/Flip ISP - rcam_2 & acam_2: mirrored*
- *MVC-100641 - SubscribeRoiVideo - Mirror/Flip ISP- rcam_2 & acam_2: flipped*
- *MVC-100640 - SubscribeRoiVideo - Mirror/Flip ISP - rcam_2 & acam_2: rotated*

- *MVC-100636 - SubscribeRoiVideo - Mirror/Flip ISP - acam_2s: normal*
- *MVC-100635 - SubscribeRoiVideo - Mirror/Flip ISP - acam_2s: mirrored*
- *MVC-100606 - SubscribeRoiVideo - Mirror/Flip ISP - acam_2s: flipped*
- *MVC-100637 - SubscribeRoiVideo - Mirror/Flip ISP - acam_2s: rotated*

Status	Draft
Verification Criteria	Verify that, for a stream configured without latency focus, the resulting output orientation matches the requested orientation for all possible flip and mirror combinations.

8.4.6 Aspect Ratio Conversion

MVC-96498 - Aspect Ratio Conversion - Aspect Ratio Check for Output Compatibility

The application software shall determine whether aspect ratio conversion is required by comparing the current sensor resolution and the requested Display Resolution.

Status	Draft
Verification Criteria	Verify that aspect ratio conversion is correctly triggered when the sensor and output resolutions differ.

MVC-96497 - Aspect Ratio Conversion - Reference Value Definition

The application software shall use the sensor´s fixed aspect ratio of 5:4 as a reference for aspect ratio conversion.

Status	Draft
Verification Criteria	Check in video stream that the sensor aspect ratio of 5:4 is used as reference for all aspect ratio conversions. The output image shall not appear stretched or compressed.

MVC-96500 - Aspect Ratio Conversion - Converting Mechanism

The application software shall implement a converting mechanism to align any deviating aspect ratios to the 5:4 reference.

Status	Draft
Verification Criteria	Verify that the Application Software uses the sensor´s aspect ratio to compute the rerquired aspect ratio conversion.

MVC-96499 - Aspect Ratio Conversion - Ratio Comparison Logic

The application software shall compare the aspect ratio of the requested output resolution with the reference value of the camera sensor aspect ratio to determine whether conversion is required.

Status	Draft
Verification Criteria	Verify that the requested output resolution with the 5:4 reference ratio is compared before conversion.

MVC-96502 - Aspect Ratio Conversion - No Cropping Required

When the output aspect ratio is equal to the camera sensor aspect ratio, the application software shall convert the input directly without applying any cropping.

Status	Draft
Verification Criteria	Verify that input is passed through without cropping when the requested aspect ratio equals 5:4.

MVC-96501 - Aspect Ratio Conversion - Y-Axis Cropping

When the output aspect ratio has a larger width-to-height ratio value than the camera sensor aspect ratio,
the application software shall crop the image symmetrically along the Y-axis, centered around the camera sensor midpoint.

Status	Specified
Verification Criteria	Verify that the image is cropped symmentrically along the y-axis when the output aspect ratio is higher than the sensor´s aspect ratio.

MVC-96495 - Aspect Ratio Conversion - Cropping Line Calculation Y-Direction

The application software shall calculate the:

- Cropping factor: Crop_Fac = x_Sensor_Res/x_output_res
- Number of lines read out in y direction in accordance to sensor center: lines_y =Crop_Fac * y_output_res

Note: The cropping factor scales in the axis where cropping is required, so that it fits the maximum possible sensor area.

Example:

X/Y Sensor Resolution: 1920x1536 -> X/Y Output Resolution : 1280x720

Crop_Fac: 1920/1280 = 1,5

*lines_y: 1,5*720 = 1080*

*lines_x: 1,5*1280 = 1920*

Status	Draft
Verification Criteria	Get stream image with max Sensor Resolution (1920x1536) as reference image. Compare the content of the reference image with the content of the output image (with new resolution).

MVC-96505 - Aspect Ratio Conversion -X-Axis Cropping

When the output aspect ratio has a smaller width-to-height ratio value
than the camera sensor aspect ratio, the application software shall crop the image symmetrically along the X-axis, centered around the camera sensor midpoint.

Status	Draft
Verification Criteria	Verify that the image is cropped symmentrically along the x-axis when the output aspect ratio is lower than the sensor´s.

MVC-96504 - Aspect Ratio Conversion - Cropping Line Calculation X-Direction

The application software shall calculate the:

- Cropping factor: Crop_Fac = y_Sensor_Res / y_output_res
- Number of lines read out in x direction in accordance to sensor center: lines_x = Crop_Fac * x_output_res

Note: The cropping factor scales in the axis where cropping is required, so that it fits the maximum possible sensor area.

Status	Draft
Verification Criteria	Get stream image with max Sensor Resolution (1920x1536) as reference image. Compare the content of the reference image with the content of the output image (with new resolution).

8.4.7 Zoom Functionality

Note:
Zooming is triggered on the IK API using the AmbaIK_SetDzoomInfo() function.
It is triggered with SvcViewCtrl_Exec() with the parmeter: Cmd=3 of the corresponding structure SVC_VIEW_CTRL_CMD_MSG_s.
For internal development or test purpose, the SvcIK_ImgDzoomCtrl() may be used for evaluation.
See:
Jira Ticket 98, 242, 243
and SW-S08-418_1.3_Ambarella_SDK8_API_SVC_iCam_Liveview_Configuration.pdf
chap. 3.5 Request Stage — Pan / Tilt / Zoom / Set the ROI by the IK.
and 2.8 View Ctrl Data Structure — StillCfg/Table 16. The structure Related to SVC_VIEW_CTRL_CMD_MSG_s
and SDK82-DDG-027_2.1_Ambarella_SDK82001_u04_DDG_ImageKernel.pdf
chap. 4.3.123.1 AmbaIK_SetDzoomInfo > AMBA_IK_DZOOM_INFO_s/Table 4-370. Definition of AMBA_IK_DZOOM_INFO_s for AmbaIK_SetDzoomInfo()

MVC-96481 - Zoom Functionality - Activation of Supported Zoom Factors via SOME/IP

The application software shall allow the configuration of a zoom factor from the predefined list of supported values stated in Zooming.

Status	Draft
Verification Criteria	Verify that all spezified zoom factors are accepted and correctly applied by the system.

MVC-96480 - Zoom Functionality - Calculation of ZoomROI in Sensor Area

The application software shall calculate the dimensions of the zoomed region of interest (ZoomROI) using the formula:

ZoomROI(x,y) == ReqResolution(x,y) * 1/ZoomFactor

Note: In the implementation, this is performed using:

*ZoomFactor: Value * 65536*

The value is taken from the structure SVC_VIEW_CTRL_CMD_MSG_s.

Example:

Value = 1,5 (1920x1080 ->1280x720)

65536: 16.16 fixed point arithmetic

the upper 16 bits represent the integer part

the lower 16 bits represent the fractional part

ZoomX = 1,5 65536*

ZoomY = 1,5 65536*

dec	*16.16 fixed point arithmetic*	*hex* *0 x integer and fractional part*
1.0	*1 * 2^{16} = 65536*	*0 x 0001 0000*
1.5	*1.5 * 2^{16} = 65536 + 32768 = 98304*	*0 x 0001 8000*

Status	Draft
Verification Criteria	Verify that ZoomROI is calculated according to formula and reflected in output image.

MVC-96478 - Zoom Functionality - Centering of ZoomROI in Sensor Area

The application software shall center the ZoomROI within the sensor area unless a specific panning position is applied.

Status	Draft
Verification Criteria	Verify that ZoomROI is centered in image when no panning is applied.

8.4.8 Pan Functionality

Note:
Panning is triggered on the IK API using the AmbaIK_SetDzoomInfo() function.
It is triggered with SvcViewCtrl_Exec() with the parmeter: Cmd=1 of the corresponding structure SVC_VIEW_CTRL_CMD_MSG_s.
For internal development or test purpose, the SvcIK_ImgDzoomCtrl() may be used for evaluation.
See:
Jira Ticket 98, 242, 243
and SW-S08-418_1.3_Ambarella_SDK8_API_SVC_iCam_Liveview_Configuration.pdf
chap. 3.5 Request Stage — Pan / Tilt / Zoom / Set the ROI by the IK.
and 2.8 View Ctrl Data Structure — StillCfg/Table 16. The structure Related to SVC_VIEW_CTRL_CMD_MSG_s
and SDK82-DDG-027_2.1_Ambarella_SDK82001_u04_DDG_ImageKernel.pdf
chap. 4.3.123.1 AmbaIK_SetDzoomInfo > AMBA_IK_DZOOM_INFO_s/Table 4-370. Definition of AMBA_IK_DZOOM_INFO_s for AmbaIK_SetDzoomInfo()

MVC-96488 - Pan Functionality - Navigation through Panning Positions using Center Pattern

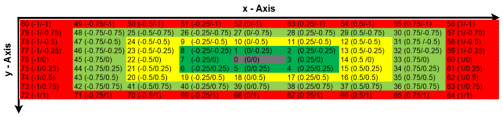
The application software shall support the navigation through panning positions based on a concentric center pattern, starting from the sensor center and expanding outward.

Status	Draft
Verification Criteria	Verify that the panning positions are navigated in the specified concentric pattern.

MVC-96487 - Pan Functionality - Use of Panning Position within Vx/Vy Grid

If the requested panning position is not 0, the application software shall allow selection of a panning position within a predefined Vx/y grid, defined as 9x9, based on Panning input received via SOME/IP.

Note:



Status	Draft
Verification Criteria	Verify that all supported panning positions can be selected and applied via SOME/IP command.

MVC-96489 - Pan Functionality - Calculation of Center Coordinates for Panning

The application software shall shall compute the center coordinates of each panning position based on the following formulas:

X_Display= ((X_AxisSensormax - X_axisZoomROI) / 2) * Vx)+ OffsetCenterSensorX

Y_Display= ((Y_AxisSensormax - Y_axisZoomROI) / 2) * Vy)+ OffsetCenterSensorY

Note: In the implementation, this is performed using:

Example:

Value = 2 (pan two pixels from the center of the ROI on the same ViewZone)

The value is taken from the structure SVC_VIEW_CTRL_CMD_MSG_s.

65536: 16.16 fixed point arithmetic

the upper 16 bits represent the integer part

the lower 16 bits represent the fractional part

*ShiftX = 2 * 65536*

*ShiftY = 0 * 65536*

dec	*16.16 fixed point arithmetic*	*hex* *0 x integer and fractional part*
1.0	*1 * 2^{16} = 65536*	*0 x 0001 0000*
2.0	*2.0 * 2^{16} = 131072*	*0 x 0002 0000*

X_Display = ((1920 - 1280) / 2) * 1 + 2 = 322

Y_Display = ((1080 - 720) / 2) * 1 + 0 = 180

ShiftX = ((ActiveOffsetX - ((RawWidth - ActiveWidth) / 2)) * 65536)

131072 = ((322 - ((1920 - 1280) / 2)) * 65536)

ShiftY = ((ActiveOffsetY - ((RawHeight - ActiveHeight) / 2)) * 65536)

65536 = ((180 - ((1080 - 720) / 2)) * 65536)

Status	Draft
Verification Criteria	Verify that panning coordinates are calculated and applied according to the specified formula.

9 GPTP

Objective: After two P2P propagation delay procedures and one subsequent SYNC/Follow_Up procedure, the "clock maintained by uC SW" shall be sychronized with <= +- 1 ms offset and syntonized with <= 200 PPM with the grandmaster clock.
At Least for A/B-Sample this could be verified via AmbaShell command "svc_net phcget".

Assumptions:

1. The SOW2-ECU shall act as time-aware end station according to IEEE Std 802.1AS-2011 with Autosar adaption according to AUTOSAR_PRS_TimeSyncProtocol FO R24-11
2. The SOW2-ECU shall be connected to a time-aware bridge of type peer-to-peer transparent clock defined in IEEE Std 1588-2008
3. The SOW2-ECU shall have the ClockIdentity = 0x112233FFFEAABBCC (dummy) according to IEEE Std 802.1AS-2011 sec. 8.5.2.2
4. The SOW2-ECU shall initiate the peer-to-peer propagation delay algorithm according to IEEE Std 802.1AS-2011 sec. 11.1.2 every 1s
5. The SOW2-ECU shall set the SequenceID = 1 for the first Pdelay_Req message according to IEEE Std 802.1AS-2011 sec. 11.3.8
6. The SOW2-ECU shall increment the SequenceID for following Pdelay_Req messages by 1 according to IEEE Std 802.1AS-2011 sec. 11.3.8
7. When the SOW2-ECU receives SYNC and Follow_Up the SOW2-ECU shall adjust its local clock and clock frequency
8. As specified in section 1.2.2 of AUTOSAR_PRS_TimeSyncProtocol FO R24-11, no Announce messages are transmitted by the grandmaster and no Pdelay_Req messages are sent to slave nodes.
   Thus the SOW2-ECU is expected to receive only the following PTP messages:
   a. SYNC (messageType = 0x0)
   b. Follow_Up (messageType = 0x8)
9. The SOW2-ECU never receives Pdelay_Req message

Note: In case of signed integers the two's complement is used according to IEEE Std 802.1AS-2011 sec. 6.3.2

9.1 gPTP Messages Handling

9.1.1 Messages Filtering

MVC-105126 - gPTP Messages Timestamping Filtering
The uC SW shall configure PHY such that it measures the Ingress/Egress timestamps only for the following gPTP messages:

Message Name	Message Type	Direction to SOW2
Sync	0x00	Ingress
PDelay_Req	0x02	Egress
PDelay_Resp	0x03	Ingress

Note: the other gPTP messages will be still sent and received but ignored for timestamping.

- For 100Mbps PHY variant (TJA1104), SW shall configure PHY register EVENT_MSG_FILT (1148h) = 0x000D
- For 1Gbps PHY variant (TJA1121), SW shall configure PHY register PTP_MESSAGE_FILTER (9010h) = 0x000D

Status	Draft
Verification Criteria	Check if UART can yield plausible time stamping values for messages Sync, Pdelay_Req, and Pdelay_Resp. (to turn on logging, use UART command: gptp_log 2)

MVC-105367 - gPTP Domain Number Filtering
The uC SW shall configure PHY to activate domain number filtering, such that only gPTP messages with domain number = 1 shall be received and sent.

- For 100Mbps PHY variant (TJA1104), SW shall configure PHY register PKT_FILT_CTRL (1140h): bit 13-6: DOMAIN_NUM = 0x01, bit 5: DOMAIN_FILT_EN = 1
- For 1Gbps PHY variant (TJA1121), SW shall configure PHY register
  - PTP_FILTER_ENABLES (9011h): bit 15: DOMAIN_NUMBER = 1
  - FILTER_PARAMETERS (9050h): bit 15-8: DOMAIN_NUMBER = 0x01

Status	Draft
Verification Criteria	Send Sync message with DOMAIN_NUMBER != 1, and then check if UART log shall not show the reception of the message (to turn on logging, use UART command: gptp_log 3)

MVC-105372 - gPTP VLAN_TAG Filtering
The uC shall configure PHY to allow the gPTP messages, which contain VLAN_TAG, being timestamped.

- For 100Mbps PHY variant (TJA1104), SW shall configure PHY register PKT_FILT_CTRL (1140h): bit 14: VLAN_TAG_ALLOW = 1
- For 1Gbps PHY variant (TJA1121), SW shall configure PHY register PTP_FILTER_ENABLES (9011h): bit 13: VLAN_TAG = 1

Status	Draft
Verification Criteria	Send Sync message with VLAN_TAG field, and then check if UART log shall show the reception of the message (to turn on logging, use UART command: gptp_log 3)

9.1.2 Messages Formatting

All gPTP messages transmitted/received by SOW2 yield the same format as following:

Offset	Bytes	Attribute
0	1	Bit[0:3]: messageType Bit[4:7]: majorSdoId
1	1	Bit[0:3]: versionPTP Bit[4:7]: minorVersionPTP
2	2	messageLength
4	1	domainNumber
5	1	minorSdoId
6	2	flags
8	8	correctionField
16	4	messageTypeSpecific
20	10	sourcePortIdentity: Byte[0:7]: ClockIdentity Byte[8:9]: portNumber
30	2	sequenceID
32	1	control
33	1	logMessageInterval

MVC-92307 - Pdelay_Req Message Transmission
The uC SW shall send Pdelay_Req message with period ~= 1s with the following format:

Offset	Bytes Length	Field	Description
0	34	Header	gPTP message Header
34	10	Reserved	not used, set to zeros
44	10	Reserved	not used, set to zeros

The header of Pdelay_Req message shall be set as the following:

Offset	Bytes	Attribute	Value	Description
0	1	Bit[0:3]: messageType Bit[4:7]: majorSdoId	0x12	transportSpecific = 0x1 messageType = 0x2 (Pdelay_Req)
1	1	Bit[0:3]: versionPTP Bit[4:7]: minorVersionPTP	0x12	indicates the version number of IEEE 1588 PTP
2	2	messageLength	0x00 36	total number of octets that form the PTP message in this case 0x36 = $(54)_{10}$, composed of 34 Bytes Header plus (10+10) Bytes reserved.
4	1	domainNumber	0x01	GVT time domain ID = 1 in T3 platform
5	1	minorSdoId	0x00	-
6	2	flags	0x00 00	all flags are set to FALSE
8	8	correctionField	0x00 00 00 00 00 00 00 00	must be 0 for an end station
16	4	messageTypeSpecific	0x00 00 00 00	-
20	10	sourcePortIdentity: Byte[0:7]: ClockIdentity Byte[8:9]: portNumber	0x [first 3-bytes of Mac Source Addr.] FF FE [last 3-bytes of Mac Source Addr.] 00 01	ClockIdentity and PortNumber as specified in system requirements
30	2	sequenceID	0xXX XX	increasing for subsequent messages starting by a random value
32	1	control	0x05	The value is 0x5
33	1	logMessageInterval	0x00	time between two messages -> 2^0s = 1s

Status	Draft
Verification Criteria	Check if Pdelay_Req message is being sent from SOW2 in about every 1s, check if all fields in message yield correct values

MVC-92296 - SequenceID of the first Pdelay_Req message after startup

The uC SW shall insert a random value into the sequenceID header field of the first Pdelay_Req message after startup of the SOW2 ethernet interface.

Note: According to IEEE Std 802.1AS-2011 sec. 11.3.8

Status	Draft
Verification Criteria	Start uC SW multiple times, check if the first Pdelay_Req sent by SOW2 yield different values in field sequenceID

MVC-92298 - SequenceID of subsequent Pdelay_Req messages

The uC SW shall increment the sequenceID header field by 1 for each subsequent egress Pdelay_Req message after the first one. Special Case: After sequenceID = 0xFFFF the next sequenceID shall be 0x0001

Note: Rollover of UInteger16 is expected behavior according to IEEE Std 802.1AS-2011 sec. 11.3.8

Note: According to IEEE Std 802.1AS-2011 sec. 11.3.8

Status	Draft
Verification Criteria	Check if the sequenceID of every Pdelay_Req message sent by SOW2 will increment by 1.

MVC-92644 - SourcePortIdentity of Pdelay_Req messages

The uC SW shall set the sourcePortIdentity in the header of theegress Pdelay_Req messages as following:

.clockIdentity = bytes[0:2] = First 3 bytes of the SOW2 Mac address
bytes[3:4] = 0xFF 0xFE
bytes[5:7] = last 3 bytes of the SOW2 Mac address

.portNumber = 0x0001

Note:

- Field defined according to IEEE Std 802.1AS-2011 sec. 8.5.2.3
- Mac address shall be obtained acc. to MVC-96158 - Zero-terminated Configuration String: MAC Address

Status	Draft
Verification Criteria	check if Pdelay_Req message sent by SOW2 yield correct value in field sourcePortIdentity

MVC-105132 - Sync Message Parsing

The uC SW shall receive and parse gPTP Sync message with following format:

Offset	Bytes Length	Field	Description
0	34	Header	gPTP message Header
34	10	Reserved	not used

The header of Sync message has the same format as MVC-111109 - gPTP Messages Common Header

Status	Draft
Verification Criteria	

MVC-105133 - Follow_Up Message Parsing

The uC SW shall receive and parse gPTP Follow_Up message with following format:

Offset	Bytes Length	Field	Description
0	34	Header	gPTP message Header, see MVC-111109 - gPTP Messages Common Header
34	10	preciseOriginTimestamp	The timestamp when gPTP master has just sent the Sync message
44	32	Follow_Up information TLV	TLV extension

preciseOriginTimestamp has following format:

Offset	Bytes Length	Field	Description
0	6	seconds	second part of the timestamp
6	4	nanoseconds	nanosecond part of the timestamp

Follow_Up information TLV has following format:

Offset	Bytes Length	Field	Description
0		tlvType	-
2		lengthField	-
4		organizationId	-
7		organizationSubType	-
10		cumulativeScaledRateOffset	(rateRatio - 1.0) * 2^41, rateRatio is the frequency ratio between master and slave.
14		gmTimeBaseIndicator	indicates if grandmaster has changed its time base.
16		lastGmPhaseChange	time of the current Grandmaster clock - time of the previous Grandmaster clock, at the time that the current Grandmaster PTP Instance became the Grandmaster PTP Instance
28		scaledLastGmFreqChange	fractional frequency offset of the current Grandmaster Clock relative to the previous Grandmaster Clock, at the time that the current Grandmaster PTP Instance became the Grandmaster PTP Instance, multiplied by 2^41

Status	Draft
Verification Criteria	

MVC-105134 - Pdelay_Resp Message Parsing

The uC SW shall receive and parse gPTP Pdelay_Resp message with following format:

Offset	Bytes Length	Field	Description
0	34	Header	gPTP message Header, see MVC-111109 - gPTP Messages Common Header
34	10	requestReceiptTimestamp	The ingress timestamp of the associated Pdelay_Req message to the gPTP neighbor
44	10	requestingPortIdentity	equals the sourcePortIdentity field of the associated Pdelay_Req message.

The header of the message has the same format as the Pdelay_req: see MVC-92307 - Pdelay_Req Message Transmission

Status	Draft
Verification Criteria	

MVC-105136 - Pdelay_Resp_Follow_Up Message Parsing

The uC SW shall receive and parse gPTP Pdelay_Resp_Follow_Up message with following format:

Offset	Bytes Length	Field	Description
0	34	Header	gPTP message Header, see MVC-111109 - gPTP Messages Common Header
34	10	responseOriginTimestamp	The egress timestamp of the associated Pdelay_Resp message from the gPTP neighbor
44	10	requestingPortIdentity	equals the sourcePortIdentity of the associated Pdelay_Req message header

The header of Sync message has the same format as the Pdelay_req: see MVC-92307 - Pdelay_Req Message Transmission

Status	Draft
Verification Criteria	

9.2 Messages Error Handling

MVC-111172 - Plausibility Check for Pdelay_Resp Message

On Receipt of Pdelay_Resp Message, the uC SW shall check if all of the following data of this message are correct:

1. if Pdelay_Resp.head.sequenceId == Pdelay_Req.head.sequenceId
2. if Pdelay_Resp.requestingPortIdentity.clockIdentity equals to EUI-64 clock identity derived from SOW2 source MAC address
    a. byte [5:7] = macAddr[3:5]
    b. byte [3:4] = 0xFF 0xFE
    c. byte [0:2] = macAddr[0:2]
3. if Pdelay_Resp.requestingPortIdentity.portNumber == 1

where Pdelay_Req is the Pdelay_Req message being sent just before the receipt of the current Pdelay_Resp.

If any of the checks failed, SW shall discard this Pdelay_Resp message and start a new peer-to-peer delay cycle (see MVC-111187 - Peer-to-Peer Delay Measurement Cycle for details) by resending a new Pdelay_Req.

Status	Draft
Verification Criteria	After the test computer received Pdelay_Req from SOW2, it shall send Pdelay_Resp and inject errors in either sequenceId, clockIdentity, or portNumber. Check if SOW2 will sends a new Pdelay_Req as response and if the UART message for neighborRateRatio and neighborPropDelay will not appear..

MVC-111171 - Plausibility Check for Pdelay_Resp_Follow_Up Message: sequenceId Error

On Receipt of Pdelay_Resp_Follow_Up Message, the uC SW shall check if all of the following data of this message are correct:

1. if Pdelay_Resp_Follow_Up.head.sequenceId == Pdelay_Req.head.sequenceId
2. if Pdelay_Resp_Follow_Up.requestingPortIdentity.clockIdentity equals to EUI-64 clock identity derived from SOW2 source MAC address
    a. byte [5:7] = macAddr[3:5]
    b. byte [3:4] = 0xFF 0xFE
    c. byte [0:2] = macAddr[0:2]
3. if Pdelay_Resp.requestingPortIdentity.portNumber == 1
4. if Pdelay_Resp_Follow_Up.head.sourcePortIdentity == Pdelay_Resp.head.sourcePortIdentity

where

- Pdelay_Resp is the Pdelay_Resp received just before the receipt of the current Pdelay_Resp_Follow_Up message
- Pdelay_Req is the Pdelay_Req message being sent just before the receipt of the current Pdelay_Resp message.

If check 1 failed (sequenceId unmatch), SW shall discard this Pdelay_Resp_Follow_Up message and start a new peer-to-peer delay cycle (see MVC-111187 - Peer-to-Peer Delay Measurement Cycle for details) by resending a new Pdelay_Req.

Status	Draft
Verification Criteria	After the test computer sent correct Pdelay_Resp message to SOW2, it shall send Pdelay_Resp_Follow_Up and inject error in sequenceId. Check if SOW2 will sends a new Pdelay_Req as response, and if the UART message for neighborRateRatio and neighborPropDelay will not appear.

MVC-111191 - Plausibility Check for Pdelay_Resp_Follow_Up Message: Other Errors

On Receipt of Pdelay_Resp_Follow_Up Message, the uC SW shall check if all of the following data of this message are correct:

1. if Pdelay_Resp_Follow_Up.head.sequenceId == Pdelay_Req.head.sequenceId
2. if Pdelay_Resp_Follow_Up.requestingPortIdentity.clockIdentity equals to EUI-64 clock identity derived from SOW2 source MAC address
    a. byte [5:7] = macAddr[3:5]
    b. byte [3:4] = 0xFF 0xFE
    c. byte [0:2] = macAddr[0:2]
3. if Pdelay_Resp.requestingPortIdentity.portNumber == 1
4. if Pdelay_Resp_Follow_Up.head.sourcePortIdentity == Pdelay_Resp.head.sourcePortIdentity

where

- Pdelay_Resp is the Pdelay_Resp received just before the receipt of the current Pdelay_Resp_Follow_Up message
- Pdelay_Req is the Pdelay_Req message being sent just before the receipt of the current Pdelay_Resp message.

If (check 1 passed) AND (any of checks 2, 3, 4 failed), SW shall discard this Pdelay_Resp_Follow_Up message and wait for a new Pdelay_Resp_Follow_Up message with correct data.

Status	Draft
Verification Criteria	After the test computer sent correct Pdelay_Resp message to SOW2, it shall send Pdelay_Resp_Follow_Up and inject error in requestingPortIdentity or sourcePortIdentity. Check if the UART message for neighborRateRatio and neighborPropDelay will not appear.

MVC-111173 - Timeout Check for Pdelay_Resp Message

The uC SW shall start a timeout counter after it sends out Pdelay_Req message.

if no Pdelay_Resp message has been received within 1 second of this timeout counter, SW shall resend a new Pdelay_Req message with its sequenceId field incremented by 1.

Status	Draft
Verification Criteria	Check if SOW2 will resend Pdelay_Req, whose sequenceId increments by 1, when no Pdelay_Resp message is received within 1s since the last sending of Pdelay_Req message.

MVC-111177 - Timeout Check for Pdelay_Resp_Follow_Up Message
The uC SW shall start a timeout counter after it sends out Pdelay_Req message.
If no Pdelay_Resp_Follow_Up message has been received within 1 second of this timeout counter, SW shall resend a new Pdelay_Req message with its sequenceId field incremented by 1.

Status	Draft
Verification Criteria	Check if SOW2 will resend Pdelay_Req, whose sequenceId increments by 1, when no Pdelay_Resp_Follow_Up message is received within 1s since the last sending of Pdelay_Req message.

MVC-111176 - Plausibility Check for Sync Message
On receipt of Sync Message, the uC SW shall check if all of the following data of this message are correct:

1. Sync.head.flags[0] & 0x2 != 0 (two-step bit flag shall be set)

If any of the checks failed, SW shall

- discard this Sync message, and
- **NOT** perform slave clock synchronization acc. to MVC-105532 - Update Slave Clock Time,
- wait for the new Sync message

Status	Draft
Verification Criteria	Send Sync and Follow_Up messages to SOW2 board, where the two-step flag bit of Sync Message is set to 0, Check if the calculation log for syncReceiptLocalTime and syncReceiptTime shall not appear in UART log (turn on UART log by "SX GPTP log 1")

MVC-111179 - Plausibility Check for Follow_Up Message
On receipt of Follow_Up Message, the uC SW shall check if all of the following data of this message are correct:

1. Follow_Up.head.sequenceId == Sync.head.sequenceId

where Sync is the Sync message received just before the receipt of the current Follow_up message.
If any of the checks failed, SW shall

- discard this Follow_Up message, and
- **NOT** perform slave clock synchronization acc. to MVC-105532 - Update Slave Clock Time,
- until the next Follow_Up message with successful plausibility check result is incoming

Status	Draft
Verification Criteria	Send Sync and Follow_Up messages to SOW2 board, where the sequenceId of Follow_Up is injected with erronous value, Check if the calculation log for syncReceiptLocalTime and syncReceiptTime shall not appear in UART log (turn on UART log by "SX GPTP log 1")

MVC-111178 - Timeout Check for Follow_Up Message
After receipt of Sync Message, if SOW2 has not received the plausible Follow_Up message within `upstreamSyncInterval` second, the uC SW shall

- discard the previously received Sync message, and
- **NOT** perform slave clock synchronization acc. to MVC-105532 - Update Slave Clock Time,
- wait for the new Sync message.

where `upstreamSyncInterval` is the timeout threshold in seconds and calculated by

$$\text{upstreamSyncInterval} = 2^{\text{Sync.head.logMessageInterval}}$$

Status	Draft
Verification Criteria	Send Sync message, then delay the sending of Follow_Up by more than upstreamSyncInterval seconds, Check if the calculation log for syncReceiptLocalTime and syncReceiptTime shall not appear in UART log (turn on UART log by "SX GPTP log 1")

9.3 Local Clocks Synchronisation

In SOW2 project, PHY and MAC0 are chosen as the time sources of the gPTP slave clock. The clocks of the two hardware modules must be synchronized and syntonized. This shall be achieved by the PPS signal between PHY and MAC0.

The following terms will be used in the requirements descriptions:

Name	Description
PHY LTC	The local time clock (LTC) of PHY, which is one of the time sources of gPTP slave clock.
MAC0 system time	The local time of MAC0, which is one of the time sources of gPTP slave clock.
PPS	The synchronization trigger signal between PHY and MAC0.
PPS_OUT	The pulse-per-second output from PHY, which can be used to test the synchronization quality between gPTP master and slave.

MVC-105373 - GPIO Configuration for PPS Signal between PHY and MAC0
The uC SW shall configure

- SoC GPIO39 (L17) as function ENET_PTP_PPS, such that MAC0 on SoC is able to send PPS signal to PHY
- PHY GPIO1 as function PTP_SYNC, such that PHY can process the incoming PPS signal:
  - For 100Mbps PHY variant (TJA1104), SW shall configure PHY register GPIO1_FUNC_CONFIG (2C41h) = 0xA040
    bit 15: ENABLE = 1
    bit 13: PIN_SNIF = 1 (Pin status readable by register GPIO_STATUS)
    bit 7-5: FUNC_SELECT = 0b010(PTP)
    bit 4-0: SIGNAL_SELECT = 0b00000(PTP_SYNC)
  - For 1Gbps PHY variant (TJA1121), SW shall configure PHY register GPIO1_FUNC_CONFIG (2C41h) = 0xA040
    bit 15: ENABLE = 1
    bit 13: PIN_SNIF = 1 (Pin status readable by register GPIO_STATUS)
    bit 7-5: FUNC_SELECT = 0b010(PTP)
    bit 4-0: SIGNAL_SELECT = 0b00000(PTP_SYNC)

Status	Draft
Verification Criteria	Check if PHY register GPIO1_FUNC_CONFIG yields the correct setting value

MVC-105448 - GPIO Configuration for PPS_OUT Signal
The uC SW shall configure SoC GPIO98 (D10) as GPIO input, and configure PHY GPIO9 as PPS_OUT function:

- For 100Mbps PHY variant (TJA1104), SW shall configure PHY register GPIO9_FUNC_CONFIG (2C49h):
  .ENABLE(bit15) = 1
  .PIN_SNIF(bit 13) = 1 (Pin status readable by register GPIO_STATUS)
  .FUNC_SELECT(bit7:5) = 0b010(PTP)
  .SIGNAL_SELECT(bit4:0) = 0b10010(PPS_OUT)
- For 1Gbps PHY variant (TJA1121), SW shall configure PHY register GPIO9_FUNC_CONFIG (2C49h):
  .ENABLE(bit15) = 1
  .PIN_SNIF(bit 13) = 1 (Pin status readable by register GPIO_STATUS)
  .FUNC_SELECT(bit7:5) = 0b010(PTP)
  .SIGNAL_SELECT(bit4:0) = 0b10010(PPS_OUT)

Note: The purpose of configuring PPS_OUT is to provide a testability such that the tester can utilize this signal to measure the synchronization accuracy between gPTP master and slave.

Status	Draft
Verification Criteria	Check if PHY register GPIO9_FUNC_CONFIG yields the correct value. Check if PHY GPIO9 outputs positive pulses in every 1s

MVC-105374 - Periodic PPS Trigger between MAC0 and PHY
The uC SW shall utilize MAC0 to send PPS single pulses via SoC GPIO39 with period = 25ms at the time base of MAC0 system time, and configures MAC0 to trigger an interrupt request (IRQ) on each PPS rising edge.

Status	Draft

Verification Criteria	Check if positive pulses can be detected at PHY GPIO1 in every 25ms.

MVC-105375 - Syntonization between PHY and MAC0

The uC SW shall enable the hardware tracking (phase lock) functionality of PHY, such that the frequency of PHY LTC can be automatically aligned to the MAC0 system clock on the incoming PPS signal.

- For 100Mbps PHY variant (TJA1104), SW shall configure PHY register HW_LTC_LOCK_CTRL(1115h):
  - bit 0: HW_LTC_LOCK_EN = 1
- For 1Gbps PHY variant (TJA1121), SW shall configure PHY register GPTP_CONFIG (1006h):
  - bit 2: LTC_AUTO_LOCK = 1

Status	Draft
Verification Criteria	Check if PHY register HW_LTC_LOCK_CTRL (for 100Mbps variant) and GPTP_CONFIG (for 1Gbps variant) yield correct value

MVC-105376 - Synchronization Accuracy between PHY and MAC0

At every incoming PPS IRQ, the uC shall align PHY LTC time to MAC0 system time.

In combination with MVC-105375 - Syntonization between PHY and MAC0 < 11_P_01467_MEKRA_Vision_Controller, the time difference between PHY LTC and MAC0 system time shall not exceed **200ns**, when no Sync message is being received from gPTP master.

Note: the alignment of PHY LTC can be achieved by writing to the target LTC registers, and then latch the write on the next PPS rising edge.

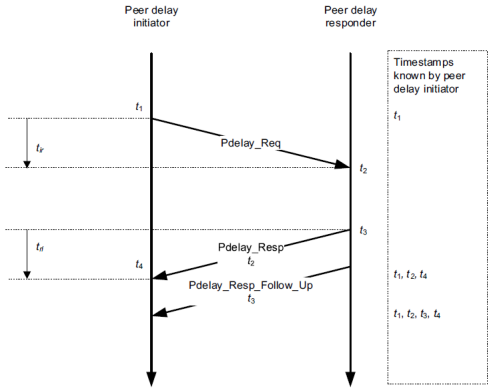
- For 100Mbps PHY variant (TJA1104), SW shall write following PHY register:
  LTC_WR_SEC_0 (1108h)
  LTC_WR_SEC_1 (1109h)
  LTC_WR_NSEC_0 (1106h)
  LTC_WR_NSEC_1 (1107h)
  the write latch register:
  LTC_LOAD_CTRL (1105h): bit 1: LOAD_LTC_PPS
- For 1Gbps PHY variant (TJA1121), SW shall write following PHY register
  LTC_LOAD_DATA_0 (1040h)
  LTC_LOAD_DATA_1 (1041h)
  LTC_LOAD_DATA_2 (1042h)
  LTC_LOAD_DATA_3 (1043h)
  the write latch register:
  GPTP_CONTROL (1000h): bit 3: LOAD_LTC_ON_SYNC

Status	Draft
Verification Criteria	Give UART commands: "gptp_log 2" and "ppssync SetPPSTraceEnabled" and "ppssync ShowPPSSyncTrace" to show the synchronization log between PHY and MAC0. Check if the time diff < 200ns

9.4 Peer-to-peer (P2P) propagation delay algorithm

In order to correct the receiving time from grandmaster via SYNC/Follow_Up messages the propagation delay to the SOW2's neighbor must be known. Therefore the peer-to-peer (P2P) propagation delay algorithm according to IEEE Std 802.1AS-2011 sec. 11.1.2 will be used.

A Peer-to-Peer delay measurement cycle starts with the initiator issuing a Pdelay_Req message and generating a timestamp t1. The responder receives this message and timestamps it with time t2. The responder returns a Pdelay_Resp message and timestamps it with time t3. The responder returns the time t2 in the Pdelay_Resp message and the time t3 in a Pdelay_Resp_Follow_Up message. The initiator generates a timestamp t4, upon receiving the Pdelay_Resp message.



MVC-111208 - Prerequisites for neighborRateRatio and neighborPropDelay Calculation

The uC SW shall perform neighborRateRatio and neighborPropDelay Calculation, only if all of following conditions are fulfilled:

1. Pdelay_Resp and Pdelay_Resp_Follow_Up message are received in the correct sequence within 1s after sending out Pdelay_Req, see MVC-111173 - Timeout Check for Pdelay_Resp Message and MVC-111177 - Timeout Check for Pdelay_Resp_Follow_Up Message
2. MVC-111172 - Plausibility Check for Pdelay_Resp Message passed
3. MVC-111171 - Plausibility Check for Pdelay_Resp_Follow_Up Message: sequenceId Error passed
4. MVC-111191 - Plausibility Check for Pdelay_Resp_Follow_Up Message: Other Errors passed

If any of the checks above has failed, neighborRateRatio and neighborPropDelay variables shall not be updated.

Status	Draft
Verification Criteria	Inject error to any of the checks in description, check if the calculation UART message for neighborRateRatio and neighborPropDelay shall NOT appear (turn on UART log by "SX GPTP log 1")

MVC-92317 - Calculation of neighborRateRatio

When the MVC-111208 - Prerequisites for neighborRateRatio and neighborPropDelay Calculation is fulfilled at the end of MVC-111187 - Peer-to-Peer Delay Measurement Cycle. The uC SW shall calculate the neighborRateRatio using the following formula (F1):

$$\text{neighborRateRatio} = \frac{\text{correctedResponderEventTimestamp}_j - \text{correctedResponderEventTimestamp}_i}{\text{pdelayRespEventIngressTimestamp}_j - \text{pdelayRespEventIngressTimestamp}_i} \quad \text{(F1)}$$

where

$$\begin{aligned} & j = i + 1 \\ & i = 0, 1, 2 \ldots \\ \text{correctedResponderEventTimestamp}_i = {} & \text{Pdelay_Resp_Follow_Up}_i.\text{responseOriginTimestamp.seconds} \times 10^9 \\ & + \text{Pdelay_Resp_Follow_Up}_i.\text{responseOriginTimestamp.nanoseconds} \\ & + \text{Pdelay_Resp_Follow_Up}_i.\text{Header.correctionFiled} \times 2^{-16} \end{aligned}$$

The terms in the formula are described below:

Name	Description
$\text{Pdelay_Resp_Follow_Up}_i$	the Pdelay_Resp_Follow_Up message in the i-th P2P Pdelay_Req cycle. The bytes fields are defined in MVC-105136 - Pdelay_Resp_Follow_Up Message Parsing
$\text{pdelayRespEventIngressTimestamp}_i$	the ingress timestamp when Pdelay_Resp is received by SOW2 PHY, in response to the i-th received Pdelay_Req message

Note: neighborRateRatio is a gPTP global variable per PTP instance per PTP port. It describes the ratio of the frequency of the clock entity of the time-aware system at the other end of the link attached to this port (P2P-TC), to the frequency of the local clock

entity of this time-aware system (SOW2-ECU) (see IEEE802.1AS-2011 §10.2.5.7).

Status	Draft
Verification Criteria	check if the calculated variable neighborRateRatio yields correct result acc. to the formula (The result can be shown on UART using command: SX GPTP log 1)

MVC-92318 - Calculation of neighborPropDelay

When the MVC-111208 - Prerequisites for neighborRateRatio and neighborPropDelay Calculation is fulfilled at the end of MVC-111187 - Peer-to-Peer Delay Measurement Cycle, the uC SW shall calculate the neighborPropDelay using the following formula (F2):

$$\text{neighborPropDelay} = \frac{\text{r} \cdot (t_4 - t_1) - (t_3 - t_2)}{2} \qquad \text{(F2)}$$

where

$$\begin{aligned} r &= \text{neighborRateRatio} \\ t_4 &= \text{pdelayRespEventIngressTimestamp} \\ t_1 &= \text{pdelayReqEventEgressTimestamp} \\ t_2 &= \text{Pdelay_Resp.requestReceiptTimestamp.nanoseconds} \\ &\quad + \text{Pdelay_Resp.requestReceiptTimestamp.seconds} \times 10^9 \\ &\quad + \text{Pdelay_Resp.Header.correctionField} \times 2^{-16} \\ t_3 &= \text{Pdelay_Resp_Follow_Up.responseOriginTimestamp.nanoseconds} \\ &\quad + \text{Pdelay_Resp_Follow_Up.responseOriginTimestamp.seconds} \times 10^9 \\ &\quad + \text{Pdelay_Resp_Follow_Up.Header.correctionField} \times 2^{-16} \end{aligned}$$

The terms in the formula are clarified below

Name	Description
pdelayRespEventIngressTimestamp	the ingress timestamp when Pdelay_Resp is received by SOW2 PHY, in response to the received Pdelay_Req message
pdelayReqEventEgressTimestamp	the egress timestamp when Pdelay_Req is sent by SOW2 PHY.
Pdelay_Resp	the bytes fields of Pdelay_Resp message defined in MVC-105134 - Pdelay_Resp Message Parsing
Pdelay_Resp_Follow_Up	the bytes fields of Pdelay_Resp_Follow_Up message defined in MVC-105136 - Pdelay_Resp_Follow_Up Message Parsing
neighborRateRatio	Ratio of clock frequency of the neighbor and SOW2, whose calculation is defined by MVC-92317 - Calculation of neighborRateRatio

Note: neighborPropDelay is a global variable per PTP instance per PTP port. It is defined as the mean propagation delay on the link attached to the port (SOW2 PHY), relative to the port of the neighbor at the other end of the link (IEEE802.1AS-2011 §10.2.5.8).

Status	Draft
Verification Criteria	check if the calculated variable neighborPropDelay yields correct result acc. to the formula (The result can be shown on UART using command: SX GPTP log 1)

9.5 Master/Slave Synchronization & Syntonization

The Grandmaster requests all time-aware end stations via SYNC/Follow_Up messages to adjust their local clocks.

MVC-111213 - Prerequisites for Master/Slave Synchronization & Syntonization

The uC SW shall perform gPTP master/slave synchronization and syntonization actions, including

- MVC-92327 - Calculation of rateRatio
- MVC-92330 - Calculation of syncReceiptTime
- MVC-92329 - Calculation of syncReceiptLocalTime
- MVC-105532 - Update Slave Clock Time
- MVC-111214 - Update Slave Clock Frequency

ONLY if all of following conditions are fulfilled,

1. Sync message is received and MVC-111176 - Plausibility Check for Sync Message passed
2. Follow_Up is received within specified timeout threshold after the receipt of Sync message, see MVC-111178 - Timeout Check for Follow_Up Message
3. MVC-111179 - Plausibility Check for Follow_Up Message passed

if any of the condition is not fulfileed, SW shall perform **NONE** of the actions mentioned above.

Status	Draft
Verification Criteria	Inject errors to any of the conditions in the Req. description, check if NO Uart messages regarding the synchronization and syntonization actions shall appear (turn on UART log by "SX GPTP log 1")

MVC-92327 - Calculation of rateRatio

When one Sync/Follow_Up messages pair is received in sequence and all MVC-111213 - Prerequisites for Master/Slave Synchronization & Syntonization are fulfilled, the uC SW shall calculate the rateRatio using the following formula (F3):

$$\begin{aligned} \text{rateRatio} = {} & \text{Follow_Up.TLV.cumulativeScaledRateOffset} \times 2^{-41} \\ & + \text{neighborRateRatio} \end{aligned} \qquad \text{(F3)}$$

The terms of the formula are described below:

Name	Description
Follow_Up	The bytes fields of Follow_Up message are defined in MVC-105133 - Follow_Up Message Parsing
neighborRateRatio	The clock frequency ratio calculated by MVC-92317 - Calculation of neighborRateRatio

Note: The rateRatio is a variable in PortSyncSync state machine (see IEEE802.1AS-2011 §10.2.2.3). It is defined as the ratio of the frequency of the Grandmaster Clock to the frequency of the Slave Clock (SOW2).

Status	Draft
Verification Criteria	check if the calculated variable rateRatio yields correct result acc. to the formula (The result can be shown on UART using command: SX GPTP log 1)

MVC-92330 - Calculation of syncReceiptTime

When one Sync/Follow_Up messages pair is received in sequence and all MVC-111213 - Prerequisites for Master/Slave Synchronization & Syntonization are fulfilled, the uC shall calculate syncReceiptTime:

$$\begin{aligned} \text{syncReceiptTime} = {} & \text{Follow_Up.preciseOriginTimestamp.seconds} \times 10^9 \\ & + \text{Follow_Up.preciseOriginTimestamp.nanoseconds} \\ & + \text{followUpCorrectionField} \times 2^{-16} \\ & + \text{neighborPropDelay} \cdot \frac{\text{rateRatio}}{\text{neighborRateRatio}} \end{aligned} \qquad \text{(F4)}$$

where

$$\begin{aligned} \text{followUpCorrectionField} = {} & \text{Sync.header.correctionField} \\ & + \text{Follow_Up.header.correctionField} \end{aligned}$$

The terms in the formula are described below:

Name	Description
Sync	The bytes fields of Sync message, defined by MVC-105132 - Sync Message Parsing
Follow_Up	The bytes fields of Follow_Up message, defined by MVC-105133 - Follow_Up Message Parsing
rateRatio	frequency ratio between the Grandmaster Clock and the slave clock (SOW2), calculated by MVC-92327 - Calculation of rateRatio
neighborRateRatio	Ratio of clock frequency of the neighbor and SOW2, whose calculation is defined by MVC-92317 - Calculation of neighborRateRatio
neighborPropDelay	propagation delay between SOW2 and the neighbor, calculated by MVC-92318 - Calculation of neighborPropDelay

Note: syncReceiptTime is a gPTP global variable of per PTP instance. It is defined as the reception timestamp of Sync message relative to the gPTP grandmaster clock (see IEEE802.1AS-2011 §10.2.4.4).

Status	Draft
Verification Criteria	check if the calculated variable syncReceiptTime yields correct result acc. to the formula (The result can be shown on UART using command: SX GPTP log 1)

MVC-92329 - Calculation of syncReceiptLocalTime

When one Sync/Follow_Up messages pair is received in sequence and all MVC-111213 - Prerequisites for Master/Slave Synchronization & Syntonization are fulfilled, the uC SW shall calculate syncReceiptLocalTime using the following formula (F5):

$$\text{syncReceiptLocalTime} = \text{syncEventIngressTimestamp} \qquad (F5)$$

The terms in the formula are described below:

Name	Description
syncEventIngressTimestamp	The ingress timestamp of Sync message to PHY

Note: syncReceiptLocalTime is a gPTP global variable of per PTP instance. It is defined as the reception timestamp of Sync message relative to the gPTP Slave Clock (see IEEE802.1AS-2011 §10.2.4.5).

Status	Draft
Verification Criteria	check if the calculated variable syncReceiptLocalTime yields correct result acc. to the formula (The result can be shown on UART using command: SX GPTP log 1)

MVC-105532 - Update Slave Clock Time

When one Sync/Follow_Up messages pair is received in sequence and all MVC-111213 - Prerequisites for Master/Slave Synchronization & Syntonization are fulfilled, the uC SW shall update the slave clock (PHY LTC and MAC0 system time) time as following:

- $\mathrm{T}_i = \mathrm{T}_{i-i} + (\text{syncReceiptTime}_i - \text{syncReceiptLocalTime}_i)$

if the time difference `abs(syncReceiptTime - syncReceiptLocalTime)` between the gPTP grandmaster and slave (SOW2) exceeds 100ns.

The terms in the formula are described below:

Name	Description
syncReceiptTime_i	calculated by MVC-92330 - Calculation of syncReceiptTime after receiving the i-th Sync and FollowUp pair.
$\text{syncReceiptLocalTime}_i$	calculated by MVC-92329 - Calculation of syncReceiptLocalTime after receiving the i-th Sync and FollowUp pair.

Status	Draft
Verification Criteria	Check if the message about slave clock time update will appear when the diff between syncReceiptTime and syncReceiptLocalTime exceeds 100ns (command to turn on UART log for GPTP: SX GPTP log 1)

MVC-111214 - Update Slave Clock Frequency

When **EVENRY SECOND** Sync/Follow_Up messages pair is received in sequence and all MVC-111213 - Prerequisites for Master/Slave Synchronization & Syntonization are fulfilled, the uC SW shall update the slave clock (PHY LTC and MAC0 system time) frequency as following:

- $\mathrm{F}_i = \mathrm{F}_{i-1} \cdot \text{rateRatio}_i$

The terms in the formula are described below:

Name	Description
rateRatio_i	calculated by MVC-92327 - Calculation of rateRatio after receiving the i-th Sync and FollowUp pair.

Status	Draft
Verification Criteria	Check if the message about slave clock frequency update will appear after SOW2 received every second Sync/Follow_Up messages pair (command to turn on UART log for GPTP: SX GPTP log 1)

MVC-111216 - Master/Slave Synchronization Accuracy

When SOW2 has received and processed 5 valid gPTP Sync/Follow_Up messages pairs, the calculated gPTP master/slave clocks time difference `syncReceiptTime - syncReceiptLocalTime` shall converge within range (-10us, 10us).

Note: "valid" means that the received Sync/Follow_Up messages pairs fulfill all checks in MVC-111213 - Prerequisites for Master/Slave Synchronization & Syntonization.

Status	Draft
Verification Criteria	Watch the calculated difference syncReceiptTime - syncReceiptLocalTime in UART log, check if the value falls within range (-10us, 10us) as of the 6th received Sync/Follow_Up messages pairs (turn on UART log by "SX GPTP log 1")

MVC-111217 - Master/Slave Syntonization Accuracy

When SOW2 has received and processed 5 valid gPTP Sync/Follow_Up messages pairs, the calculated gPTP master/slave clocks frequency difference `rateRatio - 1.0` shall converge within range (-1000PPB, 1000PPB).

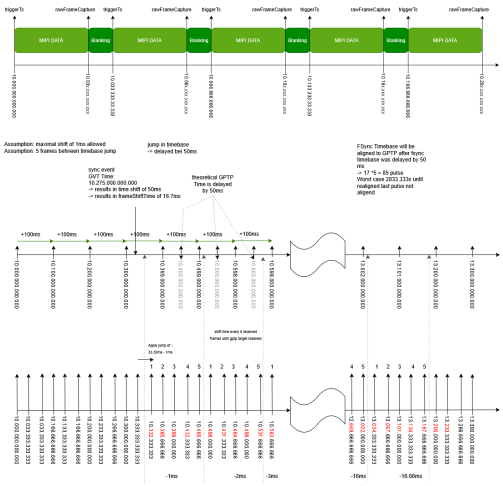
Note:

- "valid" means that the received Sync/Follow_Up messages pairs fulfill all checks in MVC-111213 - Prerequisites for Master/Slave Synchronization & Syntonization.
- `rateRatio` is calculated by MVC-92327 - Calculation of rateRatio

Status	Draft
Verification Criteria	Watch the calculated frequency difference rateRatio - 1.0 in UART log, check if the value falls within range (-10us, 10us) as of the 6th received Sync/Follow_Up messages pairs (turn on UART log by "SX GPTP log 1")

10 FSYNC

The following requirements are defined in FSYNC for now, because they are needed to get FSYNC working however it should be discussed if they need to be moved to their corresponding Component.



In this Example there is a syn event received at 10.325.000.000.000 that time should be 10.275.000.000.000. This results in a timeshift of -50ms meaning the timebase needs to be delayed by 50ms. The raster it self needs to be shifted by

$$rasterShift_{rasterAligned} = (50ms \% 33.33ms) \cdot 33.33ms = 33.33ms$$



$$rasterShift_{inBetweenRaster} = 50ms - rasterShift_{rasterAligned} = 16.66ms$$

The rasterShift_{rasterAligned} of 33.333ms can be applyed instantly, because its aligned to the fsync pulse raster. The rest rasterShift_{inBetweenRaster} must be aligned be reasjusting the raster. This must be done step by step as show above by 1ms steps highlighted in red. After the the raster was adjusted there needs to be a number of frames received until the next jump can occure. After a few time jumps the gptp time and fsync time are realigned again.

10.1 Serdes Config

Fsync needs a pin to be forwared over serdes: MVC-103085 - SerDes - Configuration - GPIO Forwarding of FsyncTrigger see also: MVC-102438 - GPIO Forwarding

Fsync will use this pin to trigger the Fsync sensor input, therefore the internal frameSync generation in serdes should be disabled

MVC-105529 - FSYNC - Disable the SerDes internal FrameSync generation

Disable the internal genertaion of FsyncPulses by writing setting FSYNC_MODE=0b11.

Note: see MVC-100620 - SerDes configuration signal routing

Status	Draft
Verification Criteria	Read back register from sensor

10.2 Image Sensor Config

MVC-103081 - FSYNC - Slave Mode Configuration register configuration

The application software shall config the Sensor to slave mode by writing the register as show in MVC-100496 - Sensor configuration (Slave mode)

Satisfyes: MVC-100496 - Sensor configuration (Slave mode)

Status	Draft
Verification Criteria	read back register from Sensor

MVC-103084 - FSYNC - Timepoint when to reconfig the Sensor to Slave mode

The application software shall reconfig the sensor to Slave after the mode has been set and before the sensor will start streaming.

Satisfyes: MVC-100496 - Sensor configuration (Slave mode)

Status	Draft
Verification Criteria	Review code seqeunce

MVC-105518 - FSYNC - Change Sensor standbymode of Sensor

The application software shall change the Sensor streaming mode from standby on to standby off, after the first fsync pulse has been send out from fsync

Status	Draft
Verification Criteria	read out sensor standaby status over i2c before fsync component triggeres first pulse (sont start fsync) and validate by using Oscilloscope to see there i sno pulse generated, then start fsync and wait for first pulse validate with Oscilloscope and read sensor standby mode

10.3 Configuration and Initialization of secondary MAC

MVC-105542 - FSYNC - Set communication interface of secondary MAC to RMII

The application software shall set the communication interface of the secondary MAC to RMII.

Status	Draft
Verification Criteria	check if mac time is increasing after setting

MVC-105541 - FSYNC - Config secondary mac to use internal clock

The application software shall configure the secondary mac to use the gclk clock

Status	Draft
Verification Criteria	check if mac time is increasing

MVC-105544 - FSYNC - Minimalistic initialisation of MAC

The application software shall initialize the MAC periphery with minimal steps neccesary .

Status	Draft
Verification Criteria	check if mac time is increasing

MVC-105545 - FSYNC - FrameSync Pulse forming

The application software shall configure the PPS generator within the secondary mac to send out a "high" signal for 1ns.

Status	Draft
Verification Criteria	check pulse length by using an Oscilloscope

10.4 Synchronization with gptp timebase

MVC-105547 - FSYNC - Syncronization and syntonizise of timebase between GPTP and FSYNC at the start of FSYNC

When the application software starts FSYNC component, it shall adjust the timebase of fsync to the timebase of GPTP, by shifting the the absoulte time of fsync to be identical to gptp time and set the PPB of GPTP timebase to FSYNC timebase.

Status	Draft
Verification Criteria	measure the delay between pps output of the phy and fsync output, if the delay doesnt add up its syncronizing

MVC-105549 - FSYNC - Syncronization of FSYNC timebase to GPTP timebase when FSYNC is running

When the application software determins with the help of gptp that the gptp timebase is running out of sync to the GVT Master the application software shall also adjust the FSync timebase by creating a jump in the FSYNC Timebase.

Status	Draft
Verification Criteria	start gptp and let gptp sync to fsync, then manipulate gptp to simualte a timejump, record the generated pulses with Oscilloscope, validate there is a time shift

MVC-105591 - FSYNC - Neccesary timebase drift to syncronization between FSYNC Timebase and GPTP Timebase

The application software shall adjust the FSYNC Timebase only if either

- the absolute time delta between absolute Time from GPTP and Fsync is bigger than **1us**
- the absolute PPB delta between GPTP MAC and FSYNC MAC is bigger than **100000**

Note: There should be a significant drift between these timebases otherwise a lot of frame sync pulses are wastet to realign the Sensor to Gptp, when they could be used to realign the sensor to the traget frameSyncRefTime

Status	Draft
Verification Criteria	manipulate gpt server to jump in 1us and clock speed and check if pulse raster jump, if it s not jumping test is passed

MVC-106548 - FSYNC - Calculate raster aligned time jump

The application shall calculate the initial timejump to be raster aligned:

$$rasterShift_{rasterAligned} = \Delta T_{gPTP} \% \frac{1}{frameRate}$$

Status	Draft
Verification Criteria	the raster aligned timestamp can not be measured by analysing fsync rigger pulses

MVC-106553 - FSYNC - Apply the raster aligned time stamp

The application software shall apple the rasterShiftRasterAligned by shifting the fsync timebase directly after a gptp sync request was handeld when calculating a new fsync Pulse Timestamp.

Status	Draft
Verification Criteria	

MVC-105592 - FSYNC - Limit the time the timbase can jump with pulseraster

The application software shall limit the jump between two fsync pulses to realign for gptp to 1.5ms. Therefore it can multiple jumps in the timebase until FSYNC is realigned to GPTP.

Note: The time jump is limited to the blanking time, otherwise the sensor will get out of sync.

Status	Draft
Verification Criteria	analyse the fsync trigger impulses wenn realigning to gptp with Oscilloscope, search for jump in timebase this jump must not be bigger than specified

MVC-105599 - FSYNC - Limit how often the timebase jump can occure

The application shall check if 5 frames after a timebase shift has occured could be received bevor a new timbase jump shall occure.

Note: From image sensor Datasheet we know the sensor needs up to 4 frames to readjust the internal frmaesync therefore the 5th frmae is the first stable frame. Otherwise there is possibility the sensor will loose sync if the timbase will jump to often.

Status	Draft
Verification Criteria	analyse the fsync trigger impulses wenn realigning to gptp with Oscilloscope, search how often jump will ocure

10.5 Ethernet Control

MVC-102673 - FSync - Ethernet Control - getDataBlock

When the application software receives a **getDataBlock** Service Method with **deviceId** = 0x0001 and **blockId** = 0x0000, it shall return the following values for the payload:

- blockSize = **0x00000008**
- blockData = currently set **franeSyncTimeRef** as absolute timestamp in `ns`

Note: The default value for blockData shall be **0x00000000 00000000**, but it might never be returned

Status	Ready for review
Verification Criteria	Set specific frameSyncTimeRef with SOME/IP setDataBlock Messsage, send SOME/IP getDataBlock Message and check if the correct values are returned

MVC-102674 - FSync - Ethernet Control - setDataBlock

When the application software receives a **setDataBlock** Service Method with **deviceId** = 0x0001, **blockId** = 0x0000 and **blockSize** = 0x00000008, it shall reconfigure the FSync timing frameSyncTimeRef applied to the sensor to the new rasterization of time using the value in **blockData**.

Status	Ready for review
Verification Criteria	Set specific frameSyncTimeRef with SOME/IP setDataBlock Messsage, send SOME/IP getDataBlock Message and check if the expected values are returned

MVC-105613 - FSYNC - Ethernet Control - setDataBlock acceptable values

When the application software receives a setDataBlock Service as defined in MVC-102674 - FSync - Ethernet Control - setDataBlock it shall check if the new Value is bigger than the old value.

If the new value is bigger than the old value it shall return no error.
If the new value is smaller than the old value ist shall return a error.

Note: The new Value must be always bigger otherwise there is a problem with time continuity which can case unforseable problems.

Status	Draft
Verification Criteria	Set a smaller frameSyncTimeRef than prevously set with SOME/IP setDataBlock Messsage, send SOME/IP getDataBlock Message and check if the function returns notok

MVC-105615 - FSYNC - Default frameTimeSyncRef

The application software shall use a default frameSyncRefTime of 0.0306s.

Note: The default frameSyncRefTime was chossen to be identical to the mean measured time from sending out a trigger pulse until the complete frame was received. Therefore by setting this delay the FSync trigger pulse must be aligned to the PPS out of GPTP at every full second. This allows to check if fsync is syncronized.

Status	Draft
Verification Criteria	Get frameSyncTimeRef with SOME/IP getDataBlock Messsage to get default value at startup

MVC-105616 - FSYNC - Calculate the rastershift in between raster from frameSyncTimeRef

The application software shall calculate the target raster shift from the frameTimeSyncRef by the follwoing calculation:

$$rasterShift_{inBetweenRaster} = (frameTimeSyncRef \% (\frac{1}{frameRate})) \cdot \frac{1}{frameRate}$$

Status	Draft
Verification Criteria	review formula to calculate rasterShiftInBetweenRaster

MVC-106150 - FSYNC - Use new frameRimeSyncRef when gptp is running

The application software shall only use the new frameTimeSyncRef value set by MVC-105613 - FSYNC - Ethernet Control - setDataBlock acceptable values , if fsync is syncronized to gptp and gptp is working.

Status	Draft
Verification Criteria	send new frameSyncTimeref with setDataBlock when gptp i s not running. Observe pulses generated by fsync for jumps, when no jump passed

MVC-106149 - FSYNC - Use new frameRimeSyncRef when gptp is not running

The application software shall use the already used frameTImeSyncRef value set by MVC-105613 - FSYNC - Ethernet Control - setDataBlock acceptable values , if fsync is not syncronized to gptp or gpt is not running.

Status	Draft
Verification Criteria	send new frameSyncTimeref with setDataBlock when gptp is running. Observe pulses generated by fsync for jumps, when jump observed passed

MVC-106158 - FSYNC - Limit time jump when align to new frameSyncTimeRef

The application software shall limit the time jump between two frames, when ist realigning to a new frameSynTimeRef to 1ms.

Status	Draft
Verification Criteria	send new frameSyncTimeref with setDataBlock when gptp is running. Observe pulses generated by fsync for jumps, measure the jump if below max passed

10.6 FSYNC general behavior

MVC-105625 - FSYNC - Calculate timestamp to trigger Sensor

The application software shall calculate the absolute timestamp neccesary to account for the system delay to receive the image frame at the exact moment specified by MVC-105616 - FSYNC - Calculate the rastershift in between raster from frameSyncTimeRef .

Status	Draft
Verification Criteria	Analyse code, the delay is only internaly used

MVC-106156 - FSYNC - Set calculated timestamp to trigger Sensor

The application software shall set the calculated timestamp (MVC-105625 - FSYNC - Calculate timestamp to trigger Sensor) to the secondary mac to generate an accure timedriven pulse.

Status	Draft
Verification Criteria	

MVC-106155 - FSYNC - Generate a pulse raster by multiple pulses

The application software shall create multiple pulses each exactly spaced apart as one frame interval, when there is no gptp realignment or change in frameSyncTimeRef.

$$\Delta T_{frameInterval} = \frac{1}{frameRate}$$

Status	Draft
Verification Criteria	observe frame pulse generation, gptp off, validate that time between pulses is constant

MVC-106157 - FSYNC - Directly set timestamp to mac after previous ts was triggered

The software shall calculate a new Timestamp to generate a pulse, when the secondray mac generated the pulse.

Status	Draft
Verification Criteria	check code

MVC-105626 - FSYNC - Measure the system delay of FSYNC

The application software shall measure the time frome from sending out the pulse until a complete frame is received in rawbuffer named **frameSyncSytemDelay**

Status	Draft
Verification Criteria	measure delay between from fsync pulse to end of mipi data transmission of frame with Oscilloscope and compare against measured value(not shure how to get the claue currently)

MVC-106159 - FSYNC - Compensate for drift in systemDelay

The application software shall calculate the average systemDelay of the last 10 frames and use this to compensate possible drifts in systemDelay.

Note: This is currently not implemented and needs to be discussed, if it is neccessary

Status	Draft
Verification Criteria	

MVC-106160 - FSYNC - Calculate average systemdelay only with aligned recived Frame systemDelay

The application software shall not use the system delay of a frame to calculate the average systemDelay, when the raster was realigned to gptp or frameSyncTimeref

Note: This is currently not implemented and needs to be discussed, if it is neccessary

Status	Draft
Verification Criteria	

11 Frame Tagging

11.1 SEI Message in Video Stream

MVC-105552 - SEI Messages for Streams

The uC SW shall send metadata via SEI NAL unit (NALU) to each video frame data for both HV and CV streams.

Within each frame RTP packet, SEI NALU shall be sent after SPS and PPS NALU and before the actual video data.

Status	Draft
Verification Criteria	Check if SEI NALU is being sent for each HV and CV frame, and if SEI NALU is sent after SPS and PPS NALU.

MVC-105610 - SEI Messages Codec

The uC SW shall encode each SEI NALU with H.264 Codec acc. to ITU-T H.264 (V15) (08/2024): Advanced video coding for generic audiovisual services

In particular, the emulation_prevention_three_byte (§7.4.1.1 Encapsulation of an SODB within an RBSP) has to be implemented in SEI NALU.

Status	Draft
Verification Criteria	Check if the transmitted SEI message can be correctly parsed with each data fields being recognized by H.264 decoder (e.g. ffmpeg) at receiver side.

MVC-107762 - SEI Messages: Insersion of emulation_prevention_three_byte

Acc. to ITU-T H.264 (V15) (08/2024): Advanced video coding for generic audiovisual services, the uC SW shall produce the SEI NAL unit with emulation_prevention_three_byte by the following procedure:

1. Copy the NALU header byte (forbidden_zero_bit + nal_ref_idc + nal_unit_type), see 11.1-3 - SEI Messages Format: NALU Syntax
2. search the RBSP data (Raw Bytes Sesquence Payload), byte by byte, for the following bits sequence: `0b00000000 00000000 000000xx` (where xx = 00, 01, 10 or 11).
3. If the byte-aligned sequence in step 2 occurred, a byte 0x03 shall be inserted to replaced this sequence with pattern: `0b00000000 00000000 00000011 000000xx`

Note: since the cabac_zero_word = 0x0000 will not be applied at the end of RBSP, the emulation_prevention_three_byte = 0x03 shall not be appended at the end of RBSP.

Status	Draft
Verification Criteria	Check if emulation_prevention_three_byte is correctly inserted in the raw SEI NALU message, and if all data fields can be correctly parsed by H.264 decoder.

MVC-105618 - SEI Messages Format: NALU Syntax

The software shall construct SEI NALU in following format:

forbidden_zero_bit (1bit)	nal_ref_idc (2bit)	nal_unit_type (5bit)	SEI Raw Bytes Sesquence Payload (RBSP) (variable size)
0b0	0b00	0b00110 for Supplemental enhancement information (SEI) (see Rec. ITU-T H.264 (V15) (08/2024) Table 7-1)	rbsp_bytes (see MVC-105617 - SEI Message Format: RBSP Syntax)

Status	Draft
Verification Criteria	check if all fields of the received SEI NALU can be recognized and decoded, and if the NALU header yields correct values (forbidden_zero_bit = 0, nal_ref_idc = 0, nal_unit_type = 0x6)

MVC-105617 - SEI Message Format: RBSP Syntax

The uC SW shall construct the SEI raw bytes sequence payload (RBSP) with following contents:

payloadType (1byte)	payloadSize (1byte if payload less then 255 bytes)	uuid_iso_iec_11578 (16 bytes)	user_data_payload_bytes (variable size)	rbsp_trailing_bits (1byte)
0b00101 for user_data_unregistred (see Rec. ITU-T H.264 (V15) (08/2024) Table 7-1)	size of payload (uuid_iso_iec_11578 + user_data_payload_bytes	see MVC-105620 - SEI Messages Payload: UUID	this are the actual user/meta data to provide to vehicle: see MVC-105621 - User Data List in Metadata	shall be equal to 0x80, which consists of: • rbsp_stop_one_bit =0b1 • rbsp_trailing_bits = 0b000 0000

Status	Draft
Verification Criteria	check if all fields of the received SEI RBSP can be recognized and decoded, and if the SEI header yields correct values (payloadType = 0x5, payloadSize = 16 + user_data_payload_bytes), and if rbsp_trailing_bits = 0b1000 0000

MVC-105620 - SEI Messages Payload: UUID

The uC SW shall construct UUID field of the SEI RBSP with following contents:

Byte Range	Bit Range	size in Bytes	Field	Value (Hex)	Notes
0-3	0-31	4	unix timestamp of UUID creation	0x67C8C580	2025 03 01 12:00:00 UTC
4-6	32-55	3	Random Entropy	017f2a	randomly choosen
7	56-63	1	Variant (RFC4122)	0xa1	Contains variant bits (10xxxxxx)
8-9	64-75	2	Mekra UUID Version	0x001	Version 1
	76-79		Version of UUID	0x8	Version (8) → binary 1000 to to comply with Version 8
10-11	80-95	2	Random entropy	0xd911	randomly choosen

Byte Range	Bit Range	size in Bytes	Field	Value (Hex)	Notes
12-15	96-127	4	Application ID	0x534F5732	SOW2 in ASCII

For each field, the most significant bit shall be transmitted first.

Status	Draft
Verification Criteria	check if the received SEI message yields the correct values in all UUID fields.

11.2 User Data in SEI Payload

MVC-105621 - User Data List in Metadata

The uC SW shall send following data in the user_data_payload_byte field of the SEI RBSP:

Data Name	size [in bytes]	used format	description
CaptureTimestamp	8	uint64_t	see MVC-105630 - User Data: CaptureTimestamp
FrameID	4	uint32_t	see MVC-105629 - User Data: FrameID
Frame exposure time LPD	2	uint16_t	see MVC-105631 - User Data: Frame exposure time LPD
Frame gain Long LCG	4	uint32_t	see MVC-105633 - User Data: Frame gain Long LCG
Scene illuminance	8	float	see MVC-105632 - User Data: Scene illuminance
Error bitmask	1	uint8_t	see MVC-105641 - User Data: Error bitmask
Cam_ID	1	uint8_t	see MVC-105640 - User Data: Cam ID
CRC-16	2	uint16_t	see MVC-105643 - User Data: CRC-16
Frame exposure time SPD	2	uint16_t	see MVC-105635 - User Data: Frame exposure time SPD
Frame gain Long HCG	2	uint16_t	see MVC-105634 - User Data: Frame gain Long HCG
Frame gain Middle	2	uint16_t	see MVC-105637 - User Data: Frame gain Middle
Frame gain Short	2	uint16_t	see MVC-105636 - User Data: Frame gain Short
PreviewDataReadyTimestamp	8	uint64_t	see MVC-105642 - User Data: PreviewDataReadyTimestamp
EncodeReadyTimestamp	8	uint64_t	see MVC-105639 - User Data: EncodeReadyTimestamp
EncoderFrameNumber	4	uint8_t	see MVC-105638 - User Data: EncoderFrameNumber
Total Size	58		

For each data, the most significant bit has to be sent first.

Status	Draft
Verification Criteria	check if the user_data_payload_byte field in the SEI NALU yields the correct length, and if each field is sent with big endianess (MSB first)

MVC-105630 - User Data: CaptureTimestamp

The uC SW shall send user data CaptureTimestamp in byte[0:7] of SEI user_data_payload_bytes field:

- The value shall be equal to the timestamp relative to MAC0 system time, when a full frame is received from the sensor and stored in raw buffer.
- The value shall be represented in nanoseconds with datatype uint64.

Note: MAC0 system time shall be synchronized to gPTP grandmaster time (see MVC-105532 - Update Slave Clock Time)

TODO: how to test the correctness of the CaptureTime by software test?

Status	Draft
Verification Criteria	Check if the sensor raw capture time sent at the dedicated position of SEI user_data_payload_bytes field yields plausible value (e.g. the time diff between two adjacent frames shall equal to frame period)

MVC-105629 - User Data: FrameID

The uC SW shall send user data FrameID in byte[8:11] of SEI user_data_payload_bytes field:

- The value shall be saved with datatype uint32
- The value shall be equal to the variable frame_cnt, which can be obtained from following sensor register or sensor embedded header location:

Variable name	Register/bits	Register Addr.	Byte Offset in Embedded Header	Bits in Embedded Header
frame_cnt	tgr_frame_cnt[8:15]	0x3000	56	[0:7]
	tgr_frame_cnt[0:7]	0x3000	57	[0:7]

Status	Draft
Verification Criteria	Check if FrameID at the dedicated position of SEI user_data_payload_bytes yields plausible value (the diff between adjacent frames shall be equal to 1).

MVC-105631 - User Data: Frame exposure time LPD

The uC SW shall send user data Frame exposure time LPD in byte[12:13] of SEI user_data_payload_bytes field:

- The value shall be saved with datatype uint16 and represented as µs
- The value shall be calculated by following formula:

$$\text{exposure_time_LPD} = \frac{\text{coarse_integration_time_long} \cdot \text{line_length_pck} + \text{fine_integration_time_long}}{\text{sysclk}}$$

$$\text{sysclk} = \text{MCLK} \times 2^{-(\text{pll_sys_s}+\text{pll_sys_div_pcs})} \times \frac{\text{pll_sys_m} + \text{pll_sys_k} \cdot 2^{-16}}{\text{pll_sys_p} \cdot \text{pll_sys_div_pre_adc} \cdot \text{pll_sys_div_dbs}}$$

$$\text{MCLK} = 25\text{MHz}$$

- The variables in the formula can be obtained from the following positions in sensor embedded header:

Variable name	Register/bits	Register Addr.	Byte Offset in Embedded Header	Bits in Embedded Header
coarse_integration_time_long	tgr_coarse_integration_time[8:15]	0x0202	86	[0:7]
	tgr_coarse_integration_time[0:7]	0x0202	87	[0:7]
fine_integration_time_long	tgr_fine_integration_time[8:15]	0x0200	92	[0:7]
	tgr_fine_integration_time[0:7]	0x0200	93	[0:7]
line_length_pck	tgr_line_length_pck[8:15]	0x0342	74	[0:7]
	tgr_line_length_pck[0:7]	0x0342	75	[0:7]
pll_sys_m	sys_pll_sys_m[8:9]	0x101E	42	[0:1]
	sys_pll_sys_m[0:7]	0x101E	43	[0:7]

Variable name	Register/bits	Register Addr.	Byte Offset in Embedded Header	Bits in Embedded Header
pll_sys_k	sys_pll_sys_k[8:15]	0x1022	48	[0:7]
	sys_pll_sys_k[0:7]	0x1022	49	[0:7]
pll_sys_p	sys_pll_sys_p[0:5]	0x101C	44	[0:5]
pll_sys_div_pre_adc	sys_pll_sys_div[0:3]	0x1026	47	[0:3]
pll_sys_div_pcs	sys_pll_sys_div[9:8]	0x1026	47	[4:5]
pll_sys_div_dbs	sys_pll_sys_div2[4:7]	0x1028	46	[4:7]
pll_sys_s	sys_pll_sys_s[0:2]	0x1020	45	[0:2]

Status	Draft
Verification Criteria	check if Frame exposure time LPD sent at the dedicated position of SEI user_data_payload_bytes yield the correct value acc. to calculation. (illuminate the camera with constant light flux, check if the value in SEI matches the value calculated from the input registers which can be read via I2C)

MVC-105633 - User Data: Frame gain Long LCG
The uC SW shall send user data Frame gain long LCG in byte[14:17] of SEI user_data_payload_bytes field:

- The value shall be saved with datatype uint32.
- The value shall be equal to the variable digital_gain_long_lcg, which is stored at the following register or the byte offset of sensor embedded header.

Variable name	Register/bits	Register Addr.	Byte Offset in Embedded Header	Bits in Embedded Header
digital_gain_long_lcg	tgr_digital_gain_long_lcg[8:15]	0x0234	194	[0:7]
	tgr_digital_gain_long_lcg[0:7]	0x0234	195	[0:7]

Status	Draft
Verification Criteria	check if Frame gain long LCG is sent at the dedicated position of SEI user_data_payload_bytes yield the correct value. (illuminate the camera with constant light flux, check if the value in SEI matches the value read from register via I2C)

MVC-105632 - User Data: Scene illuminance
The uC SW shall send user data Scene illuminance in byte[18:25] of SEI user_data_payload_bytes field:

- The value shall be saved with datatype float64 (double)
- The value shall be estimated by following pseudo code:

```
numTiles = TileNumCol * TileNumRow;
numPixelsTile = TileHeight * TileWidth;
totalNumPixels = numTiles * numPixelsTile;

totalY = 0;
for (tileIdx = 0; tileIdx < numTiles; tileIdx++)
{
    /* get the sum of Y in the tile. */
    yTile = PseudoY[tileIdx] << YShift;
    /* The outliers shall be added with the values clamped by thresholds */
    yTile += CountMin[tileIdx] << MinMaxShift * MinThresh;
    yTile += CountMax[tileIdx] << MinMaxShift * MaxThresh;

    totalY += yTile;
}
exposureTime_in_S = exposure_time_LPD / 1000000;
gain_normalized = digital_gain_long_lcg / 256

SceneIlluminance = totalY / (totalNumPixels * exposureTime_in_S * gain_normalized);
```

The variables are illustrated below:

Variable	Description
Input variables:	
`TileNumCol`	number of tiles columns within sensor capture ROI, may be extracted from Ambarella CFA AAA statistics event data.
`TileNumRow`	number of tiles rows within sensor capture ROI, may be extracted from Ambarella CFA AAA statistics event data
`TileHeight`	pixel height within a tile, may be extracted from Ambarella CFA AAA statistics event data
`TileWidth`	pixel width within a tile, may be extracted from Ambarella CFA AAA statistics event data
`PseudoY[tileIdx]`	Pseudo Luma (Y) as the summation of all pixel values within the tile of index `tileIdx`, averaged by the 4 bayer pattern, then right shifted by `YShift`. PseudoY = ((R + Gr + Gb + B) / 4) >> YShift. Note: pixels which yield values beyond the threshold `MinThresh, MaxThresh` are excluded from summation. Data may be extracted from Ambarella CFA AAA statistics event data.
`CountMin[tileIdx]`	Number of pixels in the tile of index `tileIdx` whose Pseudo Luma is smaller than `MinThresh`. right shifted by `MinMaxShift`. Data may be extracted from Ambarella CFA AAA statistics event data.
`CountMax[tileIdx]`	Number of pixelsin the tile of index `tileIdx` whose Pseudo Luma is larger than `MaxThresh`. right shifted by `MinMaxShift`. Data may be extracted from Ambarella CFA AAA statistics event data
`MinThresh`	Minimum threshold of Pseudo Luma. i.e. if a pixel value is smaller than this threshold, it will not be added to `PseudoY`. Data may be extracted from Ambarella CFA AAA statistics event data
`MaxThresh`	Maximum threshold of Pseudo Luma. i.e. if a pixel value is larger than this threshold, it will not be added to `PseudoY`. Data may be extracted from Ambarella CFA AAA statistics event data
`YShift`	The amount of right shift bits for `PseudoY[tileIdx]`. Data may be extracted from Ambarella CFA AAA statistics event data
`MinMaxShift`	The amount of right shift bits for `CountMin[tileIdx]` and `CountMax[tileIdx]`. Data may be extracted from Ambarella CFA AAA statistics event data.
`exposure_time_LPD`	exposure time LPD calculated by MVC-105631 - User Data: Frame exposure time LPD
`digital_gain_long_lcg`	digital gain long LCG calculated by MVC-105633 - User Data: Frame gain Long LCG
Intermediate variables:	
`yTile`	Total Pseudo Luma (Y) in a tile, as summation of `PseudoY[tileIdx]`, `CountMin[tileIdx]` and `CountMax[tileIdx]`. without shifting `YShift` and `Min/MaxShift`.
`totalY`	Total Pseudo Luma (Y) over all tiles.
output variables:	
`SceneIlluminance`	The estimated Scene illuminance, which is currently not normalized to physical lumen (TODO: the value shall be in future adjusted by Lookup Table between pseudo luma and physical luma)

Status	Draft
Verification Criteria	check if the scene illuminance sent at the dedicated position of SEI user_data_payload_bytes yields plausible value (the value shall decrease at darker scene captured by camera)

MVC-105641 - User Data: Error bitmask
The uC SW shall send user data error bitmask in byte[26] of SEI user_data_payload_bytes field:
TODO: the source of this data is still unknown.

Status	Draft
Verification Criteria	

MVC-105640 - User Data: Cam ID
The uC SW shall send user data Cam ID in byte[27] of SEI user_data_payload_bytes field:
TODO: the source of this data is still unknown.

Status	Draft
Verification Criteria	

MVC-105643 - User Data: CRC-16
The uC SW shall send user data CRC-16 in byte[28:29] of SEI user_data_payload_bytes field:
TODO: the source and algorithm for this data is still unknown.

Status	Draft
Verification Criteria	

MVC-105635 - User Data: Frame exposure time SPD
The uC SW shall send user data Frame exposure time SPD in byte[30:31] of SEI user_data_payload_bytes field:

- The value shall be saved with datatype uint16 and represented as µs
- The value shall be calculated by following formula:

$$\text{exposure_time_SPD} = \frac{\text{coarse_integration_time_middle} \cdot \text{line_length_pck} + \text{fine_integration_time_middle}}{\text{sysclk}}$$

$$\text{sysclk} = \text{MCLK} \times 2^{-(\text{pll_sys_s}+\text{pll_sys_div_pcs})} \times \frac{\text{pll_sys_m} + \text{pll_sys_k} \cdot 2^{-16}}{\text{pll_sys_p} \cdot \text{pll_sys_div_pre_adc} \cdot \text{pll_sys_div_dbs}}$$

$$\text{MCLK} = 25\text{MHz}$$

- The variables in the formula can be obtained from the following positions in sensor embedded header:

Variable name	Register/bits	Register Addr.	Byte Offset in Embedded Header	Bits in Embedded Header
coarse_integration_time_middle	tgr_coarse_integration_time_spd[8:15]	0x0216	88	[0:7]
	tgr_coarse_integration_time_spd[0:7]	0x0216	89	[0:7]
fine_integration_time_middle	tgr_fine_integration_time_spd[8:15]	0x0300	94	[0:7]
	tgr_fine_integration_time_spd[0:7]	0x0300	95	[0:7]
line_length_pck	tgr_line_length_pck[8:15]	0x0342	74	[0:7]
	tgr_line_length_pck[0:7]	0x0342	75	[0:7]
pll_sys_m	sys_pll_sys_m[8:9]	0x101E	42	[0:1]
	sys_pll_sys_m[0:7]	0x101E	43	[0:7]
pll_sys_k	sys_pll_sys_k[8:15]	0x1022	48	[0:7]
	sys_pll_sys_k[0:7]	0x1022	49	[0:7]
pll_sys_p	sys_pll_sys_p[0:5]	0x101C	44	[0:5]
pll_sys_div_pre_adc	sys_pll_sys_div[0:3]	0x1026	47	[0:3]
pll_sys_div_pcs	sys_pll_sys_div[9:8]	0x1026	47	[4:5]
pll_sys_div_dbs	sys_pll_sys_div2[4:7]	0x1028	46	[4:7]
pll_sys_s	sys_pll_sys_s[0:2]	0x1020	45	[0:2]

Status	Draft
Verification Criteria	check if Frame exposure time SPD sent at the dedicated position of SEI user_data_payload_bytes yield the correct value acc. to calculation. (illuminate the camera with constant light flux, check if the value in SEI matches the value calculated from the input registers which can be read via I2C)

MVC-105634 - User Data: Frame gain Long HCG
The uC SW shall send user data Frame gain long HCG in byte[32:33] of SEI user_data_payload_bytes field:

- The value shall be saved with datatype uint16.
- The value shall be equal to the variable digital_gain_long_hcg, which is stored at the following register or the byte offset of sensor embedded header.

Variable name	Register/bits	Register Addr.	Byte Offset in Embedded Header	Bits in Embedded Header
digital_gain_long_hcg	tgr_digital_gain_long_hcg[8:15]	0x0232	192	[0:7]
	tgr_digital_gain_long_hcg[0:7]	0x0232	193	[0:7]

Status	Draft
Verification Criteria	check if Frame gain long HCG is sent at the dedicated position of SEI user_data_payload_bytes yield the correct value. (illuminate the camera with constant light flux, check if the value in SEI matches the value read from register via I2C)

MVC-105637 - User Data: Frame gain Middle
The uC SW shall send user data Frame gain Middle in byte[34:35] of SEI user_data_payload_bytes field:

- The value shall be saved with datatype uint16.
- The value shall be equal to the variable digital_gain_middle, which is stored at the following register or the byte offset of sensor embedded header.

Variable name	Register/bits	Register Addr.	Byte Offset in Embedded Header	Bits in Embedded Header
digital_gain_middle	tgr_digital_gain_middle[8:15]	0x0236	196	[0:7]
	tgr_digital_gain_middle[0:7]	0x0236	197	[0:7]

Status	Draft
Verification Criteria	check if Frame gain middle sent at the dedicated position of SEI user_data_payload_bytes yield the correct value. (illuminate the camera with constant light flux, check if the value in SEI matches the value read from register via I2C)

MVC-105636 - User Data: Frame gain Short
The uC SW shall send user data Frame gain long HCG in byte[36:37] of SEI user_data_payload_bytes field:

- The value shall be saved with datatype uint16.
- The value shall be equal to the variable digital_gain_short, which is stored at the following register or the byte offset of sensor embedded header.

Variable name	Register/bits	Register Addr.	Byte Offset in Embedded Header	Bits in Embedded Header
digital_gain_short	tgr_digital_gain_short[8:15]	0x0238	198	[0:7]
	tgr_digital_gain_short[0:7]	0x0238	199	[0:7]

Status	Draft
Verification Criteria	check if Frame gain short sent at the dedicated position of SEI user_data_payload_bytes yield the correct value. (illuminate the camera with constant light flux, check if the value in SEI matches the value read from register via I2C)

MVC-105642 - User Data: PreviewDataReadyTimestamp
The uC SW shall send user data PreviewDataReadyTimestamp in byte[38:45] of SEI user_data_payload_bytes field:

- The value shall be equal to the timestamp relative to MAC0 system time, when ISP processing for a full frame has finished.
- The value shall be represented in nanoseconds with datatype uint64.

Note: MAC0 system time shall be synchronized to gPTP grandmaster time (see MVC-105532 - Update Slave Clock Time)
TODO: how to test the correctness of the PreviewDataReadyTimestamp by software test?

Status	Draft
Verification Criteria	Check if PreviewDataReadyTimestamp sent at the dedicated position of SEI user_data_payload_bytes field yields plausible value (e.g. the time diff between two adjacent frames shall equal to frame period)

MVC-105639 - User Data: EncodeReadyTimestamp
The uC SW shall send user data EncodeReadyTimestamp in byte[46:53] of SEI user_data_payload_bytes field:

- The value shall be equal to the timestamp relative to MAC0 system time, when the encoding for a full frame has finished.
- The value shall be represented in nanoseconds with datatype uint64.

Note: MAC0 system time shall be synchronized to gPTP grandmaster time (see MVC-105532 - Update Slave Clock Time)
TODO: how to test the correctness of the EncodeReadyTimestamp by software test?

Status	Draft
Verification Criteria	Check if EncodeReadyTimestamp sent at the dedicated position of SEI user_data_payload_bytes field yields plausible value (e.g. the time diff between two adjacent frames shall equal to frame period)

MVC-105638 - User Data: EncoderFrameNumber
The uC SW shall send user data EncoderFrameNumber in byte[54:57] of SEI user_data_payload_bytes field:

- The value shall be equal to the frame number returned by the event data for Amba event AMBA_DSP_EVENT_VIDEO_DATA_RDY.
- The value shall be represented with datatype uint32.

Status	Draft
Verification Criteria	Check if EncoderFrameNumber sent at the dedicated position of SEI user_data_payload_bytes yields plausible value (shall increment by 1 for each encoder frame)

12 External SPI Flash Memory

12.1 Configuration of SPI-NOR controller driver

MVC-92789 - SPI flash interface via SPI-NOR controller driver
The application software shall configure the SPI-NOR controller driver ensuring proper interfacing with the external SPI flash with the following parameters:

Parameter	Value
DevName	ISSI IS25WP01G_128MB
ManufactuerID	0x9D
DeviceID	0x1A
TotalByteSize	128 * 1024 * 1024
PageSize	256
EraseBlockSize	64 * 1024
EraseSectorSize	4 * 1024
SpiFrequncy	104000000
AddrByte	4 (= AMBA_NORSPI_4BYTE_ADDR)
LSBFirst	0
Read.CmdIndex	0x6B
Read.DummyCycles	8
Read.CmdType	3 (= AMBA_NORSPI_QOR_READ)
Read.DataLane	2 (= AMBA_NORSPI_DATA_4_LANE)
Quad.CmdIndex	0x32
Quad.DummyCycles	0x0
Quad.CmdType	4 (= AMBA_NORSPI_QOR_WRITE)
Quad.DataLane	2 (= AMBA_NORSPI_DATA_4_LANE)
ResetEnable	0x66
Reset	0x99
WriteEnable	0x6
WriteDisable	0x4
WriteReg	0x1
ReadID	0x9f
ReadStatus0	0x5
ReadStatus1	0x81
ReadFlagStatus	0x81
ClearStatus	0x8F
EraseBlk	0xD8
EraseSector	0x20
EraseChip	0
Enable4ByteMode	0xB7
EraseFailFlag	0x08
ProgramFailFlag	0x04
Quad.EnableQuadMode	0x01
Quad.ReadQuadMode	0x05
Quad.ExitQuadMode	0x01
Quad.QuadFlag	0x40
Quad.ExitQuadFlag	0x00
Quad.QuadCmdArgLen	0x1
FlowControl	0x1
HoldTime	0x0
ChipSelect	0x0
RxSampleDelay	0x4
TxThresholdLevel	0x1f
RxThresholdLevel	0x1f

Status	Ready for review
Verification Criteria	Refer to integration test

MVC-98157 - SPI-NOR driver configuration
When entering software state INIT the application software shall configure the SPI-NOR controller driver

Status	Ready for review
Verification Criteria	Refer to integration test

13 NVM

13.1 NVM - Production data block

MVC-95865 - NVM production data block

When entering the state PERIPHERAL_CONFIGURATION the application software shall read the production data block from NVM.

See MVC-94698 - NVM Production Data Block Memory Map

Note: The block from `0x03C2 003A` to `0x03C2 FFFF` is reserved for production data and may be partially read as well.

Status	Ready for review
Verification Criteria	Check SPI trace the read command of production data block

13.2 NVM - Software configuration block

MVC-95861 - NVM software configuration block

When entering the state PERIPHERAL_CONFIGURATION the application software shall read the software configuration data block from NVM.

See MVC-94611 - NVM Software Configuration Data Block Memory Map

Note: The block from `0x03C3 012C` to `0x03C3 FFFF` is currently reserved but not yet defined and therefore omitted. It may still be partly read due to constraints in allowed read ranges.

Status	Ready for review
Verification Criteria	Check SPI trace whether software configuration block is read

13.3 NVM - ImVer

MVC-113469 - NVM ImVer

When entering the state PERIPHERAL_CONFIGURATION the application software shall read the `imVer` from NVM.

Status	Ready for review
Verification Criteria	Cannot be tested on its own, test together with MVC-113471 - Readout ImVer

MVC-113471 - Readout ImVer

The application software shall read out `imVer` from SYS_DATA partition (rom.bin).
The SYS_DATA partition is a read-only filesystem readable using AmbaNVM Ambarella software interface which contains several files.
The `imVer` is located in file iq_version.txt (ASCII-coded) as third entry split by newline character.

Status	Ready for review
Verification Criteria	AmbaNVM interface not testable. Therefore: Create a custom rom.bin file and read out imVer via getDataSheet method.

14 State Handling

MVC-92814 - State Handling - Enter INIT

When the reset vector is triggered the Application Software shall switch to software state `INIT` and and perform the following sequence:

1. Apply the general I/O config
2. SPI-NOR configuration
3. PMIC Configuration (MVC-96166 - PMIC Configuration)

Note:
The application of this I/O config is done through the line `AmbaGPIO_LoadDefaultRegVals(&GpioPinGrpConfig)`
Currently this is covered by general IO Config (MVC-94055) referring to System Allocation Requirements HSI (see "used GPIO Settings at startup").

Status	Ready for review
Verification Criteria	After boot, ensure that correct order is implemented in code

MVC-96111 - State Handling - Transition INIT to PERIPHERAL_CONFIGURATION

While in software state `INIT`, when the sequence in `INIT` is finished, the Application Software shall switch to software state `PERIPHERAL_CONFIGURATION` and perform the following sequence:

1. NVM readout
   a. MVC-95865 - NVM production data block
   b. MVC-95861 - NVM software configuration block
   c. MVC-113469 - NVM ImVer
2. Camera Interface Setup
   a. MVC-96065 - Camera Interface - Setup Sequence
3. ADC Configuration
   a. MVC-100206 - SerDes - Configuration - ADC Configuration
4. Ethernet Configuration
   a. MVC-96408 - PHY RGMII-ID (RX) Configuration
   b. MVC-96407 - PHY TC10 Enable Configuration
   c. MVC-96409 - PHY Identification and Link Speed Configuration

Status	Ready for review
Verification Criteria	After last action in INIT, observe correct order of sequence before first action in PHYLINKCHECK

MVC-96162 - State Handling - Transition PERIPHERAL_CONFIGURATION to PHYLINKCHECK

While in software state `PERIPHERAL_CONFIGURATION`, when the sequence in `PERIPHERAL_CONFIGURATION` is finished, the Application Software shall switch to software state `PHYLINKCHECK` and start monitoring the phy link according to MVC-96189 - Ethernet - Monitoring of PHY Link Status .

Status	Ready for review
Verification Criteria	After last action in PERIPHERAL_CONFIGURATION, observe correct start of monitoring of PHY Link Status (MVC-96189).

MVC-96163 - State Handling - Transition PHYLINKCHECK to IPCONFIG

While in software state `PHYLINKCHECK`, when the Application Software verfies the presence of a physical link according to MVC-96189 , the Application Software shall switch to software state `IPCONFIG` and perform the specified DHCP functionality according to MVC-96167 - DHCP .

Status	Ready for review
Verification Criteria	Trace MDIO traffic, (Consider test for MVC-96189) when link is up we should send a DHCP request

MVC-96172 - State Handling - Transition IPCONFIG to OPERATIONAL

While in software state `IPCONFIG`, when the Application Software sets the IP address acknowledged by the DHCP server, the Application Software shall switch to software state `OPERATIONAL` and perform the following steps:

1. Start Heating
   a. MVC-92885 - Heating control - Startup
2. Start Shutter
   a. MVC-92034 - Shutter control - Startup functionality
3. Start SOME/IP Camera Service
   a. MVC-95858 - Camera Service Startup

4. Start SOME/IP Service Discovery
    a. MVC-95855 - Service Discovery Startup

Status	Ready for review
Verification Criteria	Indirekt Test: When reveiving DHCP ACK, check the sequence is executed as specified

MVC-98197 - State Handling - Leaving OPERATIONAL

When leaving software state OPERATIONAL, the Application Software shall stop the following functionalities:

1. Stop SOME/IP Service Discovery - MVC-95856 - Service Discovery Shutdown
2. Stop SOME/IP Camera Service - ~~MVC-95857~~ - Camera Service Shutdown
3. Stop Shutter - MVC-98194 - Shutter control - Stop
4. Stop Heating - MVC-98193 - Heating control - Stop

Status	Ready for review
Verification Criteria	Trigger child transition, oberserve sequence

MVC-96178 - State Handling - Transition OPERATIONAL to IPCONFIG - Lease Time expires

While in software state `OPERATIONAL`, when the lease time of the ip address expires, the Application Software shall switch to software state `IPCONFIG` and perform the specified DHCP functionality according to MVC-96167 - DHCP .

Status	Ready for review
Verification Criteria	Indirekt Test: When lease time expires, check if there comes a DHCP request

MVC-99421 - State Handling - OPERATIONAL - DHCP Renew and Rebind

While in software state OPERATIONAL, the Application Software shall support the renewal and rebinding process according to RFC2131 (see RFC2131 chapter 4.4.5).

Status	Ready for review
Verification Criteria	Check if T1 and T2 are complied according to RFC2131

MVC-96443 - State Handling - Transition OPERATIONAL to IPCONFIG - DHCPACK different IP Address

While in software state `OPERATIONAL`, when the Application Software receives DHCPACK with a different IP address as result of RENEW or REBIND process, the Application Software shall switch to software state `IPCONFIG` and perform the specified DHCP functionality according to MVC-96167 - DHCP.

Status	Ready for review
Verification Criteria	Immitate this case with a DHCP server

MVC-96182 - State Handling - Transition OPERATIONAL to PHYLINKCHECK

While in software state `OPERATIONAL`, if the Application Software detects the abcense physical link, the Application Software shall switch to software state `PHYLINKCHECK`.

Status	Ready for review
Verification Criteria	While not in DHCP handshake, disconnect ethernet cable, check if SoC tries to poll phy link status according to MVC-96189

MVC-96184 - State Handling - Transition IPCONFIG to PHYLINKCHECK

While in software state `IPCONFIG`, if the Application Software detects the abcense physical link, the Application Software shall switch to software state `PHYLINKCHECK`.

Status	Ready for review
Verification Criteria	While in DHCP handshake, disconnect ethernet cable, check if SoC tries to poll phy link status according to MVC-96189

MVC-96235 - State Handling - Unexpeced Behavior after PMIC Configuration

While not in software state `INIT`, if the Application Software detects unexpected behavior, the Application Software shall swich to software state `UNEXPECTED_BEHAVIOR` restart SoC by setting `pmicErrorIn` to active**.**

Status	Ready for review
Verification Criteria	Perform error injection (e.g. according to MVC-95754) during startup, observe "pmicErrorIn" Signal.

MVC-96234 - State Handling - Unexpeced Behavior bevor PMIC Configuration

While in software state `INIT`, if the Application Software detects unexpected behavior, the Application Software shall enter an endless loop.

Note: Because PMIC is not configured yet

Status	Rejected
Verification Criteria	Perform error injection after Startup, observe power rails (TP500-TP504).

15 Camera Interface Setup

The term **Camera Interface** refers to the entire connection between the **camera sensor** and the **System-on-Chip (SoC)**. This includes not only the camera sensor itself but also the **serializer-deserializer (SerDes) link** and the **VIN (Video Input) interface** of the SoC.

In other words, the Camera Interface includes all hardware components and communication links involved in transferring video data from the image source (the camera sensor) to the processing unit (the SoC).

This chapter specifies the **setup** of this components and interfaces involved in the video data input path. In this context, setup refers to the overall process of preparing components for operation, which includes both **initialization** and **configuration**.

Initialization refers to the process of bringing a component into a valid starting state. This involves enabling the power supply and preparing hardware access, such as activating interfaces or clocks.

Configuration is the process of setting parameters or options to customize how a component behaves, adapting it to specific requirements or environments.

The following requirement is superordinate to the following ones and defines the order in which the individual initialization and configuration steps must be performed. Following this sequence is essential for the successful setup of the camera interface.

For now we dont specify, that Sensor is constantly streaming and VIN is constantly capturing images.
TODO: in a later phase we should specify at least that the entire processing pipline up to encoder is constantly running.

MVC-96065 - Camera Interface - Setup Sequence

When entering the software state PERIPHERAL_CONFIGURATION, the Application Software shall execute the following camera interface setup sequence:

1. MVC-95871 - VIN MIPI PHY Initialization
    a. MVC-95882 - VIN - Initialization - Reset to MIPI Mode
2. MVC-96014 - SerDes Setup
    a. MVC-95778 - SerDes - Setup Sequence
3. MVC-96016 - Sensor Setup
    a. MVC-96061 - Sensor Setup - Static register configuration
4. MVC-95881 - VIN Configuration
    a. MVC-95974 - VIN - Configuration

Status	Draft
Verification Criteria	Check order of single requirements

15.1 Sensor Setup

The development of the sensor driver is not planned for the B-sample (CW 7). Therefore, it is sufficient at this stage to specify that the Application Software shall make use of the sensor driver. Currently, the software architecture is structured such that the sensor driver's configuration function also handles the initialization and configuration of the associated components, including the serializer, deserializer, and VIN interface, in the correct sequence.

MVC-96061 - Sensor Setup - Static register configuration

The Application Software shall write the provided static sensor configuration consisting of key-value paired register description to the corresponding image sensor registers via I2C.

TODO: Um dieses Req auf specified zu bekommen, muss die statische Sensorkonfiguration benannt werden und eingeführt werden.

Note: Regarding Initialization there is nothing to do here, as power-up and clock enable is handled within SerDes Setup. (MVC-95778 - SerDes - Setup Sequence).

Currently the key-value paired register description is defined in AmbaSensor_MAX96717_96712_SKE3B6Table.c

Status	Draft
Verification Criteria	Check I2C trace

15.1.1 EEPROM Data Readout

MVC-102919 - EEPROM - Data readout

During Camera Interface Setup as described in MVC-96111 - State Handling - Transition INIT to PERIPHERAL_CONFIGURATION, the application software shall read the following blocks of data from the EEPROM of the SOW1 via `serDesI2c` (see MVC-94043 - HSI - Camera Interface):

Start Address	Size in Bytes	Description
0x0020	224	Label Data Block
0x0100	352	Intrinsic Data Block

Notes:

- The read out data shall be stored to be used by the `getDataBlock` SOME/IP Method.
- If SOW1 data couldn't be retrieved during startup the block can be filled with zeros.

Status	Ready for review
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-102921 - EEPROM - getDataBlock - Label Data Block

When the application software receives a valid `getDataBlock` SOME/IP `REQUEST` message with **deviceId** = 0x0000 and **blockId** = 0x0000, it shall return the following values for the payload:

- blockSize = **0x000000E0**
- blockData = **Label Data Block** as described in MVC-102919 - EEPROM - Data readout

Status	Ready for review
Verification Criteria	Send getDataBlock message with valid block definitions, receive response message and check for expected output parameters and values

MVC-102922 - EEPROM - getDataBlock - Intrinsic Data Block

When the application software receives a valid `getDataBlock` SOME/IP `REQUEST` message with **deviceId** = 0x0000 and **blockId** = 0x0001, it shall return the following values for the payload:

- blockSize = **0x00000160**
- blockData = **Intrinsic Data Block** as described in MVC-102919 - EEPROM - Data readout

Status	Ready for review
Verification Criteria	Send getDataBlock message with valid block definitions, receive response message and check for expected output parameters and values

15.2 Sensor Functional Requirements

MVC-102322 - Sensor Runtime Configurations

The Software shall adjust the sensor runtime parameters over the GMSL2 backchannel based on the output parameters of the AE and AWB algorithms.
The runtime parameters that are adjusted are:

- white balance gain
- Exposure time

Status	Approved
Verification Criteria	Verify through interface and code review that AE/AWB outputs are correctly mapped to sensor I2C runtime configuration commands over the GMSL2.

MVC-108188 - Default 30 fps configuration set

The software shall configure camera sensor as defined in **30 fps Configurations** **S5K3B6SA_MAXIM_HDR4_RSSR_16B_MCLK25_30fps_1920x1550_MIPI4_481.25Mbps_wGamma_wEMBD_wSTAT6line_wFuSa_wSECU_wDS_20251029.sset**

Status	Approved
Verification Criteria	Check by testing that the configurations that are intended to be configured with in this mode are the ones during the image streaming .

MVC-108189 - Sensor Slave address

The software shall configure the sensor camera I2C slave address to 0x20.

Status	Approved
Verification Criteria	Verify by ensuring that the sensor has a response upon being called with this slave address

MVC-102323 - Camera Sensor Input clock

The software shall set sensor input clock with 25MHZ.

Status	Approved
Verification Criteria	Check that the sensor input clock is 25MHZ

MVC-108640 - VIN Clock

The software shall configure the VIN clock with the same value as the sensor input clock.

Status	Approved
Verification Criteria	Verify by checking that VIN has clock of 25MHZ

MVC-102325 - Default 30 fps mode

The software shall configure camera sensor in the default 30 fps mode at camera startup.

Status	Approved
Verification Criteria	Verify by makin sure that sensor is working in 30 fps at camera startup

15.3 SerDes Setup

MVC-97239 - SerDes - Interface

The Application Software shall communicate with serializer and deserializer by using the I2C interface `serDesI2c` (see MVC-94043):

- Deserializer I2C address: 0x50
- Serializer I2C address: 0x80
- Speed mode: 100kb/s

Status	Specified
Verification Criteria	Check if I2C traffic has specified properties

MVC-97823 - SerDes - Setting a Bit

The application software shall set one or multiple bits in an arbitrary register by performing the following sequence:

1. Read current register value
2. Modify the desired bits to 1
3. Write modified value back to register

Status	Specified

Verification Criteria	Check if there is a read and write access to the affected register

MVC-97824 - SerDes - Clearing a Bit

The application software shall clear one or multiple bits in an arbitrary register by performing the following sequence:

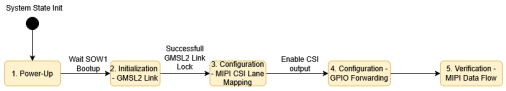
1. Read current register value
2. Modify the desired bits to 0
3. Write modified value back to register

Status	Specified
Verification Criteria	Check if there is a read and write access to the affected register

The following provides a description of the setup and validation sequence for the data link between sensor and SoC via the GMSL2 interface. Proper system functionality requires execution of the steps in the specified order.
Note: When there is a concrete state handling Steps 1-5 could be substates.

MVC-95778 - SerDes - Setup Sequence

The application software shall execute the following SerDes setup sequence in the specified 5 phases:



Phase 1: MVC-95707 - Power-Up

Activate deserializer and SOW1 (including serializer and sensor):

1. MVC-95716 - SerDes - Power-Up - Activate Deserializer
2. MVC-95724 - SerDes - Power-Up - Activate SOW1
3. MVC-95727 - SerDes - Power-Up - Wait SOW1 Bootup (Last Action before next phase)

Phase 2: MVC-95726 - Initialization - GMSL2 Link

Initialization of GMSL2 link parameters, execution of link reset and monitoring for lock acquisition:

1. MVC-95731 - SerDes - Initialization - Disable CSI Output during Initialization and Configuration
2. MVC-95732 - SerDes - Initialization - Disable Video Transmit Pipe Z during Initialization and Configuration
3. MVC-95739 - SerDes - Initialization - Serializer Tunnel Mode
4. MVC-95741 - SerDes - Initialization - Deserializer One-Shot Reset
5. MVC-95742 - SerDes - Initialization - Wait for GMSL2 Link Lock
6. MVC-95754 - SerDes - Initialization - Handling of missing GMSL2 Link Lock
7. MVC-95780 - SerDes - Initialization - Successfull GMSL2 Link Lock (Last Action before next phase)

Phase 3: MVC-95708 - Configuration - MIPI CSI Lane Mapping

Definition of lane mapping between Serializer and Sensor, definition of lane mapping between Deserializer and SoC

1. MVC-95786 - SerDes - Configuration - Deserializer MIPI Lane Mapping
2. MVC-95765 - SerDes - Configuration - Serializer MIPI Lane Mapping
3. MVC-95772 - SerDes - Configuration - Serializer Enable 25 MHz RCLK on MFP4
4. MVC-95775 - SerDes - Configuration - Enable Video Transmit Pipe Z
5. MVC-95774 - SerDes - Configuration - Enable CSI Output after Configuration of MIPI CSI Lane Mapping (Last Action before next phase)

Phase 4: MVC-95711 - Configuration - GPIO Forwarding

Configuration of MFP pin assignments and internal pull-up/pull-down resistors

1. MVC-95777 - SerDes - Configuration - Deserializer Disable Output of LOCK signal on MFP5
2. MVC-95787 - SerDes - Configuration - GPIO Forwarding of imagerError
3. MVC-95788 - SerDes - Configuration - GPIO Forwarding of camHeatingEn
4. MVC-95789 - SerDes - Configuration - GPIO Forwarding of camPmicError
5. MVC-95790 - SerDes - Configuration - GPIO Forwarding of imagerEn

Phase 5: MVC-95710 - Verification - MIPI Data Flow

Verification of MIPI CSI packet output from Deserializer to SoC (VIN interface)

1. MVC-95801 - SerDes - Verification - MIPI Data Flow Error Handling

Status	Specified
Verification Criteria	Check the correct Sequence by I2C trace. Phase 1 has to be checked by measurements

15.3.1 Power-Up

MVC-95716 - SerDes - Power-Up - Activate Deserializer

The application software shall activate the deserializer by setting the level of signal **`desEn`** to **`high`**.

Status	Specified
Verification Criteria	Check if desEn was set to high by observing corresponding testpad

MVC-95724 - SerDes - Power-Up - Activate SOW1

The application software shall activate the SOW1 (including the serializer and image sensor) by setting the level of signal **`powerOverCoaxEn`** to **`low`**.

Status	Specified
Verification Criteria	Check if powerOverCoaxEn was set to low by observing corresponding testpad

MVC-95727 - SerDes - Power-Up - Wait SOW1 Bootup

The application software shall wait at least `100ms` after activating the SOW1 before continue executing phase 2: 15.3.2 - Initialization - GMSL2 Link .

Note: The SOW1 requires this time to boot properly.

Status	Specified
Verification Criteria	Between powerOverCoaxEn was set to low (MVC-95724) and MVC-95731 there shall be at least 100ms no I2C traffic

15.3.2 Initialization - GMSL2 Link

MVC-95731 - SerDes - Initialization - Disable CSI Output during Initialization and Configuration

When entering phase 2 (MVC-95726 - Initialization - GMSL2 Link), the application software shall disable CSI output by clearing the `CSI_OUT_EN` (bit 1) in register `BACKTOP12 (0x313)` of the deserializer.

Note: CSI output will be enabled at the end of MIPI CSI Lane Mapping by execution of MVC-95774 - SerDes - Configuration - Enable CSI Output after Configuration of MIPI CSI Lane Mapping

Status	Specified
Verification Criteria	Check if bit 1 in register 0x313 at slave id 0x50 was cleared via I2C access (Clearing as specified in MVC-97824 (read, manipulate, write))

MVC-95732 - SerDes - Initialization - Disable Video Transmit Pipe Z during Initialization and Configuration

The application software shall disable the video transmit pipe Z by clearing the `VID_TX_EN_Z` (bit 6) in register `REG2 (0x02)` of the serializer.

Note: Video transmit pipe Z will be enabled at the end of configuration of MIPI CSI Lane Mapping by execution of MVC-95775 - SerDes - Configuration - Enable Video Transmit Pipe Z

Status	Specified

Verification Criteria	Check if bit 6 in register 0x02 at slave id 0x80 was cleared via I2C access (Clearing as specified in MVC-97824 (read, manipulate, write))

MVC-95739 - SerDes - Initialization - Serializer Tunnel Mode

The application software shall set the serializer to tunnel mode by setting the `Tun_Mode` (bit 7) in register `EXT11 (0x383)`.

Status	Specified
Verification Criteria	Check if bit 7 in register 0x383 at slave id 0x80 was set via I2C access (Setting as specified in MVC-97823 (read, manipulate, write))

MVC-95741 - SerDes - Initialization - Deserializer One-Shot Reset

The application software shall trigger a one-shot reset on the deserializer by setting the self-clearing `RESET_ONESHOT` (bit 5) in register `CTRL0 (0x10)`.

Status	Specified
Verification Criteria	Check if bit 5 in register 0x10 at slave id 0x50 was set via I2C access (Setting as specified in MVC-97823 (read, manipulate, write))

MVC-95742 - SerDes - Initialization - Wait for GMSL2 Link Lock

The application software shall wait for the GMSL2 link to be locked on the deserializer by polling the `LOCKED` (bit 3) in register `CTRL3 (0x13)` at a `25ms` interval with a `200ms` timeout.

Status	Specified
Verification Criteria	Observe correct polling: There shall be read accesses to deserializer id 0x50, register 0x13 with 25ms interval with a 200ms timeout.

MVC-95754 - SerDes - Initialization - Handling of missing GMSL2 Link Lock

If the `200ms` timeout exprires and the LOCKED bit is not set, the application software shall switch to software state `UNEXPECTED_BEHAVIOR`.

Status	Specified
Verification Criteria	Here we need some kind of error injection (e.g. manipulate LOCKED bit in I2C traffic) and observe if software switches to state UNEXPECTED_BEHAVIOR (SoC performes reboot according to MVC-96235)

MVC-95780 - SerDes - Initialization - Successfull GMSL2 Link Lock

While waiting for link GMSL2 link lock, when the LOCKED bit is set, the application software shall continue executing phase 3: 15.3.3 - Configuration - MIPI CSI Lane Mapping .

Status	Specified
Verification Criteria	Observe polling: When LOCKED is set the next I2C access shall be as specified in MVC-95786

15.3.3 Configuration - MIPI CSI Lane Mapping

MVC-95786 - SerDes - Configuration - Deserializer MIPI Lane Mapping

The application software shall configure MIPI lane swapping on the deserializer by writing the value `0x4E` register `MIPI_PHY3 (0x333)`.

Status	Specified
Verification Criteria	Check I2C trace of ISP_DES_I2C Slave ID: 0x50 (Deserializer); Reg 0x333 (MIPI_PHY3); value 0x4e

MVC-95765 - SerDes - Configuration - Serializer MIPI Lane Mapping

The application software shall configure MIPI lane mapping on the serializer by writing the following registers:
`MIPI_RX2 (0x332)` = `0xE0`, `MIPI_RX3 (0x333)` = `0x01`, `MIPI_RX4 (0x334)` = `0x10` and `MIPI_RX5 (0x335)` = `0x07`.

Status	Specified
Verification Criteria	Check I2C trace of ISP_DES_I2C Slave ID = 0x80: MIPI_RX2 (0x332) = 0xE0, MIPI_RX3 (0x333) = 0x01, MIPI_RX4 (0x334) = 0x10 and MIPI_RX5 (0x335) = 0x07 (4 Transactions in total)

MVC-95772 - SerDes - Configuration - Serializer Enable 25 MHz RCLK on MFP4

The application software shall enable the configured 25 MHz clock on MFP4 by setting the `RCLKEN` (bit 5) in register `REG6 (0x06)` of the serializer.

Status	Specified
Verification Criteria	Check if bit 5 in register 0x06 at slave id 0x80 was set via I2C access (Setting as specified in MVC-97823 (read, manipulate, write))

MVC-95775 - SerDes - Configuration - Enable Video Transmit Pipe Z

The application software shall enable the video transmit pipe Z on the serializer by setting the `VID_TX_EN_Z` (bit 6) in register `REG2 (0x2)`.

Status	Specified
Verification Criteria	Check if bit 6 in register 0x2 at slave id 0x80 was set via I2C access (Setting as specified in MVC-97823 (read, manipulate, write))

MVC-95774 - SerDes - Configuration - Enable CSI Output after Configuration of MIPI CSI Lane Mapping

The application software shall enable CSI output on the deserializer by setting the `CSI_OUT_EN` (bit 1) in register `BACKTOP12 (0x313)`.

Status	Specified
Verification Criteria	Check if bit 1 in register 0x313 at slave id 0x50 was set via I2C access (Setting as specified in MVC-97823 (read, manipulate, write))

15.3.4 Configuration - GPIO Forwarding

MVC-95777 - SerDes - Configuration - Deserializer Disable Output of LOCK signal on MFP5

The application software shall disable the output of the LOCK signal on MFP5 by clearing the `LOCK_EN` (bit 7) in register `REG5 (0x5)` of the deserializer.

Status	Specified
Verification Criteria	Check if bit 7 in register 0x5 at slave id 0x50 was cleared via I2C access (Clearing as specified in MVC-97824 (read, manipulate, write))

MVC-95787 - SerDes - Configuration - GPIO Forwarding of imagerError

The application software shall configure the deserializer and serializer registers defined in MVC-94705 - SerDes GPIO Forwarding Configuration - Imager Error in order to realize the forwarding of the signal *`imagerError`*.

Status	Specified
Verification Criteria	Check I2C trace: 0x50 (Deserializer); register: 0x2B3; value: 0x84; 0x50 (Deserializer); register: 0x2B4; value: 0xA0 0x50 (Deserializer); register: 0x2B5; value: 0x47 0x80 (Serializer); register: 0x2D3; value: 0x83 0x80 (Serializer); register: 0x2D4; value: 0xA7

MVC-95788 - SerDes - Configuration - GPIO Forwarding of camHeatingEn

The application software shall configure the deserializer and serializer registers defined in MVC-94711 - SerDes GPIO Forwarding Configuration - Camera Heating Enable in order to realize the forwarding of the signal *`camHeatingEn`*.

Status	Specified
Verification Criteria	Check I2C trace: 0x80 (Serializer); register: 0x2C7; value: 0x84 0x80 (Serializer); register: 0x2C8; value: 0xA3 0x50 (Deserializer); register: 0x2C9; value: 0x45 0x50 (Deserializer); register: 0x2BF; value: 0x83 0x50 (Deserializer); register: 0x2C0; value: 0xA5

MVC-95789 - SerDes - Configuration - GPIO Forwarding of camPmicError

The application software shall configure the deserializer and serializer registers defined in MVC-94712 - SerDes GPIO Forwarding Configuration - Camera PMIC Error in order to realize the forwarding of the signal *`camPmicError`*.

Status	Specified
Verification Criteria	Check I2C trace: 0x50 (Deserializer); register: 0x2C2; value: 0x84 0x50 (Deserializer); register: 0x2C3; value: 0xA6 0x50 (Deserializer); register: 0x2C4 ; value: 0x45 0x80 (Serializer); register: 0x2D3; value: 0x83 0x80 (Serializer); register: 0x2D4; value: 0xA5

MVC-95790 - SerDes - Configuration - GPIO Forwarding of imagerEn

The application software shall configure the deserializer and serializer registers defined in MVC-94717 - SerDes GPIO Forwarding Configuration - Imager Enable in order to realize the forwarding of the signal *`imagerEn`*.

Status	Specified
Verification Criteria	Check I2C trace: 0x80 (Serializer); register: 0x2BE; value: 0x84 0x80 (Serializer); register: 0x2BF; value: 0xA0 0x80 (Serializer); register: 0x2C0; value: 0x47 0x50 (Deserializer); register: 0x2C5; value: 0x83 0x50 (Deserializer); register: 0x2C6; value: 0xA7

MVC-95793 - SerDes - Configuration - GPIO of camPmicError

The application software shall configure GPIO2 as an input (resistor configuration to be defined) in order to receive the *`camPmicError`* signal from SOW1 PMIC.

Note: Here we have an explicit requirement, because the initial GPIO pin config for GPIO2 is output. (This could be changed in the future)

Status	Specified
Verification Criteria	Check GPIO configuration per UART (:\> gpio show <pin>)

MVC-103085 - SerDes - Configuration - GPIO Forwarding of FsyncTrigger

The application software shall configure the deserializer and serializer registers defined in MVC-94717 - SerDes GPIO Forwarding Configuration - Imager Enable in order to realize the forwarding of the signal *FsyncTrigger*.

Note:
- *MVC-95778 - SerDes - Setup Sequence is related to this*
- *MVC-95778 - SerDes - Setup Sequence needs to adjusted for fsync*
- *MVC-98066 - SerDes Setup - Successfull verifys this req*
- *MVC-98066 - SerDes Setup - Successfull needs to be adjusted to check fsync config too*

Status	Draft
Verification Criteria	

15.3.5 Configuration - ADC

MVC-100206 - SerDes - Configuration - ADC Configuration

The application software shall configure the following registers on the MAX96717 Serializer in order to set up the ADC:

Step	Register	Bitfield Name	Value	Description
1	0x0534	adc_rr_run	write 0b0	hold round robin state machine in idle
2	0x0500	adc_pu	write 0b0	ADC powered off
3	0x0500	buf_pu	write 0b0	ADC internal buffer off
4	0x0500	adc_refbuf_pu	write 0b0	ADC reference buffer off
5	0x0500	adc_chgpump_pu	write 0b0	ADC charge pump off
5	0x0501	adc_clk_en	write 0b0	Disable ADC clock
6	0x001E	adc_int_oen	write 0b1	Enable global ADC interrupt
7	0x0501	adc_clk_en	write 0b1	Enable ADC clock
8	0x0501	adc_refsel	write 0b0	Select 1.25V internal ref
9	0x0502	adc_xref	write 0b1	Select internal ref for ADC
10	0x0509	bypass_volttempt_corr	write 0b0	Disable correction bypass for internal ref
11	0x050C	adc_ref_ready_ie	write 0b1	Enable ADC ready interrupt
12	0x050C	adc_done_ie	write 0b1	Enable ADC done interrupt
13	0x050C	adc_calDone_ie	write 0b1	Enable ADC calibration done interrupt
14	0x0510 0x0511 0x0512 0x0513	adc.intr0 adc.intr1 adc.intr2 adc.intr3	read read read read	Read to clear registers of ADC interrupts
15	0x0502	adc_refbuf_pu	write 0x0	Select internal of external ref
16	0x0500	adc_chgpump_pu	write 0b1	Power up ADC charge pump
17	0x0500	adc_pu	write 0b1	Power up ADC
18	0x0500	adc_refbuf_pu	write 0b1	Power up ADC ref buffer
19	0x0500	buf_pu	write 0b1	Power up ADC internal buffer
20	0x0510	adc_ref_ready_if	read 0b1 (via polling)	Wait for ready interrupt to be asserted

Status	Draft
Verification Criteria	Read Serializer register access and check for correct configuration order.

MVC-100207 - SerDes - ADC - Die Temperature readout

The application software shall allow on-demand Die Temperature readout of the Serializer via the following steps:

Step	Register	Bitfield Name	Value	Description
1	0x1D28	run_tmon_cal	write 0b1	Initiate temperature conversion
2	0x0510	adc_done_if	read 0b1	Wait for ADC done to assert
3	0x0510	adc_calDone_if	read 0b1	Wait for ADC calibration done to assert
4	0x1D3B 0x1D3C	t_est_out_b0 t_est_out_b1	read	Read internal Die temperature registers

Status	Draft
Verification Criteria	Check that during temperature read the order is followed

15.3.6 Verification - MIPI Data Flow

MVC-95801 - SerDes - Verification - MIPI Data Flow Error Handling

If the application software reads equal values for two sequencial reads of the two fields

1. `csi2_txl_pkt_cnt` (bit 3 until bit 0) in register `MIPI_PHY18 (0x342)` and
2. `phy1_pkt_cnt` (bit 7 until bit 4) in register `MIPI_PHY20 (0x344)`,

the application software shall switch to software state `UNEXPECTED_BEHAVIOR`.

Status	Specified
Verification Criteria	Perform error injection (e.g. manipulate I2C traffic) so that at least one of the specified conditions apply software switches so state UNEXPECTED_BEHAVIOR (SoC reboot).

15.4 SerDes Functional Requirements

MVC-102423 - SerDes I2c Commuincation

The SerDes software shall support I2C read and write access to:

- the deserializer via direct SoC I2C connection (Address: 0x50)
- the serializer via GMSL2 backchannel routing through the deserializer (Address: 0x80).

Status	Approved
Verification Criteria	Communicate with serializer/deserialzer using mentioned I2c addresses.

MVC-102430 - Deserializer Configuration

The SerDes software shall configure the deserializer to enable GMSL2 link operation.

Status	Approved
Verification Criteria	Reading the deseriaziler configuration registers that specified in the HSI

MVC-108450 - GMSL2 LOCK pin assertion

If the deserializer GMSL2 LOCK status pin asserted, the SerrDes software shall complete the serial communication powerup sequence.

Status	✔ Approved
Verification Criteria	Reading the deseriaziler GMSI lock status register that specified in the HSI

MVC-108445 - SOC-Deserializer interface pins

The SerDes software shall configure the interface pins between the SOC and the deserializer.

Status	✔ Approved
Verification Criteria	Reading the deseriaziler configuration registers that specified in the HSI

MVC-102432 - Serializer Configuration

The SerDes software shall configure the serializer.

Status	✔ Approved
Verification Criteria	Reading the serializer configuration registers that specified in the HIS

MVC-102435 - Tunnel Mode Configuration

The SerDes software shall configure the serializer to operate in tunnel mode according to MVC-94696 - Serializer Tunnel Mode Selection.

Status	✔ Approved
Verification Criteria	Reading the serializer tunnel register that specified in the HSI

MVC-102436 - CSI-2 MIPI Lane Order/ polarity Configuration

The SerDes software shall configure serializer and deserializer MIPI lane order and polarity according to the MVC-94702 - Deserializer - MIPI CSI-2 Lane Configuration and MVC-94708 - Serializer - MIPI CSI-2 Lane Configuration.

Imager Output		Serializer Input		Deserializer Output		SoC/ISP Input	
Lane	Polarity	Lane	Polarity	Lane	Polarity	Lane	Polarity
0	N	1	P	2	P	0	P
	P		N		N		N
1	N	0	P	3	P	1	P
	P		N		N		N
2	N	2	P	0	P	2	P
	P		N		N		N
3	N	3	P	1	P	3	P
	P		N		N		N
CLK	N	CLK	P	CLK	P	CLK	P
	P		N		N		N

.

Status	✔ Approved
Verification Criteria	Reading the serializer MIPI Lane registers that specified in the HSI.

MVC-102437 - CSI-2 MIPI datatypes Configuration

The SerDes software shall configure MIPI data type and bit width according to MVC-94043 - HSI - Camera Interface.

Status	✘ Rejected
Verification Criteria	Reading the serialzer MIPI Lane registers that specified in the HSI

MVC-102438 - GPIO Forwarding

The SerDes software shall configure GMSL2 GPIO forwarding for signals:

- imagerEn MVC-94717 - SerDes GPIO Forwarding Configuration - Imager Enable
- camHeatingEn MVC-94711 - SerDes GPIO Forwarding Configuration - Camera Heating Enable
- camPmicError MVC-94712 - SerDes GPIO Forwarding Configuration - Camera PMIC Error
- imagerError MVC-94705 - SerDes GPIO Forwarding Configuration - Imager Error
- Frame Synchronization MVC-103086 - SerDes GPIO Forwarding Configuration - Fsync Trigger

including direction, MFP selection as following:

Signal	Forwarding Setting Serialierzer MFP/direction/Deserializer MFP	Target configrutation of MFP @Serializer	Target configrutation of MFP @Deserializer
`imagerEn`	MFP0 <- MFP7	GPO (Push-pull)	GPI (with internal pull down 1MΩ)
`fsyncTrigger`	MFP1 <- MFP0	GPI (with internal pull down 1MΩ)	GPO (Push-Pull)
`camHeatingEn`	MFP3 <- MFP2	GPO (Push-pull)	GPI (with internal pull down 1MΩ
`camPmicError`	MFP5 -> MFP6	GPI (with internal pull down 1MΩ)	GPO (Push-Pull)
`imagerError`	MFP7 -> MFP1	GPI (with internal pull down 1MΩ)	GPO (Push-Pull)

Status	✔ Approved
Verification Criteria	Checking data forwarding on related pins.

MVC-102439 - Srializer - sensor clock

The SerDes software shall enable a 25 MHz clock output on serializer MFP4 according to MVC-94726 - Serializer - Enable RCLK Output .

Status	✔ Approved
Verification Criteria	Checking the clock that output on MFP4

MVC-102440 - Temperature - ADC initialization

The SerDes software shall initialize the serializer ADC according to MVC-99301 - Ser ADC Configuration.

Status	✔ Approved
Verification Criteria	Reading the serializer ADC register

MVC-108488 - Temperature Readout

The SerDes software shall provide the demand temperature according to MVC-99302 - Ser On-Demand Die Temperature read out.

Status	✔ Approved
Verification Criteria	Reading the serializer ADC register

MVC-102448 - Video Data Flow statue

The SerDes software shall detect the link lock establishment status.

Status	✔ Approved
Verification Criteria	Reading the link lock register of deseriazlier

15.5 VIN Setup

The VIN has to be configured to match the mode and paramters set in the sensor.

15.5.1 VIN MIPI PHY Initialization

MVC-95882 - VIN - Initialization - Reset to MIPI Mode

The Application Software shall reset the VIN to MIPI mode.

Note for architecture: This is achieved by calling `AmbaVIN_MipiReset` with `AMBA_VIN_MIPI_PAD_CONFIG_s` data structure parameter.

Status	Draft
Verification Criteria	Check Amba's VIN Global Register: 0xED1C0800[1:0] == 0b11 via debugger or programmed debuginterface

MVC-95870 - VIN - Initialization - Continous Clock Mode

The Application Software shall configure the MIPI PHY of VIN to operate in continuous clock mode.

Note for software architecture: This is achieved by setting `ClkMode = 1` in the `AMBA_VIN_MIPI_PAD_CONFIG_s` data structure and call `AmbaVIN_MipiReset`.

Status	Draft
Verification Criteria	We need to implement Testsoftware, which calls AmbaCSL_VinGetMipiForceClkHs and check for return value to be 0b1

MVC-95873 - VIN - Initialization - Data Rate

The Application Software shall configure the input data rate of the VIN MIPI PHY to the value `DataRate` specified by the `VideoInputConfiguration`.

Note for architecture: In Case of 700MBps per lane this is achieved by setting `DataRate = 700000000` in the `AMBA_VIN_MIPI_PAD_CONFIG_s` data structure and call `AmbaVIN_MipiReset`.

Status	Draft
Verification Criteria	Hard to test. Within AmbaVIN data rate is converted into MIPI PHY Timing register values. Theres no 1:1 register value to be read

MVC-95875 - VIN - Initialization - Enable Data Pins

The Application Software shall enable the data pins SD_LVDS_P/N 0 to 3 of the VIN MIPI PHY according to the value `NumActiveLanes` specified by the `VideoInputConfiguration`.

Note for architecture: This is achieved by setting the `EnabledPin` = `0xF` in the `AMBA_VIN_MIPI_PAD_CONFIG_s` data structure and call `AmbaVIN_MipiReset`.

Status	Draft
Verification Criteria	Test by review

15.5.2 VIN Configuration

MVC-95974 - VIN - Configuration

The Appliacation Software shall configure the VIN MIPI parameters.

Note for architecture: This is achieved by calling `AmbaVIN_MipiReset` with `AMBA_VIN_MIPI_PAD_CONFIG_s` data structure parameter.

Status	Draft
Verification Criteria	

MVC-95883 - VIN - Configuration - Frame Rate

The Application Software shall configure the frame rate of VIN MIPI interface to the value `FrameRate` specified by the `VideoInputConfiguration`.

Note for software architecture: In case of 60 FPS this is achieved by setting `Interlace = 0`, `NumUnitsInTick = 1` and `TimeScale = 60` in the `FrameRate` field in `AMBA_VIN_MAIN_CONFIG_s` data structure and call `AmbaVIN_MipiConfig`.

Status	Draft
Verification Criteria	Checked by calling AmbaVIN_GetInfo()

MVC-95887 - VIN - Configuration - Color Space

The Application Software shall configure the color space of VIN to the value `ColorSpace` specified by the `VideoInputConfiguration`.

Note for software architecture: In case of RGB this is achieved by setting `ColorSpace = 0` in the `AMBA_VIN_MIPI_CONFIG_s` data structure and call `AmbaVIN_MipiConfig`.

Status	Draft
Verification Criteria	Checked by calling AmbaVIN_GetInfo()

MVC-95888 - VIN - Configuration - Bayer Pattern

The Application Software shall configure the bayer pattern of VIN to the value `BayerPattern` specified by the `VideoInputConfiguration`.

Note for architecture: In case of GRBG this is achieved by setting `BayerPattern = 0` in the `AMBA_VIN_MIPI_CONFIG_s` data structure and call `AmbaVIN_MipiConfig`.

Status	Draft
Verification Criteria	Checked by calling AmbaVIN_GetInfo()

MVC-95889 - VIN - Configuration - Number of Data Bits

The Application Software shall configure the resolution of the input pixel data bits of VIN to the value `NumDataBits` specified by the `VideoInputConfiguration`.

Note for architecture: In case of 12 bits per pixel this is achieved by setting `NumDataBits = 12` in the `AMBA_VIN_MIPI_CONFIG_s` data structure and call `AmbaVIN_MipiConfig`.

Status	Draft
Verification Criteria	Checked by calling AmbaVIN_GetInfo()

MVC-95890 - VIN - Configuration - Number of Skip Frames

The Application Software shall configure the number of frames to be skipped when VIN starts to capture to 1.

Note for architecture: This is achieved by setting `NumSkipFrame = 1` in the `AMBA_VIN_MIPI_CONFIG_s` data structure and call `AmbaVIN_MipiConfig`.

Status	Draft
Verification Criteria	Checked by calling AmbaVIN_GetInfo()

MVC-95891 - VIN - Configuration - Input H/V Sync Signal Format.

The Application Software shall configure the input H/V sync signal format of VIN to the following values specified by the `VideoInputConfiguration`:

- Active region width to the value `NumActivePixels`
- Active region height to the value `NumActiveLines`
- Horizontal total width to the value `NumTotalPixels - 1` (TODO: Warum -1 ?)
- Vertical total height to the value `NumTotalLines`

Note for architecture: This is achieved by setting `RxHvSyncCtrl` in the `AMBA_VIN_MIPI_CONFIG_s` data structure and call `AmbaVIN_MipiConfig`.

Status	Draft
Verification Criteria	Hard to test. Within AmbaVIN these values are converted into MIPI PHY Timing register values. Theres no 1:1 register value to be read

MVC-95999 - VIN - Configuration - Number of active Data Lanes

The Application Software shall set the number of data lanes according to the value `NumActiveLanes` specified by the `VideoInputConfiguration`.

Note for architecture: In case of 4 lanes this is achieved by setting the `NumActiveLanes` = 4 in the `AMBA_VIN_MIPI_CONFIG_s` data structure and call `AmbaVIN_MipiConfig`.

Status	Draft
Verification Criteria	In Case of 4 data lanes check register 0xED1C0001[3:0] == 0xF

MVC-96011 - VIN - Configuration - Data Type

The Application Software shall configure the data type of the input pixel data bits of VIN according to the value `NumDataBits` specified by the `VideoInputConfiguration`.

Note for architecture: In case of 12 bits per pixel this is achieved by setting the `Datatype` = 0x2C (RAW12) in the `AMBA_VIN_MIPI_CONFIG_s` data structure and call `AmbaVIN_MipiConfig`.

Status	Draft
Verification Criteria	Check Amba's VIN Main Register: 0xED1C0078[15:10] == 0x2C via debugger or programmed debuginterface

MVC-96012 - VIN - Configuration - Data Type Mask

The Application Software shall configure the data type mask of the input pixel data bits of VIN according to the value `NumDataBits` specified by the `VideoInputConfiguration`.

Note for architecture: This is achieved by setting the `DatatypeMask` = 0x00 in the `AMBA_VIN_MIPI_CONFIG_s` data structure and call `AmbaVIN_MipiConfig`.

Status	Draft
Verification Criteria	Check Amba's VIN Main Register: 0xED1C0078[9:4] == 0x00 via debugger or programmed debuginterface

16 AE/AWB

16.1 Auto Exposure

MVC-107741 - AE ISP Frame Statistics Acquisition

The AE software shall acquire frame statistics from ISP for each image sensor frame

Status	Draft
Verification Criteria	Verify by test that AE software acquires image frame statistics from ISP for each camera sensor frame captured

MVC-105501 - AE CFA Statistics Passing

The AE software shall pass CFA statistics data to AE algorithm

Status	Draft
Verification Criteria	Verify by test that AE receives one CFA statistics set per frame

MVC-105502 - AE PG Statistics Passing

The AE software shall pass PG aka, RGB statistics data to AE algorithm

Status	Draft
Verification Criteria	Verify by test that AE receives one PG statistics set per frame

MVC-105504 - Frame-Consistent CFA Statistics

The software shall provide identical CFA frame statistics to both AE and AWB algorithms for the same image sensor captured frame

Status	Draft
Verification Criteria	Verify by test that CFA frane statistics are identical for both AE and AWB algorithms

MVC-105503 - Frame-Consistent PG Statistics

The software shall provide identical PG frame statistics to both AE and AWB algorithms for the same image sensor captured frame

Status	Draft
Verification Criteria	Verify by test that PG frame statistics are identical for AE and AWB algorithms

MVC-105506 - Frame-Based AE Execution

The software shall execute AE algorithm once per frame upon reception of new statistics data

Status	Draft
Verification Criteria	Verify by test that AE algorithm is executed once a new frame statistics is available

MVC-105510 - AE Exposure Time and Digital Gain Control

The AE software shall control sensor exposure time and digital gain to stay within configured target brightness

Status	Draft
Verification Criteria	Verify by test that AE software computes sensor exposure time and digital gain to reach target brightness

MVC-105512 - AE Exposure Time Conversion

The AE software shall convert calculated exposure time to a valid sensor value range

Status	Draft
Verification Criteria	Verify by test that AE calculated exposure time is converted to sensor valid value

MVC-105511 - AE Digital Gain Conversion

The AE software shall convert calculated digital gain value to a valid sensor value range

Status	Draft
Verification Criteria	Verify by test that AE calculated digital gain is converted to sensor valid value

MVC-105514 - AE Exposure Time and Digital Gain Sensor Update

The AE software shall output sensor-converted exposure time and digital gain values to image sensor

Status	Draft
Verification Criteria	Verify by test that AE computed expsoure time and digital gain values are written to image sensor output interface (I2C)

MVC-105516 - AE Minimum Exposure Time Limitation

The AE software shall ensure that converted exposure time doesn't go below image sensor defined minimum value

Status	Draft
Verification Criteria	Verify by test that converted exposure time doesn't go below sensor minimum value for very bright scene frames

MVC-105515 - AE Minimum Digital Gain Limitation

The AE software shall ensure that converted digital gain doesn't go below image sensor defined mimimum value

Status	Draft
Verification Criteria	Verify by test that converted digital gain value doesn't go below sensor minimum value for very bright scene frames

MVC-105507 - AE Maximmum Exposure Time Limitation

The AE software shall ensure that converted exposure time doesn't exceed image sensor defined maximum value

Status	Draft
Verification Criteria	Verify by test that converted exposure time doesn't exceed sensor maximum value for very dark scene frames

MVC-105508 - AE Maximum Digital Gain Limitation

The AE software shall ensure that converted digital gain doesn't exceed image sensor defined maximum value

Status	Draft
Verification Criteria	Verify by test that converted digital gain doesn't exceed sensor maximum value for very dark scene frames

16.2 Auto White Balance

MVC-105940 - AWB CFA Statistics Acquisition

The AWB software shall acquire CFA statistics from ISP every image sensor frame

Status	Draft
Verification Criteria	Verify by test that AWB receives one CFA statistics set per frame

MVC-105941 - AWB PG Statistics Acquisition

The AWB software shall acquire PG statistics from ISP every image sensor frame

Status	Draft
Verification Criteria	Verify by test that AWB receives one PG statistics set per frame

MVC-105942 - Frame-Based AWB Execution

The software shall execute AWB algorithm once per frame upon reception of new statistics data

Status	Draft
Verification Criteria	

MVC-105524 - AWB Channel Gain Control

The AWB software shall control sensor channel gains (Red, Blue gains) to achieve natural white balance

Status	Draft
Verification Criteria	Verify by test that AWB compute channel gains opposite to scene color cast

MVC-105523 - AWB Red Channel Gain Conversion

The AWB software shall convert red channel gain to sensor valid value

Status	Draft
Verification Criteria	Verify by test that AWB converts red channel gain to valid senosr value

MVC-105526 - AWB Blue Channel Gain Conversion

The AWB software shall convert blue channel gain to sensor valid value

Status	Draft
Verification Criteria	Verify by test that AWB converts blue channel gain to sensor valid value

MVC-105525 - AWB Channel Gains Sensor Update

The AWB software shall output converted channel gains to image sensor

Status	Draft
Verification Criteria	Verify by test that AWB channel gain values are written to sensor output interface I2C

MVC-105527 - AWB Minimum Red Channel Gain Limitation

The AWB software shall limit converted red channel gain to sensor defined minimum value

Status	Draft
Verification Criteria	Verify by test that converted red channel gain doesn't go below sensor minimum threshold for warm scene frames

MVC-105519 - AWB Maximum Red Channel Gain Limitation

The AWB software shall limit converted red channel gain to sensor defined maximum value

Status	Draft
Verification Criteria	Verify by test that converted red channel gain doesn't exceed sensor maximum threshold for cold scene frames

MVC-105521 - AWB Minimum Blue Channel Gain Limitation

The AWB software shall limit converted blue channel gain to sensor defined minimum value

Status	Draft
Verification Criteria	Verify by test that converted blue channel gain doesn't go below sensor minimum threshold for cold scene frames

MVC-105520 - AWB Maximum Blue Channel Gain Limitation

The AWB software shall limit converted blue channel gain to sensor defined maximum value

Status	Draft
Verification Criteria	Verify by test that converted blue channel gain doesn't exceed sensor mximum threshold for warm scene frames

17 Encoder

Note encoder has 2 parameter sets on for stream index 0 (HV-stream) and stream index 1 (CV-stream)

17.1 Static Encoder Config

MVC-107674 - Encode Standard H.264/AVC.

The application software shall configure the encoder to encode the video stream 0 and 1 using H.264/AVC.

Status	Draft
Verification Criteria	

MVC-107675 - Encode Window Resoltion Stream 0 (HV-Stream)

The application software shall configure the encoder window resolution on index 0 to the encode window resolution of ROI preset 9.

Status	Draft
Verification Criteria	

MVC-107676 - Encode Window Resoltion Stream 1 (CV-Stream)

The application software shall configure the encoder window resolution on index 1to the encode window resolution of ROI preset 10.

Status	Draft
Verification Criteria	

MVC-107677 - Encode Framecrop Stream 0 (HV-Stream)

The application software shall configure the encoder framecrop on index 0 to the encode framecrop of ROI preset 9.

Status	Draft
Verification Criteria	

MVC-107678 - Encode Framecrop Stream 1 (CV-Stream)

The application software shall configure the encoder framecrop on index 1 to the encode framecrop of ROI preset 10.

Status	Draft
Verification Criteria	

MVC-107679 - Encode Framerate Stream 0 (HV-Stream)

The application software shall configure the encoder framecrop on index 0 to the encode framecrop of ROI preset 9.

Status	Draft
Verification Criteria	

MVC-107680 - Encode Framerate Stream 1 (CV-Stream)

The application software shall configure the encoder framerate on index 1 to the encode framerate of ROI preset 10.

Status	Draft
Verification Criteria	

MVC-107682 - Encode Bitrate Stream 0 (HV-Stream)

The application software shall configure the encoder bitrate on index 0 to the bitrate framerate of ROI preset 9.

Status	Draft
Verification Criteria	

MVC-107681 - Encode Bitrate Stream 1 (CV-Stream)

The application software shall configure the encoder bitrate on index 1 to the bitrate framerate of ROI preset 10.

Status	Draft
Verification Criteria	

MVC-107683 - Encode GOP Structure

The application software shall configure the encoder GOP structure to 0 on both streams.

Note: This in absence of temporal prediction between frames.

Status	Draft
Verification Criteria	

MVC-107684 - Encode GOP Structure Parameter M

The application software shall configure the encoder GOP parameter M to 1 on both streams.

Note: This results in a GOP structure without B-frames.

Status	Draft
Verification Criteria	

MVC-107685 - Encode GOP Structure Parameter N

The application software shall configure the encoder GOP parameter N to 1 on both streams.

Note: This results in every encoded frame being an I-frame.

Status	Draft
Verification Criteria	

MVC-107686 - Encode GOP Structure Parameter IDR Interval

The application software shall configure the encoder IDR interval to 1 on both streams.

Note: This results in every I-frame being encoded as an IDR frame.

Status	Draft
Verification Criteria	

MVC-107687 - Encode H.264 Profile

The application software shall configure the H.264 encoder to use the High Profile (ProfileIDC = 100).

Note: H.264 profile 100, which corresponds to the High Profile. High Profile supports features such as CABAC and is commonly used for high-quality video.

Note: Value randomly chosen, should come from system requirement

Status	Draft
Verification Criteria	

MVC-107688 - Encode H.264 Level

The application software shall configure the H.264 encoder to use Level 4.2 (LevelIDC = 42).

Note: H.264 Level 4.2, defining decoder capability limits such as maximum bitrate, macroblock processing rate, and picture size

Status	Draft
Verification Criteria	

MVC-107689 - Encode CABAC Entropy Coding

The application software shall configure the H.264 encoder to enable CABAC entropy coding (IsCabac = 1).

Note: Enables CABAC (Context-Adaptive Binary Arithmetic Coding) entropy coding, providing better compression efficiency than CAVLC at the expense of higher computational complexity.

Status	Draft
Verification Criteria	

MVC-107690 - Encode Enable Deblocking Filter

The application software shall configure the encoder to enable the in-loop deblocking filter.

Note: The deblocking filter reduces blocking artifacts and improves prediction efficiency.

Status	Draft
Verification Criteria	

MVC-107691 - Encode Deblocking Filter Alpha Offset

The application software shall configure the deblocking filter alpha offset to 0.

Note: The default H.264 alpha filtering strength is used.

Status	Draft
Verification Criteria	

MVC-107692 - Encode Deblocking Filter Beta Offset

The application software shall configure the deblocking filter beta offset to 0.

Note: The default H.264 beta filtering strength is used.

Status	Draft
Verification Criteria	

MVC-107693 - Encode Enable Video Usability Information (VUI)

The application software shall configure the encoder to enable Video Usability Information (VUI).

Note: VUI information is included in the SPS.

Status	Draft
Verification Criteria	

MVC-107694 - Encode VUI Enable Aspect Ratio

The application software shall configure the encoder to signal aspect ratio information.

Note: The encoded stream contains aspect ratio information.

Status	Draft
Verification Criteria	

MVC-107695 - Encode VUI Aspect Ratio Square Pixels

The application software shall configure the encoder aspect ratio to Square Pixels (AspectRatioIDC = 1).

Note: The sample aspect ratio is 1:1.

Status	Draft
Verification Criteria	

MVC-107696 - Encode VUI Enable Video Type

The application software shall configure the encoder to signal the video signal type.

Note: Video signal information is present in the VUI.

Status	Draft
Verification Criteria	

MVC-107697 - Encode VUI Enable Full-Range Video Samples

The application software shall configure the encoder to signal full-range video samples.

Note: Pixel values use the full sample range.

Status	Draft
Verification Criteria	

MVC-107698 - Encode VUI Video Format

The application software shall configure the encoder video format to 5 (Unspecified).

Note: The video format is signalled as unspecified.

Status	Draft
Verification Criteria	

MVC-107699 - Encode VUI Enable Timing Information

The application software shall configure the encoder to signal timing information.

Note: Decoder timing information is included in the VUI.

Status	Draft
Verification Criteria	

MVC-107700 - Encode VUI Enable Framerate

The application software shall configure the encoder to signal a fixed frame rate.

Note: The frame rate remains constant throughout the stream.

Status	Draft
Verification Criteria	

MVC-107701 - Encode VUI Enable NAL HRD

The application software shall configure the encoder to signal NAL HRD parameters.

Note: Hypothetical Reference Decoder information is included for the NAL layer.

Status	Draft
Verification Criteria	

MVC-107702 - Encode VUI Enable VCL HRD

The application software shall configure the encoder to signal VCL HRD parameters.

Note: Hypothetical Reference Decoder information is included for the VCL layer.

Note: Maybe not needed, to be checked

Status	Draft
Verification Criteria	

MVC-107703 - Encode VUI Enable Picture Structure Information

The application software shall configure the encoder to signal picture structure information.

Note: Picture structure information is present in the VUI.

Status	Draft
Verification Criteria	

MVC-107704 - Encode VUI Enable One Recorder Frame

The application software shall configure the encoder to signal one reorder frame.

Note: The decoder may reorder up to one frame.

Status	Draft
Verification Criteria	

MVC-107705 - Encode VUI Enable Maximum Decoded Picture Buffer Size

The application software shall configure the encoder to signal a maximum decoded picture buffer size of 4 frames.

Note: Specifies the decoder buffering capability.

Status	Draft
Verification Criteria	

MVC-107706 - Encode Bitrate Control Mode

The application software shall configure the encoder bitrate control mode to 1.

Note: Selects constant bitrate (CBR) operation according to the encoder API.

Status	Draft
Verification Criteria	

MVC-107707 - Encode Bitrate Quality Level

The application software shall configure the encoder bitrate quality level to 0.

Note: Uses the encoder default quality setting for the selected bitrate mode.

Note: Maybe not needed because of default

Status	Draft
Verification Criteria	

MVC-107709 - Encode Bitstream Buffer Address

The application software shall configure the encoder bitstream address to 0x34A2 C480 on both streams.

Note: This address is fixed and is shared memory buffer between encoder and software.

Both bitstreams are available there and are somehow interleafed

Status	Draft
Verification Criteria	

MVC-107708 - Encode Bitstream Buffer Size

The application software shall configure the encoder bitstream buffer size to 33,554,432 bytes (32 MiB).

Note: Defines the maximum memory available for encoded output data.

Status	Draft
Verification Criteria	

MVC-107710 - Encode Slicing

The application software shall configure the encoder to generate one slice per picture.

Note: Each encoded picture consists of a single slice.

Status	Draft
Verification Criteria	

MVC-107711 - Encode Tiling

The application software shall configure the encoder to generate one tile per picture.

Note: Tile-based picture partitioning is disabled.

Status	Draft
Verification Criteria	

17.2 Dynamic Encoder Config

MVC-107716 - Enable Encoder

The application software shall enable the encoder by activating the responsible clock source.

Note: This is done via AmbaSYS API which internally sets 0x08C of RCT to 0x10

Status	Draft
Verification Criteria	

MVC-107713 - Encode Start Sequence

The application software shall start the encoder by executing the following sequence:

1. invoke encoder config request
2. invoke start request to the dsp encoder
3. wait for the encoder to be started (signaled via event)

Status	Draft
Verification Criteria	

MVC-107714 - Encode Stop Sequence

The application software shall stop the encoder by executing the following sequence:

1. invoke stop request to the dsp encoder
2. wait for the encoder to be stopped (signaled via event)

Status	Draft
Verification Criteria	

MVC-107715 - Encoder Reconfiguration Sequence

The application software shall reconfigure the encoder by executing the following sequence:

1. Stop encoder acc. to MVC-107714 - Encode Stop Sequence
2. Modify the desired values
3. Start encoder acc. to MVC-107713 - Encode Start Sequence

Status	Draft
Verification Criteria	

18 Hardware Abstraction Layer (HAL)

18.1 Microcontroller Interfaces

MVC-93001 - HAL - Microcontroller Interfaces

The uC SW shall control the following microcontroller interfaces by using the corresponding HAL function:

Interface	Connected system component
GPIO	SDRAM, SPI flash, Deserializer, PHY, CAM, Shutter, PMIC
I2C	PMIC, Deserializer, IMU
SPI	external SPI flash
RGMII	PHY
MIPI-CSI2	Deserializer
LPDDR4	SDRAM

Status	Draft
Verification Criteria	

19 Maintenance

We need to achieve several goals in order to allow for effective Software development:

- We need a development build logging mechanism that can be disabled for release builds.
- We need an error handling mechanism that allows for easy mocking while unit testing.
- We need the error handling to allow for the necessary diagnosis features (those are currently undefined)

We do not have the ability to provide fitting stakeholder requirements. So i will try to write simple requirements with the upper goals in mind.

MVC-93055 - Debug build logging

The software shall implement a logging mechanism that is only available in debug builds.

Status	Draft
Verification Criteria	

MVC-95807 - Leistungoptimierung für Minimierung der Hitzeentwicklung

The System on a Chip shall following settings (so the power consumption is alligned/reduced):

- Frequencies settings (reduction to earlier):

Cortex	DDR	Core	IDSP	VPU	SPU
600	1068	552	552	0	0

- Using of just on Cortex core (other disabled)

Status	Draft
Verification Criteria	Test via UART by reading out the frequencies and usage of the cores

MVC-93054 - Error handling

When an error occurs the software shall invoke an error handling mechanism that interfaces to diagnosis and stop software execution, when no other error handling is specified.

Status	Draft
Verification Criteria	

Diagnosis

Mekra will define what, when and how diagnosis is to be done. Solectrix will then be able to implement a feature "diagnosis". Until then Soelctrix will implement

- a debug logging mechanism
- a debug error handling mechanism
- a defined state to enter in case of errors and stop execution in release build

MVC-96449 - Diagnostic output via UART

The application software shall print diagnostic messages via UART.

Status	Draft
Verification Criteria	Connect to UART and wait for logging messages after the board was powered on.

MVC-96453 - Diagnostic output of the heartbeat counter

Each 5 seconds the application software shall print a heartbeat counter via the diagnostic interface.
Each cycle, the heartbeat counter is incremented.

Note:
Output format:
`HeartbeatLogger: <N> beats have passed.`
`<N>`: *Counter that is incremented each cycle.*

Status	Draft
Verification Criteria	Every 5 seconds the heartbeat counter is incremented and printed via the diagnostic interface.

MVC-95798 - Diagnostic output of the SoC temperature

Each 5 seconds the application software shall print the temperature of the System on a Chip (SoC) via the diagnostic interface.

Note:
*Currently the temperature is measured from the SoC **Sensor0 center**.*

Output format:
`SOCtemp<SENSOR> in Celsius <TEMP>`

`<SENSOR>`: *ID of the sensor that measures the temperature on the SOC*
`<TEMP>`: *The measured temperature as a floating number*

Status	Draft
Verification Criteria	Every 5 seconds a temperature value is given out on the diagnostic interface.

MVC-111478 - Diagnostic test interface for I2C

The application software shall provide an interface to simulate I2C communication for testing purposes.

Status	Draft
Verification Criteria	

20 Non-testable items

Ort für eine temporäre Liste an SW-Requirements die das Software Team oder das Testteam für nicht testbar halten.

Entweder finden wir einen Weg diese indirekt zu testen oder wir definieren Informationen die für das Testteam zugänglich gemacht werden muss. Die Idee dazu ist eine UART Schnittstelle mit der man gezielt Informationen auslesen kann.

20.1 UART Format

Das Format für die UART Übertragung könnte so aussehen:

- Generell:
  - Zeichen werden übertragen im ASCII Format,
  - Ein Kommando/Antwort pro Zeile
  - Zeilenende markiert mit `\r`
  - Antwortformat definieren! Todo. Vorschlag: `DATA\r`
    - Bei einem Schreibzugriff ist das einfach ein Echo des Datums, dass geschrieben werden soll, bei einem Lesezugriff ist dies der Wert des gelesenen Registers.
    - Edit_NE: Eventuell wäre eine Bestätigung für erfolgreichen Schreibzugriff interessant (ggf. ist es das Echo?). Ist Rückmeldung zu neg. Schreib-/Lesezugriff machbar (z.B. unbekannte Adresse; Schreiben auf read only; ...)?
- Schreiben: `ww ADDRESS DATA\r`, wobei gilt:
  - ADDRESS ist eine 32 Bit Adresse im hexadezimal Format gepadded mit 0. Adresse 31 wäre dann z.B. `0000001F`
  - DATA ist ein 32 Bit Wert im hexadezimal Format gepadded mit 0. z.B: `01234567`
- Lesen: `rw ADDRESS\r`, wobei gilt:
  - ADDRESS ist eine 32 Bit Adresse im hexadezimal Format gepadded mit 0. Adresse 10 wäre dann z.B. `0000000A`
- Edit_NE: Für höhere Geschwindikeiten (z.B. Zugriff je Frame), sollte die SW auch Adressen & Daten ohne padded 0 annehmen können.

Das ist ein sehr primitives Format, dass wir auch in der Vergangenheit schon benutzt haben und zunächst nur als Vorswchlag zu verstehen.

20.2 Liste an noch nicht testbaren SW-Requirements

Hier bitte zunächst einfach nur eine Liste pflegen.

Liste:

- SW state handling:
  - MVC-96166 - PMIC Configuration
  - MVC-95855 - Service Discovery Startup
  - MVC-95856 - Service Discovery Shutdown
  - MVC-95794 - Service Discovery Startup - Initial Notification
  - MVC-95796 - Service Discovery Startup - Cyclic Repetition
- SOW1 sensor temperature:
  - MVC-92908 - Heating control - Calculate temperature value (t_image)
  - MVC-96464 - Heating control - Startup - Read value between T_imager_auto_off and T_imager_auto_on
  - MVC-92884 - Heating ModeAuto - Turn on condition
  - ...
- SerDes setup

- hier Beispiele (aus MVC-95778) ergänzen

Für jedes dieser Requirements wollen wir uns überlegen wie wir das testen wollen und ob wir dafür die Testschnittstelle benötigen.

20.3 use case für den Einsatz mit dem Testinterface

20.3.1 Manipulation der I2C SerDes-Daten

20.3.1.1 Manipulation nach abgeschlossener Boot Phase

Tests (Beispiel MVC-92920), welche die Auswertung der Temperaturdaten vom Sensor validieren, manipulieren gezielt die am SoC einkommenden Daten. Hierfür wird das Testinterface verwendet, um die vom externen Sensor real ankommenden Daten gezielt zu überschreiben, bevor sie an die SW weitergeben werden.

20.3.1.2 Manipulation während / kurz nach der Boot Phase

Grundsätzlich sind Tests an dieser Schnittstelle (Beispiel SW-RQ MVC-95754) in einer sehr frühen Phase nach dem Booten durchzuführen. Hier ist zu berücksichtigen, dass die Fehlerinjektion ebenfalls in dieser frühen Phase durchführbar sein muss.

20.3.1.3 Bitweise Manipulation

Einige Tests (Beispiel SW-RQ MVC-95754 für Manipulation vom LOCKED-Bits) benötigen die Funktionalität im Testinterface, das Datum bitweise verändert an den Empfänger weitersenden zu können. Es ist somit erforderlich die ursprünglichen Daten maskiert zu überschreiben.

20.3.2 Auswerten der I2C PMIC-Daten

Tests (Beispiel MVC-96166), welche die I2C PMIC-Daten auswerten, benötigen Lesezugriffe auf diese. Ein Schreibzugiff auf die Daten zum PMIC ist nach bisherigem Stand (29.01.2026) nicht für den SWQTest erforderlich.

20.3.3 Auswertung der gPTP-Daten

ToDo (hier ist noch nicht klar, wie getestet werden kann): Tests (Beispiel MVC-93072), welche ...

20.3.4 Auswertung der MDIO-Daten zum ETH-PHY

ToDo (hier ist noch nicht klar, wie auch in der Serie getestet werden kann): Tests (Beispiel MVC-96476), welche die MDIO-Daten auswerten, benötigen Lesezugriffe auf diese. Stand 29.01.2026 ist der Lesezugriff per AmbaShell möglich. Es ist zu klären, ob diese Shell auch in der Finalen SW für den Test erhalten bleibt.

20.4 Definition von UART Zugriffen

Hier bitte die Wünsche für UART Zugriffe beschreiben, die obige Requirements testbar machen würden.

- Wert im Temperatur-Register (atopif_tmc_cal_top) auslesen
- Wert im Temperatur-Register (atopif_tmc_cal_top) manipulieren

20.5 Ideensammlung für die Umsetzung

1. Verwendung einer neuen, **separaten UART** Schnittstelle
   a. Erstes Feedback: Die andere UART ist bereits belegt. Unter Umständen ist diese Lösung aber dennoch möglich.
2. Verwendung der **existierenden UART** & Abschaltbares Logging per UART-Kommando
   a. neg: Während dem Bootvorgang kurzzeitig eingeschrängt, bis das Kommando gesendet wurde.
   b. pos: Flexible Lösung.
3. Verwendung der **existierenden UART** & Abschaltbares Logging per externen Pin (high/low)
   a. neg: Lötarbeit je HW erforderlich => Zeitaufwand je Testaufbau
   b. pos: jeder Zeit einsatzbereit
4. Verwendung der **existierenden UART** & Abschaltbares Logging per UART-Kommando oder externen Pin (high/low)
   a. pos: Flexibel für spätere Testlösungen
   b. neg: Infomationen aus dem "normalen" Logging ist für den Test nicht mehr verfügbar, ohne HW-Wechsel (Fädeldraht).
5. Verwendung der **ETH** Schnittstelle & Abschaltbares Logging per externen Pin (high/low)
   a. Die Verwendung der ETH Schnittstelle verhindert die Kommunikation/Rückmeldungen während des Boot-Vorgangs **==> Idee aktuell nicht mehr Favorisiert**

21 Rejected items

21.1 Reading from SPI Flash

MVC-92767 - Reading from SPI flash - flush the FIFO

The uC SW shall flush the FIFO

Status	✗ Rejected
Verification Criteria	

21.2 Writing to SPI Flash

MVC-92766 - Write to SPI flash - return value

The uC SW shall check the status register of the external SPI flash using the SPI-NOR controller.

Note: See ~~MVC-92726~~ · Reading the status register

(source: IS25LP(WP)01G.pdf Rev A6 table 6.1, p. 21)

Status	✗ Rejected
Verification Criteria	

MVC-92850 - Write to SPI flash - return value handling

The uC SW shall perform a return value handling

TODO: Error detection (evtl. return value, number of retries, timeouts, etc.) and handling

Status	✗ Rejected
Verification Criteria	

21.3 Erasing

MVC-92764 - Erasing operation on SPI flash - return value

The uC SW shall check the status register of the external SPI flash using the SPI-NOR controller.

Note: See ~~MVC-92726~~ · Reading the status register

(source: IS25LP(WP)01G.pdf Rev A6 table 6.1, p. 21)

Status	✗ Rejected
Verification Criteria	

MVC-92788 - Erasing operation on SPI flash - return value handling

The uC SW shall perform a return value handling

Status	✗ Rejected
Verification Criteria	

MVC-98155 - Enable 4 byte addressing

The application software shall configure the SPI-NOR controller driver with the value specified in MVC-94568 to enable 4 byte addressing mode on SPI-Flash.

Status	✗ Rejected
Verification Criteria	Check SPI Trace for 4 byte addresses or refer to integration test

MVC-98166 - SPI-Flash device identification

When entering software state INIT the application software shall check the ManufactuerID and the DeviceID for the values specified in MVC-94573.

Status	✗ Rejected
Verification Criteria	Check SPI for ManfucatuerID(0x9D) and DeviceID(0x1A)

MVC-92857 - Reading

The application software shall configure the SPI-NOR controller driver to use the fast read quad output operation (FRQO = 0x6B) for reading.

Note: This operation includes 8 dummy cycles between transmission of 4 byte address and transmission of read data.
(source: IS25LP(WP)01G.pdf Rev A6 section 8.8, p. 73 and Table 6.11 Read Dummy Cycles, p. 29)

Status	✗ Rejected
Verification Criteria	Verify correct operation on SPI bus line (indirect functional test). Additionally, check correct SPI-NOR controller driver configuration by review.

MVC-113291 - The application software shall configure the SPI-NOR controller driver for 8 dum...

The application software shall configure the SPI-NOR controller driver for 8 dummy cycles.

Status	✗ Rejected
Verification Criteria	

MVC-92773 - Programming

The uC SW shall configure the SPI-NOR controller to use the quad input page program operation (PPQ = 0x32) for programming.
Note: This operation will program the whole page which is 256 bytes
(source: IS25LP(WP)01G.pdf Rev A6 section 8.11, p. 86)

Status	✗ Rejected
Verification Criteria	Check SPI for 0x32.

MVC-98150 - Reset

The application software shall configure the SPI-NOR controller driver to reset the SPI flash according to MVC-94603

Status	✗ Rejected
Verification Criteria	Refer to integration test

MVC-98156 - LSB first addressing

The application software shall configure the SPI-NOR controller driver to address the SPI-Flash using least sigificant bit (LSB) first.

Status	✗ Rejected
Verification Criteria	Check if address is transmitted LSB first on SPI interface.

MVC-92858 - Erasing

The application software shall configure the SPI-NOR controller driver to use the following erase operations:

1. Erasing a sector using sector erase operation (SER = 0x20)
2. Erasing a block using block erase operation (BER64K = 0xD8)

Note:
(source: IS25LP(WP)01G.pdf Rev A6 section 8.13, p. 88 and section 8.14, p. 90)

Status	✗ Rejected
Verification Criteria	Refer to integration test

MVC-92726 - Reading the status register

The application software shall configure the SPI-NOR controller driver to use the read status register operation (RDSR = 0x05) for reading the status register.

Note:
(source: IS25LP(WP)01G.pdf Rev A6 section 8.18, p. 96)

Status	✗ Rejected
Verification Criteria	Check SPI for 0x05.

MVC-92870 - Write enable

The uC SW shall configure the SPI-NOR controller to use the write enable operation (WREN = 0x06) before writing to the external SPI flash.
Note:
(source: IS25LP(WP)01G.pdf Rev A6 section 8.16, p. 94

Status	✗ Rejected
Verification Criteria	Refer to integration test

21.4 PMIC

MVC-97846 - PMIC Read Register

The application software shall read data from a specified I^2C register of the PMIC by performing the following steps:

1. Prepare the I^2C transaction by specifying the PMIC Slave ID and the address of the register to read.
2. Use the I^2C communication protocol (with the specified speed and channel) to send the read request to the PMIC.
3. Receive the data from the specified register and store it in a provided buffer

Status	✗ Rejected
Verification Criteria	Perform read access to a known PMIC register and verify that the correct data volume is received.

Note:
Register overview: see p.82 f. MPQ70165FS-xxxx-AEC1 - Published Datasheet (MPS Rev#{3} 0.8) (002).pdf REGISTER MAP

21.5 Heating

21.5.1 Heating Modes

MVC-99829 - Heating Control - Manual Mode Off - No Change

While in Manual Mode Off, when the application software receives the Service Method `setCamControl` with camControlIndex = 0x09, it shall not change its mode for the following payloads:

- controlMode = 0x02 (ModeManual)
- controlMode = 0x03 (ModeManualModified) AND controlValue = 0x00

Status	✗ Rejected
Verification Criteria	Send SOME/IP Method setCamControl with correct payloads during Manual Mode Off. Check that the heater remains off and check internal state via getCamControl.

MVC-99833 - Heating Control - Manual Mode Off - Change to Manual Mode On

While in Manual Mode Off, when the application software receives the Service Method `setCamControl` with camControlIndex = 0x09, it shall change to Manual Mode On if cotrolMode = 0x03 (ModeManualModified) AND controlValue = 0x01.

Status	✗ Rejected
Verification Criteria	Send SOME/IP setCamControl with correct payloads during Manual Mode Off. Check that the heater turns on and check internal state via getCamControl.

MVC-99832 - Heating Control - Manual Mode On - No Change

While in Manual Mode On, when the application software receives the Service Method `setCamControl` with camControlIndex = 0x09, it shall not change its mode for the following payloads:

- controlMode = 0x02 (ModeManual)

- controlMode = 0x03 (ModeManualModified) AND controlValue = 0x01

Status	❌ Rejected
Verification Criteria	Send SOME/IP Method setCamControl with correct payloads during Manual Mode On. Check that the heater remains on and check internal state via getCamControl.

MVC-99834 - Heating Control - Manual Mode On - Change to Manual Mode Off

While in Manual Mode On, when the application software receives the Service Method `setCamControl` with camControlIndex = 0x09, it shall change to Manual Mode Off if controlMode = 0x03 (ModeManualModified) AND controlValue = 0x00.

Status	❌ Rejected
Verification Criteria	Send SOME/IP setCamControl with correct payloads during Manual Mode On. Check that the heater turns off and check internal state via getCamControl.

MVC-99838 - Heating Control - Automatic Mode - Change to Manual Mode Off

While in Automatic Mode, when the application software receives the Service Method `setCamControl` with camControlIndex = 0x09, it shall change to Manual Mode Off if:

- controlMode = 0x02 (ModeManual) AND it is in Automatic Mode Off
- controlMode = 0x03 (ModeManualModified) AND controlValue = 0x00

Status	❌ Rejected
Verification Criteria	Send SOME/IP setCamControl with ModeManual during Automatic Mode Off. Check that the heater remains off and check internal state via getCamControl. Repeat Test with Automatic Mode (any) with ModeManualModified and controlValue 0x00.

MVC-99837 - Heating Control - Automatic Mode - Change to Manual Mode On

While in Automatic Mode, when the application software receives the Service Method `setCamControl` with camControlIndex = 0x09, it shall change to Manual Mode On if:

- controlMode = 0x02 (ModeManual) AND it is in Automatic Mode On
- controlMode = 0x03 (ModeManualModified) AND controlValue = 0x01

Status	❌ Rejected
Verification Criteria	Send SOME/IP setCamControl with ModeManual during Automatic Mode On. Check that the heater remains on and check internal state via getCamControl. Repeat Test with Automatic Mode (any) with ModeManualModified and controlValue 0x01.

MVC-99876 - Heating Control - Automatic Mode - No Change

While in Automatic Mode, when the application software receives the Service Method `setCamControl` with camControlIndex = 0x09, it shall not change its mode if controlMode = 0x04 (ModeAuto).

Status	❌ Rejected
Verification Criteria	Send SOME/IP setCamControl with ModeAuto during Automatic Mode (any). Check that the heater remains as is and check getCamControl for no changes to before.

MVC-92874 - Heating control - Automatic heating mode

While in manual mode, when the application software receives the service request setCamControl with (controlMode = 0x04: ModeAuto), the application software shall switch heater control to automatic mode.

Status	❌ Rejected
Verification Criteria	Check if automatic mode can be entered from manual mode with the correct setCamControl request.

MVC-99893 - Heating Control - setCamControl - Invalid Control Mode

When the application software receives a `setCamControl` Service Method with camControlIndex = 0x09, it shall send an Error Message with Return Code `E_INVALID_CONTROL_MODE` if the value of controlMode is **not** one of the following:

- 0x02 (ModeManual)
- 0x03 (ModeManualModified)
- 0x04 (ModeAuto)

Status	❌ Rejected
Verification Criteria	Try invalid values for controlMode, expect SOME/IP ERROR Message being returned with Return Code E_INVALID_CONTROL_MODE.

MVC-99897 - Heating Control - setCamControl - Invalid Control Value

When the application software receives a `setCamControl` Service Method with camControlIndex = 0x09 and controlMode = 0x03 (ModeManualModified), it shall send an Error Message with Return Code `E_INVALID_CONTROL_VALUE` if the value of controlValue is **neither** 0x00 (Heating Off) **nor** 0x01 (Heating On).

Status	❌ Rejected
Verification Criteria	Try invalid values for controlValue, expect SOME/IP ERROR Message being returned with Return Code E_INVALID_CONTROL_VALUE.

21.6 Shutter

21.6.1 Shutter modes

MVC-92045 - Shutter control - Operation mode - Enable automatic mode

When the uC SW receives a SOME/IP setCamControl message for the automatic mode then the uC SW shall enable the automatic mode.

Status	❌ Rejected
Verification Criteria	Set the shutter control state via SOME/IP to automatic mode. Verify shutter reaction while streaming and while not streaming.

MVC-92031 - Shutter control - Operation mode - Enable manual mode

When the uC SW receives a SOME/IP setCamControl message for the manual mode then the uC SW shall enable the manual mode.

Status	❌ Rejected
Verification Criteria	Send SOME/IP messages and see if shutter reacts

MVC-92037 - Shutter control - SOME/IP setCamControl message - Manual mode modified

The uC SW shall accept SOME/IP setCamControl messages for the manual mode modified with all of the following properties:

- `controlIndex` set to `0x50` (Shutter)
- `controlMode` set to `ModeManualModified`
- `controlValue` set to:
  - `0` to close shutter
  - `1` to open shutter

Status	❌ Rejected
Verification Criteria	Send SOME/IP messages and see if shutter reacts

MVC-92732 - Shutter control - SOME/IP setCamControl message - Error response message - Invalid camControlIndex

If the uC SW receives a SOME/IP setCamControl message with an `camControlIndex` that is out of range or not supported then the uC SW shall send an error response message with the error code `E_INVALID_CONTROL_INDEX`.

Status	❌ Rejected
Verification Criteria	Send a SOME/IP setCamControl message with an camControlIndex that is out of range or not supported and check if an error response message with the error code E_INVALID_CONTROL_INDEX was returned.

MVC-92044 - Shutter control - SOME/IP setCamControl message - Manual mode

The uC SW shall accept SOME/IP setCamControl messages for the manual mode with all of the following properties:

- `controlIndex` set to `0x50` (shutter)
- `controlMode` set to `ModeManual`
- `controlValue` is ignored

Status	❌ Rejected
Verification Criteria	Send a SOME/IP setCamControl message and verify behavior of the shutter.

MVC-92046 - Shutter control - Operation mode - Enable manual mode modified

When the uC SW receives a SOME/IP setCamControl message for the manual mode modified then the uC SW shall enable the manual mode modified.

Status	❌ Rejected
Verification Criteria	Send SOME/IP messages and see if shutter reacts

21.7 SOME/IP

21.7.1 General

21.7.1.1 Data Structures

MVC-95632 - Supported Service IDs

The application software shall support the following values for `Service ID`:

- 0x433F = 17215

Status	❌ Rejected
Verification Criteria	Ensure that values other than described here are not supported by the application software

MVC-95633 - Supported Method IDs

The application software shall support the following values for `Method ID`:

- 0x0001 - getDataSheet
- 0x0010 - getCamIdentification
- 0x0131 - subscribeRoiVideo
- 0x0132 - unSubscribeRoiVideo

Status	❌ Rejected
Verification Criteria	Ensure that values other than described here are not supported by the application software

MVC-95635 - Supported Protocol Versions

The applicaiton software shall support the following values for `Protocol Version`:

- 0x01

Status	❌ Rejected
Verification Criteria	Ensure that values other than described here are not supported by the application software

MVC-95634 - Supported Interface Versions

The application software shall support the following values for `Interface Version`:

- 0x01

Status	❌ Rejected
Verification Criteria	Ensure that values other than described here are not supported by the application software

MVC-95637 - Supported Message Types

The applicaiton software shall support the following values for `Message Types`:

- 0x00 - `REQUEST`
- 0x80 - `RESPONSE`
- 0x81 - `ERROR`

Status	❌ Rejected
Verification Criteria	Ensure that values other than described here are not supported by the application software

MVC-95636 - Supported Message Types - Incoming

The application software shall only support incoming Messages of type `REQUEST`.

Status	❌ Rejected
Verification Criteria	Ensure that values other than described here are not supported by the application software for incoming SOME/IP Messages

MVC-95630 - Supported Message Types - Outgoing

The application software shall only support outgoing Messages of type `RESPONSE` and `ERROR`.

Status	❌ Rejected
Verification Criteria	Ensure that values other than described here are not used by the application software for sending SOME/IP Messages

MVC-95631 - Supported Return Codes

The application software shall support the following values for `Return Codes` for all Messages:

- 0x00 - `E_OK`
- 0x01 - `E_NOT_OK`
- 0x02 - `E_UNKNOWN_SERVICE`
- 0x03 - `E_UNKNOWN_METHOD`
- 0x04 - `E_NOT_READY`
- 0x05 - `E_NOT_REACHABLE`
- 0x06 - `E_TIMEOUT`
- 0x07 - `E_WRONG_PROTOCOL_VERSION`
- 0x08 - `E_WRONG_INTERFACE_VERSION`
- 0x09 - `E_MALFORMED_MESSAGE`

Note: The following Return Codes cannot be actively tested:

- `E_NOT_READY`
- `E_NOT_REACHABLE`
- `E_TIMEOUT`

Status	❌ Rejected
Verification Criteria	Ensure that values other than described here are not supported by the application software

21.7.2 Communication

MVC-95858 - Camera Service Startup

When entering software state OPERATIONAL, the application software shall start listening for incoming UDP Messages on port `cameraServicePort` read from the NVM.

Status	❌ Rejected
Verification Criteria	After the application software obtained a valid IP Address (e.g. via DHCP), send a valid SOME/IP REQUEST message to cameraServicePort. Expect a SOME/IP RESPONSE message to be received from cameraServicePort.

MVC-95857 - Camera Service Shutdown

When exiting software state OPERATIONAL, the application software shall stop listening for incoming SOME/IP Messages on port `cameraServicePort`.

Status	❌ Rejected

Verification Criteria	After the application software looses a valid IP Address (e.g. the IP assigned via DHCP expires without renewal), send a valid SOME/IP REQUEST message to cameraServicePort. Expect no SOME/IP RESPONSE message to be received from cameraServicePort.

MVC-95853 - Camera Service - SOME/IP Response - Message Destination

The Camera Service shall send its SOME/IP `RESPONSE` and `ERROR` messages to the destination IP and Port that sent the initial SOME/IP message.

Status	✗ Rejected
Verification Criteria	Ensure that the application software sends the SOME/IP RESPONSE and ERROR messages to the correct destination

21.7.3 Service Discovery

21.7.3.1 Startup Behaviour

MVC-95792 - Service Discovery Initialisation

When entering software state `PERIPHERAL_CONFIGURATION`, the application software shall initialise the SOME/IP Service Discovery functionality.

Status	✗ Rejected
Verification Criteria	When entering software state PERIPHERAL_CONFIGURATION, ensure that the application software initialises the Service Discovery

MVC-96350 - Service Discovery Initialisation - Purpose

During initialisation the Service Discovery shall allocate all required data structures and read and store the following values from Software Configuration Data:

- `someIpSdConfigurationOption`
- `someIpSdSourcePort`
- `someIpSdDestinationPort`
- `someIpSdDestinationIp`

Status	✗ Rejected
Verification Criteria	During initialisation of the Service Discovery, ensure that the application software reads cameraServicePort from the software configuration data

MVC-95697 - Startup states

The application software shall use the following states during startup of the Service Discovery:

- INITIAL
- REPETITION
- MAIN

Status	✗ Rejected
Verification Criteria	Ensure that the application software goes through all mentioned states during startup

MVC-95698 - Startup states - Initial

When the application software starts the Service Discovery Task, it shall be in the startup state INITIAL.

Status	✗ Rejected
Verification Criteria	Ensure that the application software reaches the startup states in the correct order

MVC-95699 - Startup states - Repetition

After the Service Discovery sends the first OfferService message in startup state INITIAL, it shall switch to state REPETITION.

Status	✗ Rejected
Verification Criteria	Ensure that the application software reaches the startup states in the correct order

MVC-95700 - Startup states - Main

After the Service Discovery sends REPETITION_MAX OfferService Messages in state REPETITION, it shall switch to state MAIN.

Status	✗ Rejected
Verification Criteria	Ensure that the application software reaches the startup states in the correct order

MVC-95817 - Exiting Main

After the Service Discovery reached state MAIN it shall not leave it until the Task is stopped.

Status	✗ Rejected
Verification Criteria	Ensure that the application software does not perform any state changes after reaching MAIN while Service Discovery is running

21.7.4 Camera service

21.7.4.1 Startup behaviour

MVC-95838 - Camera Service Initialisation

When entering software state `PERIPHERAL_CONFIGURATION`, the application software shall initialise the SOME/IP Camera Service functionality by allocating all required data structures, including the following values from the Software Configuration Data:

- `cameraServicePort`

Status	✗ Rejected
Verification Criteria	Ensure that the application software initialises the Camera Service as described when entering software state PERIPHERAL_CONFIGURATION

MVC-96346 - Camera Service Initialisation - Purpose

During initialisation the Camera Service shall allocate all required data structures and read and store the following values from Software Configuration Data:

- `cameraServicePort`

Status	✗ Rejected
Verification Criteria	During initialisation of the Camera Service, ensure that the application software reads cameraServicePort from the software configuration data

21.7.4.2 Method getCamIdentification

MVC-93024 - getCamIdentication - Negative Response

If the application software receives a SOME/IP request message with SOME/IP header fields **Method ID** set to `0x0010`, **payload** unequal `0` and **length** unequal 8,
then the application software shall send a negative SOME/IP response message with the following SOME/IP message header and no payload:

SOME/IP header:

Service ID	Method ID	Length	Protocol Version	Interface Version	Message Type	Return Code
0x433F	0x0010	8 (0x08)	0x01	0x01	0x81	0x09 (E_MALFORMED_MESSAGE)

Status	✗ Rejected
Verification Criteria	Client sends an invalid SOME/IP message (message with payload unequal 0) with methodID getCamIdentificaiton then application software responds with a negative response as described in requirement

21.7.5 Method getCamStatus

MVC-107718 - getCamStatus - Method validation
When the application software receives a valid SOME/IP request message with SOME/IP header fields **Method ID** set to `0x0002`, **Length** set to `8` and **no payload**.

The application software shall accept the request as valid.

Status	✗ Rejected
Verification Criteria	This requirement shall be verified in conjunction with Requirements that yield a response message by application software.

MVC-95694 - getCamStatus - Positive Response
When the application software receives a valid getCamStatus SOME/IP request, a response with message type `RESPONSE (0x80)`, return code `E_OK (0x00)` and the following fields as payload shall be sent:

ByteIndex	Output Parameter	BaseType	Description
0	lockedBy	uint32	User of the camera (e.g. IP address) is set/unset by specific methods (i.e. MVC-95703 - setCamExclusive - Storing request IP address)
4	powerStatus	uint8	Value mirrored to the camera API, but managed by the generic device
5	temperature	uint16	Actual temperature of the camera

Status	✗ Rejected
Verification Criteria	Send a valid getCamStatus message and check SOME/IP response message.

21.7.5.1 Method getDataSheet

MVC-95689 - getDatasheet - SOME/IP Payload Field - deviceIdentification - Encoding
The application software shall encode the field `deviceIdentification` as UTF-8.

Note: As long as provided data is encoded as ASCII, which is a subset of UTF-8 as 1 Byte per character then the software does not need to perform UTF-8 encoding.

Status	✗ Rejected
Verification Criteria	

MVC-95655 - getDatasheet - SOME/IP Payload Field - standardVersion
The application software shall include the field `standardVersion` of type `uint32` as part of the payload in the positive response message of the method `getDatasheet`.

Status	✗ Rejected
Verification Criteria	Response message payload at byte index 256 contains a 4 byte value which is 0x00000001

MVC-95657 - getDatasheet - SOME/IP Payload Field - imagerCharacteristic
The application software shall include the field `imagerCharacteristic` of type `sImagerCharacteristic` as part of the payload in the positive response message of the method getDatasheet.

Status	✗ Rejected
Verification Criteria	Response message payload at byte index 260 up to byte index 304 is filled with zeros

MVC-95660 - getDatasheet - SOME/IP Payload Field - intrinsicCamParam
The application software shall include the field `intrinsicCamParam` of type `sIntrinsicCamParam` as part of the payload in the positive response message of the method getDatasheet.

Status	✗ Rejected
Verification Criteria	Response message payload at byte index 305 up to byte index 404 is filled with zeros

MVC-95663 - getDatasheet - SOME/IP Payload Field - extrinsicCamParam
The application software shall include the field `exstrincsicCamParam` of type `sExtrinsicCamParam` as part of the payload in the positive response message of the method getDatasheet.

Status	✗ Rejected
Verification Criteria	Response message payload at byte index 405 up to byte index 500 is filled with zeros

MVC-95671 - getDatasheet - SOME/IP Payload Field - defectPixelMap
The application software shall include the field `defectPixelMap` of type `sPixelMap` as part of the payload in the positive response message of the method getDatasheet.

Status	✗ Rejected
Verification Criteria	Response message payload at byte index 501 up to byte index 504 is filled with zeros

MVC-95674 - getDatasheet - SOME/IP Payload Field - numOfTemperatures
The application software shall include the field `numOfTemperatures` of type `uint32` as part of the payload in the positive response message of the method getDatasheet.

Status	✗ Rejected
Verification Criteria	Response message payload at byte index 505 contains a 4 byte value which is 0x00000001

MVC-95677 - getDatasheet - SOME/IP Payload Field - numOfRegionOfInterest
The application software shall include the field `numOfRegionOfInterest` of type `uint32` as part of the payload in the positive response message of the method getDatasheet.

Status	✗ Rejected
Verification Criteria	Response message payload at byte index 509 contains a 4 byte value which is defined in NVM

MVC-95680 - getDatasheet - SOME/IP Payload Field - maxLengthOfPersistentEntry
The application software shall include the field `maxLengthOfPersistentEntry` of type `uint32` as part of the payload in the positive response message of the method getDatasheet.

Status	✗ Rejected
Verification Criteria	Response message payload at byte index 513 up to byte index 516 is filled with zeros

21.7.5.2 Method SubscribeROIVideo

MVC-96358 - SubscribeROIVideo - Message validation
When the application software receives a SOME/IP request message with SOME/IP header fields **Method ID** set to `0x0131`, **Length** set to `12` and the following field as payload:

ByteIndex	Input Parameter	BaseType	Description
0	roiIndex	uint32	Index of the requested ROI

then it shall validate all SOME/IP protocol fields as described in MVC-95604 - Validation sequence and roiIndex as described in MVC-95735 - subscribeRoiVideo - Payload Validation - roiIndex.

Status	✗ Rejected
Verification Criteria	

MVC-96315 - SubscribeROIVideo - roiIndex check - negative
If the application software evaluates the roiIndex to be invalid it shall not change any of the video streams before sending the `ERROR` message with Return Code `E_INVALID_ROI_INDEX`, independent of whether any streams are running or not.

Status	✗ Rejected
Verification Criteria	When sending a SOME/IP SubscribeROIVideo REQUEST with an invalid ROI Index, ensure that the application software does not change any streaming states before sending an ERROR Message with Return Code E_INVALID_ROI_INDEX

MVC-96314 - SubscribeROIVideo - roiIndex check - positive - stream not running
if the application software evaluates the roiIndex to be valid and the requested stream is not running, it shall start the requested stream and store the Client ID of the SOME/IP `REQUEST` message before sending the `RESPONSE` message.

Status	✗ Rejected

Verification Criteria	

MVC-96317 - SubscribeROIVideo - roiIndex check - positive - stream running - same client

If the application software evaluates the roiIndex to be valid and the requested stream is already running, it shall stop and start the requested stream before sending the `RESPONSE` message, if the Client ID of the SOME/IP `REQUEST` message is the same as the one stored for that stream.

Status	✗ Rejected
Verification Criteria	

MVC-96318 - SubscribeROIVideo - roiIndex check - positive - stream running - different client

If the application software evaluates the roiIndex to be valid and the requested stream is already running, it shall not change the video stream and send an `ERROR` message with Return Code `E_NOT_OK`, if the Client ID of the SOME/IP `REQUEST` message is not the same as the one stored for that stream.

Status	✗ Rejected
Verification Criteria	

MVC-95749 - SubscribeROIVideo - SOME/IP Payload Field - roiIndex - evaluation - Input parameter - Settings - T3 Settings

If the `roiIndex` is evaluated as valid and T3 Settings are selected then the application software shall execute the Display Resolution, Panning, Zooming, and Framerate adjustments.

Status	✗ Rejected
Verification Criteria	Check if supported binary value 0b111 is considered a valid input parameter which yields a valid roiIndex if all other input parameters are valid.

MVC-95752 - SubscribeROIVideo - SOME/IP Payload Field - roiIndex - evaluation - Input parameter - Framerate

The application software shall support the following binary values:

- 0b00 = 60 Hz
- 0b01 = 30 Hz
- 0b10 = 15 Hz
- 0b11 = 1 Hz

Status	✗ Rejected
Verification Criteria	Check if supported binary values are considered a valid input parameter which yields a valid roiIndex if all other input parameters are valid.

MVC-95753 - SubscribeROIVideo - SOME/IP Payload Field - roiIndex - evaluation - Input parameter - Zooming

The application software shall support the following binary values:

- 0b000 = 100%
- 0b001 = 150%
- 0b010 = 200%
- 0b011 = 300%
- 0b100 = 400%

All other binary values are considered invalid.

Status	✗ Rejected
Verification Criteria	Check if supported binary values are considered a valid input parameter which yields a valid roiIndex if all other input parameters are valid.

MVC-95757 - SubscribeROIVideo - SOME/IP Payload Field - roiIndex - evaluation - Input parameter - Panning

The application software shall support the following binary values:

From `0b0000000` (decimal: 0) for Position 0 to `0b1010000` (decimal: 80) Position 80

All other binary values are considered invalid.

Status	✗ Rejected
Verification Criteria	Check if supported binary values are considered a valid input parameter which yields a valid roiIndex if all other input parameters are valid.

MVC-95756 - SubscribeROIVideo - SOME/IP Payload Field - roiIndex - evaluation - Input parameter - Display resolution

The application software shall support the following binary values:

- 0b000000000 = 1920 x 1536
- 0b000000001 = 1920 x 1080
- 0b000000010 = 1280 x 720
- 0b000000011 = 952 x 536
- 0b000000100 = 640 x 360
- 0b000000101 = 1920 x 720

All other binary values are considered invalid.

Status	✗ Rejected
Verification Criteria	Check if supported binary values are considered a valid input parameter which yields a valid roiIndex if all other input parameters are valid.

MVC-95760 - SubscribeROIVideo - SOME/IP Payload Field - roiIndex - evaluation - Input parameter - Stream selection

The application software shall support the following binary values:

- 0b00 = HV ISP + IP Config 1
- 0b01 = HV ISP + IP Config 2

All other binary values are considered invalid.

Status	✗ Rejected
Verification Criteria	Check if supported binary values are considered a valid input parameter which yields a valid roiIndex if all other input parameters are valid.

21.8 Video streaming

21.8.1 Video stream/s

MVC-95985 - UnSubscribeROIVideo - Stop of stream

The application software shall stop a video stream to the destination IP based on configuration when a valid UnSubscribeROIVideo SOME/IP request (MVC-95808 - UnSubscribeROIVideo - Positive Response - no camera stream/s running) is received.

Possible configurations are:
MVC-95980 - RTP - Destination IP/Port 1 Configured
MVC-95983 - RTP - Destination IP 1 Unconfigured
MVC-95982 - RTP - Destination IP/Port 2 Configured
MVC-95975 - RTP - Destination IP 2 Unconfigured

Status	✗ Rejected
Verification Criteria	Subscribe to a video stream and check network trace that RTP packets are received to the correct configured destination ip. Send a valid SOME/IP UnsubscribeROIVideo message and check network trace that no RTP packets are transmitted to configured destination ip address.

MVC-95770 - SubscribeROIVideo - State switch

The application software shall switch from system state Standby (~~MVC-92864~~ - System state: Standby) to system state Running (~~MVC-92762~~ - System state: Running) when a valid SubscribeROIVideo SOME/IP request is received.

TODO: Check whether this requirement is needed since states are not final.
TODO: Hier soll das State Handling von RUNNING und STREAMING nach /wiki/40_System_Allocation_Requirements/System_Allocation_Requirements_SW?selection=MVC-94742 spezifiziert werden.

Status	✗ Rejected

Verification Criteria	

Für ganzes substateHandling (RUNNING/STREAMING): Was ist mit zwei Streams?

Idee: Die Substates RUNNING und STREAMING sind eine im Streaming Kontext relevant, deshalb hier eingeführt.
In some ip:
Hier Bezug zum globalen State Handling:
When entering software state OPERATIONAL, the application software shall switch to streaming-substate RUNNING
Beschreibung des lokalen (Sub)State Handlings:
While in software substate RUNNING, when receiving valid subscribeROI, the application software shall switch to substate STREAMING.
While in software substate STREAMING, when receiving valid unsubscribeROI **mit roi von laufendem Stream(?),** the application software shall switch to substate RUNNING

Im Video Streaming:

While in streaming the software shall packetize encoded video data according to:

- 8.2-2 - RTP packet structure
- 8.2-2.2 - RTP packet field - sequence number
- ... SW-Reqs aus 8.2 verlinken

Notiz von Lawrence: RQs zu "streaming-substates" sollten im Kapitel Video Streaming und nicht im Kapitel SOME/IP liegen, da die SOME/IP nichts davon weis bzw. wissen soll.

MVC-100831 - Supported ROI Presets
The application software shall support the ROI presets 0, 1, 2, 3, 4, 5, 9 and 10 for video streaming.
Note: Implementation status -> implemented
Currently, it is not possible to add a link to the relevant system requirements, so the allocation is listed here:

- *MVC-100013 - Supported ROIIndexes*
- *MVC-100010 - Overview - Implemented ROIs*
- *MVC-100035 - subscribeRoiVideo - Reception*

Status	❌ Rejected
Verification Criteria	Send SubscribeROIVideo request with ROI presets 0, 1, 2, 3, 4, 5, 9 and 10. Verify that each request is accepted and that the corresponding video stream starts.

MVC-100830 - Unsupported ROI Presets
The application software shall reject requests for unsupported ROI presets with return code E_INVALID_ROI_INDEX.
Note: Implementation status -> implemented
Currently, it is not possible to add a link to the relevant system requirements, so the allocation is listed here:

- *MVC-100013 - Supported ROIIndexes*
- *MVC-100010 - Overview - Implemented ROIs*
- *MVC-100035 - subscribeRoiVideo - Reception*

Status	❌ Rejected
Verification Criteria	Send SubscribeROIVideo request with unsupported ROI presets, (e.g. 6, 7, 8). Verify that the software returns E_INVALID_ROI_INDEX and that no video stream is started.

21.8.2 Video Streaming Functions

MVC-96508 - Flip / Mirror Functionality - Application of Flip/Mirror Based on Sensor Orientation
The Application Software shall verify and apply mirror/flip settings in accordance with sensor´s physical orientation and the coordinate system definition.

Status	❌ Rejected
Verification Criteria	Check if flip/mirror settings are aligned with the sensor orientation and coordinate system.

MVC-96479 - Zoom Functionality - Use of Zoom API Interface
The application software shall use the Ambarella image kernel API in order to zoom the image.
Note: The application software shall create and pass a AMBA_IK_DZOOM_INFO_s structure to the image kernel API AmbaIK_SetDzoomInfo() via SvcViewCtrl module.

Status	❌ Rejected
Verification Criteria	Verify that the Application Software calls the Ambarella image kernel API to trigger zoom operations.

MVC-96485 - Pan Functionality - Use of Panning API Interface
The application software shall use the Ambarella image kernel API in order to pan the image.
Note: The application software shall create and pass a AMBA_IK_DZOOM_INFO_s structure to the image kernel API AmbaIK_SetDzoomInfo() via SvcViewCtrl module.

Status	❌ Rejected
Verification Criteria	Verify that the Application Software calls the Ambarella image kernel API to trigger pan operations.

21.9 gPTP

MVC-92310 - Destination MAC address for Pdelay_Req messages
The uC SW shall send the Pdelay_Req messages to the LLDP multicast MAC address (MAC = 01:80:C2:00:00:0E)

Status	Draft
Verification Criteria	

MVC-92315 - Pdelay_Req message transmission
When the uC SW sends the Pdelay_Req message, it shall execute the following steps:

1. read and store T_PHY_PDREQ_EG from PHY
2. start Timer to wait for Pdelay_Resp (????? should be discussed)

For SW-Tester: Contains details of architecture/detailed design which are not testable - likely to be reworked

Status	Draft
Verification Criteria	

MVC-92316 - Pdelay_Resp message handling
When the uC SW receives a Pdelay_Resp message, whose SequenceID matches the previously sent Pdelay_Req message, it shall execute the following steps:

1. Read and store T_PHY_PDRESP_IG from PHY
2. Extract and store T_NB_PDRESP_PL from the payload of the Pdelay_Resp message
3. Calculate and store the neighborRateRatio using formula (F1)
4. start Timer to wait for Pdelay_Resp_Follow_Up (????? should be discussed)

For SW-Tester: Contains details of architecture/detailed design which are not testable - likely to be reworked

Status	Draft
Verification Criteria	

MVC-92319 - Pdelay_Resp_Follow_Up message handling
When the uC SW receives a Pdelay_Resp_Follow_Up message, whose SequenceID matches the previously sent Pdelay_Req message, it shall execute the following steps:

1. Extract and store T_NB_PDRESPFOLLOWUP_PL from the payload of the Pdelay_Resp_Follow_Up message
2. Calculate and store the neighborPropDelay using formula (F2)

For SW-Tester: Contains details of architecture/detailed design which are not testable - likely to be reworked

Status	Draft
Verification Criteria	

MVC-92324 - SYNC message handling

When the uC SW receives a SYNC message, it shall execute the following steps:

1. Read and store T_PHY_SYNC_IG from PHY
2. Start Timer to wait for Follow_Up (????? should be discussed)

Note: T_PHY_SYNC_IG can be used to calculate the time offset to the grandmaster. As were just setting the absolute time this timestamp will eventually not be needed

For SW-Tester: Contains details of architecture/detailed design which are not testable - likely to be reworked

Status	Draft
Verification Criteria	

MVC-92326 - Calculation of responderRateRatio

The uC SW shall calculate the responderRateRatio using the following formula (F3):

$$\text{responderRateRatio} = \text{cumulativeScaledRateOffset} \cdot 2^{-41} + 1 \qquad (F3)$$

Note: The value of cumulativeScaledRateOffset is equal to (rateRatio - 1) · (2^{41}), truncated to the next smaller signed integer, where rateRatio is the ratio of the frequency of the grandmaster to the frequency of the responder (neighbor of SOW2-ECU) that sends the message.

The above scaling allows the representation of fractional frequency offsets in the range [-(2^{-10} - 2^{-41}); 2^{-10},2^{-41}],

with granularity of 2^{-41}. This range is approximately [–9.766 · 10^{-4}; 9.766 · 10^{-4}].

Status	Draft
Verification Criteria	

MVC-92325 - Follow_Up message handling

When the uC SW receives a Follow_up message, it shall execute the following steps:

1. Read and store T_PHY_FOLLOWUP_IG from PHY
2. Extract cumulativeScaledRateOffset from the TLV of the message, which is located at byte offset 83 of the header and 4 bytes long
3. Calculate and store responderRateRatio using formula (F3)
4. Calculate and store rateRatio using formula (F4)
5. Calculate and store T_DIFF using formula (F5)
6. Extract and store correctionField from the header of the Follow_Up message
7. Extract and store T_GM_FOLLOWUP_PL from the payload of the Follow_Up message
8. Calculate and store corrected synchronized time T using formula (F6)
9. Set the system time to the value T
10. Adjust the counting frequency of the clock by rateRatio

For SW-Tester: Contains details of architecture/detailed design which are not testable - likely to be reworked

Status	Draft
Verification Criteria	

21.10 FSYNC

21.10.1 Ethernet Control

MVC-100770 - FSync - Ethernet Control - getCamControl

When the application software receives a `getCamControl` Service Method with camControlIndex = 0x62, it shall return the following values for the payload:

- controlSupportedMode = 0x01 (`StatRW_nA_nO`)
- controlMode = 0x02 (ModeManual)
- controlValue = [0x0000..0x8234]

Note: The default value mentioned in ~~MVC-100289~~ · controlMode and controlValue as output parameter - FrameSync will never be returned

Status	Rejected
Verification Criteria	Set specific frameSyncTimeRef with SOME/IP setCamControl Messsage, send SOME/IP getCamControlMessage and check if the correct values are returned

MVC-100771 - FSync - Ethernet Control - setCamControl

When the application software receives a `setCamControl` Service Method with camControlIndex = 0x62, it shall reconfigure the FSync timing *'frameSyncTimeRef'* applied to the sensor to the new rasterization of time using the value in controlValue as offset to 1s of GVTtime, if:

- controlSupportedMode = any
- controlMode = 0x03 (ModeManualModified)
- controlValue = [0x0000..0x8234]

Status	Rejected
Verification Criteria	Set specific frameSyncTimeRef with SOME/IP setCamControl Messsage, send SOME/IP getCamControlMessage to validate with the obtained controlValue

MVC-100772 - FSync - Ethernet Control - setCamControl - Invalid Control Mode

When the application software receives a `setCamControl` Service Method with camControlIndex = 0x62, it shall respond with a SOME/IP ERROR message with Return Code **E_INVALID_CONTROL_MODE**, if controlMode != 0x03 (ModeManualModified).

Status	Rejected
Verification Criteria	Send SOME/IP setCamControl Message with camControlIndex = 0x62 and controlMode != 0x03, expect SOME/IP ERROR message with Return Code E_INVALID_CONTROL_MODE

MVC-100773 - FSync - Ethernet Control - setCamControl - Invalid Control Value

When the application software receives a `setCamControl` Service Method with camControlIndex = 0x62, it shall respond with a SOME/IP ERROR message with Return Code **E_INVALID_CONTROL_VALUE**, if controlMode == 0x03 (ModeManualModified) and controlValue outside of valid range [0x0000..0x8235].

Status	Rejected
Verification Criteria	Send SOME/IP setCamControl Message with camControlIndex = 0x62, controlMode == 0x03 and controlValue outside of valid range, expect SOME/IP ERROR message with Return Code E_INVALID_CONTROL_VALUE

21.11 External SPI Flash Memor

MVC-92872 - Configuration of SPI-NOR controller

The uC SW shall configure the SPI-NOR controller for the following functionality:

1. Disable SM error detection

Offset	Name	Bits	Field	Reset	Val	Descripton
0x5C	SPI-NOR Fault Injection	1	sm_error_disable	0x0	0x1	0x1 = disable SM error (TBD)

2. Set DMA channel to SPI-NOR RX and TX
3. Set SPI-NOR clock divider to match 100 MHz

Offset	Name	Bits	Field	Reset	Val	Descripton
0x08	SPI-NOR Flow Control	17:10	CLKDIVIDER	0x1E	0xXX	to be calculated

4. Disable multiple chip select

Offset	Name	Bits	Field	Reset	Val	Descripton
0x08	SPI-NOR Flow Control	25:18	CHIPSELECT#	0xFE	0x00	disable CEN for multiple devices

5. Enable flow control

Offset	Name	Bits	Field	Reset	Val	Descripton
0x08	SPI-NOR Flow Control	31	FLOWCONTROL	0x1	0x1	0x1 = enable flow control

6. Configure hold clock for flow control

Offset	Name	Bits	Field	Reset	Val	Descripton
0x08	SPI-NOR Flow Control	30:28	HOLD	0x0	0x0	0x0 = gate clock to perform flow control

7. Set RX sample delay

Offset	Name	Bits	Field	Reset	Val	Descripton
0x08	SPI-NOR Flow Control	4:0	RXSAMPLEDELAY	0x0	0x3	0x3 = 3 clock cycle shift

8. Reset FIFO

Offset	Name	Bits	Field	Reset	Val	Descripton
0x40	SPI-NOR TX FIFO Reset	0	TXFIFORESET	0x0	0x1	0x1 = Software reset the TX FIFO
0x44	SPI-NOR RX FIFO Reset	0	RXFIFORESET	0x0	0x1	0x1 = Software reset the RX FIFO

9. Set TX FIFO threshold level

Offset	Name	Bits	Field	Reset	Val	Descripton
0x20	SPI-NOR RX FIFO Threshold	8:0	RXFIFOTHLV	0x0	0x1F	0x1F = RX FIFO Threshold level

10. Set RX FIFO threshold level

Offset	Name	Bits	Field	Reset	Val	Descripton
0x1C	SPI-NOR TX FIFO Threshold	8:0	TXFIFOTHLV	0x0	0x1F	0x1F = TX FIFO Threshold level

11. Configure the SPI flash to operate in 4 byte addressing mode by setting EXTADD=1 by issuing Enter 4-byte Address Mode (EN4B = 0xB7)

Status	❌ Rejected
Verification Criteria	

MVC-92760 - Reading from SPI flash

The uC SW shall read data using the SPI-NOR controller by executing the following sequence:

1. Configure DMA
2. Clear the remain data on receive FIFO if not empt

Offset	Name	Bits	Field	Reset	Val	Descripton
0x40	SPI-NOR TX FIFO Reset	0	TXFIFORESET	0x0	0x1	0x1 = Software reset the TX FIFO
0x44	SPI-NOR RX FIFO Reset	0	RXFIFORESET	0x0	0x1	0x1 = Software reset the RX FIFO

3. Set the SPI-NOR Command Format Restister

Offset	Name	Bits	Field	Reset	Val	Descripton
0x00	SPI-NOR Command Format Restister	31:30	CMDLENGTH	0x1	0x1	command length
		29:27	ADDRLENGTH	0x3	0x4	address length = 4 byte
		26:22	DUMMYLENGTH	0x0	0x7	8 dummy cycles
		21:0	DATALENGTH	0x18	0xX	length to be read

4. Set the SPI-NOR Address Register

Offset	Name	Bits	Field	Reset	Val	Descripton
0x14	SPI-NOR Address Register	31:0	ADDR3:0	0x0	0xX	address to be read

5. Set the SPI-NOR Command Register according to ~~MVC-92857~~ - Reading

Offset	Name	Bits	Field	Reset	Val	Descripton
0x0C	SPI-NOR Command Register	7:0	CMD0	0x0	0x6B	0x6B = FRQO

6. Set threshold to receive FIFO

Offset	Name	Bits	Field	Reset	Val	Descripton
0x20	SPI-NOR RX FIFO Threshold	8:0	RXFIFOTHLV	0x0	0x1F	0x1F = RX FIFO Threshold level

7. Enable DMA

Offset	Name	Bits	Field	Reset	Val	Descripton
0x18	SPI-NOR DMA Enable	1:0	TXFIFORESET	0x0	0x3	0x3 = enable TX and RX DMA

8. Set transaction done interrupt mask

Offset	Name	Bits	Field	Reset	Val	Descripton
0x30	SPI-NOR Interrupt Mask	5	DATALENTHREACHMSK	0x0	0x1	0x1 = Mask this interrupt

9. Enable transmit/receive bit

Offset	Name	Bits	Field	Reset	Val	Descripton
0x50	SPI-NOR TRX Start	0	STRTTRX	0x0	0x1	0x1 = Start transaction

10. Wait for interrupt transaction done interrupt
11. Read FIFO

Offset	Name	Bits	Field	Reset	Val	Descripton
0x200 - 0x2FF	SPI-NOR RX Data	31:0	RXDATA	-	-	Data

Status	❌ Rejected
Verification Criteria	

MVC-92728 - Writing to SPI flash

The uC SW shall write data to the external SPI flash using the SPI-NOR controller by executing the following sequence:

1. Sending write enable command according to ~~MVC-92870~~ - Write enable
2. Configure DMA
3. Clear the remain data on receive FIFO if not empt

Offset	Name	Bits	Field	Reset	Val	Descripton
0x40	SPI-NOR TX FIFO Reset	0	TXFIFORESET	0x0	0x1	0x1 = Software reset the TX FIFO
0x44	SPI-NOR RX FIFO Reset	0	RXFIFORESET	0x0	0x1	0x1 = Software reset the RX FIFO

4. Set the SPI-NOR Command Format Restister

Offset	Name	Bits	Field	Reset	Val	Descripton
0x00	SPI-NOR Command Format Restister	31:30	CMDLENGTH	0x1	0x1	command length
		29:27	ADDRLENGTH	0x3	0x4	address length = 4 byte
		26:22	DUMMYLENGTH	0x0	0x0	0 dummy cycles

Offset	Name	Bits	Field	Reset	Val	Descripton
		21:0	DATALENGTH	0x18	0xX	length to be written

5. Set the SPI-NOR Address Register

Offset	Name	Bits	Field	Reset	Val	Descripton
0x14	SPI-NOR Address Register	31:0	ADDR3:0	0x0	0xX	address to be written

6. Set the SPI-NOR Command Register according to ~~MVC-92773~~ - Programming

Offset	Name	Bits	Field	Reset	Val	Descripton
0x0C	SPI-NOR Command Register	7:0	CMD0	0x0	0x32	0x32 = PPQ

7. Set threshold to transmit FIFO

Offset	Name	Bits	Field	Reset	Val	Descripton
0x1C	SPI-NOR TX FIFO Threshold	8:0	TXFIFOTHLV	0x0	0x1F	0x1F = TX FIFO Threshold level

8. DMA??

Offset	Name	Bits	Field	Reset	Val	Descripton
0x18	SPI-NOR DMA Enable	1:0	TXFIFORESET	0x0	0x3	0x3 = enable TX and RX DMA

9. Write FIFO

Offset	Name	Bits	Field	Reset	Val	Descripton
0x100 - 0x1FF	SPI-NOR TX Data	31:0	TXDATA	-	-	Data

10. Set transaction done interrupt mask

Offset	Name	Bits	Field	Reset	Val	Descripton
0x30	SPI-NOR Interrupt Mask	5	DATALENTHREACHMSK	0x0	0x1	0x1 = Mask this interrupt

11. Enable transmit/receive bit

Offset	Name	Bits	Field	Reset	Val	Descripton
0x50	SPI-NOR TRX Start	0	STRTTRX	0x0	0x1	0x1 = Start transaction

12. Wait for interrupt transaction done interrupt
13. Check if TXFIFO is empty and if not delay until it is empty

Status	✗ Rejected
Verification Criteria	

MVC-92912 - Erasing operation on SPI flash

The uC SW shall erase data on the external SPI flash using the SPI-NOR controller by executing the following sequence:

1. Sending write enable command according to ~~MVC-92870~~ - Write enable
2. Set the SPI-NOR Command Format Restister

Offset	Name	Bits	Field	Reset	Val	Descripton
0x00	SPI-NOR Command Format Restister	31:30	CMDLENGTH	0x1	0x1	command length
		29:27	ADDRLENGTH	0x3	0x4	address length = 4 byte
		26:22	DUMMYLENGTH	0x0	0x0	0 dummy cycles
		21:0	DATALENGTH	0x18	0x0	length

3. Set the SPI-NOR Address Register

Offset	Name	Bits	Field	Reset	Val	Descripton
0x14	SPI-NOR Address Register	31:0	ADDR3:0	0x0	0xX	start address of sector to be erased

4. Set the SPI-NOR Command Register according to ~~MVC-92858~~ - Erasing

Offset	Name	Bits	Field	Reset	Val	Descripton
0x0C	SPI-NOR Command Register	7:0	CMD0	0x0	0xXX	0x21 = SER, 0xD8 = BER64

5. Set transaction done interrupt mask

Offset	Name	Bits	Field	Reset	Val	Descripton
0x30	SPI-NOR Interrupt Mask	5	DATALENTHREACHMSK	0x0	0x1	0x1 = Mask this interrupt

6. Enable transmit/receive bit

Offset	Name	Bits	Field	Reset	Val	Descripton
0x50	SPI-NOR TRX Start	0	STRTTRX	0x0	0x1	0x1 = Start transaction

7. Wait for interrupt transaction done interrupt

Status	✗ Rejected
Verification Criteria	

21.12 NVM

21.12.1 NVM - Production data block

MVC-95867 - NVM production data block - macAddress

The application software shall store the value at address `0x03C2 0000` with a length of `6` bytes without conversion in RAM as **macAddress**.

Status	✗ Rejected
Verification Criteria	

MVC-95869 - NVM production data block - batchNumber

The application software shall store the value at address `0x03C2 0006` with a length of `10` bytes as ASCII string in RAM as **batchNumber**.

Status	✗ Rejected
Verification Criteria	

MVC-95872 - NVM production data block - partNumber

The application software shall store the value at address `0x03C2 0010` with a length of `12` bytes as ASCII string in RAM as **partNumber**.

Status	✗ Rejected
Verification Criteria	

MVC-95874 - NVM production data block - partRev

The application software shall store the value at address `0x03C2 001C` with a length of `2` bytes without conversion in RAM as **partRev**.

Status	✗ Rejected
Verification Criteria	

MVC-95876 - NVM production data block -serialNumber

The application software shall store the value at address `0x03C2 001E` with a length of `8` bytes as ASCII string in RAM as **serialNumber**.

Status	❌ Rejected
Verification Criteria	

MVC-95877 - NVM production data block - Timestamp

The application software shall store the value at address `0x03C2 0026` with a length of `8` bytes as ASCII string in RAM as **Timestamp**.

Status	❌ Rejected
Verification Criteria	

MVC-95878 - NVM production data block - epShort

The application software shall store the value at address `0x03C2 002E` with a length of `4` bytes as ASCII string in RAM as **epShort**.

Status	❌ Rejected
Verification Criteria	

MVC-95879 - NVM production data block - hwVer

The application software shall store the value at address `0x03C2 0032` with a length of `4` bytes as ASCII string in RAM as **hwVer**.

Status	❌ Rejected
Verification Criteria	

21.12.2 NVM - Software configuration block

MVC-95907 - NVM software configuration block - someIpSdConfigurationOption

The application software shall read the values at address `0x03C3 0000` with a length of up to 100 bytes as ASCII strings. The values shall be referred to as **someIpSdConfigurationOption**.

Each entry shall follow the format:

- 1 byte: Length (n)
- n bytes: ASCII data

For the final entry:

- 1 byte: Length (n)
- n bytes: ASCII data
- 1 byte: Null terminator ('\0')

Status	❌ Rejected
Verification Criteria	

MVC-95910 - NVM software configuration block - someIpSdSourcePort

The application software shall read the value at address `0x03C3 0100` with a length of `2` bytes without conversion. The value shall be referred to as **someIpSdSourcePort**.

Status	❌ Rejected
Verification Criteria	

MVC-95909 - NVM software configuration block - someIpSdDestinationPort

The application software shall read the value at address `0x03C3 0102` with a length of `2` bytes without conversion. The value shall be referred to as **someIpSdDestinationPort**.

Status	❌ Rejected
Verification Criteria	

MVC-95911 - NVM software configuration block - someIpSdDestinationIp

The application software shall read the value at address `0x03C3 0104` with a length of `4` bytes without conversion. The value shall be referred to as **someIpSdDestinationIp**.

Status	❌ Rejected
Verification Criteria	

MVC-95912 - NVM software configuration block - cameraServicePort

The application software shall read the value at address `0x03C3 0108` with a length of `2` bytes without conversion. The value shall be referred to as **cameraServicePort**.

Status	❌ Rejected
Verification Criteria	

MVC-95958 - NVM software configuration block - shutterSupported

The application software shall read the value at address `0x03C3 010A` with a length of `1` byte without conversion. The value shall be referred to as **shutterSupported**.

Status	❌ Rejected
Verification Criteria	

MVC-95959 - NVM software configuration block - T_imager_auto_on

The application software shall read the value at address `0x03C3 010B` with a length of `2` bytes without conversion. The value shall be referred to as **T_imager_auto_on**.

Status	❌ Rejected
Verification Criteria	

MVC-95960 - NVM software configuration block - T_imager_auto_off

The application software shall read the value at address `0x03C3 010D` with a length of `2` bytes without conversion. The value shall be referred to as **T_imager_auto_off**.

Status	❌ Rejected
Verification Criteria	

MVC-95962 - NVM software configuration block - T_imager_protection_on

The application software shall read the value at address `0x03C3 010F` with a length of `2` bytes without conversion. The value shall be referred to as **T_imager_protection_on**.

Status	❌ Rejected
Verification Criteria	

MVC-95961 - NVM software configuration block - T_imager_protection_off

The application software shall read the value at address `0x03C3 0111` with a length of `2` bytes without conversion. The value shall be referred to as **T_imager_protection_off**.

Status	❌ Rejected
Verification Criteria	

MVC-95863 - NVM software configuration block - numbersOfRegionOfInterest

The application software shall read the value at address `0x03C3 0113` with a length of `1` byte without conversion. The value shall be referred to as **numbersOfRegionOfInterest**.

Status	❌ Rejected
Verification Criteria	

MVC-95963 - NVM software configuration block - RtpSourcePort1

The application software shall read the value at address `0x03C3 0114` with a length of `2` bytes without conversion. The value shall be referred to as **RtpSourcePort1**.

Status	❌ Rejected
Verification Criteria	

MVC-95964 - NVM software configuration block - RtpSourcePort2

The application software shall read the value at address `0x03C3 0116` with a length of `2` bytes without conversion. The value shall be referred to as **RtpSourcePort2**.

Status	❌ Rejected
Verification Criteria	

MVC-95965 - NVM software configuration block - RtpDestinationIp1

The application software shall read the value at address `0x03C3 0118` with a length of `4` bytes without conversion. The value shall be referred to as **RtpDestinationIp1**.

Status	❌ Rejected
Verification Criteria	

MVC-95966 - NVM software configuration block - RtpDestinationIp2

The application software shall read the value at address `0x03C3 011C` with a length of `4` bytes without conversion. The value shall be referred to as **RtpDestinationIp2**.

Status	❌ Rejected
Verification Criteria	

MVC-95880 - NVM software configuration block - ParVer

The application software shall read the value at address `0x03C3 0128` with a length of `4` bytes as ASCII string. The value shall be referred to as **ParVer**.

Status	❌ Rejected
Verification Criteria	

MVC-92748 - PHY managment via MDIO

The uC SW shall manage the PHY using the Management Date Input/Output Interface (MDIO)

Status	❌ Rejected
Verification Criteria	

MVC-95800 - SerDes - Verification - MIPI Data Flow Calculation

The application software shall verify that valid MIPI data is sent from the deserializer's MIPI PHY to the SoC's VIN by at least two sequencial reads of both the following two registers

1. `csi2_tx1_pkt_cnt[3:0]` (bit 3 until bit 0) in register `MIPI_PHY18 (0x342)`
2. `phy1_pkt_cnt[3:0]` (bit 7 until bit 4) in register `MIPI_PHY20 (0x344)`

Status	❌ Rejected
Verification Criteria	Check I2C trace, there shall be at least two read accesses to each of the respective registers.

MVC-92873 - System state: IP-config

While in system state IP-config, the uC SW shall cover the following functionality:

1. Setup of network by execution of network mechanisms such as:
    a. Initiate gPTP machanisms according to 9.5 - Master/Slave Synchronization & Syntonization
    b. Request an ip-address by the DHCP server
    c. open sockets for SOME/IP and RTP

Note: Here we should wait until the DHCP server acknowledges the requested IP-address and until the sockets are opened. Allthought the gPTP synchronization is considered to be faster then DHCP mechanism, we should wait until we are syntonized and synchronized.

Status	❌ Rejected
Verification Criteria	

MVC-92864 - System state: Standby

While in system state Standby, the uC SW shall cover the following functionality:

1. Execution of cyclic network mechanisms such as gPTP, SOME/IP SD, DHCP
2. Controlling of the heating and the shutter

Note: All components have been fully initialized and configured, and the software is now waiting for stream enable

Status	❌ Rejected
Verification Criteria	

MVC-92762 - System state: Running

While in system state Running, the uC SW shall cover the following functionality:

1. Execution of cyclic network mechanisms such as gPTP, SOME/IP SD, DHCP
2. Controlling of the heating
3. Streaming of at least one video stream via RTP

Note: This state is typically entered from Standby, with the transition usually triggered by a subscribeROI SOME/IP command. In this state, the shutter shall be opened.

Status	❌ Rejected
Verification Criteria	

22 Functional Safety Software Specifications

22.1 Ambarella Chip SW Safety Requirements

22.1.1 BIST TSRs

MVC-111609 - Enabling the LBIST mechanism

the software shall enable the LBIST mechanism to run after each powerup

Status	🕒 Ready for review
Verification Criteria	Test shall ensure that the LBIST mechanism is configured as enable

MVC-111228 - LBIST Vector Transfer to IST SRAM (SM_HW_LBIST)

The software shall copy the LBIST vector data from the off chip non volatile memory (NAND flash) into the In System Test (IST) SRAM after power on reset.
Note : It is possible to run some LBIST controllers at chip shutdown to reduce boot time. If any LBIST controller is run at shutdown, the status information needs to be stored in RTC and read back and checked during boot process.

Status	🕒 Ready for review
Verification Criteria	Tests shall ensure that the software copies the LBIST vector data from the off-chip NAND Flash into the IST SRAM after every power-on reset by putting a breakpoint in the LBIST routine to make sure the routine is called at startup..

MVC-111229 - LBIST Controller Activation (SM_HW_LBIST)

The software shall write the copied LBIST vectors to a separate CONTROL register to kick off IST for each LBIST controller in the SoC

Status	🕒 Ready for review
Verification Criteria	Tests shall ensure that the software write the copied the LBIST vectors to the IST CONTROL register to initiate execution for each LBIST controller.

MVC-111608 - Enabling the MBIST mechanism

the software shall enable the MBIST mechanism to run after each powerup

Status	Ready for review
Verification Criteria	Test shall ensure that the MBIST mechanism is configured as enable

MVC-111233 - Reading LBIST and MBIST Completion Status (SM_HW_LBIST, SM_HW_PO_MBIST)

The software shall read the IST STATUS register to determine the completion and result of the LBIST and MBIST operations.

Status	Ready for review
Verification Criteria	Tests shall ensure that the software reads the IST STATUS register after test execution to determine the completion status and result of the LBIST and MBIST operations.

MVC-111232 - Reading LBIST and MBIST Completion Status (SM_HW_LBIST, SM_HW_PO_MBIST)

If the IST STATUS register indicates a PASS status for all LBIST and MBIST tests, then the software shall continue with normal operation.

Status	Ready for review
Verification Criteria	Tests shall ensure that the software continues with normal operation only when the IST STATUS register indicates a PASS status for all LBIST and MBIST tests.

MVC-111234 - Reaction to LBIST and MBIST Failure (SM_HW_LBIST, SM_HW_PO_MBIST)

If the IST controller status is FAIL, then the software shall go to safe state within 200ms.

Status	Ready for review
Verification Criteria	Tests shall ensure that the software notifies the CEHU through the AMBA bus and transitions the system to the defined safe state when the IST controller reports a FAIL status.

MVC-111237 - Reading CEHU Initial Safety Error Status (SM_SW_CHK_SAFETY_ERR)

After every power on reset or global reset, and before clearing any error flags, the software shall read the CEHU registers and ECRU registers to capture the initial safety error state of all components.

Status	Ready for review
Verification Criteria	Tests shall ensure that the software reads all required CEHU and ECRU registers after every power-on reset or global reset and before clearing any error flags to capture the initial safety error state by reviewing the reset handling source code to confirm that the first operations after reset are reads of the CEHU and ECRU registers..

MVC-111236 - Verification of Expected Post Reset Error Vector Values(SM_SW_CHK_SAFETY_ERR)

The software shall compare the values read from the CEHU error_bit_vector registers and

ECRU registers against the expected post reset pattern.

The safety error status register in CEHU should be all ones except:

• Bit [37:32]: WDT external safety errors=> These signals may be used as reset signals to external element(s), and thus is waived to avoid interfering other element(s). SW can check WDT internal safety errors at Bit 0 to Bit 5 instead.

• Bit 51: APB_DBG timeout error =>This timeout error is not flop output. SW shall check APB_DBG safety error flag at bit 31 instead.

• Bit 94: ijtag safety error =>This is not flop output but gated version of xx_jtag_rst_l which is tied to low externally.

• Bit 102: OTP safety error =>This is cleared by VROM boot code.

• Bit 107: SW-ARM safety error => This is the register used by SW only. Instead of checking initial safety error state of SW-ARM safety error, SW can check bit-102 of AMBA_ECRU safety error status, which is at the same reset domain

• Bit 108: test-mode safety error =>This is not flop output, but wired to xx_test_mode, which is tied to low externally.

• Bit 113: VROM safety error =>This is cleared by VROM boot code.

Status	Ready for review
Verification Criteria	Tests shall ensure that the software compares the values read from the CEHU error_bit_vector registers and ECRU registers with the expected post-reset pattern before entering normal operation - Inject a corrupted value to a selected CEHU or ECRU register- Run the software again- Verify that CEHU error is reported..

MVC-111239 - Reaction to Inconsistent Initial Safety Error State (SM_SW_CHK_SAFETY_ERR)

If the values read from the CEHU or ECRU registers are inconsistent with the expected post reset state, then the software shall go to safe state within 200ms.

Status	Ready for review
Verification Criteria	Tests shall ensure that the software prevents entry into operational mode and transitions the system to the defined safe state when the CEHU or ECRU register values do not match the expected post-reset pattern - Use the debugger to simulate a failure indicated to a selected CEHU or ECRU register- Run the software again- Verify that the system goes to safe state within time constraint.

MVC-111238 - Clearing of Initial Safety Error Status (SM_SW_CHK_SAFETY_ERR)

If the initial safety error state matches the expected pattern, then the software shall clear the CEHU and ECRU error status registers to prepare for runtime error monitoring.

Status	Ready for review
Verification Criteria	Tests shall ensure that the software clears the CEHU and ECRU error status registers only after confirming that the initial safety error state matches the expected post-reset pattern - Boot the system normally- After comparison is done successfully, Read the current values of the CEHU/ECRU registers - After the clear, read the registers again, verify the CEHU/ECRU registers are now cleared..

22.1.2 Clock TSRs

MVC-111243 - Initialization of the hardware clock monitors (SM_HW_CKMON)

The Software shall initialize hardware clock monitors by:

(1) verifying clock monitors error flags to CEHU after power-on reset,

(2) clearing CEHU clock monitor error bits,

(3) programming PLLs and clock dividers,

(4) setting each clock monitor's upper_bound and low_bound thresholds via configuration registers,

(5) programming CEHU to receive errors,

(6) enabling each clock monitor.

Status	Ready for review
Verification Criteria	Tests shall ensure that the software performs the complete hardware clock monitor initialization sequence after power-on reset, including verification of CEHU error flags, clearing CEHU clock monitor error bits, programming PLLs and clock dividers, configuring the upper_bound and low_bound thresholds, enabling CEHU error reporting, and enabling all required hardware clock monitors.

MVC-111242 - Reaction of the stuck clocks (SM_HW_CKMON)

the software shall configure the clock failure in the CEHU error to be handled by assert the safety interrupt

Status	Ready for review
Verification Criteria	Tests shall ensure that the safety interrupt is configured to handle the clock errors.

MVC-111241 - Reaction of the stuck clocks (SM_HW_CKMON)

If clock failure is detected, then, then the software shall transition into SafeState within 200ms.

Status	Ready for review
Verification Criteria	Tests shall ensure that the software transitions the system into the defined Safe State within 200 ms after detecting a stuck clock condition by the following steps - Run system with all clocks at nominal frequencies.- Read the CEHU error register; verify no clock monitor errors - Decrease the frequency of the monitored clock until it falls below the threshold- Start a timer immediately- Measure the time from threshold crossing to error assertion.- Verify the time constraint

MVC-111245 - Nodes to be enabled (SM_HW_CKMON)

The software shall enable clock monitoring for the following nodes:

- PLL outputs
- Clock divider outputs
- Cluster level clock gating cell output
- Right before Cortex-R52 CLKIN input
- Clock to main logic of lockstep / duplicate function

Status	Ready for review
Verification Criteria	Tests shall ensure that clock monitoring is enabled for all specified clock nodes, including PLL outputs, clock divider outputs, cluster-level clock gating outputs, the Cortex-R52 CLKIN input, and the lockstep/duplicate-function logic clock.

MVC-111244 - Steps during startup test (SM_SW_CKMON_TEST)

During startup test, The software shall test each hardware clock monitor by:

(1) enabling clock source,

(2) setting clock monitor upper_bound to zero and lower_bound to max via configuration registers,

(3) programming CEHU to receive error,

(4) Enable each clock monitor

(5) verifying CEHU receives the error.

(6) If the software receives error signal (within 99ms), the the software shall clear the relevant (CEHU error source)bits.

Note: Waiting for Amba ticket #406

Status	Ready for review
Verification Criteria	Tests shall ensure that the software performs the startup diagnostic for each hardware clock monitor by configuring the specified threshold values, enabling the monitor, injecting the diagnostic condition, and verifying that the corresponding error is reported to the CEHU for each clock monitor:- start the system- pause the system- make sure clock source is anabled - set clock monitor upper_bound to zero and lower_bound to max via configuration registers - resume start up - observe CEHU receives an error within 99ms- observe the relevant (CEHU error source)bits has been cleared.

MVC-111247 - Reaction on the missing receiving error in CEHU (SM_SW_CKMON_TEST)

If error from clock monitor not received in CEHU, then the software shall assert safety interrupt.

Status	Ready for review
Verification Criteria	Tests shall ensure that the software asserts the specified safety pin when the expected clock monitor error is not received by the CEHU.

MVC-111246 - Reaction on the missing receiving error in CEHU (SM_SW_CKMON_TEST)

If error from clock monitor not received in CEHU, then the software shall shall transition into SafeState within 200ms.

Status	Ready for review
Verification Criteria	- Before the test runs, disable the clock monitor's ability to report errors- Resume execution and allow the test to run- Verify that no error appears in CEHU- observe SoC SW transitions the System into SafeState wihin time constraints

22.1.3 Power TSRs

MVC-111249 - System reaction on power monitor reported error (SM_HW_PSMON)

Before SM_HW_PSMON configuration, the software shall configure the CEHU SAFETY_MODE
register, such that if a power monitor error is reported to CEHU, then the software shall propagate the
error to the SoC safety pin

Status	Ready for review
Verification Criteria	Tests shall ensure that the software configures the CEHU SAFETY_MODE register before SM_HW_PSMON initialization and verifies that power monitor errors reported to the CEHU are propagated to the SoC safety pin.

MVC-111248 - Initialization the Hardware Power Supply Monitor (SM_HW_PSMON)

The software shall initialize the SoC internal Hardware Power Supply Monitor by:

(1) Setup of the the related power monitor register (power down register, disable control register)

(2) Selecting the input voltage sensing levels for each power rail

(3) Programming the under voltage and over voltage comparison reference levels

(4) Clearing old result by toggling the reset control register.

(5) Enabling the sticky error function

The following power rails used by SoC shall be monitored:

- PMIC buck 1 (SoC VDDI)=> Nominal: 0.75V, Minimum: 0.73V, Maximum: 0.77V
- PMIC buck 2 (SoC VDDI)=> Nominal: 0.75V, Minimum: 0.73V, Maximum: 0.77V
- PMIC buck 3 (SoC DDR_DVDD_VDDQ, SoC DDR_DVDD_IO)=> Nominal: 1.1V, Minimum: 1.06V, Maximum: 1.17V
- PMIC buck 4 (SoC AVDD)=> Nominal: 0.8V, Minimum: 0.78V, Maximum: 0.825V
- PMIC buck 5 (SoC AVDD18)=> Nominal: 1.8V, Minimum: 1.7V, Maximum: 1.9V

Status	Ready for review
Verification Criteria	Tests shall ensure that the software initializes the SoC Hardware Power Supply Monitor by configuring the required power monitor registers, programming the specified voltage thresholds, enabling the sticky error function, and configuring monitoring for all specified power rails with the defined voltage limits.

MVC-111251 - Perform a latent fault test (SM_SW_PSMON_TEST)

The software shall perform a latent fault test for each SoC internal hardware power monitor once per driving cycle (power on test) by:

(1) Place the selected power monitor into test mode.

(2) Force an out of range condition by programming illegal under voltage/over voltage thresholds.

(3) Trigger a measurement cycle.

(4) Reading the observation status register and the CEHU safety error status register.

(5) Verify that the expected error is correctly reported to the CEHU.

(6) Repeat above procedure for 4 times.programming illegal under voltage/over voltage thresholds.

Status	Rejected
Verification Criteria	Tests shall ensure that the software performs the latent fault test for each SoC hardware power monitor once per driving cycle by executing the complete diagnostic sequence, verifying that the expected error is reported to the CEHU, and repeating the test four times.

MVC-111250 - Reaction on the missing receiving error in CEHU (SM_SW_PSMON_TEST)

If an error from any SoC powered up internal power supply monitor is not received by CEHU immediately after the test is performed, then the software shall assert the safety pin within 50ms.

Status	Draft
Verification Criteria	Tests shall ensure that the software asserts a safety error through the CEHU and transitions the system to the defined safe state when the expected power monitor error is not reported by the CEHU immediately after completion of the diagnostic test.

22.2 DRAM Safety Requirements

MVC-111258 - Configurations of DRAM WDT (SM_HW_DRAM_WDT)

The software shall program WATCHDOG_TIMER_0_CONFIG and WATCHDOG_TIMER_1_CONFIG registers with 0xFFFF

Status	Ready for review
Verification Criteria	Tests shall ensure that the software programs the WATCHDOG_TIMER_0_CONFIG and WATCHDOG_TIMER_1_CONFIG registers with the value 0xFFFF during the initialization.

MVC-111257 - Configurations of the the hardware ECC (SM_HW_HECC)

The software shall set the HW ECC engine enable bit REG_ECCH_MODE to enable the hardware SECDED ECC function for the external DRAM interface during the system startup.

Status	Ready for review
Verification Criteria	Tests shall ensure that the software sets the REG_ECCH_MODE enable bit to activate the hardware SECDED ECC function for the external DRAM interface during system startup.

MVC-115027 - If an uncorrectable error (double-bit or multi-bit error) is fault is reported t...

If an uncorrectable error (double-bit or multi-bit error) is fault is reported to CEHU, the
software shall go to safe state within 200m

Status	Ready for review
Verification Criteria	- Use debugger to simulate an double-bit error reported to CEHU.- Verify the safe state is reached within the time constraint.

MVC-115026 - The configurations of Data RAM segmentation (SM_HW_DRAM_SEGMENTATION)

The software shall configure the Data RAM segmentation hardware registers of the SoC during the startup phase

Status	Ready for review
Verification Criteria	- Check by reading back the Data RAM segmentation register configuration

MVC-111259 - Allocate non-overlapped memory region for DRAM segmentation(SM_HW_DRAM_SEGMENTATION)

The software shall configure the DRAM memory segmentation (one segment is 256 KB) using a DRAM memory map that explicitly identifies the memory regions allocated to safety-critical (ASIL) and non-safety (QM) software components.

Status	Ready for review
Verification Criteria	Tests shall ensure that the DRAM memory segmentation is configured according to the defined DRAM memory map, with 256 KB segment boundaries, and that the configured ASIL and QM memory regions match the documented allocation.

MVC-111262 - Allocate non-overlapped memory region for DRAM segmentation(SM_HW_DRAM_SEGMENTATION)

The software shall configure the DRAM memory segmentation during system startup before any application-level data processing begins.

Status	Ready for review

Verification Criteria	Tests shall ensure that the DRAM memory segmentation is configured before any application-level data processing begins.

MVC-111261 - Allocate non-overlapped memory region for DRAM segmentation(SM_HW_DRAM_SEGMENTATION)

The software shall initialize the DRAM memory segmentation by invoking the memory protection configuration function during the safety application startup sequence

Status	Ready for review
Verification Criteria	Tests shall ensure that the memory protection configuration function is invoked during the safety application startup sequence to initialize the DRAM memory segmentation.

MVC-111255 - Ensure that each DRAM client is only permitted to access its assigned memory region(s) (SM_HW_DRAM_SEGMENTATION)

The software shall be configured to ensure that each DRAM client can only access its permitted memory region(s).

Status	Ready for review
Verification Criteria	Tests shall ensure that each DRAM client can access only its configured memory region(s) and that access to unauthorized memory regions is prevented.

MVC-111254 - Safe state transition due to the faulty client (SM_HW_DRAM_SEGMENTATION)

The software shall read the REG_DRAM_SEG_FAULT register upon receiving of CEHU interrupt to check if there is segmentation access violation.

Status	Ready for review
Verification Criteria	Tests shall ensure that the software reads the REG_DRAM_SEG_FAULT register upon receipt of a CEHU interrupt to determine whether a DRAM segmentation access violation has occurred by: - After system initialization, select an address outside its assigned region.

MVC-111256 - Safe state transition due to the faulty client (SM_HW_DRAM_SEGMENTATION)

If REG_DRAM_SEG_FAULT is set, then the software shall trigger safe state within 200ms.

Status	Ready for review
Verification Criteria	Tests shall ensure that the software initiates the defined safe state within 200 ms when the REG_DRAM_SEG_FAULT register indicates a DRAM segmentation access violation by: - Execute a read or write instruction to that address.- Read REG_DRAM_SEG_FAULT and verify that the corresponding fault bit is set.- Start timer once corresponding fault bit is set.- Stop the timer when Ethernet transmission is stopped.- Verify the measured time is within limit.

MVC-111264 - Execution of the hardware Address Stuck Test at every startup. (SM_HW_ASTUCK_TEST)

The software shall enable and execute the hardware Address Stuck Test (DTTE_ASTUCK_TEST_EN) every 100ms

Status	Ready for review
Verification Criteria	Tests shall ensure that the software enables the DTTE_ASTUCK_TEST_EN function and executes the hardware Address Stuck Test at intervals of 100 ms.

MVC-111263 - Configuration of the Address Stuck Test Engine (SM_HW_ASTUCK_TEST)

The software shall configure the Address Stuck Test Engine registers (DTTE_DUAL_DIE, DTTE_DUAL_CHANNEL, DTTE_MR15, DTTE_MR20, DTTE_EXPECTED_PATTERN, DTTE_ASTUCK_PERIOD, DTTE_ASTUCK_PERIOD_MSB) according to the specific LPDDR4 device used in the system.

Status	Ready for review
Verification Criteria	Tests shall ensure that the Address Stuck Test Engine registers are configured with values corresponding to the LPDDR4 device used in the target system by : - Initialize the system- Read the Address Stuck Test Engine registers- Verify the values are correct according to LPDDR4 device parameters.

MVC-111253 - Configuration of the Initialization function of the DRAM BIST and Scrubbing Engine.

The software shall configure the DRAM BIST and Scrubbing Engine (DBSE) by setting DBSE_MODE to 0 and DBSE_BIST_PATTERN to 0x1 to initialize all addressable memory in the external DDR device before any client performs a read operation from DRAM that relies on ECC protection.

Status	Ready for review
Verification Criteria	Tests shall ensure that the software configures the DBSE_MODE register to 0 and the DBSE_BIST_PATTERN register to 0x1, and that all addressable DRAM locations are initialized before any ECC-protected DRAM read operation is performed by : - Review boot code to confirm that the DBSE configuration routine is called before any code that accesses DRAM - Verify that the registers are written with the correct values (DBSE_MODE=0, DBSE_BIST_PATTERN=0x1).

22.3 Flash Safety Requirements

MVC-111293 - Calculate and compare checksums with stored checksums at each startup (Startup use case)

The software shall load all safety-relevant code and data sections required for startup verification into RAM before performing integrity verification.

Status	Ready for review
Verification Criteria	Tests shall ensure that all safety-relevant code and data sections required for startup verification are loaded into RAM before the integrity verification process begins.

MVC-111292 - Calculate and compare checksums with stored checksums at each startup (Startup use case)

The software shall calculate a checksum for each loaded safety-relevant code and data section and compare the calculated checksum with the corresponding stored checksum during system startup.

Status	Ready for review
Verification Criteria	Tests shall ensure that the software calculates a checksum for each loaded safety-relevant code and data section and compares it with the corresponding stored checksum during system startup.

MVC-111295 - Calculate and compare checksums with stored checksums at each startup (Startup use case)

The software shall permit the use of safety-relevant code and data only if the calculated checksum matches the corresponding stored checksum.

Status	Ready for review
Verification Criteria	Tests shall ensure that safety-relevant code and data are permitted for execution or use only when the calculated checksum matches the corresponding stored checksum.

MVC-111294 - Checksums error detection at startup (Startup use case)

The software shall compare the calculated checksum against the one in the RAM before using safety-relevant code or data from flash.

Status	Ready for review
Verification Criteria	Tests shall ensure that the software compares the calculated checksum with the checksum stored in RAM before using any safety-relevant code or data from flash memory.

MVC-111297 - Checksums error detection at startup (Startup use case)

If there is CRC-32 mismatch at startup, the software shall indicate a CRC-32 error and shall not start
application SW.

Status	Ready for review
Verification Criteria	Tests shall ensure that the software indicates a CRC-32 error and prevents application software from starting when a CRC-32 mismatch is detected during system startup.

MVC-111296 - Store the data and checksum in flash (Storing use case after startup)

The software shall store the safety-related data together with its CRC-32 checksum in flash memory.

Status	Ready for review
Verification Criteria	Tests shall ensure that the software stores each safety-related data item together with its corresponding CRC-32 checksum in flash memory.

22.4 Deserializer Safety Requirements

22.4.1 Configuartion

MVC-111299 - At system start-up the SoC software shall configure the serializer in the tunelling mode.

At system start-up the software shall configure the serializer in the tunnelling mode.

Status	Ready for review
Verification Criteria	Tests shall ensure that the serializer is configured in tunnelling mode during system start-up by checking the suitable register (EXT11(0x383)) of the serializer if the Tun_Mode bit is set to 1m (i.e.: EXT11(0x383)::Tun_Mode == 1).

22.4.1.1 ERRB Signal

MVC-111328 - At the system startup the SoC SW shall check if the ERRB pin in the deserializer is enabled. If not, the SoC SW shall enable ERRB pin by setting REG5 (0x5)::ERRB_EN := 0b1

At the system startup, the Software shall check if the ERRB pin in the deserializer is enabled. If not, then the software shall enable ERRB pin by setting REG5 (0x5)::ERRB_EN := 0b1

Status	Ready for review
Verification Criteria	Tests shall ensure that the software checks that the ERRB pin in the deserializer is enabled. If not,then the software shall enable ERRB pin by setting REG5 (0x5)::ERRB_EN := 0b1 during system start-up - start the system- pause the system before the check is performed- set REG5 (0x5)::ERRB_EN to 0b0 - resume the system - observe the REG5 (0x5)::ERRB_EN has been set to 0b1.

MVC-111338 - When ERRB becomes disabled, the SoCSW shall...

The software shall check if the ERRB signal is disabled within 100ms

Status	Ready for review
Verification Criteria	Tests shall ensure that the software checks that the ERRB signal is disabled within 100ms during system start-up.

MVC-111321 - When ERRB becomes disabled, the SoCSW shall...

When ERRB becomes disabled, the Software shall transition the system into the SafeState within 100ms.

Status	Ready for review
Verification Criteria	Tests shall ensure that when ERRB becomes disabled, the Software shall transition the system into the SafeState within 100ms - start the system - disable ERRB- observe that system transitioned into safeState within the time constraints.

MVC-116382 - Sending erroneous i2c message to the deserializer

After the boot-up/configuration phase the software shall send erroneous i2c message to the deserializer.

Status	Ready for review
Verification Criteria	- start system - send from SoC erroneous i2c message to the deserializer

MVC-116381 - Receiving CRC error signal

the software shall receive CRC error (ERRB and relevant bits set) within 100ms.

Status	Ready for review
Verification Criteria	- verify the error is signaled by deserializer (ERRB and relevant bit set) . verify time constraints - verify SoC received the aswer and proceeded with normal operation

MVC-116383 - Error Reaction

If the signaling time was exceeded, the software shall transition into SafeState within 300ms.

Status	Ready for review
Verification Criteria	- start system - send from SoC erroneous i2c message to the deserializer - disrupt ERRB connection - verify SoC transitions into SafeState - . verify time constraints

22.4.1.2 VSYNC Signal

MVC-116393 - VSYNC sent via MFP5-pin

The software shall enable the VSYNC indication on the deserlializer's MFP5-pin by setting:
CFG_1 (0x541)[VS_OUT2] := 1
REG5 (0x5)[LOCK_EN] := 0
(i.e.: disabling LOCK being sent to any pins of the deserializer).

Status	Ready for review
Verification Criteria	- start the system- observe VSYNC being sent to the software via MFP5-pin

22.4.1.3 I2C communication - primary channel

MVC-111357 - primary I2C Channel

The software shall communicate with the SOW1 components by means of 'primary-I2C' bus.

Note (BOSCH): "To arrive at the peripherials of the SOW1 (e.g.: imager) the main/primary (MFP9, MFP10) channel should be used from the Deserializer"

Status	Ready for review
Verification Criteria	- start the SOW2 + SOW1 system - try to connect with all components that can be reached from SoC via primary-I2C bus - verfiy that the connection was established Note: for component list and their I2C addresses refer to §3 of "Camera Interface Specification", Version 1.0, Document Number: 0220DN00JX

MVC-111298 - The SoC SW shall check REG1(0x1):: DIS_LOCAL_CC

During start-up the Software shall check if REG1 (0x1):: DIS_LOCAL_CC==0b0.

Status	Ready for review
Verification Criteria	Tests shall ensure that the software checks that REG1 (0x1):: DIS_LOCAL_CC==0b0 during system start-up.

MVC-111334 - The SoC SW shall check REG1(0x1):: DIS_LOCAL_CC

If the check fails then the software shall log a trouble code.

Status	Ready for review
Verification Criteria	Tests shall ensure that the software logs a trouble code when the check fails - start the system - pause the boot-process - set REG1(0x1):: DIS_LOCAL_CC to 1 - resume the boot-process - observe the System logging a trouble code.

MVC-111332 - The SoC SW shall check REG1(0x1):: DIS_REM_CC

During start-up the Software shall check if REG1 (0x1):: DIS_REM_CC==0b0.

Status	Ready for review
Verification Criteria	Tests shall ensure that the software checks that REG1 (0x1):: DIS_REM_CC==0b0 during system start-up.

MVC-111337 - The SoC SW shall check REG1(0x1):: DIS_REM_CC

If the check fails the software shall log a trouble code.

Status	Ready for review
Verification Criteria	Tests shall ensure that the software logs a trouble code when the check fails - start the system - pause the boot-process - set REG1(0x1):: DIS_REM_CC to 1 - resume the boot-process - observe the System logging a trouble code.

MVC-111317 - The primary-I2C bus shall use CRC scheme for the communication.

The software shall use CRC scheme for the communication.

Status	Ready for review
Verification Criteria	Tests shall ensure that the software shall use CRC scheme for the communication.

MVC-115571 - Configuring the threshold for CRC Error Counter

The software shall configure the threshold for the CRC Error Counter in the register I2C_UART_CRC1 in address(0x3009)

Status	Ready for review
Verification Criteria	- consult the relevant part of the source code.

MVC-117360 - Confirm the CRC error after the threshold value

The fault in the CRC shall be confirmed after (three) CRC errors.

Status	Ready for review
Verification Criteria	The test shall ensure that the CRC error is reported after three trials (three sequential errors in the CRC)

MVC-111316 - The SoC SW shall use Message Counter for primary-I2C.

The Software shall use Message Counter for primary-I2C.

Status	Ready for review
Verification Criteria	Tests shall ensure that the Software shall use Message Counter for primary-I2C.

MVC-115572 - Configuring the threshold for Message Counter Error Counter

The software shall configure the threshold for the Message Counter Error counter in the register I2C_UART_CRC1 in address(0x3009)

Status	Ready for review
Verification Criteria	- consult the relevant part of the source code.

MVC-117359 - Confirm the Message counter error after the threshold value

The fault in the message counter shall be confirmed after (three) message counter errors.

Status	Ready for review
Verification Criteria	The test shall ensure that the message counter error is reported after three trials (three sequential errors in the message counter)

MVC-111308 - SoC SW shall check if tunneling mode is still set once per driver cycle.

At the end of the startup phase the Software shall check if tunneling mode is still set.

Status	Ready for review
Verification Criteria	Tests shall ensure that the software checks that tunneling mode is still set during system start-up- start the system- pause the system before check begins- Inject and write to register 0x474 and clear bit 0 to 0 - resume the system.

MVC-111311 - SoC SW shall check if tunneling mode is still set once per driver cycle.

If software detects that deserializer and serializer are not in the tunneling mode, then the software shall log this issue and reset the system.

Status	Ready for review
Verification Criteria	Tests shall ensure that if software detects that deserializer and serializer is not in the tunneling mode, then the software shall log this issue and reset the system - observe the condition being detected and log being generated- observe system being reset.

MVC-111310 - GMSL2 and I2C selection shall remain as set during startup.

the software shall check if the configuration changes in REG6::GMSL2_A or REG6::I2CSEL within 100ms

Status	Ready for review
Verification Criteria	Tests shall ensure that the software checks that the configuration changes within 100ms during system start-up.

MVC-111303 - GMSL2 and I2C selection shall remain as set during startup.

When REG6::GMSL2_A becomes 0b0 or REG6::I2CSEL becomes 0b0 (i.e. configuration changes), the Software shall transition the system into the SafeState within 100ms.

Status	Ready for review
Verification Criteria	Tests shall ensure that when REG6::GMSL2_A becomes 0b0 or REG6::I2CSEL becomes 0b0 (i.e. configuration changes), the Software shall transition the system into the SafeState within 100ms - start the system- set REG6::GMSL2_A := 0b0- observe SoC SW detecting the change within 100ms- observe SoC SW transitioning the system into SafeState within time constraints - start the system- set REG6::I2CSEL := 0b0- observe SoC SW detecting the change within 100ms- observe SoC SW transitioning the system into SafeState within time constraints.

MVC-111319 - Coax cable select

the software shall check if the configuration changes in CTRL1::CXTP_A within 100ms

Status	Ready for review
Verification Criteria	Tests shall ensure that the software checks that the configuration changes within 100ms during system start-up.

MVC-111301 - Coax cable select

When CTRL1::CXTP_A becomes 0b0 (i.e. configuration changes), the software shall transition the system into the SafeState within 100ms .

Status	Ready for review
Verification Criteria	Tests shall ensure that when CTRL1::CXTP_A becomes 0b0 (i.e. configuration changes) the software shall transition the system into the SafeState within 100ms - start the system - set CTRL1::CXTP_A := 0b0 - observe SoC SW detecting the change within 100ms - observe SoC SW transitioning the system into SafeState within time constraints.

MVC-111333 - Read back data from deserializer

After having set a parameter in the deserializer, the software shall read back the value of the parameter and compare it with the intended (original 'set'-value) within 100ms.

Status	Ready for review
Verification Criteria	Tests shall ensure that after having set a parameter in the deserializer, the software shall read back the value of the parameter and compare it with the intended (original 'set'-value) - startup the SOW2 system- set parameter in the deserializer- change the value of the pramater in the deserializer before the read attempt occurs- observe the system entering the SafeSatate within 300ms (detection + transition).

MVC-111327 - Read back data from deserializer Error Reaction

If the comparison fails, the software shall transition to the SafeState within 200ms.

Note: The transition phase follows the detection phase.

Status	Ready for review
Verification Criteria	Tests shall ensure that if the comparison fails, the software shall transition to the SafeState within 200ms. The transition phase follows the detection phase. The starting time for measurement of detection phase shall conicide with issuing the write command. The detection time shall be less than 100ms.

MVC-111325 - Read back data from serializer

After having set a parameter in the SOW1-component, the Software shall read back the value of the parameter and compare it with the intended (original 'set'-value) within 100ms.

Status	Ready for review
Verification Criteria	Tests shall ensure that after having set a parameter in the SOW1-component, the Software shall read back the value of the parameter and compare it with the intended (original 'set'-value).

MVC-111331 - Read back data from serializer Error Reaction

If the comparison fails, the software shall transition to the SafeState within 200ms.

Note1: for component list and their I2C addresses refer to §3 of "Camera Interface Specification", Version 1.0, Document Number: 0220DN00JX

Note2: ATTENTION! the channels for write and read can be different depending on the parameter and its "host" (e.g.: parameter == 'exposure time', host == imager)

Note3: The transition phase follows the detection phase.

Status	Ready for review
Verification Criteria	- startup the SOW2 system- set parameter in the SOW1-component - change the value of the paramater in the SOW1-component before the read attempt occurs- observe the system entering the SafeState within 300ms (detection + transition)

MVC-111323 - If the read command (issued by SoC SW via I2C bus) of FuSa related parametes...

If the read command (issued by SoC SW via I2C bus) of FuSa related parametes does not yield result within 50ms, the software shall transition the system into SafeState within 200ms.

Status	Ready for review
Verification Criteria	Tests shall ensure that if the read command (issued by SoC SW via I2C bus) of FuSa related parametes does not yield result within 50ms, the software shall transition the system into SafeState within 200ms - start the system- disturb I2C-communication - issue command to serializer (e.g. read temperature)- wait for timeout- check time costraint <=50ms- observe transition into SafeState within time constraints.

MVC-111322 - When, while performing read action via I2C bus, the SoC SW detects CRC error signaled by Deserializer, the SoC SW shall repeat the read action within 50ms

When, while performing read action via I2C bus, if the software detects CRC error signaled by Deserializer, the software shall repeat the read action within 50ms.

Note (BOSCH): "To arrive at the peripherials of the SOW1 (e.g.: imager) the main/primary (MFP9, MFP10) channel should be used from the Deserializer" (ref. to MVC-100121)

Status	Ready for review
Verification Criteria	- start the system- issue read command from SoC SW- inject answer with CRC fault - observe a repeated raed command being sent within time constraints

MVC-111315 - if the re-read action via I2C bus also results in CRC error, the SoC shall transition into SafeState within 200ms (detection + reaction)

If the re-read action via I2C bus (from deserializer) also results in CRC error, the software shall transition into SafeState within 200ms (detection + reaction)

Status	Ready for review
Verification Criteria	Tests shall ensure that if the re-read action via I2C bus (from deserializer) also results in CRC error, the software shall transition into SafeState within 200ms (detection + reaction) Note: this is a continuation of MVC-106125 (alternative path - in case of failure) - start the system- cause two consecutive CRC-fails in the context of read request- observe that the system transitions into SafeState within time constraints

22.4.1.4 Safety Mechanism Activation (ASIL B Scope) and Fault Identification

22.4.1.4.1 SM3 - CRC for Control Channel

MVC-116406 - SM3 enabling

The software shall check if: TR0(0x70)[TX_CRC_EN] == 1 and TR0(0x70)[RX_CRC_EN] == 1 (are set by default)

Status	Ready for review
Verification Criteria	- review relevant code section

MVC-116404 - SM3 enabling

If the values TR0(0x70)[TX_CRC_EN] == 1 and TR0(0x70)[RX_CRC_EN] == 1 are not set, the software shall set them accordingly.

Status	Ready for review
Verification Criteria	review relevant code section

MVC-116408 - SM3 forwarding

The software shall set the FAULT_OEN (0x57B) [RBB_CC_CRC_ERR_OEN] := 1 in order to enable SM3 relevant faults to be forwarded to ERRB.

Status	Ready for review
Verification Criteria	- review relevant code section

MVC-116407 - SM3 fault identification

In order to identify the SM3 relevant fault, the software shall check if FAULT_STAT (0x57C)[RBB_CC_CRC_ERR_FLAG] > 0 every 100ms.

Note: In order to use the 'CRC for Control Channel/GMSL2 link-layer CRC' there is a need for enabling it on the serializer side (i.e.: TR0 (0x88)[TX_CRC_EN] == 1 and TR0 (0x88)[RX_CRC_EN] == 1) – it is enabled by default.

Status	Ready for review
Verification Criteria	- start the system- set relevant software variable(s)

MVC-116410 - SM3 fault Reaction

if FAULT_STAT (0x57C)[RBB_CC_CRC_ERR_FLAG] > 0, then the software shall log the error.

Status	Ready for review
Verification Criteria	- start the system- set relevant SoC SW variable(s)- verify the system logs an error

22.4.1.4.2 SM4 - CRC for Forward Video Channel

MVC-116401 - SM4 checking enabling and forwarding

The software shall check if VIDEO_RX0(0x112)[LINE_CRC_EN] ==1 and INTR6 (0x1E)[LCRC_ERR_OEN] == 1 (are set by default).

Note1: Enabling the optional 'Video Packet CRC' (each packet carrying 36 pixels is protected by a CRC) is not considered necessary since the 36 pixels change seems to be fine grained; the line-grained check is performed by the enabled mechanism
Note2: in order to use the 'Video Line CRC' there is a need for enabling it on the serializer side (i.e.: VIDEO_TX0(0x110)[LINE_CRC_EN] == 1) – it is enabled by default

Status	Ready for review
Verification Criteria	- review relevant code section

MVC-116418 - SM4 enabling and forwarding

If the values VIDEO_RX0(0x112)[LINE_CRC_EN] ==1 and INTR6 (0x1E)[LCRC_ERR_OEN] == 1 are not set, the software shall set them.

Status	Ready for review
Verification Criteria	- review relevant code section

MVC-116416 - SM4 fault identification

In order to identify the SM4 relevant fault, the software shall check if INTR7 (0x1F)[LCRC_ERR_FLAG] == 1 or VIDEO_RX0 (0x112)[LCRC_ERR] == 1 every 100ms.

Status	Ready for review
Verification Criteria	- start the system- set relevant software variable(s)

MVC-116421 - SM4 fault Reaction

If INTR7 (0x1F)[LCRC_ERR_FLAG] == 1 or VIDEO_RX0 (0x112)[LCRC_ERR] == 1, the software shall transition to SafeState.

Status	Ready for review
Verification Criteria	- start the system- set relevant software variable(s)- verify the system transitions into SafeState

22.4.1.4.3 SM5 - Error Correction through Re-transmission on Control Channel

MVC-116411 - SM5 enabling and forwarding

The software shall check if: TR0(0x70)[TX_CRC_EN] == 1 and TR0(0x70)[RX_CRC_EN] == 1 and ARQ0(0x75)[EN] == 1 and ARQ0(0x75)[DIS_DBL_ACK_RETX] == 0 and ARQ1(0x76)[MAX_RT_ERR_OEN] == 1 and ARQ1(0x76)[RT_CNT_OEN] == 0 (should be default values). If these bits do not take value as supposed, the software shall set them to the correct values.

Status	Ready for review
Verification Criteria	review relevant code section

MVC-116415 - SM5 fault identification

In order to identify the SM5 relevant fault, the software shall check if ARQ2 (0x77)[MAX_RT_ERR] ==1 or FAULT_STAT (0x57C)[RBB_CC_CRC_ERR_FLAG] == 1 every 100ms.

Status	Ready for review
Verification Criteria	- start the system - set relevant software variable(s)

MVC-116414 - SM5 fault reaction

If ARQ2 (0x77)[MAX_RT_ERR] ==1 or FAULT_STAT (0x57C)[RBB_CC_CRC_ERR_FLAG] == 1. , the software shall transition into SafeState (unrecoverable i2c control communication error occurred).

Status	Ready for review
Verification Criteria	- start the system - set relevant software variable(s) - verify the system transitions into SafeState

22.4.1.4.4 SM6 - LOCK indicator of Forward Channel

MVC-116423 - SM6 enabling and forwarding

The software shall enable LOCK signal indication (link lock status) on ERBB by setting bit INTR0(0x18)[LOCK_OEN]:=1

Status	Ready for review
Verification Criteria	- review relevant code section

MVC-116405 - NOT LOCKED

When the software receives the information that GSML2 link is NOT locked (ERRB asserted and CTRL3 (0x13)[LOCKED]==0) the software shall transition into SafeState (SW-LinkDisable-SafeSate) within 100ms (100ms for detection + 100ms for transition)

Status	Ready for review
Verification Criteria	- start the system- disrupt GMSL2 link- verify the system transitions into SafeState (SW-LinkDisable-SafeSate)- verify time constraints

MVC-116419 - LOCKED

When the software receives the information that GSML2 link is locked (ERRB event and CTRL3 (0x13)[LOCKED]==1) within 200ms, the software may start streaming video (depending on the System's StateMachine current state)

Status	Ready for review
Verification Criteria	- send "video subscribe" command- bring system into SafeState (SW-LinkDisable-SafeSate) due to LOCK loss- resume GMSL2 link- verify the system starts streaming video

22.4.1.4.5 SM7 - ERRB indicator of Forward Channel

MVC-116422 - SM7 identificaton

In order to identify the SM7 relevant fault, the software shall check if CTRL3[ERROR] ==1. If it is the case, the software shall proceed with surveying relevant (status, fault, error, SM-related) registers.

Status	Ready for review
Verification Criteria	- review relevant code section

22.4.1.4.6 SM15 - OV/UV comparators

MVC-116413 - SM15 enabling and forwarding

At Startup, The software shall set the INTR6 (0x1E)[VDDCMP_INT_OEN]:= 1, INTR6 (0x1E)[VDDBAD_INT_OEN]:= 1, PM_OV_STAT2(0x578)[VTERM_OV_OEN] := 1, PM_OV_STAT2(0x578)[VREG_OV_OEN] := 1 and PM_OV_STAT4(0x57A)[VDDIO_OV_OEN] := 1 in order to enable SM15 relevant faults to be forwarded to ERRB.

Status	Ready for review
Verification Criteria	- review relevant code section-inject overvoltage failure and verify ERRB will be asserted low (in the absence of other errors)

MVC-116431 - SM15 fault identification

In order to identify the SM15 relevant fault, the software shall check if INTR7(0x1F)[VDDCMP_INT_FLAG] ==1 and INTR7(0x1F)[VDDBAD_INT_FLAG] == 1 and PM_OV_STAT3(0x579)[VTERM_OV_FLAG] == 1 and PM_OV_STAT3(0x579)[VREG_OV_FLAG] == 1 and PM_OV_STAT3(0x579)[VDDIO_OV_FLAG] == 1 every 100ms. If at least one bit is set, the software shall transition into SafeState.

Status	Ready for review
Verification Criteria	- start the system- set relevant software variable(s)- verify the system transitions into SafeState

22.4.1.4.7 SM18 - Logic BIST

MVC-116433 - SM18 fault identification

At startup, In order to identify the SM18 relevant fault, the software shall check if REG_POST0(0x3020)[POST_DONE] == 1 (POST has completed) and REG_POST0(0x3020)[POST_RUN_LBIST] == 1 (confirms LBIST actually ran) and REG_POST0(0x3020)[POST_LBIST_PASSED] ==1 (Logic BIST passed). If it is NOT the case, the software shall transition into SafeState.

Note: no configuration needed. “The POST produces a pass/fail result that can be read through a register for each test. A failure does not prevent the device from operation but does assert ERRB.” (p. 43 of document “MAX96714/MAX96714F/MAX96714R Single GMSL2/GMSL1 to CSI-2 Deserializer“, 19-101134; Rev 2; 6/23)

Status	Ready for review
Verification Criteria	- start the system- set relevant software variable(s)- verify the system transitions into SafeState

22.4.1.4.8 SM19 - Memory BIST

MVC-116429 - SM19 fault identification

At startup, In order to identify the SM18 relevant fault, the software shall check if REG_POST0 (0x3020)[POST_DONE] ==1 (POST has completed) and REG_POST0 (0x3020)[POST_RUN_MBIST] ==1 (Confirms MBIST actually ran) and REG_POST0 (0x3020)[POST_MBIST_PASSED] ==1 (MBIST passed). If it is NOT the case, the software shall transition into SafeState.

Note: no configuration needed. “The POST produces a pass/fail result that can be read through a register for each test. A failure does not prevent the device from operation but does assert ERRB.” (p. 43 of document “MAX96714/MAX96714F/MAX96714R Single GMSL2/GMSL1 to CSI-2 Deserializer“, 19-101134; Rev 2; 6/23)

Status	Ready for review
Verification Criteria	- start the system- set relevant software variable(s)- verify the system transitions into SafeState

22.4.1.4.9 SM20 - End-to-end CRC during video transmission using CSI-2 forwarding

MVC-116430 - MIPI CSI-2 CRC calculation

The software shall calculate and evaluate CSI-2 CRC. If the CRC evaluation fails, the software shall enter the SafeState.

Status	Ready for review
Verification Criteria	- start the system- set relevant software variable(s)- verify the system transitions into SafeState

22.4.1.4.10 SM21 - CRC on I2C and UART transactions

MVC-111309 - During startup the SoC SW shall test if CRC scheme for primary- I2C bus is working.

During startup the software shall enable SM21 in the deserializer.

Status	Ready for review
Verification Criteria	- start the system- at SW prepare message with WRONG CRC attached- observe NACK response being sent from Deserializer- observe ERRB being asserted- Start the system- prepare an answer (with wrong CRC attached) from Deserializer and inject it into SW- observe SW detects the error

MVC-116436 - SM21 enabling and forwarding

The software shall set the I2C_UART_CRC7 (0x300F)[CC_CRC_EN] := 1 and I2C_UART_CRC7 (0x300F)[CC_CRC_MSGCNTR_OVR] := 1 in order to configure SM21 and set FS_INTR0 (0x3010)[I2C_UART_CRC_ERR_OEN] :=1 in order to enable SM21 relevant faults to be forwarded to ERRB.

Status	Ready for review
Verification Criteria	- review relevant code section -inject failure and verify that the ERRB will be asserted low (in the absence of other errors)

MVC-116435 - SM21 fault identification

In order to identify the SM21 relevant fault, the software shall check if FS_INTR1(0x3011)[I2C_UART_CRC_ERR_INT] == 1 every 100ms, If it is the case, the software shall reset the crc error counter by I2C_UART_CRC1(0x3009)[RESET_CRC_ERR_CNT] := 1 and transition into SafeState (SW-LinkDisable-SafeSate).

Status	Ready for review
Verification Criteria	- start the system- set relevant software variable(s)- verify the system transitions into SafeState

22.4.1.4.11 SM22 - Message counter

MVC-116440 - SM22 enabling and forwarding

The software shall set the I2C_UART_CRC7 (0x300F)[CC_MSGCNTR_EN] := 1 and I2C_UART_CRC7 (0x300F)[CC_CRC_MSGCNTR_OVR] := 1 in order to configure SM22 and set FS_INTR0 (0x3010)[I2C_UART_MSGCNTR_ERR_OEN] :=1 in order to enable SM22 relevant faults to be forwarded to ERRB

Status	Ready for review
Verification Criteria	- review relevant code section -inject failure and verify that the ERRB will be asserted low (in the absence of other errors)

MVC-116439 - SM22 fault identification

In order to identify the SM22 relevant fault, the software shall check every 100ms if FS_INTR1(0x3011)[I2C_UART_MSGCNTR_ERR_INT] == 1. If it is the case, the software shall :
reset the device’s counter to 0 by I2C_UART_CRC0(0x3008)[RESET_MSGCNTR] := 1,
reset software own counter to 0,
reset the msg error counter by I2C_UART_CRC1(0x3009)[RESET_MSGCNTR_ERR_CNT] := 1
transition into SafeState (SW-LinkDisable-SafeSate.

Status	Ready for review
Verification Criteria	- start the system- set relevant SoC SW variable(s)- verify the system transitions into SafeState

22.4.1.4.12 SM26 - Memory ECC

MVC-116441 - SM26 enabling and forwarding

The software shall set FS_INTR0 (0x3010)[MEM_ECC_ERR2_OEN] := 1in order to enable SM22 relevant faults to be forwarded to ERRB.

Status	Ready for review
Verification Criteria	- review relevant code section -inject failure and verify that the ERRB will be asserted low (in the absence of other errors)

MVC-116444 - SM26 fault identification

In order to identify the SM26 relevant fault, the software shall check every 100ms if FS_INTR1(0x3011)[MEM_ECC_ERR2_INT] == 1 If it is the case, the softwrae shall:
reset error counter to 0 by MEM_ECC0 (0x3016)[RESET_MEM_ECC_ERR2_CNT] := 1
and transition into SafeState (SW-LinkDisable-SafeSate).

Status	Ready for review
Verification Criteria	- start the system- set relevant SoC SW variable(s)- verify the system transitions into SafeState

22.4.1.4.13 SM27 - Configuration register CRC

TBD

22.4.1.4.14 SM29 - CRC of non-volatile memory

MVC-116445 - SM29 enabling and forwarding
During start-up the software shall check if FS_INTR0(0x3010)[EFUSE_CRC_ERR_OEN] == 1 (is set by default). If the value is not set, the software shall set it.

Status	Ready for review
Verification Criteria	- review relevant code section - inject failure and verify ERRB will be asserted low (in the absence of other errors)

MVC-116438 - SM29 fault identification
At startup, In order to identify the SM29 relevant fault, the software shall check if FS_INTR1(0x3011)[EFUSE_CRC_ERR_FLAG] == 1. If it is the case, the software transition into SafeState.

Status	Ready for review
Verification Criteria	- start the system- set relevant SoC SW variable(s)- verify the system transitions into SafeState

22.4.1.4.15 SM34 - DPLL lock detector

TBD

two options:

- Check with FMEA if the fault of this functional block will be caught on some other level (frame rate?, MIPI CSI-2 CRC?)-
- check with the manufacturer of deserializer

22.4.2 Run-time behavior

MVC-111300 - When SoC SW receives an ERRB-signal the SoC SW shall check...
The software shall read the ERRB-signal every 50ms.

Status	Ready for review
Verification Criteria	Tests shall ensure that the software receives the ERRB-signal within 50 ms.

MVC-111302 - When SoC SW receives an ERRB-signal the SoC SW shall check...
When the software read an ERRB-signal, the software shall check the meaning of the signal (i.e.: origin thereof) by querying the relevant registers in the deserializer via I2C bus within 50ms.

Status	Ready for review
Verification Criteria	Tests shall ensure that the software queries the relevant deserializer registers via the I2C bus within 50 ms after receiving the ERRB-signal.

MVC-111305 - When SoC SW receives an ERRB-signal the SoC SW shall check...
When the assertion is detected in the ERRB-signal, the software shall enter SafeState within 100ms.

Status	Ready for review
Verification Criteria	Tests shall ensure that the software transitions to the SafeState within the specified time after the condition is detected.

MVC-116450 - False positive ERRB
When the software receives ERRB signal but this signal cannot be associated with any subscribed event, the software shall transition into the SafeState within 300ms (detection + reaction).

Status	Ready for review
Verification Criteria	- start system- set ERRB to low- verify system transitions into SafeState- verify time constraints

MVC-116449 - VSYNC plausibility check
The software shall check that the VSYNC indication occurs congruent with the set frame rate. If the software detects discrepancy of more than 20% * nominalFrameRate within 300ms , the software shall transition into the SafeState within 100ms.

Status	Draft
Verification Criteria	- start system, - start video streaming (30 fps)- generate VSYNC of 23 fps for longer than 300ms- verify System tranistioned into SafeState- verify time costraint- start system, - start video streaming (30 fps)- generate VSYNC of 37fps for longer than 300ms- verify System tranistioned into SafeState- verify time costraint

MVC-111335 - Changes to GMSL link
When any changes to the GMSL link were performed, the Software shall perform a link reset to reinitialize the link (toggle RESET_LINK HIGH and then LOW or write 1 to RESET_ONESHOT bit of the CTRL0 (0x10) register).

Status	Ready for review
Verification Criteria	Tests shall ensure that when any changes to the GMSL link were performed, the Software shall perfom a link reset to reinitialize the link (toggle RESET_LINK HIGH and then LOW or write 1 to RESET_ONESHOT bit of the CTRL0 (0x10) register) - start the system- introduce changes to GMSL link: disable I2C pass-through Channel 2 by setting REG1::IIC_2_EN := 0- watch RESET_LINK HIGH being toggled or CTRL0::RESET_ONESHOT being set - the I2C pass-through channel becomes disabled.

MVC-111307 - If SoC SW receives an LOCK-signal (assertion) the SoC SW shall...
The software shall receive the LOCK-signal within 50ms

Status	Rejected
Verification Criteria	Tests shall ensure that the software receives the LOCK-signal within 50 ms.

MVC-111304 - If SoC SW receives an LOCK-signal (assertion) the SoC SW shall...
If software receives an LOCK-signal (deassertion), then the software shall check the cause of the deassertion (i.e.: origin thereof) by querying the relevant registers in the deserializer via I2C bus within 50ms.

Status	Rejected
Verification Criteria	

MVC-111306 - If SoC SW receives an LOCK-signal (assertion) the SoC SW shall...
When the deassertion is detected, the software shall enter SafeState within 100ms.

Status	Rejected
Verification Criteria	Tests shall ensure that the software transitions to the SafeState within the specified time after the condition is detected.

MVC-111336 - The SoC SW shall check once per driver cycle if REG1(0x1):: IIC_1_EN ==0b1. If the check fails the system shall log a trouble code.
The Software shall check once per driver cycle if REG1(0x1):: IIC_1_EN ==0b1.

Status	Rejected
Verification Criteria	Tests shall ensure that the Software shall check once per driver cycle if REG1(0x1):: IIC_1_EN ==0b1.

MVC-111326 - The SoC SW shall check once per driver cycle if REG1(0x1):: IIC_1_EN ==0b1. If the check fails the system shall log a trouble code.
If the check fails the software shall log a trouble code.

Status	Rejected
Verification Criteria	Tests shall ensure that the software logs a trouble code when the check fails.

MVC-111324 - The SoC SW shall check once per driver cycle if REG1(0x1):: IIC_2_EN ==0b1. If the check fails the system shall log a trouble code.
The Software shall check once per driver cycle if REG1(0x1):: IIC_2_EN ==0b1.

Status	Rejected
Verification Criteria	Tests shall ensure that the Software shall check once per driver cycle if REG1(0x1):: IIC_2_EN ==0b1.

MVC-111330 - The SoC SW shall check once per driver cycle if REG1(0x1):: IIC_2_EN ==0b1. If the check fails the system shall log a trouble code.
If the check fails the software shall log a trouble code.

Status	Rejected
Verification Criteria	Tests shall ensure that the software logs a trouble code when the check fails.

MVC-111339 - At the system startup the SoC SW shall check if the LOCK pin in the deserializer is enabled. If not, the SoC SW shall enable LOCK pin by setting REG5 (0x5)::LOCK_EN := 0b1

At the system startup the Software shall check if the LOCK pin in the deserializer is enabled. If not, then the software shall enable LOCK pin by setting REG5 (0x5)::LOCK_EN := 0b1

Status	✗ Rejected
Verification Criteria	Tests shall ensure that the software checks that the LOCK pin in the deserializer is enabled. If not, then the software shall enable LOCK pin by setting REG5 (0x5)::LOCK_EN := 0b1 during system start-up - start the system - pause the system before the check is performed - set REG5 (0x5)::LOCK_EN to 0b0 - resume the system - observe the REG5 (0x5)::LOCK_EN has been set to 0b1.

MVC-111314 - When LOCK becomes disabled, the SoC SW shall...

The software shall check if the LOCK signal is disabled within 100ms

Status	✗ Rejected
Verification Criteria	Tests shall ensure that the software checks that the LOCK signal is disabled within 100ms during system start-up.

MVC-111312 - When LOCK becomes disabled, the SoC SW shall...

When LOCK becomes disabled, the Software shall transition the system into the SafeState within 100ms.

Status	✗ Rejected
Verification Criteria	Tests shall ensure that when LOCK becomes disabled, the Software shall transition the system into the SafeState within 100ms - start the system - disable LOCK- observe that system transitioned into safeState within the time constraints..

MVC-111329 - Read back data from imager

After having set a parameter in the imager, the Software shall read back the value of the parameter and compare it with the intended (original 'set'-value)

(detection time shall be less than 100ms).

Status	✗ Rejected
Verification Criteria	Tests shall ensure that after having set a parameter in the imager, the Software shall read back the value of the parameter and compare it with the intended (original 'set'-value) (detection time shall be less than 100ms).

MVC-111320 - Read back data from imager

If the comparison fails, the software shall enter the SafeState within 200ms.

Status	✗ Rejected
Verification Criteria	Tests shall ensure that if the comparison fails, the software shall enter the SafeState within 200ms.

MVC-111313 - At startup the Deserializer shall run...

At startup the software shall run Logic BIST test, Memory BIST test and at run BIST for Serial Link test. If at least one of the tests fails, then the Software shall transition into SafeState within 350ms (test execution + error signaling + error detection + reaction)

Status	✗ Rejected
Verification Criteria	Tests shall ensure that at startup the Deserializer shall run Logic BIST test, Memory BIST test and at run BIST for Serial Link test. If at least one of the tests fails, the System shall transition into SafeState within 350ms (test execution + error signaling + error detection + reaction)

MVC-111318 - During startup the SoC SW shall enable reporting of line-fault interrupt.

During startup the software shall enable reporting of line-fault interrupt.

Status	✗ Rejected
Verification Criteria	Tests shall ensure that during startup the SoC SW shall enable reporting of line-fault interrupt.

MVC-114524 - Check after Logic BIST test

After Logic BIST test has been performed, the software shall check the REG_POST0 (0x3020) for the test results. If results indicate failure, then the software shall transition into SafeState within 350ms (detection + reaction time).

Status	✗ Rejected
Verification Criteria	- run the software- observe the SoC SW checks the bits: REG_POST0[POST_RUN_LBIST] == 1 and REG_POST0[POST_LBIST_PASSED] == 1 and REG_POST0[POST_DONE] == 1 - system does NOT transition into SafeState - run the boot software - halt the system - manipulate the SoC SW variable responsible for REG_POST0[POST_RUN_LBIST] and REG_POST0[POST_LBIST_PASSED] and REG_POST0[POST_DONE] to indicate an error - observe System transitioning to SafeState - verify time constraint was satisfied

MVC-114523 - Check after Memory BIST test

After Memory BIST test has been performed, the software shall check the REG_POST0 (0x3020) for the test results. If results indicate failure, then the software shall transition into SafeState within 350ms (detection + reaction time).

Status	✗ Rejected
Verification Criteria	- run the software - observe the software checks the bits: REG_POST0[POST_RUN_MBIST] == 1 and REG_POST0[POST_MBIST_PASSED] == 1 and REG_POST0[POST_DONE] == 1 - system does NOT transition into SafeState - run the software - manipulate the software variable responsible for REG_POST0[POST_RUN_MBIST] and REG_POST0[POST_MBIST_PASSED] and REG_POST0[POST_DONE] to indicate an error - observe System transitioning to SafeState - verify time constraint was satisfied

22.5 Ethernet PHY Safety Requirements

22.5.1 Configuration

MVC-117972 - Set config enable.

Before starting configuration of PHY, the software shall set the CONFIG_ENABLE bit to 1 (to unlock for reconfiguration).

Status	Draft
Verification Criteria	- Set CONFIG_ENABLE register- Attempt to change a configuration register- Read back the register value and check it has been changed.

MVC-111290 - After the safety-critical configuration is complete, the software driver shall r...

After the configuration is complete, the software shall reset the CONFIG_ENABLE bit to protect the configuration from unintended modifications during runtime.

Status	Draft
Verification Criteria	- Reset CONFIG_ENABLE register- Attempt to change a configuration register- Read back the register value and check it is not changed.

MVC-117971 - Activating the Interrupt Signaling within PHY

After PHY startup, the software shall activate interrupt signaling within PHY by configuring (set active) relevant IRQ sources at PHY by doing:

a. Set PORT_IRQ_ENABLES[PORT1_IRQ] :=1
   i. Set PORT_FUNC_IRQ_ENABLES[SERDES_MII_IRQ] :=1
      1. Set PORT_LEVEL_IRQ_ENABLES[EPHY_CLK_WARN] :=1
      2. Set PORT_LEVEL_IRQ_ENABLES[EPHY_LF] :=1
      3. Set PORT_LEVEL_IRQ_ENABLES[TXC_CLK_WARN] :=1
      4. Set PORT_LEVEL_IRQ_ENABLES[RXC_CLK_WARN] :=1
   ii. Set PORT_FUNC_IRQ_ENABLES[INFRA_IRQ] :=1
      1. Set INFRA_IRQ_ENABLES[FIFO_ERROR]:=1
      2. Set INFRA_IRQ_ENABLES[RGMII_ID_LOCK_ERROR]:=1
   iii. Set PORT_FUNC_IRQ_ENABLES[EPHY_IRQ] :=1
      1. Set EPHY_CAT_IRQ_ENABLES[EPHY_CONFIG_REGXS_IRQ]:=1
         a. Set PHY_CONFIG_REGXS_IRQ_ENABLES[RECONFIG_DENIED] :=1
         b. Set PHY_CONFIG_REGXS_IRQ_ENABLES[INIT_ERROR] :=1
         c. Set PHY_CONFIG_REGXS_IRQ_ENABLES[SMI_ERROR] :=1
      2. Set EPHY_CAT_IRQ_ENABLES[EPHY_FUNCTIONAL_IRQ]:=1
         a. Set EPHY_FUNCTIONAL_IRQ_ENABLE[LINK_AVAILABLE_EVENT] := 1
b. Set PORT_IRQ_ENABLES[GLOBAL_IRQ] :=1
   i. GLOBAL_CAT_IRQ_ENABLE[CONFIG_REGXS_IRQ]:=1
      1. Set CONFIG_REGXS_IRQ_ENABLE[SMI_ERROR]:= 1
   ii. GLOBAL_CAT_IRQ_ENABLE[OP_CONDITIONS_IRQ]:=1
      1. Set OP_CONDITIONS_IRQ_ENABLE[TEMP_WARN]:=1
      2. Set OP_CONDITIONS_IRQ_ENABLE[OSC_WARN]:=1
      3. Set OP_CONDITIONS_IRQ_ENABLE[VDDA_TRX_SUPPLY_WARN]:=1
      4. Set OP_CONDITIONS_IRQ_ENABLE[VREGA_SUPPLY_WARN]:=1
      5. Set OP_CONDITIONS_IRQ_ENABLE[VREGD_SUPPLY_WARN]:=1
      6. Set OP_CONDITIONS_IRQ_ENABLE[VDDIO_SUPPLY_WARN]:=1
      7. Set OP_CONDITIONS_IRQ_ENABLE[CORE_SUPPLY_WARN]:=1
      8. Set OP_CONDITIONS_IRQ_ENABLE[AO_SYSTEM_SUPPLY_WARN]:=1

Note: The PHY shall report potential safety hazard by means of interrupt signal to the SoC.

Status	Draft
Verification Criteria	- configure the relevant registers- read back if the configuration was performed correctly

22.5.2 Boot-Up

MVC-111291 - The SoC shall confirm the successful completion of the Ethernet PHY startup BIST by eit...

The software shall confirm the successful completion of the Ethernet PHY startup BIST by either: a) Waiting for and receiving the FUSA_PASS_IRQ interrupt, OR b) Polling the DEVICE_STATUS[FUSA_STATUS] bit until it reads as '1'.

Status	Draft
Verification Criteria	

22.5.3 Run Time

22.5.4 Miscellaneous

22.5.5 Requirements to Volvo

22.5.6 Communication with PHY

22.5.7 FuSa checks by PHY

22.5.8 System reaction on FuSa error

22.5.8.1 Communication with PHY

22.6 PMIC Safety Requirements

MVC-115363 - Activation of the forwarding of the PG# and ALERT# towards SoC's POR_L pin.

After PG#/ALERT# toggling tests have been completed, the software shall activate the forwarding of the PG# and ALERT# towards SoC's POR_L pin.

Status	Ready for review
Verification Criteria	Start the system After toggling tests have been completed, observe the activation signal is sent from SoC.

MVC-111275 - The system shall toggle the ALERT# at power up to ensure the ALERT# function is working properly in case of relevant failures.

The software shall toggle the ALERT# line of MPQ70165FS' (by Write 1b to this bit (called ALERT#_FORCE_ASSERT) of ALERT_PG_FORCE (0xC0) register) at power up to ensure the ALERT# function is working properly in case of relevant failures.

Status	Draft
Verification Criteria	Tests shall ensure that the software toggles the ALERT#_FORCE_ASSERT bit in the ALERT_PG_FORCE (0xC0) register during power-up and verifies that the ALERT# signal changes state accordingly.

MVC-111286 - The system shall toggle the ALERT# at power up within time constrains

Time between assertion of bit1 which called (ALERT#_FORCE_ASSERT) of ALERT_PG_FORCE (0xC0) register) and detection of the ALERT# line change shall be less than 50ms.

Status	Draft
Verification Criteria	Tests shall ensure that the elapsed time between asserting the ALERT#_FORCE_ASSERT bit and detecting the corresponding ALERT# signal transition is less than 50 ms by:- start the system- observe ALERT# having value high- observe ALERT#_FORCE_ASSERT bit having value low- set ALERT#_FORCE_ASSERT bit to High- observe ALERT# having value low- observer time constraints- set ALERT#_FORCE_ASSERT bit to Low- observe ALERT# having value high - observer time constraints.

MVC-111274 - During startup, if the SoC SW detects that ALERT# pin is not properly pulled low…

During startup, if ALERT# power up test fails then, then the software shall remain in its safe state and no normal operation is allowed.

Status	Draft
Verification Criteria	Tests shall ensure that the software remains in the defined safe state and prevents entry into normal operation when the ALERT# pin is not correctly asserted during startup by - start the system- force ALERT# signal to low- observe SoC detecting ALERT# signal having low value -> system remains/transition into SafeState- observe time constraints- force ALERT# signal to high- observe SoC detecting ALERT# signal having high value -> system remains in the operational mode- observe time constraints.

MVC-111287 - During startup, if the SoC SW detects that ALERT# pin is not properly pulled low…

The software shall detect the ALERT# signal in less than 50ms.

Status	Draft
Verification Criteria	Tests shall ensure that the software detects an ALERT# signal transition within 50 ms.

MVC-111277 - The system shall toggle the PG# line of MPQ70165FS' ...

The software shall toggle the PG# line of MPQ70165FS' (by toggling bit 0 (called PG_FORCE_ASSERT) of ALERT_PG_FORCE (0xC0) register) at power up to ensure the PG# function is working properly in case of power good failures.

Status	Draft
Verification Criteria	Tests shall ensure that the software toggles the PG_FORCE_ASSERT bit in the ALERT_PG_FORCE (0xC0) register during power-up and verifies that the PG# signal changes state accordingly.

MVC-111273 - The system shall toggle the PG# line of MPQ70165FS' ...

Time between assertion of the bit and detection of the PG# line change shall be less than 50ms.

Status	Draft
Verification Criteria	Tests shall ensure that the elapsed time between asserting the PG_FORCE_ASSERT bit and detecting the corresponding PG# signal transition is less than 50 ms.

MVC-111276 - During startup, if the SoC SW detects that PG# pin is not properly pulled low, t…

During startup, if the software detects that PG# pin is not properly pulled low, the software shall remain in its safe state and no normal operation is allowed.

Status	Draft
Verification Criteria	Tests shall ensure that the software remains in the defined safe state and prevents entry into normal operation when the PG# pin is not correctly asserted during startup.

MVC-111280 - During startup, if the SoC SW detects that PG# pin is not properly pulled low, t…

The software shall detect the PG# signal in less than 50ms.

Status	Draft
Verification Criteria	Tests shall ensure that the software detects a PG# signal transition within 50 ms.

MVC-111284 - The SoC SW shall use CRC for I2C communication (Tx and Rx) with PMIC using the f...

The software shall use CRC-8 for I2C communication (Tx and Rx) with PMIC using the following parameters:

- Number of bits to check: 32 bits
- Polynomial: 0x83
- Polynomial: C(x) = x^8 + x^2 + x + 1

Status	Ready for review
Verification Criteria	Tests shall ensure that the software performs CRC checking on all I²C transmit and receive communications with the PMIC using the specified CRC polynomial and configuration.

MVC-111279 - Initialization of I2C communication in PMIC

During startup the software shall configure the I2C SoC to PMIC communication check with the following parameters:

- set bit no. 2 (PMBUS_CRC_ALERT#_MAPPING) of the register ALERT#_MAPPING (C4h) to zero (i.e. ALERT#_MAPPING[2] := 0b0) to enable ALERT# signaling
- set bit no. 2 (PMBUS_CRC_PG_MAPPING) of the register PG_MAPPING (C6h) to zero (i.e. PG_MAPPING[2] := 0b0) to enable PG signaling
- set bits no. 0 - 3 (PG_DELAY and ALERT#_DELAY) of the register PG_ALERT#_DELAY (C8h) to zero (PG_ALERT#_DELAY[0-4] := 0b000) to set the delay to 0
- set bit no. 0 (PEC_REQ) of the register SAFETY_CONFIG (C2h) to one (SAFETY_CONFIG[0] := 1) to enable packet error checking

Status	Ready for review
Verification Criteria	Tests shall ensure that the software configures the PMIC communication registers with the specified values during startup, including ALERT# mapping, PG# mapping, delay configuration, and PEC_REQ enablement - start the system - wait till the system boots up completely - verify the values of the bits/registers given above.

MVC-111278 - The SoC SW shall transition to safe state within FRTI (200ms) if it detects tha...

While in runtime,if the software detects that PG# is pulled low, then the software shall transition to safe state within FRTI (200ms) .

Status	❌ Rejected
Verification Criteria	Tests shall ensure that the software transitions the system to the defined safe state within 200 ms after detecting that the PG# signal is asserted low during runtime.

MVC-111281 - The SoC SW shall transition to safe state within FRTI (200ms) if it detects tha...

While in runtime, if the software detects that ALERT# is pulled low, then the software shall transition to safe state within FRTI (200ms).

Status	❌ Rejected
Verification Criteria	Tests shall ensure that the software transitions the system to the defined safe state within 200 ms after detecting that the ALERT# signal is asserted low during runtime.

MVC-116550 - The SoC SW shall activate continuous OTP CRC

The software shall activate continuous OTP CRC in PMIC by setting the bit MFR_OTP_MEM_CHK[MFR_OTP_CRC_CHK_EN] := 1 and bit MFR_OTP_MEM_CHK[MFR_OTP_MEM_CHK] := 1

Note: this SM is not enabled by default (see: p. 67 of document: MPQ70165FS Rev. 1.0 11/3/2025)

Status	Ready for review
Verification Criteria	1- Boot up system 2- Verify the MFR_OTP_MEM_CHK[MFR_OTP_CRC_CHK_EN] == 1 3- Verify the MFR_OTP_MEM_CHK[MFR_OTP_MEM_CHK] == 1

MVC-116551 - The SoC SW shall activate continuous Register CRC Check

The software shall activate continuous Register CRC Check in PMIC by setting the bit MFR_OTP_MEM_CHK[MFR_MEM_CRC_CHK_EN] := 1 and bit MFR_OTP_MEM_CHK[MFR_OTP_MEM_CHK] := 1

Note: this SM is not enabled by default (see: p. 67 of document: MPQ70165FS Rev. 1.0 11/3/2025)

Status	Ready for review
Verification Criteria	1- Boot up system 2- Verify the MFR_OTP_MEM_CHK[MFR_MEM_CRC_CHK_EN] == 1 3- Verify the MFR_OTP_MEM_CHK[MFR_OTP_MEM_CHK] == 1

MVC-111289 - If the CRC check fails in SoC, the SoC SW shall transition to safe state within...

If the CRC check fails in SoC, the software shall transition to safe state within FRTI (200ms)

Status	Ready for review
Verification Criteria	Tests shall ensure that the software transitions the system to the defined safe state within 200 ms when a CRC error is detected during SoC-to-PMIC I²C communication - Run system normally - Verify no CRC failures are detected- Inject CRC Error- Measure the time from CRC injection to PHY inactivity.- Verify this time is <=200 ms..

MVC-111288 - After each write command to PMIC, the SoC SW shall read back the written value f...

After each write command to PMIC, the software shall read back the written value from the PMIC register .The process of writing, reading back and checking shall take less than 50ms.

Status	Ready for review
Verification Criteria	Tests shall ensure that the software retries the PMIC register write within 50 ms when the read-back value does not match the commanded value - Run the system normally - SOC performs write to given register- Verify that the software performs the read-back.- verify time constraint.

MVC-111283 - If the SoC SW detects that an incorrect register value has been read back from P…

If the software detects that an incorrect register value has been read back from PMIC, then the software shall re-write the register value within 50ms.

Status	Ready for review
Verification Criteria	Tests shall ensure that the software retries the PMIC register write within 50 ms when the read-back value does not match the commanded value - Run the system normally - write given value - halt execution of SoC SW task resposible for communication with PMIC- Inject Fault for a PMIC Register into the SoC - Resume execution..

MVC-111282 - If the SoC SW detects that an incorrect PMIC register value has been read back f...

If the software detects that an incorrect PMIC register value has been read back for the second

time, the software shall transition to safe state within FRTI (200ms)

Status	Ready for review
Verification Criteria	Tests shall ensure that the software transitions the system to the defined safe state within 200 ms when the PMIC register read-back value remains incorrect after the second write attempt by : - Run the system normally - write given value - halt execution of SoC SW task resposible for communication with PMIC- Inject Fault for a PMIC Register into the SoC- Resume execution.- Verify that the software performs the read-back and detects the mismatch.- Verify if the software performs the read-back - halt execution of SoC SW task resposible for communication with PMIC- Inject Fault for a PMIC Register into the SoC- Resume execution.- verify that system deactivates PHY- Measure the time from second detection to PHY become inactive is <= 200ms.

MVC-111550 - The SoC SW shall configure PMIC at startup. PMIC buck 1 and buck 2 outputs shall...

The software shall configure PMIC at startup(use PAGE command 00h to configure different bucks). PMIC buck 1 and buck 2 outputs shall be monitored for OV with +4.5% threshold and UV with -3.5% threshold using the following register configurations:

PG_CONFIG (0xCA) of Buck 1, bit[0] = 1 (Buck 1 thresholds: +4.5% / - 3.5%)

PG_CONFIG (0xCA) of Buck 2, bit[0] = 1 (Buck 2 thresholds: +4.5% / - 3.5%)

Status	Ready for review
Verification Criteria	- Normal Operation at nominal voltage, verify that both of PG pin and ALERT# pin are high- Decrease the output voltage of buck 1,2 (UV)- Verify that ALERT# pin and PG pin become low- Increase the output voltage of buck 1,2 (OV)- Verify that ALERT# pin and PG pin become low - Normal Operation at nominal voltage, verify that both of PG pin and ALERT# pin are high- Decrease the output voltage of buck 1,2 (UV)- Verify that ALERT# pin and PG pin become low- Increase the output voltage of buck 1,2 (OV)- Verify that ALERT# pin and PG pin become low

MVC-111551 - The SoC SW shall configure PMIC at startup. PMIC buck 3 output shall be monitore...

The software shallconfigure PMIC at startup(use PAGE command 00h to configure different bucks). PMIC buck 3 output shall be monitored for OV with +4.5% threshold and UV with -3.5% threshold using the following register configurations:

PG_CONFIG (0xCA) of Buck 3, bit[0] = 1 (Buck 3 thresholds: +4.5% / - 3.5%).

Status	Ready for review
Verification Criteria	- Normal Operation at nominal voltage, verify that both of PG pin and ALERT# pin are high- Decrease the output voltage of buck 3 (UV)- Verify that ALERT# pin and PG pin become low- Increase the output voltage of buck 3 (OV)- Verify that ALERT# pin and PG pin become low

MVC-111552 - The SoC SW shall configure PMIC at startup. PMIC buck 4 output shall be monitore...

The software shall configure PMIC at startup(use PAGE command 00h to configure different bucks). PMIC buck 4 output shall be monitored for OV with +4.5% threshold and UV with -3.5% threshold using the following register configurations:

PG_CONFIG (0xCA) of Buck 4, bit[0] = 1 (Buck 4 thresholds: +4.5% / - 3.5%)

Status	Ready for review
Verification Criteria	- Normal Operation at nominal voltage, verify that both of PG pin and ALERT# pin are high- Decrease the output voltage of buck 4 (UV)- Verify that ALERT# pin and PG pin become low- Increase the output voltage of buck 4 (OV)- Verify that ALERT# pin and PG pin become low

MVC-111553 - The SoC SW shall configure PMIC at startup. PMIC buck 5 output shall be monitore...

The software shall configure PMIC at startup(use PAGE command 00h to configure different bucks). PMIC buck 5 output shall be monitored for OV with +4.5% threshold and UV with -3.5% threshold using the following register configurations:

PG_CONFIG (0xCA) of Buck 5, bit[0] = 1 (Buck 5 thresholds: +4.5% / - 3.5%)

Status	Ready for review
Verification Criteria	- Normal Operation at nominal voltage, verify that both of PG pin and ALERT# pin are high- Decrease the output voltage of buck 5 (UV)- Verify that ALERT# pin and PG pin become low- Increase the output voltage of buck 5 (OV)- Verify that ALERT# pin and PG pin become low

MVC-111554 - The SoC SW shall configure PMIC at startup. PMIC buck 6 output shall be monitore...

The software shall configure PMIC at startup(use PAGE command 00h to configure different bucks). PMIC buck 6 output shall be monitored for OV with +4.5% threshold and UV with -3.5% threshold using the following register configurations:

PG_CONFIG (0xCA) of Buck 6, bit[0] = 1 (Buck 6 thresholds: +4.5% / - 3.5%)

Status	Draft
Verification Criteria	- Normal Operation at nominal voltage, verify that both of PG pin and ALERT# pin are high- Decrease the output voltage of buck 6 (UV)- Verify that ALERT# pin and PG pin become low- Increase the output voltage of buck 6 (OV)- Verify that ALERT# pin and PG pin become low

22.7 Safe State Requirements

MVC-111285 - If the SoC SW detects ALERT# is set to low, the SoC SW shall transition into SafeState (detection: 50ms + reaction: 300ms)

If the software detects within 50ms that ALERT# is set to low, then the software shall transition into SafeState within 300ms

Status	Draft
Verification Criteria	Tests shall ensure that the software detects the ALERT# signal within 50 ms and transitions the system to the defined safe state within 300 ms of the detection by the following steps: - start up the system- start timer- set ALERT# to low- verify that system have entered a SafeState- stop timer- verify fullfilment of time constraints.

23 Cybersecurity Software Specifications

23.1 Secure Communication (SOW2 - Vehicle ECU)

23.1.1 Local Preparation

23.1.1.1 PSK Loading

MVC-107801 - The software shall retrieve the configured PSK from secure storage before initia...

The software shall retrieve the configured PSK from secure storage before initiating the DTLS 1.3 handshake.

Status	Draft
Verification Criteria	Verify that the PSK is retrieved before transmission of the ClientHello message. Verify that handshake establishment is aborted if PSK retrieval fails.

MVC-107804 - The software shall retrieve the configured PSK identity associated with the comm...

The software shall retrieve the configured PSK identity associated with the communication interface before initiating the DTLS handshake.

Status	Draft
Verification Criteria	Verify that the configured PSK identity is available before the ClientHello message is transmitted. Verify that handshake establishment is rejected when no valid PSK identity exists.

MVC-107803 - The software shall use the provisioned unique PSK associated with the configured...

The software shall use the provisioned unique PSK associated with the configured SOW2–Vehicle ECU pairing during DTLS handshake key derivation.

Status	Draft
Verification Criteria	

MVC-107805 - The software shall ensure that the retrieved PSK remains within the secure crypt...

The software shall ensure that the retrieved PSK remains within the secure cryptographic boundary throughout the handshake process.

Status	Draft
Verification Criteria	Verify that the PSK is never exposed outside the configured secure cryptographic boundary.

23.1.1.2 Random Generation

MVC-107796 - The software shall obtain random values required for DTLS cryptographic operatio...

The software shall obtain random values required for DTLS cryptographic operations from a Cryptographically Secure Pseudo-Random Number Generator (CSPRNG) seeded by the hardware TRNG.

Status	Draft
Verification Criteria	Verify that random values are obtained from the approved random generation mechanism.

MVC-107795 - The software shall generate a new DTLS Client Random value for every DTLS handsh...

The software shall generate a new DTLS Client Random value for every DTLS handshake before transmitting the ClientHello message.

Status	Draft
Verification Criteria	Verify that a Client Random value is generated before the ClientHello message is transmitted.

MVC-107798 - The software shall not reuse a previously generated Client Random value for any...

The software shall not reuse a previously generated Client Random value for any subsequent DTLS handshake.

Status	Draft
Verification Criteria	Verify that a unique Client Random value is generated for every handshake. Verify that previously generated Client Random values are not reused.

MVC-107797 - The software shall prevent reuse of random values across independent DTLS sessio...

The software shall prevent reuse of random values across independent DTLS sessions.

Status	Draft
Verification Criteria	Verify that random values are unique for independent DTLS sessions. Verify that DTLS processing is terminated when random number generation fails.

23.1.1.3 Ephemeral ECDHE Generation

MVC-107807 - The software shall generate a new ephemeral ECDHE key pair before constructing t...

The software shall generate a new ephemeral ECDHE key pair before constructing the ClientHello message.

Status	Draft
Verification Criteria	Verify that a new ECDHE key pair is generated for every handshake.

MVC-107811 - The software shall generate the ephemeral ECDHE key pair using the configured se...

The software shall generate the ephemeral ECDHE key pair using the configured secp256r1 (P-256) named group.

Status	Draft
Verification Criteria	

MVC-107809 - The software shall ensure that the generated ephemeral ECDHE key pair is associa...

The software shall ensure that the generated ephemeral ECDHE key pair is associated only with the current DTLS handshake and not reuse an ephemeral ECDHE private key or its corresponding public key for any subsequent DTLS handshake.

Status	Draft
Verification Criteria	Verify that the corresponding public key is included in the DTLS key_share extension. Verify that each DTLS handshake uses a unique ephemeral ECDHE key pair. Verify that previously generated ephemeral ECDHE key pairs are never reused.

MVC-107812 - The software shall ensure that the ephemeral ECDHE private key remains within th...

The software shall ensure that the ephemeral ECDHE private key remains within the configured secure cryptographic boundary throughout the DTLS handshake.

Status	Draft
Verification Criteria	Verify that the ephemeral private key remains protected within the secure cryptographic boundary.

23.1.2 ClientHello Construction

MVC-107813 - The software shall operate as the DTLS 1.3 Client for the SOW2–Vehicle ECU secur...

The software shall operate as the DTLS 1.3 Client for the SOW2–Vehicle ECU secure communication interface.

Status	Draft
Verification Criteria	Verify that SOW2 initiates the DTLS handshake as the DTLS Client.

MVC-107806 - The software shall include DTLS 1.3 as the supported protocol version in the sup...

The software shall include DTLS 1.3 as the supported protocol version in the supported_versions extension in the ClientHello message.

Status	Draft
Verification Criteria	Verify that the ClientHello advertises DTLS 1.3. Verify that the handshake is terminated if DTLS 1.3 is not negotiated.

MVC-107821 - The software shall include only supported DTLS 1.3 AEAD cipher suites in the Cli...

The software shall include only supported DTLS 1.3 AEAD cipher suites in the ClientHello message.

Status	Draft
Verification Criteria	Verify that only configured cipher suites are advertised.

MVC-107819 - The software shall include TLS_AES_128_GCM_SHA256 in the ClientHello message as...

The software shall include TLS_AES_128_GCM_SHA256 in the ClientHello message as a supported cipher suite for DTLS session establishment with the Vehicle ECU.

Status	Draft
Verification Criteria	Verify that TLS_AES_128_GCM_SHA256 is advertised

MVC-107825 - The software shall advertise the configured PSK-DHE key exchange mode with psk_d...

The software shall advertise the configured PSK-DHE key exchange mode with psk_dhe_ke in the ClientHello message.

Status	Draft
Verification Criteria	Verify that the ClientHello advertises the configured PSK-DHE key exchange mode. Verify that unauthorized key exchange modes are not advertised. Verify that handshake establishment fails when the negotiated key exchange mode differs from the configured mode.

MVC-107823 - The software shall encode the configured PSK identity using the configured ident...

The software shall encode the configured PSK identity using the configured identity format, encoding, and length before including it in the pre_shared_key extension of the ClientHello message.

Status	Draft
Verification Criteria	

MVC-107816 - The software shall include the generated ephemeral ECDHE public key in the DTLS...

The software shall include the generated ephemeral ECDHE public key in the DTLS key_share extension of the ClientHello message.

Status	Draft
Verification Criteria	Verify that the generated ECDHE public key is included in the key_share extension. Verify that the transmitted public key corresponds to the generated key pair. Verify that only the public key is transmitted.

MVC-107815 - The software shall terminate the DTLS handshake if the generated ECDHE public ke...

The software shall terminate the DTLS handshake if the generated ECDHE public key cannot be included in the key_share extension.

Status	Draft
Verification Criteria	Verify that the handshake is terminated if the key_share extension cannot be generated.

MVC-107818 - The software shall include the generated ClientRandom value in the ClientHello m...

The software shall include the generated ClientRandom value in the ClientHello message

Status	Draft
Verification Criteria	

MVC-107817 - The software shall maintain the DTLS handshake transcript from the ClientHello u...

The software shall maintain the DTLS handshake transcript from the ClientHello until handshake completion by appending handshake messages in the order transmitted or successfully received.

Status	Draft
Verification Criteria	

23.1.3 HelloRetryRequest

MVC-107810 - The software shall maintain a retransmission buffer for transmitted DTLS handsha...

The software shall maintain a retransmission buffer for transmitted DTLS handshake records until acknowledgement is received or retransmission is no longer required.

Status	Draft
Verification Criteria	Verify that transmitted handshake records are retained in the retransmission buffer until acknowledged or no longer required.

MVC-107822 - The software shall start a retransmission timer after transmitting each DTLS han...

The software shall start a retransmission timer after transmitting each DTLS handshake flight requiring acknowledgement.

Status	Draft
Verification Criteria	Verify that the retransmission timer starts after transmission of a handshake flight requiring acknowledgement.

MVC-107820 - The software shall retransmit unacknowledged DTLS handshake records when the ret...

The software shall retransmit unacknowledged DTLS handshake records when the retransmission timer expires.

Status	Draft
Verification Criteria	Verify that unacknowledged handshake records are retransmitted after retransmission timeout expiration.

MVC-107826 - The software shall apply the configured retransmission timeout, timeout backoff...

The software shall apply the configured retransmission timeout, timeout backoff policy, maximum retransmission attempts, and handshake timeout during retransmission processing.

Status	Draft
Verification Criteria	Verify that the configured timeout, timeout backoff policy, retry limit, and handshake timeout are applied during retransmission processing.

MVC-107824 - The software shall terminate the DTLS handshake if the configured handshake time...

The software shall terminate the DTLS handshake if the configured handshake timeout expires or the maximum retransmission attempts are exceeded.

Status	Draft
Verification Criteria	Verify that the DTLS handshake is terminated when the configured retry limit or handshake timeout is exceeded.

23.1.4 ServerHello Validation

MVC-107964 - The software shall validate in ServerHello:

The software shall validate in ServerHello:
the selected PSK identity received from the Vehicle ECU against the configured PSK identity.
the selected cipher suite matches one of the configured supported cipher suites.
the selected key exchange mode matches the configured PSK-DHE mode.
the Vehicle ECU ECDHE public key before computing the shared secret.

Status	Draft
Verification Criteria	Verify successful validation of the selected PSK identity. Verify rejection when the selected PSK identity is invalid. Verify acceptance of configured cipher suites. Verify rejection of unsupported cipher suites. Verify successful validation of PSK-DHE negotiation. Verify rejection of unauthorized key exchange modes. Verify successful validation of valid public keys. Verify rejection of malformed or invalid public keys.

MVC-107969 - The software shall verify that the cipher suite selected by the Vehicle ECU is o...

The software shall verify that the cipher suite selected by the Vehicle ECU is one of the advertised and configured cipher suites.

Status	Draft
Verification Criteria	Verify that deprecated or unauthorized cipher suites are not advertised. Verify that the handshake is terminated when an unauthorized cipher suite is negotiated.

MVC-107968 - The software shall terminate the DTLS handshake if the Vehicle ECU selects a cip...

The software shall terminate the DTLS handshake if the Vehicle ECU selects a cipher suite that was not advertised or is not supported.

Status	Draft
Verification Criteria	

MVC-107971 - The software shall terminate the DTLS handshake if any mandatory ServerHello val...

The software shall terminate the DTLS handshake if any mandatory ServerHello validation fails.

Status	Draft
Verification Criteria	

MVC-107970 - The software shall record a DTLS handshake failure in the security log when prot...

The software shall record a DTLS handshake failure in the security log when protocol version negotiation fails due to an unsupported or unauthorized protocol version.

Status	Draft
Verification Criteria	Verify generation of a security event for protocol version negotiation failure like recording.

23.1.5 Secret Derivation

MVC-107993 - The software shall compute the ECDHE shared secret using the locally generated E...

The software shall compute the ECDHE shared secret using the locally generated ECDHE private key and the validated Vehicle ECU public key.

Status	Draft
Verification Criteria	Verify successful shared secret generation. Verify that shared secret generation fails when public key validation fails.

MVC-107991 - The software shall derive the DTLS handshake secrets using the configured PSK, t...

The software shall derive the DTLS handshake secrets using the configured PSK, the ECDHE shared secret, and the DTLS handshake transcript before processing encrypted handshake messages.

Status	Draft
Verification Criteria	Verify successful handshake secret derivation. Verify that derivation fails when any required input is invalid. Verify that encrypted handshake messages are processed only after handshake traffic secrets are derived.

MVC-107982 - The software shall derive the DTLS handshake write keys, write IVs, and sequence...

The software shall derive the DTLS handshake write keys, write IVs, and sequence number encryption keys from the handshake traffic secrets.

Status	Draft
Verification Criteria	

23.1.6 EncryptedExtensions

MVC-107995 - The software shall receive the DTLS EncryptedExtensions message after successful...

The software shall receive the DTLS EncryptedExtensions message after successful derivation of the server handshake traffic keys.

Status	Draft
Verification Criteria	Verify that the software processes the EncryptedExtensions message only after the server handshake traffic keys have been derived.

MVC-107994 - The software shall decrypt the received EncryptedExtensions message using the se...

The software shall decrypt the received EncryptedExtensions message using the server handshake traffic keys before processing its contents.

Status	Draft
Verification Criteria	Verify that the software decrypts the EncryptedExtensions message using the server handshake traffic keys. Verify that the software validates the decrypted EncryptedExtensions message before continuing the DTLS handshake.

MVC-107980 - The software shall terminate the DTLS handshake if the EncryptedExtensions messa...

The software shall terminate the DTLS handshake if the EncryptedExtensions message cannot be successfully decrypted or validated.

Status	Draft
Verification Criteria	Verify that the DTLS handshake is terminated if EncryptedExtensions decryption or validation fails.

23.1.7 Server Finished Verification

MVC-107977 - The software shall receive and process the encrypted Finished message transmitte...

The software shall receive and process the encrypted Finished message transmitted by the Vehicle ECU after successful completion of the preceding DTLS handshake messages.

Status	Draft
Verification Criteria	Verify that the encrypted Finished message is processed only after the required handshake secrets have been successfully derived. Verify successful decryption of a valid Finished message. Verify rejection when decryption fails.

MVC-107976 - The software shall decrypt the received Finished message using the derived serve...

The software shall decrypt the received Finished message using the derived server handshake traffic keys before performing verification.

Status	Draft
Verification Criteria	Verify that the DTLS handshake is terminated when Finished message decryption fails.

MVC-107973 - The software shall ensure that the verification confirms that the Vehicle ECU su...

The software shall ensure that the verification confirms that the Vehicle ECU successfully derived the same DTLS handshake secrets and processed the same handshake messages.

Status	Draft
Verification Criteria	Verify that the DTLS handshake is terminated when Finished message verification fails.

MVC-107972 - The software shall discard the received Finished message when decryption or veri...

The software shall discard the received Finished message when decryption or verification fails and prevent establishment of the DTLS secure session.

Status	Draft
Verification Criteria	Verify that the secure session is not established following Finished message verification failure.

MVC-107974 - The software shall return only a generic handshake failure indication to the com...

The software shall return only a generic handshake failure indication to the communication peer and shall not disclose whether the failure was caused by decryption, integrity verification, authentication failure, or handshake transcript validation.

Status	Draft
Verification Criteria	Verify that only a generic handshake failure indication is returned to the communication peer. Verify that detailed failure information is not disclosed through the communication interface.

23.1.8 Client Finished

MVC-107966 - The software shall send the DTLS Finished message after successful verification...

The software shall send the DTLS Finished message after successful verification of the Vehicle ECU Finished message.

Status	Draft
Verification Criteria	Verify that the DTLS Finished message is sent only after successful verification of the Vehicle ECU Finished message.

MVC-107965 - The software shall protect the DTLS Finished message using the client handshake...

The software shall protect the DTLS Finished message using the client handshake traffic keys.

Status	Draft
Verification Criteria	Verify that the DTLS Finished message is protected using the client handshake traffic keys.

MVC-107990 - The software shall terminate the DTLS handshake if generation or transmission of...

The software shall terminate the DTLS handshake if generation or transmission of the DTLS Finished message fails.

Status	Draft
Verification Criteria	Verify that the DTLS handshake is terminated if generation or transmission of the DTLS Finished message fails.

MVC-107989 - The software shall wait for acknowledgement of the final DTLS handshake flight b...

The software shall wait for acknowledgement of the final DTLS handshake flight before considering the handshake successfully completed or shall retransmit the final handshake flight according to the configured retransmission procedure.

Status	Draft
Verification Criteria	

23.1.9 Session Establishment

MVC-107984 - The software shall establish the DTLS secure session only after successful excha...

The software shall establish the DTLS secure session only after successful exchange and verification of Finished messages.

Status	Draft
Verification Criteria	Verify that the secure session is established only after successful Finished message exchange. Verify that the secure session is not established when either Finished message verification fails.

MVC-107983 - The software shall derive DTLS application traffic keys after successful complet...

The software shall derive DTLS application traffic keys after successful completion of the DTLS handshake using the DTLS 1.3 HKDF-SHA256 key schedule.

Status	Draft
Verification Criteria	Verify that application traffic keys are derived only after successful handshake completion. Verify that application traffic keys are not derived following handshake failure.

MVC-107987 - The software shall perform key derivation using the configured PSK, the ECDHE sh...

The software shall perform key derivation using the configured PSK, the ECDHE shared secret, and the DTLS handshake transcript.

Status	Draft
Verification Criteria	Verify that key derivation uses the configured PSK, ECDHE shared secret, and handshake transcript. Verify that DTLS processing is terminated when key derivation fails.

MVC-107986 - The software shall derive the DTLS application write keys, write IVs, and sequen...

The software shall derive the DTLS application write keys, write IVs, and sequence number encryption keys from the application traffic secrets after successful Finished verification.

Status	Draft
Verification Criteria	

MVC-107979 - The software shall securely erase the ephemeral ECDHE private key after successf...

The software shall securely erase the ephemeral ECDHE private key after successful derivation of the DTLS session keys or upon termination of the DTLS handshake.

Status	Draft
Verification Criteria	Verify that the ephemeral private key is securely erased after session key derivation or handshake termination.

MVC-107981 - The software shall enable protected SOME/IP communication and video streaming on...

The software shall enable protected SOME/IP communication and video streaming only after DTLS application traffic keys become available.

Status	Draft
Verification Criteria	Verify that protected communication is enabled only after application traffic keys are available. Verify that protected communication remains disabled when handshake completion fails.

23.1.10 Application Data protection

MVC-108000 - The software shall encapsulate application payloads within DTLS application reco...

The software shall encapsulate application payloads within DTLS application records before applying DTLS record protection.

Status	Draft
Verification Criteria	

MVC-107999 - The software shall use the active DTLS application write state, including the ap...

The software shall use the active DTLS application write state, including the application write key, write IV, epoch, and send sequence number, for transmission of protected application records.

Status	Draft
Verification Criteria	

MVC-108003 - The software shall encrypt all DTLS application records before transmission usin...

The software shall encrypt all DTLS application records before transmission using the negotiated AES-GCM cipher suite using the active DTLS application traffic keys and ensure that plaintext application data is not transmitted after DTLS session establishment.

Status	Draft
Verification Criteria	Verify that AES-GCM encryption is applied to transmitted DTLS records. Verify that encryption uses the active application traffic keys.

MVC-108002 - The software shall discard plaintext records received after establishment of the...

The software shall discard plaintext records received after establishment of the DTLS secure session.

Status	Draft
Verification Criteria	Verify that plaintext records are rejected after secure session establishment.

MVC-108005 - The software shall decrypt received DTLS protected records using the negotiated...

The software shall decrypt received DTLS protected records using the negotiated DTLS application traffic keys.

Status	Draft
Verification Criteria	Verify successful decryption of valid protected DTLS records. Verify that unauthenticated application data is not delivered to upper software layers.

MVC-108004 - The software shall verify the AEAD authentication tag before accepting the decry...

The software shall verify the AEAD authentication tag before accepting the decrypted record.

Status	Draft
Verification Criteria	Verify authentication tag verification for every received protected record. Verify rejection of records with invalid authentication tags.

MVC-107997 - The software shall accept Vehicle ECU application records only when they belong...

The software shall accept Vehicle ECU application records only when they belong to the active authenticated DTLS session.

Status	Draft
Verification Criteria	

23.1.10.1 Record Protection

MVC-108026 - The software shall generate a unique IV/nonce for each AES-GCM encryption operat...

The software shall generate a unique IV/nonce for each AES-GCM encryption operation performed during an active DTLS session.

Status	Draft
Verification Criteria	Verify generation of a unique IV/nonce for each protected DTLS record.

MVC-108024 - The software shall ensure that IV/nonce values are not reused with the same cryp...

The software shall ensure that IV/nonce values are not reused with the same cryptographic key.

Status	Draft
Verification Criteria	Verify that IV/nonce values are not reused with the same application traffic key.

MVC-108017 - The software shall terminate record protection processing if a valid IV/nonce ca...

The software shall terminate record protection processing if a valid IV/nonce cannot be generated.

Status	Draft
Verification Criteria	Verify that record protection fails when IV/nonce generation fails.

MVC-108015 - The software shall prevent export of DTLS cryptographic key material outside the...

The software shall prevent export of DTLS cryptographic key material outside the secure cryptographic boundary and prevent unauthorized software components from accessing DTLS cryptographic key material.

Status	Draft
Verification Criteria	Verify that DTLS cryptographic operations are performed within the configured secure cryptographic boundary. Verify that cryptographic key material is not exported outside the secure cryptographic boundary. Verify that unauthorized access to DTLS cryptographic key material is prevented. Verify that DTLS processing is terminated following an unrecoverable cryptographic service error.

23.1.10.2 Sequence Numbers

MVC-108029 - The software shall maintain an independent DTLS record sequence number for each...

The software shall maintain an independent DTLS record sequence number for each active DTLS session and initialize the DTLS record sequence number when a new DTLS session is established.

Status	Draft
Verification Criteria	

MVC-108028 - The software shall assign a unique sequence number to every transmitted DTLS pro...

The software shall assign a unique sequence number to every transmitted DTLS protected record.

Status	Draft
Verification Criteria	

MVC-108014 - The software shall prevent reuse of DTLS record sequence numbers within the same...

The software shall prevent reuse of DTLS record sequence numbers within the same DTLS session.

Status	Draft
Verification Criteria	Verify that sequence numbers are not reused within an active DTLS session.

23.1.10.3 Anti-Replay

MVC-108011 - The software shall maintain an anti-replay receive window for each active DTLS s...

The software shall maintain an anti-replay receive window for each active DTLS session.

Status	Draft
Verification Criteria	Verify that an independent anti-replay receive window is maintained for each active DTLS session.

MVC-108010 - The software shall verify that the received DTLS record sequence number falls wi...

The software shall verify that the received DTLS record sequence number falls within the configured anti-replay receive window before accepting the record.

Status	Draft
Verification Criteria	Verify that received DTLS record sequence numbers are validated against the configured anti-replay receive window.

MVC-108007 - The software shall reject duplicated DTLS records and records outside the config...

The software shall reject duplicated DTLS records and records outside the configured anti-replay receive window.

Status	Draft
Verification Criteria	Verify that duplicated or out-of-window DTLS records are rejected.

MVC-108006 - The software shall update the anti-replay receive window after successful accept...

The software shall update the anti-replay receive window after successful acceptance of a DTLS record.

Status	Draft
Verification Criteria	Verify that the anti-replay receive window is updated after successful acceptance of a DTLS record.

23.1.10.4 Record Size

MVC-108012 - The software shall limit the DTLS application record payload size according to t...

The software shall limit the DTLS application record payload size according to the configured DTLS record size and path MTU.

Status	Draft
Verification Criteria	Verify that DTLS application record payload size complies with the configured maximum DTLS record size and path MTU.

MVC-108001 - The software shall prevent transmission of DTLS application records that exceed...

The software shall prevent transmission of DTLS application records that exceed the configured runtime record size limits.

Status	Draft
Verification Criteria	Verify that transmission of DTLS application records exceeding the configured runtime record size limits is prevented.

23.1.10.5 Freshness Protection

MVC-108022 - The software shall generate the freshness value as either a monotonic counter or...

The software shall generate the freshness value as either a monotonic counter or a timestamp in outgoing SOME/IP responses and streaming video frames

Status	Draft
Verification Criteria	Verify that a freshness value is included in outgoing SOME/IP responses and streaming video frames when required. Verify that the configured freshness mechanism (counter or timestamp) is used. Verify that transmission is prevented when freshness generation fails.

MVC-108027 - The software shall ensure that freshness values are not reused within an active...

The software shall ensure that freshness values are not reused within an active DTLS session.

Status	Draft
Verification Criteria	Verify that freshness values are not reused within an active DTLS session.

MVC-108025 - The software shall verify the freshness value contained in received SOME/IP cont...

The software shall verify the freshness value contained in received SOME/IP control messages and streaming video frames.

Status	Draft
Verification Criteria	Verify that freshness values are validated upon reception.

MVC-108018 - The software shall verify that the received freshness value satisfies the config...

The software shall verify that the received freshness value satisfies the configured ordering requirements and reject messages or streaming video frames whose freshness value is duplicated, expired, or outside the configured acceptance window.

Status	Draft
Verification Criteria	Verify rejection of duplicated freshness values. Verify rejection of expired or out-of-window freshness values.

MVC-108016 - The software shall discard rejected messages or streaming video frames before de...

The software shall discard rejected messages or streaming video frames before delivery to the upper software layer and continue processing subsequent valid messages after rejecting an invalid freshness value.

Status	Draft
Verification Criteria	Verify that rejected messages are not delivered to the application. Verify that valid subsequent messages continue to be processed.

23.1.11 DTLS Fragmentation

MVC-108013 - The software shall fragment outgoing DTLS handshake messages that exceed the con...

The software shall fragment outgoing DTLS handshake messages that exceed the configured DTLS handshake record size or path MTU.

Status	Draft
Verification Criteria	Verify that DTLS handshake messages exceeding the configured record size or path MTU are fragmented before transmission.

MVC-108036 - The software shall populate the DTLS handshake fragmentation fields, including f...

The software shall populate the DTLS handshake fragmentation fields, including fragment_offset and fragment_length, for each transmitted handshake fragment.

Status	Draft
Verification Criteria	Verify that fragment_offset and fragment_length are correctly populated for transmitted handshake fragments.

MVC-108035 - The software shall reassemble fragmented DTLS handshake messages before performi...

The software shall reassemble fragmented DTLS handshake messages before performing handshake message processing.

Status	Draft
Verification Criteria	Verify that fragmented handshake messages are successfully reassembled before handshake processing.

MVC-108038 - The software shall reject incomplete, malformed, overlapping, or inconsistent ha...

The software shall reject incomplete, malformed, overlapping, or inconsistent handshake fragments.

Status	Draft
Verification Criteria	Verify that incomplete, malformed, overlapping, or inconsistent handshake fragments are rejected.

MVC-108037 - The software shall terminate the DTLS handshake if handshake message reassembly...

The software shall terminate the DTLS handshake if handshake message reassembly cannot be successfully completed.

Status	Draft
Verification Criteria	Verify that the DTLS handshake is terminated when handshake message reassembly fails.

23.1.12 Error Handling and Security Logging

MVC-108041 - The software shall terminate the DTLS handshake when negotiation of an unsupport...

The software shall terminate the DTLS handshake when negotiation of an unsupported DTLS feature is detected.

Status	Draft
Verification Criteria	Verify that the DTLS handshake is terminated when negotiation of an unsupported DTLS feature is detected.

MVC-108040 - The software shall generate a log entry immediately after detection of a securit...

The software shall generate a log entry immediately after detection of a security DTLS event.

Status	Draft
Verification Criteria	Verify that DTLS security events are recorded. Verify that a log entry is generated immediately after event detection.

MVC-108030 - The software shall generate a security event log entry for malformed records, in...

The software shall generate a security event log entry for malformed records, invalid epoch, invalid sequence number, replay detection, AEAD authentication failure, decryption failure, freshness verification failure, and unauthorized command delivery attempts.

Status	Draft
Verification Criteria	

MVC-108032 - The software shall ensure that failure to record one security event does not pre...

The software shall ensure that failure to record one security event does not prevent processing of subsequent security events.

Status	Draft
Verification Criteria	Verify that failure to record one security event does not prevent processing of subsequent security events.

MVC-108031 - The software shall prevent modification or deletion of recorded DTLS security lo...

The software shall prevent modification or deletion of recorded DTLS security logs by unauthorized entities.

Status	Draft
Verification Criteria	Verify that unauthorized modification of security logs is prevented. Verify that unauthorized deletion of security logs is prevented.

MVC-108034 - The software shall ensure that DTLS security logs do not contain PSKs, DTLS sess...

The software shall ensure that DTLS security logs do not contain PSKs, DTLS session keys, handshake traffic keys, application traffic keys, ECDHE private keys, shared secrets, or other confidential cryptographic material.

Status	Draft
Verification Criteria	Verify that confidential cryptographic material is not stored in security logs.

MVC-108033 - The software shall permit access to DTLS security logs only for authorized roles...

The software shall permit access to DTLS security logs only for authorized roles, authorized diagnostic sessions, or authorized service functions.

Status	Draft
Verification Criteria	Verify that unauthorized users cannot access protected log contents.

23.2 Secure Communication (SOW2 - SOW1)

23.2.1 Root Trust Initialization

MVC-108143 - The software shall store the trusted Root RSA public key used for SOW1 device au...

The software shall store the trusted Root RSA public key used for SOW1 device authentication in protected non-volatile storage.

Status	Draft
Verification Criteria	Verify that the Root RSA public key is stored in protected non-volatile storage.

MVC-108142 - The software shall prevent unauthorized modification, replacement, deletion, or...

The software shall prevent unauthorized modification, replacement, deletion, or disclosure of the stored Root RSA public key.

Status	Draft
Verification Criteria	Verify that unauthorized modification or disclosure of the Root RSA public key is prevented.

MVC-108144 - The software shall reject SOW1 device authentication if the Root RSA public key...

The software shall reject SOW1 device authentication if the Root RSA public key cannot be successfully loaded or validated.

Status	Draft
Verification Criteria	Verify that authentication is rejected if the Root RSA public key cannot be loaded or validated.

23.2.2 Authentication Data Request

MVC-108133 - The software shall request the SOW1 authentication data after system reset or be...

The software shall request the SOW1 authentication data after system reset or before initiating the secured SOW1 video stream.

Status	Draft
Verification Criteria	Verify that authentication data is requested after reset or before secured video streaming.

MVC-108131 - The software shall read the Device_n, KID, and Sig_device values from SOW1 throu...

The software shall read the Device_n, KID, and Sig_device values from SOW1 through the I2C interface.

Status	Draft
Verification Criteria	Verify successful reception of Device_n, KID, and Sig_device.

MVC-108126 - The software shall verify that all required authentication fields are successful...

The software shall verify that all required authentication fields are successfully received before initiating device authentication.

Status	Draft
Verification Criteria	Verify rejection of incomplete authentication data.

MVC-108124 - The software shall terminate the authentication procedure if the authentication...
The software shall terminate the authentication procedure if the authentication data cannot be completely received.

Status	Draft
Verification Criteria	Verify that session establishment does not begin before authentication data acquisition completes.

23.2.3 Device Authentication

MVC-108136 - The software shall verify that the received KID corresponds to a supported crypt...
The software shall verify that the received KID corresponds to a supported cryptographic configuration.

Status	Draft
Verification Criteria	Verify selection of the cryptographic algorithm associated with the received KID.

MVC-108135 - The software shall select the RSA signature verification scheme and hash algorit...
The software shall select the RSA signature verification scheme and hash algorithm corresponding to the received KID.

Status	Draft
Verification Criteria	Verify selection of the cryptographic algorithm associated with the received KID.

MVC-108120 - The software shall perform RSA signature verification using the RSA verification...
The software shall perform RSA signature verification using the RSA verification scheme and hash algorithm selected for the received KID.

Status	Draft
Verification Criteria	

MVC-108113 - The software shall authenticate SOW1 by verifying Sig_device over Device_n and K...
The software shall authenticate SOW1 by verifying Sig_device over Device_n and KID using the trusted Root RSA public key.

Status	Draft
Verification Criteria	Verify signature verification using the trusted Root RSA public key. Verify that trusted communication is not permitted before successful authentication.

MVC-108111 - The software shall reject device authentication if Device_n, KID, Sig_device, or...
The software shall reject device authentication if Device_n, KID, Sig_device, or the selected cryptographic configuration is invalid, missing, malformed, unsupported, or if signature verification fails.

Status	Draft
Verification Criteria	Verify rejection of malformed or unsupported authentication data. Verify rejection when signature verification fails.

23.2.4 Secure Session Establishment

23.2.4.1 Session Key Generation

MVC-108107 - The software shall generate a fresh symmetric session key for each new SOW1–SOW2...
The software shall generate a fresh symmetric session key for each new SOW1–SOW2 secure session following successful completion of SOW1 device authentication.

Status	Draft
Verification Criteria	Verify that a new session key is generated for every new secure session.

MVC-108106 - The software shall generate the session key using a Cryptographically Secure Pse...
The software shall generate the session key using a Cryptographically Secure Pseudo-Random Number Generator (CSPRNG) seeded from the hardware TRNG.

Status	Draft
Verification Criteria	Verify that the session key is generated using the approved entropy source.

MVC-108110 - The software shall prevent reuse of session keys across independent SOW1–SOW2 se...
The software shall prevent reuse of session keys across independent SOW1–SOW2 secure sessions.

Status	Draft
Verification Criteria	Verify that session keys are not reused across independent secure sessions.

MVC-108109 - The software shall terminate secure session establishment if session key generat...
The software shall terminate secure session establishment if session key generation cannot be completed successfully.

Status	Draft
Verification Criteria	Verify that secure session establishment is terminated when session key generation fails.

23.2.4.2 Row Selection Generation

MVC-108101 - The software shall generate row selection information for AES-CMAC calculation a...
The software shall generate row selection information for AES-CMAC calculation after successful session key generation

Status	Draft
Verification Criteria	Verify that row selection information is associated with the active secure session.

MVC-108166 - The software shall generate row selection information including the start row, r...
The software shall generate row selection information including the start row, row interval, and total number of selected rows.

Status	Draft
Verification Criteria	Verify generation of start row, row interval, and total selected rows.

MVC-108165 - The software shall generate two independent row selection information sets (Row_...
The software shall generate two independent row selection information sets (Row_info_00 and Row_info_01) for the active secure session

Status	Draft
Verification Criteria	

MVC-108170 - The software shall maintain the generated row selection information unchanged th...
The software shall maintain the generated row selection information unchanged throughout the active secure session unless a new secure session is established.

Status	Draft
Verification Criteria	Verify that row selection information remains unchanged during the active session.

MVC-108168 - The software shall terminate secure session establishment if row selection infor...
The software shall terminate secure session establishment if row selection information cannot be successfully generated.

Status	Draft
Verification Criteria	Verify that session establishment is terminated when row selection generation fails.

23.2.4.3 Session Key Protection

MVC-108164 - The software shall store the active session key only in protected memory allocat...
The software shall store the active session key only in protected memory allocated for the active secure session (DRAM).

Status	Draft
Verification Criteria	Verify that the session key is stored only within the authorized security processing context.

MVC-108163 - The software shall prevent unauthorized software components from accessing the g...

The software shall prevent unauthorized software components from accessing the generated session key.

Status	Draft
Verification Criteria	Verify that only authorized software components can access the active session key.

MVC-108154 - The software shall prevent modification of the active session key after successf...
The software shall prevent modification of the active session key after successful generation.

Status	Draft
Verification Criteria	Verify that unauthorized modification of the session key is prevented.

MVC-108157 - The software shall prevent storage of the session key in application logs, diagn...
The software shall prevent storage of the session key in application logs, diagnostic responses, debug interfaces, memory dumps, or non-secure storage.

Status	Draft
Verification Criteria	Verify that the session key is not exposed through logs, diagnostic responses, debug interfaces, trace outputs, memory dumps, or non-secure storage.

23.2.4.4 Key Sharing

MVC-108148 - The software shall perform encryption only after successful completion of SOW1 d...
The software shall perform encryption only after successful completion of SOW1 device authentication.

Status	Draft
Verification Criteria	Verify that encryption occurs only after successful device authentication.

MVC-108153 - The software shall encrypt the generated session key and the associated row sele...
The software shall encrypt the generated session key and the associated row selection information before transmission to SOW1 using the authenticated SOW1 Device public key.

Status	Draft
Verification Criteria	Verify that the session key and row selection information are encrypted before transmission. Verify that encryption uses the authenticated SOW1 Device public key.

MVC-108152 - The software shall terminate secure session establishment if encryption of the s...
The software shall terminate secure session establishment if encryption of the session key or row selection information fails.

Status	Draft
Verification Criteria	Verify that plaintext session establishment information is never transmitted.

MVC-108146 - The software shall transmit the encrypted session key and encrypted row selectio...
The software shall transmit the encrypted session key and encrypted row selection information to SOW1 through the I2C interface after successful SOW1 authentication.

Status	Draft
Verification Criteria	

MVC-108145 - The software shall prevent transmission of unencrypted session establishment inf...
The software shall prevent transmission of unencrypted session establishment information over the I2C interface.

Status	Draft
Verification Criteria	Verify that session establishment is terminated when encryption fails.

MVC-108147 - The software shall securely erase temporary copies of the generated session key...
The software shall securely erase temporary copies of the generated session key after completion of the key-sharing procedure.

Status	Draft
Verification Criteria	Verify that temporary copies and working buffers containing the session key are securely erased after use.

MVC-108138 - The software shall prevent acceptance of authenticated video frames until succes...
The software shall prevent acceptance of authenticated video frames until successful completion of secure session establishment.

Status	Draft
Verification Criteria	Verify that authenticated video communication is blocked until secure session establishment completes.

23.2.5 Video Authentication

23.2.5.1 Authentication Tag Extraction

MVC-108140 - The software shall extract the embedded AES-CMAC authentication tag from the foo...
The software shall extract the embedded AES-CMAC authentication tag from the footer of each received SOW1 video frame.

Status	Draft
Verification Criteria	Verify extraction of the embedded AES-CMAC tag from each received frame.

MVC-108129 - The software shall verify that the authentication tag is present before initiati...
The software shall verify that the authentication tag is present before initiating video frame authentication.

Status	Draft
Verification Criteria	Verify rejection of frames with missing authentication tags.

MVC-108134 - The software shall reject the received video frame if the authentication tag is...
The software shall reject the received video frame if the authentication tag is missing or malformed.

Status	Draft
Verification Criteria	Verify rejection of malformed authentication tags.

MVC-108132 - The software shall prevent delivery of the received video frame for further proc...
The software shall prevent delivery of the received video frame for further processing until authentication tag extraction has completed successfully.

Status	Draft
Verification Criteria	Verify that frame processing does not continue before successful authentication tag extraction.

23.2.5.2 AES-CMAC Verification

MVC-108118 - The software shall select the active row selection information for AES-CMAC calc...
The software shall select the active row selection information for AES-CMAC calculation according to the configured security selection state of the active secure session

Status	Draft
Verification Criteria	

MVC-108122 - The software shall recalculate the AES-CMAC authentication tag for each received...
The software shall recalculate the AES-CMAC authentication tag for each received SOW1 video frame using the active session key.

Status	Draft
Verification Criteria	Verify recalculation of the AES-CMAC for each received frame.

MVC-108121 - The software shall calculate the AES-CMAC only over the configured image rows de...
The software shall calculate the AES-CMAC only over the configured image rows defined by the row selection information established for the active secure session.

Status	Draft

Verification Criteria	Verify that only the configured image rows are included in the calculation.

MVC-108114 - The software shall terminate video authentication processing if AES-CMAC calcula...

The software shall terminate video authentication processing if AES-CMAC calculation cannot be completed successfully and prevent acceptance of the received video frames.

Status	Draft
Verification Criteria	Verify rejection of the frame when AES-CMAC calculation fails. Verify rejection of the received video frames.

MVC-108112 - The software shall compare the recalculated AES-CMAC authentication tag with the...

The software shall compare the recalculated AES-CMAC authentication tag with the authentication tag extracted from the received video frame.

Status	Draft
Verification Criteria	Verify comparison of calculated and received AES-CMAC tags.

MVC-108117 - The software shall accept the received video frame only when the calculated and...

The software shall accept the received video frame only when the calculated and received AES-CMAC authentication tags match.

Status	Draft
Verification Criteria	Verify acceptance of frames only when the tags match.

MVC-108116 - The software shall reject and discard the received video frame if AES-CMAC verif...

The software shall reject and discard the received video frame if AES-CMAC verification fails.

Status	Draft
Verification Criteria	

MVC-108108 - The software shall generate a security event log entry when AES-CMAC verificatio...

The software shall generate a security event log entry when AES-CMAC verification of a received video frame fails.

Status	Draft
Verification Criteria	

MVC-108105 - The software shall continue processing subsequent received video frames followin...

The software shall continue processing subsequent received video frames following rejection of an individual video frame.

Status	Draft
Verification Criteria	Verify continued processing of subsequent frames when permitted

23.2.5.3 Freshness Verification

MVC-108099 - The software shall verify the freshness of frame counter contained in each recei...

The software shall verify the freshness of frame counter contained in each received SOW1 video frame.

Status	Draft
Verification Criteria	Verify that freshness information is verified when provided by SOW1.

MVC-108104 - The software shall compare the received freshness information with the previousl...

The software shall compare the received freshness information with the previously accepted freshness information associated with the active secure session.

Status	Draft
Verification Criteria	Verify detection of duplicated freshness values. Verify rejection of stale freshness values. Verify rejection of out-of-order video frames.

MVC-108102 - The software shall reject duplicated, stale, or out-of-order video frames when f...

The software shall reject duplicated, stale, or out-of-order video frames when freshness verification fails.

Status	Draft
Verification Criteria	

MVC-108167 - The software shall continue processing subsequent valid video frames after rejec...

The software shall continue processing subsequent valid video frames after rejecting an invalid frame.

Status	Draft
Verification Criteria	Verify that subsequent valid frames continue to be processed.

23.2.6 Protected Control Commands

MVC-108169 - The software shall transmit protected SOW1 control commands only after successfu...

The software shall transmit protected SOW1 control commands only after successful completion of SOW1 device authentication.

Status	Draft
Verification Criteria	Verify that protected control commands are transmitted only after successful authentication. Verify that a valid secure session exists before transmission. Verify rejection of transmission requests when no active session exists. Verify that unauthenticated protected commands are never transmitted.

MVC-108159 - The software shall include a monotonic counter in each protected SOW1 control co...

The software shall include a monotonic counter in each protected SOW1 control command before generating the AES-CMAC authentication tag.

Status	Draft
Verification Criteria	Verify inclusion of freshness information in protected control commands. Verify that freshness values are unique within the active secure session. Verify that transmission is prevented when freshness generation fails.

MVC-108161 - The software shall ensure that freshness values are not reused within the active...

The software shall ensure that freshness values are not reused within the active secure session.

Status	Draft
Verification Criteria	Verify that freshness values are not reused.

MVC-108160 - The software shall generate and append an AES-CMAC authentication tag for each t...

The software shall generate and append an AES-CMAC authentication tag for each transmitted protected SOW1 control command only when protected control command verification is supported by SOW1.

Status	Draft
Verification Criteria	Verify generation of an AES-CMAC for each protected control command. Verify that the active session key is used. Verify that the authentication tag is appended before transmission. Verify that command transmission is prevented when CMAC generation fails.

23.2.7 Failure Handling

MVC-108158 - The software shall invalidate the active SOW1–SOW2 session key upon system reset...

The software shall invalidate the active SOW1–SOW2 session key upon system reset, system restart, secure session termination, authentication failure, key sharing failure, or video authentication failure requiring session recovery.

Status	Draft
Verification Criteria	Verify session key invalidation under all specified conditions. Verify secure erasure of invalidated session keys. Verify that invalidated keys cannot be reused.

MVC-108156 - The software shall require establishment of a new SOW1–SOW2 secure session follo...

The software shall require establishment of a new SOW1–SOW2 secure session following session expiry, system reset, system restart, secure session termination, authentication failure, key sharing failure, or video authentication failure requiring session recovery.

Status	Draft
Verification Criteria	Verify that secure session re-establishment is required under all specified conditions.

MVC-108151 - The software shall perform SOW1 device authentication before establishing the ne...

The software shall perform SOW1 device authentication before establishing the new secure session.

Status	Draft
Verification Criteria	Verify that SOW1 device authentication is repeated. Verify generation of a new session key and row selection information. Verify that trusted video frames are rejected until the new secure session becomes active.

MVC-108149 - The software shall terminate the secure session establishment procedure if compl...

The software shall terminate the secure session establishment procedure if completion is not achieved within the configured timeout period or retry limit.

Status	Draft
Verification Criteria	Verify monitoring of secure session establishment duration. Verify termination when the configured timeout or retry limit is exceeded. Verify invalidation of partially established session information. Verify transition to the Inactive state. Verify initiation of recovery handling.

23.3 Secure Debug

MVC-107859 - The SW shall keep the hardware debug interface, including JTAG, in the locked st...

The SW shall keep the hardware debug interface, including JTAG, in the locked state by default when the ECU is in production mode.

Status	Draft
Verification Criteria	Verify that when the ECU is in production mode, the hardware debug interface, including JTAG, is locked by default after power-on/reset and cannot be used before successful secure debug unlock.

MVC-107863 - The SW shall permit hardware debug unlocking only through the approved UART-base...

The SW shall permit hardware debug unlocking only through the approved UART-based secure debug unlock service.

Status	Draft
Verification Criteria	Verify that hardware debug unlocking is permitted only through the approved UART-based secure debug unlock service.

MVC-107862 - The SW shall reject any hardware debug unlock attempt received through any inter...

The SW shall reject any hardware debug unlock attempt received through any interface other than the configured dedicated UART service interface [X] [UART0, UART1, dedicated service UART, OEM-defined debug UART channel].

Status	Draft
Verification Criteria	Verify that any hardware debug unlock attempt received through an interface other than the configured dedicated UART service interface is rejected and does not enable the hardware debug interface.

MVC-107865 - The SW shall accept a debug unlock challenge request only when the request forma...

The SW shall accept a debug unlock challenge request only when the request format is valid and the ECU lifecycle state permits debug unlock processing.

Status	Draft
Verification Criteria	Verify that a debug unlock challenge request is accepted only when the request format is valid and the ECU lifecycle state permits debug unlock processing. Invalid format or unauthorized lifecycle state shall be rejected.

MVC-107864 - The SW shall generate a fresh debug unlock challenge, KDF nonce, KDF salt, and S...

The SW shall generate a fresh debug unlock challenge, KDF nonce, KDF salt, and Session_ID for each valid debug unlock transaction using the configured random source [X] [platform TRNG, TRNG-seeded CSPRNG].

Status	Draft
Verification Criteria	Verify that for each valid debug unlock transaction, the SW generates a fresh Challenge, KDF Nonce, KDF Salt, and Session_ID using the configured random source.

MVC-107857 - The SW shall ensure that the generated Challenge, Nonce, Salt, and Session_ID ar...

The SW shall ensure that the generated Challenge, Nonce, Salt, and Session_ID are unique per active debug unlock transaction and shall not reuse them for another transaction.

Status	Draft
Verification Criteria	Verify that the generated Challenge, Nonce, Salt, and Session_ID are unique per active debug unlock transaction and are not reused across transactions.

MVC-107889 - The SW shall construct the debug unlock transaction context using the following...

The SW shall construct the debug unlock transaction context using the following fields: Challenge, ECU_ID, Nonce, Salt, Session_ID, and Validity_Period.

Status	Draft
Verification Criteria	Verify that the debug unlock transaction context is constructed using the required fields: Challenge, ECU_ID, Nonce, Salt, Session_ID, and Validity_Period.

MVC-107887 - The SW shall bind the debug unlock transaction context to the target ECU identit...

The SW shall bind the debug unlock transaction context to the target ECU identity format [X] [ECU serial number, ECU unique ID, VIN-bound ECU ID, OEM-defined ECU ID] and ECU identity encoding [X] [ASCII, UTF-8, binary].

Status	Draft
Verification Criteria	Verify that the debug unlock transaction context is bound to the target ECU identity using the configured ECU identity format and encoding. A response using an incorrect ECU_ID shall be rejected.

MVC-107893 - The SW shall configure the challenge validity period as [X] [30 seconds, 60 seco...

The SW shall configure the challenge validity period as [X] [30 seconds, 60 seconds, 120 seconds] and shall associate this validity period with the active debug unlock transaction context.

Status	Draft
Verification Criteria	Verify that the configured challenge validity period is associated with the active debug unlock transaction and that responses received after expiry are rejected.

MVC-107891 - The SW shall transmit to the debug tool only the debug unlock transaction contex...

The SW shall transmit to the debug tool only the debug unlock transaction context containing Challenge, ECU_ID, Nonce, Salt, Session_ID, and Validity_Period.

Status	Draft
Verification Criteria	Verify that the SW transmits to the debug tool only the debug unlock transaction context containing Challenge, ECU_ID, Nonce, Salt, Session_ID, and Validity_Period, without exposing secret material.

MVC-107881 - The SW shall not transmit, process, or expect any algorithm identifier field dur...

The SW shall not transmit, process, or expect any algorithm identifier field during the secure debug unlock transaction because the secure debug mechanism uses a single fixed cryptographic profile.

Status	Draft
Verification Criteria	Verify that no algorithm identifier field is transmitted, processed, or expected during the secure debug unlock transaction. Any unexpected algorithm identifier shall not affect the fixed cryptographic profile.

MVC-107879 - The SW shall derive the DebugUnlockSessionKey from the ECU-specific pre-shared d...

The SW shall derive the DebugUnlockSessionKey from the ECU-specific pre-shared debug authentication key using the configured KDF type [X] [CMAC-based KDF in counter mode, HKDF-SHA256, OEM-approved KDF].

Status	Draft
Verification Criteria	Verify that the DebugUnlockSessionKey is derived from the ECU-specific pre-shared debug authentication key using only the configured KDF type.

MVC-107885 - The SW shall use the following KDF input fields for DebugUnlockSessionKey deriva...

The SW shall use the following KDF input fields for DebugUnlockSessionKey derivation: PSK, Nonce, Salt, ECU_ID, Session_ID, and debug unlock context [X] [SecureDebugUnlock, OEM-defined context string].

Status	Draft
Verification Criteria	Verify that the KDF input fields include PSK, Nonce, Salt, ECU_ID, Session_ID, and the configured debug unlock context string.

MVC-107883 - The SW shall not use the ECU-specific pre-shared debug authentication key direct...

The SW shall not use the ECU-specific pre-shared debug authentication key directly as the AES-CMAC key for unlock response verification.

Status	Draft
Verification Criteria	Verify that the ECU-specific pre-shared debug authentication key is not used directly as the AES-CMAC key for unlock response verification.

MVC-107896 - The SW shall verify the debug unlock response using AES-CMAC with the derived De...

The SW shall verify the debug unlock response using AES-CMAC with the derived DebugUnlockSessionKey.

Status	Draft
Verification Criteria	Verify that a valid unlock response generated using AES-CMAC with the derived DebugUnlockSessionKey is accepted, and an invalid AES-CMAC response is rejected.

MVC-107895 - The SW shall construct the AES-CMAC protected input using the fixed field order:...

The SW shall construct the AES-CMAC protected input using the fixed field order: Challenge || ECU_ID || Session_ID || Nonce || Salt || Validity_Period.

Status	Draft
Verification Criteria	Verify that the AES-CMAC protected input is constructed using the fixed field order: Challenge || ECU_ID || Session_ID || Nonce || Salt || Validity_Period.

MVC-107876 - The SW shall encode the debug unlock transaction context using a fixed canonical...

The SW shall encode the debug unlock transaction context using a fixed canonical serialization format [X] [fixed-order binary concatenation, TLV format, OEM-defined canonical format] before AES-CMAC verification.

Status	Draft
Verification Criteria	Verify that the debug unlock transaction context is encoded using the configured fixed canonical serialization format before AES-CMAC verification.

MVC-107871 - The SW shall encode multi-byte numeric fields in the debug unlock transaction co...

The SW shall encode multi-byte numeric fields in the debug unlock transaction context using byte order [X] [big-endian / network byte order, little-endian].

Status	Draft
Verification Criteria	Verify that all multi-byte numeric fields in the debug unlock transaction context are encoded using the configured byte order.

MVC-107870 - The SW shall maintain only one active debug unlock transaction at a time unless...

The SW shall maintain only one active debug unlock transaction at a time unless another approved policy is configured by response validation policy [X] [strict single-use challenge, single active challenge only, multiple pending challenges not allowed].

Status	Draft
Verification Criteria	Verify that the SW maintains only one active debug unlock transaction at a time unless another approved response validation policy is configured.

MVC-107873 - The SW shall reject any unlock response that is expired, reused, malformed, inva...

The SW shall reject any unlock response that is expired, reused, malformed, invalid, unauthorized, or not associated with the active debug unlock transaction context.

Status	Draft
Verification Criteria	Verify that expired, reused, malformed, invalid, unauthorized, or non-associated unlock responses are rejected and do not enable the hardware debug interface.

MVC-107872 - The SW shall validate the received unlock response against the active transactio...

The SW shall validate the received unlock response against the active transaction context, including Challenge, ECU_ID, Session_ID, Nonce, Salt, and Validity_Period.

Status	Draft
Verification Criteria	Verify that the received unlock response is validated against the active transaction context, including Challenge, ECU_ID, Session_ID, Nonce, Salt, and Validity_Period.

MVC-107867 - The SW shall enable the hardware debug interface only after the debug unlock res...

The SW shall enable the hardware debug interface only after the debug unlock response has been successfully authenticated and verified inside the secure execution environment.

Status	Draft
Verification Criteria	Verify that the hardware debug interface is enabled only after the debug unlock response has been successfully authenticated and verified inside the secure execution environment.

MVC-107866 - The SW shall prevent AES-CMAC key material, KDF intermediate values, derived deb...

The SW shall prevent AES-CMAC key material, KDF intermediate values, derived debug session keys, and unlock verification material from being exposed through UART messages, logs, diagnostic responses, memory dumps, or non-secure storage.

Status	Draft
Verification Criteria	Verify that AES-CMAC key material, KDF intermediate values, derived debug session keys, and unlock verification material are not exposed through UART messages, logs, diagnostic responses, memory dumps, or non-secure storage.

MVC-107869 - The SW shall import or decrypt the ECU-specific pre-shared debug authentication...

The SW shall import or decrypt the ECU-specific pre-shared debug authentication key only inside the protected execution environment using the ECU-specific OTP DEK.

Status	Draft
Verification Criteria	Verify that the ECU-specific pre-shared debug authentication key is imported or decrypted only inside the protected execution environment using the ECU-specific OTP DEK/KEK.

MVC-107868 - The SW shall store the ECU-specific pre-shared debug authentication key only as...

The SW shall store the ECU-specific pre-shared debug authentication key only as part of an encrypted and authenticated ECU-specific key package in the configured SOW2 secure partition [X] [SPI secure partition, encrypted secure partition, secure storage partition].

Status	Draft
Verification Criteria	Verify that the ECU-specific pre-shared debug authentication key is stored only as part of an encrypted and authenticated ECU-specific key package in the configured SOW2 secure partition.

MVC-107874 - The SW shall generate derived debug unlock session keys only for the active debu...

The SW shall generate derived debug unlock session keys only for the active debug unlock attempt and shall not persistently store the derived session key.

Status	Draft
Verification Criteria	Verify that derived debug unlock session keys are generated only for the active debug unlock attempt and are not persistently stored.

MVC-107861 - The SW shall invalidate debug unlock temporary material, including KDF intermedi...

The SW shall invalidate debug unlock temporary material, including KDF intermediate values and derived session keys, after unlock verification completes, fails, expires, or when the active transaction is terminated.

Status	Draft
Verification Criteria	Verify that debug unlock temporary material, including KDF intermediate values and derived session keys, is invalidated after verification completes, fails, expires, or when the active transaction is terminated.

MVC-107860 - The SW shall unlock the hardware debug interface only for a limited authorized d...

The SW shall unlock the hardware debug interface only for a limited authorized debug session and shall automatically re-lock it after session timeout [X] [5 minutes, 10 minutes, 15 minutes, 30 minutes] or upon completion of authorized debug activities.

Status	Draft
Verification Criteria	Verify that after successful unlock, the hardware debug interface remains unlocked only for the limited authorized debug session and is automatically re-locked after the configured timeout or completion of authorized debug activities.

MVC-107890 - The SW shall return the hardware debug interface to the locked state after every...

The SW shall return the hardware debug interface to the locked state after every Power-On Reset.

Status	Draft

Verification Criteria	Verify that the hardware debug interface returns to the locked state after every Power-On Reset.

MVC-107888 - The SW shall invalidate any active debug unlock session when the lifecycle state...

The SW shall invalidate any active debug unlock session when the lifecycle state changes or when a security violation is detected.

Status	Draft
Verification Criteria	Verify that any active debug unlock session is invalidated when the lifecycle state changes or when a security violation is detected.

MVC-107894 - The SW shall maintain a failed unlock attempt counter and shall apply the config...

The SW shall maintain a failed unlock attempt counter and shall apply the configured lockout policy after the defined number of failed attempts [X] [3 attempts, 5 attempts, 10 attempts].

Status	Draft
Verification Criteria	Verify that the SW maintains a failed unlock attempt counter and applies the configured lockout policy after the defined number of failed attempts is reached.

MVC-107892 - The SW shall lock the UART-based debug unlock function for a configurable period...

The SW shall lock the UART-based debug unlock function for a configurable period [X] [30 seconds, 60 seconds, 5 minutes, until next POR] after the failed-attempt threshold is reached.

Status	Draft
Verification Criteria	Verify that after the failed-attempt threshold is reached, the UART-based debug unlock function is locked for the configured lockout period.

MVC-107882 - The SW shall return only minimum failure information over UART during failed unl...

The SW shall return only minimum failure information over UART during failed unlock attempts and shall not disclose whether the failure was caused by challenge expiry, malformed data, invalid CMAC, wrong key, freshness failure, or authorization failure.

Status	Draft
Verification Criteria	Verify that during failed unlock attempts, the SW returns only minimum failure information over UART and does not disclose whether the failure was caused by expiry, malformed data, invalid CMAC, wrong key, freshness failure, or authorization failure.

MVC-107880 - The SW shall log debug unlock security events including challenge requests, succ...

The SW shall log debug unlock security events including challenge requests, successful unlocks, failed verifications, lockout events, and relock events.

Status	Draft
Verification Criteria	Verify that debug unlock security events are logged, including challenge requests, successful unlocks, failed verifications, lockout events, and relock events.

MVC-107886 - The SW shall include in debug unlock logs the configured timestamp source [X] [s...

The SW shall include in debug unlock logs the configured timestamp source [X] [secure RTC, system monotonic counter, backend-provided timestamp], ECU identity, and transaction result.

Status	Draft
Verification Criteria	Verify that debug unlock logs include the configured timestamp source, ECU identity, and transaction result.

MVC-107884 - The SW shall restrict debug unlock log readout according to log access policy [X...

The SW shall restrict debug unlock log readout according to log access policy [X] [authorized debug session only, backend-authorized readout only, local diagnostic readout disabled].

Status	Draft
Verification Criteria	Verify that debug unlock log readout is restricted according to the configured log access policy and is not accessible through unauthorized interfaces or sessions.

MVC-107875 - The SW shall reject development or test debug authentication keys when the ECU i...

The SW shall reject development or test debug authentication keys when the ECU is configured in production locked mode.

Status	Draft
Verification Criteria	Verify that development or test debug authentication keys are rejected when the ECU is configured in production locked mode.

MVC-107878 - The SW shall support the selected secure debug key package provisioning configur...

The SW shall support the selected secure debug key package provisioning configuration, including encrypted key package location and OTP DEK location, without exposing plaintext key values during runtime access.

Status	Draft
Verification Criteria	Verify that the selected secure debug key package provisioning configuration is supported, including encrypted key package location and OTP DEK/KEK location, without exposing plaintext key values during runtime access.

MVC-107877 - The SW shall process only backend-generated unlock responses received through th...

The SW shall process only backend-generated unlock responses received through the debug tool over the approved UART debug unlock service and within the active challenge validity period.

Status	Draft
Verification Criteria	Verify that the SW processes only backend-generated unlock responses received through the debug tool over the approved UART debug unlock service and within the active challenge validity period.

23.4 SPI Flash secure interface

MVC-107901 - The SW shall support secure boot chain-of-trust establishment from the read-only...

The SW shall support secure boot chain-of-trust establishment from the read-only Root of Trust using public verification material provisioned in OTP/eFuse location [X] [SoC OTP/eFuse zone 7, other SoC-approved OTP/eFuse zone].

Status	Draft
Verification Criteria	Verify that the SW establishes the secure boot chain of trust from the read-only Root of Trust using only the approved public verification material provisioned in the configured OTP/eFuse location.

MVC-107900 - The SW shall verify the authenticity and integrity of each boot image loaded fro...

The SW shall verify the authenticity and integrity of each boot image loaded from external SPI NOR flash before execution or transfer of control to the next boot stage.

Status	Draft
Verification Criteria	Verify that each boot image loaded from external SPI NOR flash is checked for authenticity and integrity before execution or before control is transferred to the next boot stage.

MVC-107904 - The SW shall support the selected secure boot flow [X] [CV2xFS boot flow without...

The SW shall support the selected secure boot flow [X] [CV2xFS boot flow without OP-TEE, CV2xFS boot flow with OP-TEE if approved in future configuration].

Status	Draft
Verification Criteria	Verify that the SW supports only the selected secure boot flow configured for the project, such as CV2xFS boot flow without OP-TEE or the approved future OP-TEE boot flow.

MVC-107902 - The SW shall verify the configured boot stages [X] [Safety BST, Safety BLD, Safe...

The SW shall verify the configured boot stages [X] [Safety BST, Safety BLD, Safety OS, A53 BLD, ATF, OS image] according to the approved boot chain sequence.

Status	Draft
Verification Criteria	Verify that the configured boot stages are verified according to the approved boot chain sequence before each stage is executed.

MVC-107907 - The SW shall calculate the hash digest of each boot binary or firmware image usi...

The SW shall calculate the hash digest of each boot binary or firmware image using hash algorithm [X] [SHA-256].

Status	Draft
Verification Criteria	Verify that the SW calculates the hash digest of each boot binary or firmware image using the configured hash algorithm, expected to be SHA-256.

MVC-107906 - The SW shall verify the RSA digital signature of each signed boot binary or firm...

The SW shall verify the RSA digital signature of each signed boot binary or firmware image using the public verification key provisioned in SoC OTP/eFuse zone [X] [zone 7, other SoC-approved OTP/eFuse zone].

Status	Draft
Verification Criteria	Verify that the SW verifies the RSA digital signature of each signed boot binary or firmware image using the public verification key provisioned in the configured SoC OTP/eFuse zone.

MVC-107909 - The SW shall support RSA signature verification according to RSA key length [X]...

The SW shall support RSA signature verification according to RSA key length [X] [2048, 3072, 4096] bits and RSA signature scheme [X] [RSA-PSS, RSASSA-PKCS1-v1_5].

Status	Draft
Verification Criteria	Verify that the SW supports RSA signature verification using the configured RSA key length and RSA signature scheme. Unsupported key lengths or schemes shall be rejected.

MVC-107908 - The SW shall confirm that the calculated image digest matches the authenticated...

The SW shall confirm that the calculated image digest matches the authenticated digest before executing the image or transferring control to the next boot stage.

Status	Draft
Verification Criteria	Verify that the SW confirms the calculated image digest matches the authenticated digest before execution. If the digest does not match, the image shall not be executed.

MVC-107899 - The SW shall support the configured signature location/format [X] [image header,...

The SW shall support the configured signature location/format [X] [image header, authenticated metadata block, end of binary] when locating and verifying boot image signatures.

Status	Draft
Verification Criteria	Verify that the SW locates and verifies boot image signatures according to the configured signature location or format, such as image header, authenticated metadata block, or end of binary.

MVC-107926 - The SW shall fail the secure boot flow and prevent execution of the boot image i...

The SW shall fail the secure boot flow and prevent execution of the boot image if authenticity, integrity, boot metadata, hash digest verification, RSA signature verification, or anti-rollback verification fails.

Status	Draft
Verification Criteria	Verify that the secure boot flow fails and the boot image is not executed if authenticity, integrity, boot metadata, hash digest verification, RSA signature verification, or anti-rollback verification fails.

MVC-107924 - The SW shall apply the configured secure boot failure action [X] [halt boot, res...

The SW shall apply the configured secure boot failure action [X] [halt boot, reset, enter recovery mode, enter safe state] when secure boot verification fails.

Status	Draft
Verification Criteria	Verify that the configured secure boot failure action is applied when secure boot verification fails, such as halt boot, reset, recovery mode, or safe state.

MVC-107930 - The SW shall apply the configured secure boot error reporting/logging policy [X]...

The SW shall apply the configured secure boot error reporting/logging policy [X] [no external detail, secure diagnostic event only, secure internal log only] when verification fails.

Status	Draft
Verification Criteria	Verify that the configured secure boot error reporting/logging policy is applied when verification fails, and that no unauthorized external failure details are exposed.

MVC-107928 - The SW shall use authenticated boot metadata to support secure boot verification...

The SW shall use authenticated boot metadata to support secure boot verification and anti-rollback protection.

Status	Draft
Verification Criteria	Verify that authenticated boot metadata is used to support secure boot verification and anti-rollback protection.

MVC-107921 - The SW shall process the configured authenticated metadata fields [X] [image ide...

The SW shall process the configured authenticated metadata fields [X] [image identifier, image length, image version, boot stage ID, SHA-256 image digest, RSA signature information, non-volatile counter information, signature offset].
The SW shall support the configured metadata storage format [X] [image header, separate authenticated metadata block, appended footer].

Status	Draft
Verification Criteria	Verify that the SW processes the configured authenticated metadata fields, including image identifier, image length, image version, boot stage ID, SHA-256 image digest, RSA signature information, non-volatile counter information, and signature offset.Verify that the SW supports the configured metadata storage format, such as image header, separate authenticated metadata block, or appended footer.

MVC-107920 - The SW shall verify during firmware boot that the firmware version is not older...

The SW shall verify during firmware boot that the firmware version is not older than the minimum version authorized by the trusted monotonic counter.

Status	Draft
Verification Criteria	Verify that during firmware boot, the SW checks that the firmware version is not older than the minimum version authorized by the trusted monotonic counter.

MVC-107923 - The SW shall reject any boot image whose firmware version does not satisfy the c...

The SW shall reject any boot image whose firmware version does not satisfy the configured anti-rollback check.

Status	Draft
Verification Criteria	Verify that any boot image whose firmware version does not satisfy the configured anti-rollback check is rejected and not executed.

MVC-107922 - The SW shall support the configured anti-rollback counter width/location [X] [So...

The SW shall support the configured anti-rollback counter width/location [X] [SoC OTP monotonic counter, OEM-defined non-volatile monotonic counter].

Status	Draft
Verification Criteria	Verify that the SW supports the configured anti-rollback counter width and location, such as SoC OTP monotonic counter or OEM-defined non-volatile monotonic counter.

MVC-107933 - The SW shall support the configured firmware version encoding [X] [integer versi...

The SW shall support the configured firmware version encoding [X] [integer version, security version number, image version field] for anti-rollback verification.

Status	Draft
Verification Criteria	Verify that the SW supports the configured firmware version encoding for anti-rollback verification, such as integer version, security version number, or image version field.

MVC-107932 - The SW shall update the trusted anti-rollback counter only according to the conf...

The SW shall update the trusted anti-rollback counter only according to the configured counter update trigger [X] [after successful update, during production provisioning, OEM-authorized update package] and authorized updater [X] [OEM, Mekra production process, secure bootloader].

Status	Draft
Verification Criteria	Verify that the trusted anti-rollback counter is updated only according to the configured counter update trigger and only by the authorized updater. Unauthorized counter updates shall be rejected.

MVC-107919 - The SW shall restrict secure boot enable/disable controls and boot-device select...

The SW shall restrict secure boot enable/disable controls and boot-device selection configuration to the allowed development lifecycle stage [X] [development, validation].

Status	Draft
Verification Criteria	Verify that secure boot enable/disable controls and boot-device selection configuration are restricted to the allowed development lifecycle stage only.

MVC-107915 - The SW shall enforce secure boot as permanently enabled when the ECU is in the m...

The SW shall enforce secure boot as permanently enabled when the ECU is in the mass-production lifecycle stage [X] [production locked, OEM-defined production lifecycle state].

Status	Draft
Verification Criteria	Verify that secure boot is permanently enabled when the ECU is in the configured mass-production lifecycle stage.

MVC-107914 - The SW shall prevent modification, bypass, or use of alternative development boo...

The SW shall prevent modification, bypass, or use of alternative development boot paths such as USB boot in shipped products according to disable mechanism [X] [OTP/eFuse configuration, secure boot lifecycle lock, SoC boot configuration lock].

Status	Draft
Verification Criteria	Verify that modification, bypass, or use of alternative development boot paths such as USB boot is prevented in shipped products according to the configured disable mechanism.

MVC-107917 - The SW shall use external SPI NOR flash as the selected production boot device w...

The SW shall use external SPI NOR flash as the selected production boot device when the ECU is in production mode.

Status	Draft
Verification Criteria	Verify that external SPI NOR flash is selected as the production boot device when the ECU is in production mode.

MVC-107916 - The SW shall allow boot-device selection to remain configurable only during the...

The SW shall allow boot-device selection to remain configurable only during the development lifecycle stage through the supported Power-On Configuration mechanism [X] [boot strap pins, SoC Power-On Configuration, OEM-defined development configuration].

Status	Draft
Verification Criteria	Verify that boot-device selection remains configurable only during the development lifecycle stage through the supported Power-On Configuration mechanism.

MVC-107911 - The SW shall support permanent production boot-device selection from the SoC OTP...

The SW shall support permanent production boot-device selection from the SoC OTP/eFuse field [X] [SoC boot-device selection field, OEM-defined OTP/eFuse field].

Status	Draft
Verification Criteria	Verify that the SW supports permanent production boot-device selection from the configured SoC OTP/eFuse field.

MVC-107910 - The SW shall store security-sensitive assets requiring restricted access in prot...

The SW shall store security-sensitive assets requiring restricted access in protected storage type [X] [protected SECURE partition, encrypted secure partition, SoC protected storage, equivalently protected storage region].

Status	Draft
Verification Criteria	Verify that security-sensitive assets requiring restricted access are stored only in the configured protected storage type.

MVC-107913 - The SW shall enforce access to protected boot/security assets according to acces...

The SW shall enforce access to protected boot/security assets according to access control owner [X] [Secure ROM, secure bootloader, secure CPU, OEM-defined secure service].

Status	Draft
Verification Criteria	Verify that access to protected boot/security assets is enforced according to the configured access control owner, such as Secure ROM, secure bootloader, secure CPU, or OEM-defined secure service.

MVC-107912 - The SW shall use encryption/key source [X] [OTP-derived DEK, device-unique key,...

The SW shall use encryption/key source [X] [OTP-derived DEK, device-unique key, SoC secure storage key] where storage encryption is required for security-sensitive assets.

Status	Draft
Verification Criteria	Verify that the configured encryption/key source is used where storage encryption is required for security-sensitive assets.

MVC-107918 - The SW shall protect security-sensitive data stored in external SPI NOR flash ag...

The SW shall protect security-sensitive data stored in external SPI NOR flash against unauthorized disclosure using protection mechanism [X] [encryption, secure partition protection, integrity protection with device binding].

Status	Draft
Verification Criteria	Verify that security-sensitive data stored in external SPI NOR flash is protected against unauthorized disclosure using the configured protection mechanism.

MVC-107905 - The SW shall not store private keys or other long-term secrets in plaintext in e...

The SW shall not store private keys or other long-term secrets in plaintext in external SPI NOR flash.

Status	Draft
Verification Criteria	Verify that private keys or other long-term secrets are not stored in plaintext in external SPI NOR flash.

MVC-107903 - The SW shall bind security-relevant firmware or protected storage data to device...

The SW shall bind security-relevant firmware or protected storage data to device-unique identity or device-unique cryptographic material [X] [device UID, OTP secret, OTP-derived DEK, device-unique cryptographic material].

Status	Draft
Verification Criteria	Verify that security-relevant firmware or protected storage data is bound to device-unique identity or device-unique cryptographic material.

MVC-107927 - The SW shall prevent operation of security-relevant firmware or protected data o...

The SW shall prevent operation of security-relevant firmware or protected data on unauthorized cloned hardware when SPI NOR flash contents are copied to another device.

Status	Draft
Verification Criteria	Verify that copying SPI NOR flash contents to another unauthorized device does not allow operation of security-relevant firmware or protected data.

MVC-107925 - The SW shall accept for execution only OEM-approved RSA-signed production firmwa...

The SW shall accept for execution only OEM-approved RSA-signed production firmware images that pass secure boot verification.

Status	Draft
Verification Criteria	Verify that only OEM-approved RSA-signed production firmware images that pass secure boot verification are accepted for execution. Unsigned or non-approved images shall be rejected.

MVC-107931 - The SW shall use only approved public verification material [X] [RSA public veri...

The SW shall use only approved public verification material [X] [RSA public verification key, RSA public key hash, authenticated provisioning artifact] provisioned into SoC OTP/eFuse for production secure boot verification.

Status	Draft
Verification Criteria	Verify that only approved public verification material provisioned into SoC OTP/eFuse is used for production secure boot verification.

MVC-107929 - The SW shall not use, access, request, or depend on the OEM production RSA priva...

The SW shall not use, access, request, or depend on the OEM production RSA private signing key in the ECU runtime or production boot verification flow.

Status	Draft
Verification Criteria	Verify that the SW does not use, access, request, or depend on the OEM production RSA private signing key in ECU runtime or production boot verification flow.

23.5 DRAM secure interface

MVC-107937 - The SW shall enable DRAM memory scrambling according to the configured productio...

The SW shall enable DRAM memory scrambling according to the configured production policy [X] [enabled in production, disabled only in development builds].

Status	Draft
Verification Criteria	Verify that DRAM memory scrambling is enabled according to the configured production policy. In production mode, DRAM scrambling shall be enabled unless the approved policy explicitly allows disabling only for development builds.

MVC-107936 - The SW shall configure DRAM memory scrambling during the approved secure startup...

The SW shall configure DRAM memory scrambling during the approved secure startup stage [X] [BST, secure bootstrap, early bootloader] before normal runtime execution.

Status	Draft
Verification Criteria	Verify that DRAM memory scrambling is configured during the approved secure startup stage, such as BST, secure bootstrap, or early bootloader, before normal runtime execution starts.

MVC-107940 - The SW shall configure the SoC DRAM controller AES encrypt/decrypt engine such t...

The SW shall configure the SoC DRAM controller AES encrypt/decrypt engine such that all writes to external DRAM pass through the encryption path and all reads from external DRAM pass through the decryption path.

Status	Draft
Verification Criteria	Verify that the SoC DRAM controller AES encrypt/decrypt engine is configured so that all writes to external DRAM pass through the encryption path and all reads from external DRAM pass through the decryption path.

MVC-107939 - The SW shall generate the DRAM memory scrambling key during secure boot using th...

The SW shall generate the DRAM memory scrambling key during secure boot using the configured entropy source [X] [SoC internal TRNG, Ambarella TRNG module/API].

Status	Draft
Verification Criteria	Verify that the DRAM memory scrambling key is generated during secure boot using the configured entropy source, such as SoC internal TRNG or Ambarella TRNG module/API.

MVC-107942 - The SW shall generate or configure the DRAM memory scrambling key with the confi...

The SW shall generate or configure the DRAM memory scrambling key with the configured AES key length [X] [SoC default AES key length, 128-bit, 256-bit].

Status	Draft
Verification Criteria	Verify that the generated or configured DRAM memory scrambling key uses the configured AES key length, such as SoC default AES key length, 128-bit, or 256-bit.

MVC-107941 - The SW shall load the DRAM memory scrambling key into the configured write-only...

The SW shall load the DRAM memory scrambling key into the configured write-only DRAM key register [X] [SoC write-only DRAM key register, Ambarella-approved write-only DRAM key register].

Status	Draft
Verification Criteria	Verify that the DRAM memory scrambling key is loaded only into the configured write-only DRAM key register.

MVC-107935 - The SW shall not provide any runtime software path to read back the DRAM memory...

The SW shall not provide any runtime software path to read back the DRAM memory scrambling key after it has been written to the write-only DRAM key register.

Status	Draft
Verification Criteria	Verify that no runtime software path exists to read back the DRAM memory scrambling key after it has been written to the write-only DRAM key register. Any attempted read-back shall fail or return no key material.

MVC-107958 - The SW shall configure the DRAM controller Address Translation Table for each co...

The SW shall configure the DRAM controller Address Translation Table for each configured DRAM client [X] [CPU, ISP, Video Codec, ETH-DMA, DSP, other DMA-capable clients].

Status	Draft
Verification Criteria	Verify that the DRAM controller Address Translation Table is configured for each configured DRAM client, such as CPU, ISP, Video Codec, ETH-DMA, DSP, and other DMA-capable clients.

MVC-107957 - The SW shall assign each DRAM client only to its authorized physical memory rang...

The SW shall assign each DRAM client only to its authorized physical memory range [X] [start address, end address, size].

Status	Draft
Verification Criteria	Verify that each DRAM client is assigned only to its authorized physical memory range according to the configured start address, end address, and size.

MVC-107960 - The SW shall configure DRAM access ranges during secure startup before enabling...

The SW shall configure DRAM access ranges during secure startup before enabling normal runtime execution and before enabling DMA-capable clients.

Status	Draft
Verification Criteria	Verify that DRAM access ranges are configured during secure startup before normal runtime execution and before DMA-capable clients are enabled.

MVC-107959 - The SW shall define ATT access permissions for each DRAM client and memory range...

The SW shall define ATT access permissions for each DRAM client and memory range according to the configured permission set [X] [Read, Write, Read/Write, No Access].

Status	Draft
Verification Criteria	Verify that ATT access permissions are defined for each DRAM client and memory range according to the configured permission set: Read, Write, Read/Write, or No Access.

MVC-107954 - The SW shall prevent DMA-capable clients from accessing DRAM regions outside the...

The SW shall prevent DMA-capable clients from accessing DRAM regions outside their configured ATT access ranges.

Status	Draft
Verification Criteria	Verify that DMA-capable clients are prevented from accessing DRAM regions outside their configured ATT access ranges. Unauthorized access attempts shall be blocked or handled according to the configured policy.

MVC-107953 - The SW shall apply the configured DMA/ATT violation handling policy [X] [block a...

The SW shall apply the configured DMA/ATT violation handling policy [X] [block access, raise interrupt, report error signal, trigger controlled reset] when an unauthorized DRAM access attempt is detected.

Status	Draft
Verification Criteria	Verify that the configured DMA/ATT violation handling policy is applied when an unauthorized DRAM access attempt is detected, such as block access, raise interrupt, report error signal, or trigger controlled reset.

MVC-107956 - The SW shall isolate security-sensitive DRAM regions from non-authorized runtime...

The SW shall isolate security-sensitive DRAM regions from non-authorized runtime software and unauthorized DRAM clients.

Status	Draft
Verification Criteria	Verify that security-sensitive DRAM regions are isolated from non-authorized runtime software and unauthorized DRAM clients.

MVC-107955 - The SW shall include the configured security-sensitive DRAM regions in the prote...

The SW shall include the configured security-sensitive DRAM regions in the protected memory map [X] [secure boot buffers, crypto working buffers, key-derivation temporary buffers, debug-authentication temporary buffers, security-relevant runtime buffers].

Status	Draft
Verification Criteria	Verify that all configured security-sensitive DRAM regions are included in the protected memory map, including secure boot buffers, crypto working buffers, key-derivation temporary buffers, debug-authentication temporary buffers, and security-relevant runtime buffers.

MVC-107962 - The SW shall use the configured SoC-supported DRAM protection mechanisms [X] [AT...

The SW shall use the configured SoC-supported DRAM protection mechanisms [X] [ATT access control, secure boot-time memory map configuration, SoC hardware access restrictions] to enforce isolation of security-sensitive DRAM regions.

Status	Draft
Verification Criteria	Verify that the configured SoC-supported DRAM protection mechanisms are used to enforce isolation of security-sensitive DRAM regions, such as ATT access control, secure boot-time memory map configuration, and SoC hardware access restrictions.

MVC-107961 - The SW shall protect the DRAM security configuration from modification by non-au...

The SW shall protect the DRAM security configuration from modification by non-authorized runtime software after secure startup.

Status	Draft
Verification Criteria	Verify that the DRAM security configuration cannot be modified by non-authorized runtime software after secure startup.

MVC-107952 - The SW shall protect the memory scrambling configuration after secure startup us...

The SW shall protect the memory scrambling configuration after secure startup using the configured protection mechanism [X] [secure boot-time lock, SoC register protection, privileged-only access, OEM-approved configuration lock].

Status	Draft
Verification Criteria	Verify that the memory scrambling configuration is protected after secure startup using the configured protection mechanism, such as secure boot-time lock, SoC register protection, privileged-only access, or OEM-approved configuration lock.

MVC-107948 - The SW shall protect the DRAM key register configuration after secure startup us...

The SW shall protect the DRAM key register configuration after secure startup using the configured protection mechanism [X] [secure boot-time lock, SoC register protection, privileged-only access, OEM-approved configuration lock].

Status	Draft
Verification Criteria	Verify that the DRAM key register configuration is protected after secure startup using the configured protection mechanism. Unauthorized runtime modification shall be prevented.

MVC-107947 - The SW shall protect the ATT client configuration, ATT access ranges, and ATT ac...

The SW shall protect the ATT client configuration, ATT access ranges, and ATT access permissions from unauthorized modification after secure startup.

Status	Draft
Verification Criteria	Verify that ATT client configuration, ATT access ranges, and ATT access permissions are protected from unauthorized modification after secure startup.

MVC-107950 - The SW shall ensure that final ATT ranges and permissions are derived from the a...

The SW shall ensure that final ATT ranges and permissions are derived from the approved memory map before enabling DMA-capable clients.

Status	Draft
Verification Criteria	Verify that the final ATT ranges and permissions are derived from the approved memory map before enabling DMA-capable clients.

MVC-107949 - The SW shall configure unauthorized or unused DRAM regions with No Access permis...

The SW shall configure unauthorized or unused DRAM regions with No Access permissions where supported by the ATT policy.

Status	Draft
Verification Criteria	Verify that unauthorized or unused DRAM regions are configured with No Access permissions where supported by the ATT policy.

MVC-107944 - The SW shall prevent non-secure runtime software from modifying DRAM scrambling,...

The SW shall prevent non-secure runtime software from modifying DRAM scrambling, key register, ATT range, or ATT permission configuration after the secure startup configuration is locked.

Status	Draft
Verification Criteria	Verify that non-secure runtime software cannot modify DRAM scrambling, key register, ATT range, or ATT permission configuration after the secure startup configuration is locked.

MVC-107943 - The SW shall report or handle ATT violation events according to the configured v...

The SW shall report or handle ATT violation events according to the configured violation policy and platform-supported reporting mechanism.

Status	Draft
Verification Criteria	Verify that ATT violation events are reported or handled according to the configured violation policy and platform-supported reporting mechanism.

MVC-107946 - The SW shall ensure that DRAM scrambling and ATT protection are configured befor...

The SW shall ensure that DRAM scrambling and ATT protection are configured before any security-sensitive data is placed in external DRAM.

Status	Draft
Verification Criteria	Verify that DRAM scrambling and ATT protection are configured before any security-sensitive data is placed in external DRAM.

MVC-107945 - The SW shall ensure that DMA-capable clients are enabled only after their ATT ra...

The SW shall ensure that DMA-capable clients are enabled only after their ATT ranges and access permissions have been configured.

Status	Draft
Verification Criteria	Verify that DMA-capable clients are enabled only after their ATT ranges and access permissions have been configured.

MVC-107951 - The SW shall maintain the DRAM protection configuration consistently with the ap...

The SW shall maintain the DRAM protection configuration consistently with the approved production memory map and shall not allow runtime reconfiguration outside authorized secure startup flow.

Status	Draft
Verification Criteria	Verify that the DRAM protection configuration remains consistent with the approved production memory map and that runtime reconfiguration outside the authorized secure startup flow is not allowed.

MVC-107938 - The SW shall treat any access attempt outside a configured ATT range as a securi...

The SW shall treat any access attempt outside a configured ATT range as a security-relevant event and shall apply the configured violation handling response.

Status	Draft
Verification Criteria	Verify that any access attempt outside a configured ATT range is treated as a security-relevant event and that the configured violation handling response is applied.

24 New SRS

24.1 Camera Module Management Feature

24.1.1 Scope

Module Initialization & Configuration: Managing startup sequences and parameter configurations for all integrated camera module components, including the Image Sensor, Remote PMIC, I2C EEPROM, and Serializer/Deserializer (SerDes) devices.
GMSL2 Link & Transport Management: Handling I2C/GPIO tunneling over the GMSL2 backchannel, maintaining bidirectional control communication, and monitoring GMSL2 link-lock and transport integrity.

24.1.2 Functional Requirements

24.1.2.1 Configuration

24.1.2.1.1 Local De-serializer Configuration

MVC-110321 - The software shall configure the local De-serializer I2C slave address on SOW2 w...

The software shall configure the local De-serializer I2C slave address on SOW2 with the address 0x50 through MVC-110324 - SerDes_I2c_Config_Cmd interface.

Status	Draft
Verification Criteria	

MVC-110343 - The Software shall configure the I2C communication bus connecting to the local D...

The Software shall configure the I2C communication bus connecting to the local De-serializer on SOW2 through MVC-110324 - SerDes_I2c_Config_Cmd interface according to MVC-94043 - HSI - Camera Interface

Status	Draft
Verification Criteria	

24.1.2.1.2 Remote Serializer and camera components Configuration

MVC-110323 - The software shall configure the I2C slave address for the remote SOW1 component...

The software shall configure the I2C slave address for the remote SOW1 components over the GMSL2 back-channel through MVC-110324 - SerDes_I2c_Config_Cmd interface with the following address:

- Remote Serializer: MVC-110880 - REMOTE_SER_I2C_SLAVE_ADDRESS [0x80]
- Camera PMIC: MVC-110879 - CAM_PMIC_I2C_SLAVE_ADDRESS [0xE0]
- Imager: MVC-110882 - IMG_I2C_SLAVE_ADDRESS [0x20]
- Camera EEPROM: MVC-110881 - CAM_EEPROM_I2C_SLAVE_ADDRESS [0xA0]

Status	Draft
Verification Criteria	

MVC-110325 - The Software shall configure the master I2C bus speed of the remote serializer o...

The Software shall configure the master I2C bus speed of the remote serializer on SOW1 through MVC-110324 - SerDes_I2c_Config_Cmd interface to operate in "Fast mode" with a speed of 400 kb/s.

Status	Draft
Verification Criteria	Test Method: Connect logic analyzer or oscilloscope to I2C Bus

24.1.2.2 Power-up Sequence

24.1.2.2.1 Power-UP Local De-Serializer

MVC-110346 - During startup, the software shall enable "desEn" pin as High through interface...

During startup, the software shall enable "desEn" pin as High through MVC-110351 - SerDes_desEn interface to power-up the local De-serializer on SOW2 according to MVC-94043 - HSI - Camera Interface

Status	Draft
Verification Criteria	

MVC-110687 - Following power up the local De-serializer, the software shall verify that the P...

Following power up the local De-serializer, the software shall verify that the Power-On-Self-Test (POST) execution status in the De-serializer indicates successful completion after a stabilization delay of MVC-110690 - DESER_EN_STABILIZATION_DELAY [50 msec] through I2C Bus.

Status	Draft
Verification Criteria	

24.1.2.2.2 Power-UP remote Serializer (SOW1)

MVC-110353 - During startup, following the local De-serializer enabling, the software shall a...

During startup, following the local De-serializer enabling, the software shall activate the Power-Over-Coax (POC) power supply through MVC-110352 - SerDes_PowerOverCoaxEn to power-up the remote camera module SOW1 according to MVC-94043 - HSI - Camera Interface.

Status	Draft
Verification Criteria	

MVC-110693 - Following enabling the POC, the Software shall check the Serializer Lock status...

Following enabling the POC, the Software shall check the Serializer Lock status through MVC-110689 - SerDes_Lock_Status after a stabilization delay of MVC-110692 - SER_LOCK_STABILIZATION_DELAY [100 msec] according to MVC-94043 - HSI - Camera Interface

Status	Draft
Verification Criteria	

MVC-110695 - If the Serializer Lock status through is locked or established, the software sha...

If the Serializer Lock status through MVC-110689 - SerDes_Lock_Status is locked or established, the software shall configure the Serializer and De-serializer according to MVC-94679 - SerDes-Interface

Status	Draft
Verification Criteria	

SerDes Consists of a Serializer -> on SOW1 and De-serializer -> on SOW2

They Connected through a Coaxial Cable operating at default link rate.

The physical link lock only confirm that they are electrically synchronized not means that it is necessarily the video data will be received by the De-serializer Correctly (Data Parsing).

So immediately after Link Lock, the SW shall align the operation mode (in our case to "Tunnel Mose") and protocol layers between them to make sure that the video frame will

1. transmit (from the imager),
2. encode (in the serializer),
3. Decode (in De-serializer), and
4. parsing (in the SOC) correctly.

MVC-110698 - Following Confirmation of the SerDes Lock, the software shall configure the oper...

Following Confirmation of the SerDes Lock, the software shall configure the operational mode of the remote Serializer to "Tunnel mode" through I2C MVC-110324 - SerDes_I2c_Config_Cmd according to MVC-94696 - Serializer Tunnel Mode Selection.

Status	Draft
Verification Criteria	

24.1.2.3 SerDes MIPI and MFPs Configuration

To transfer the video payload with very high speed from the imager sensor to the Serializer, there are parallel lanes connected between them, Lane 0,1,2,3,4 for Data and one Lane for CLK.

Each Lane consists of 2 wires (one with +ve polarity and the other with -ve polarity).

So, between the Imager and Serializer, the video payload is in MIPI format over these 5 lanes.

Between the De-serializer and the SOC, there are also another 5 Lanes with the same structure to transfer the received video streaming in MIPI format from the De-serializer to the SOC.

To transfer the video payload from the remote serializer (in SOW1) to the local de-serializer (in SOW2),

1. The Serializer encodes the video data in a GMSL format to forward it on the coax-cable
2. The De-serializer decode the received GMSL format and convert it back to MIPI format to forward them to the SOC over the MIPI lanes.

24.1.2.3.1 MIPI Lane Mapping and Polarity Configuration

MVC-110705 - Following SerDes mode configuration, the software shall configure the MIPI CSI-2...

Following SerDes mode configuration, the software shall configure the MIPI CSI-2 data and Clock Lanes swapping and polarity inversion of the De-serializer through MVC-110324 - SerDes_I2c_Config_Cmd over the I2C according to the mapping matrix and the HSI MVC-94702 - Deserializer - MIPI CSI-2 Lane Configuration

Status	Draft
Verification Criteria	

Imager		Serializer	
Lane	Polarity	Lane	Polarity
Data Lane - 0	N	Data Lane - 1	P
	P		N
Data Lane - 1	N	Data Lane - 0	P
	P		N
Data Lane - 2	N	Data Lane - 2	P
	P		N
Data Lane - 3	N	Data Lane - 3	P
	P		N
CLK Lane	N	CLK Lane	P
	P		N

MVC-110700 - Following SerDes mode configuration, the software shall configure the MIPI CSI-2...

Following SerDes mode configuration, the software shall configure the MIPI CSI-2 data and Clock Lanes swapping and polarity inversion of the Serializer through MVC-110324 - SerDes_I2c_Config_Cmd over the I2C according to the mapping matrix and the HSI MVC-94708 - Serializer - MIPI CSI-2 Lane Configuration

Status	Draft
Verification Criteria	

De-Serailizer		SOC	
Lane	Polarity	Lane	Polarity
Data Lane - 2	P	Data Lane - 0	P
	N		N
Data Lane - 3	P	Data Lane - 1	P
	N		N
Data Lane - 0	P	Data Lane - 2	P
	N		N
Data Lane - 1	P	Data Lane - 3	P
	N		N
CLK Lane	P	CLK Lane	P
	N		N

24.1.2.3.2 Multi-Function Pins (MFPs) Forwarding Configuration

MVC-110809 - The Software shall configure the Multi-Function Pins-MFPs (imagerEn, FsyncTrigge...

The Software shall configure the Multi-Function Pins-MFPs (imagerEn, FsyncTrigger, CamHeatingEn, CamPmicError, and ImagerError) forwarding matrix and signal mapping of the De-serializer through MVC-110324 - SerDes_I2c_Config_Cmd over the I2C according to the mapping matrix and the HSI MVC-94717 - SerDes GPIO Forwarding Configuration - Imager Enable , MVC-94711 - SerDes GPIO Forwarding Configuration - Camera Heating Enable , MVC-94712 - SerDes GPIO Forwarding Configuration - Camera PMIC Error , MVC-94705 - SerDes GPIO Forwarding Configuration - Imager Error , MVC-103086 - SerDes GPIO Forwarding Configuration - Fsync Trigger

Status	Draft
Verification Criteria	

Signal Name	Local De-serializer Pin	Remote Serializer Pin	Signal Direction from SOC
ImagerEn	MFP7	MFP0	Output
fsyncTrigger	MFP0	MFP1	Output
camHeatingEn	MFP2	MFP3	Output
camPmicError	MFP6	MFP5	Input
imagerError	MFP1	MFP7	Input

MVC-110811 - The Software shall configure the Multi-Function Pins-MFPs (imagerEn, FsyncTrigge...

The Software shall configure the Multi-Function Pins-MFPs (imagerEn, FsyncTrigger, CamHeatingEn, CamPmicError, and ImagerError) forwarding matrix and signal mapping for the remote Serializer; to enable the interface with the Camera module peripherals (Imager Sensor, PMIC and Lens Heater) through MVC-110324 - SerDes_I2c_Config_Cmd over the I2C according to the mapping matrix and the HSI MVC-94717 - SerDes GPIO Forwarding Configuration - Imager Enable , MVC-94711 - SerDes GPIO Forwarding Configuration - Camera Heating Enable , MVC-94712 - SerDes GPIO Forwarding Configuration - Camera PMIC Error , MVC-94705 - SerDes GPIO Forwarding Configuration - Imager Error , MVC-103086 - SerDes GPIO Forwarding Configuration - Fsync Trigger

Status	Draft
Verification Criteria	

Signal Name	Local De-serializer Pin	Remote Serializer Pin	Signal Direction from SOC
ImagerEn	MFP7	MFP0	Output
fsyncTrigger	MFP0	MFP1	Output
camHeatingEn	MFP2	MFP3	Output

Signal Name	Local De-serializer Pin	Remote Serializer Pin	Signal Direction from SOC
camPmicError	MFP6	MFP5	Input
imagerError	MFP1	MFP7	Input

24.1.2.4 Serializer-Image Sensor Master Clock Configuration

MVC-110821 - The software shall configure the remote Serializer the master clock Mutli-Functi...

The software shall configure the remote Serializer the master clock Mutli-Function Pin-MFP4 to provide MVC-110820 - SER_IMG_SNSR_MASTER_CLK [25 MHZ] for the image sensor master Clock through MVC-110324 - SerDes_I2c_Config_Cmd over the I2C according to HSI MVC-94726 - Serializer - Enable RCLK Output

Status	Draft
Verification Criteria	

24.1.2.5 Serializer-internal ADC configuration for temperature monitoring

The serializer has internal temp sensor to read its internal temperature by using ADC so we need to configure or intiailzize the ADC and then to use this temp ,easurement to measure the temp of lens. To configure the ADC we need 2 steps sequentially:
1. Reset the ADC firstly to ensure the clean reset
2. configure the ADC to be ready to convert the current temp immediately once requested “on demand readout”

MVC-110823 - The software shall configure the internal ADC of the remote Serializer through a...

The software shall configure the internal ADC of the remote Serializer through MVC-110324 - SerDes_I2c_Config_Cmd according to the sequence defined in the HSI MVC-99301 - Ser ADC Configuration followed by MVC-99302 - Ser On-Demand Die Temperature read out to prepare for die temperature readout.

Status	Draft
Verification Criteria	

24.1.2.6 Image Sensor Configuration

24.1.2.6.1 Image Sensor Initial Powerup and Configuration

MVC-110826 - The Software shall enable the Sensor Imager MFP through for the De-serailizer to...

The Software shall enable the Sensor Imager MFP through MVC-110824 - ImagerEn_MFP for the De-serailizer to Powering-up the imager sensor in the camera module according to the HSI MVC-94043 - HSI - Camera Interface

Status	Draft
Verification Criteria	

MVC-110828 - The Software shall configure the video streaming frame rate (FRS) of the image s...

The Software shall configure the video streaming frame rate (FRS) of the image sensor through MVC-110324 - SerDes_I2c_Config_Cmd to MVC-110827 - IMG_SNSR_Frame_PER_SEC [30 FPS] according to MAXIM_HDR4_RSSR_16B_MCLK25_30fps_1920x1550_MIPI4_481.25Mbps_wSECU_wDS_20251029.sset

Status	Draft
Verification Criteria	

24.1.2.6.2 Image Sensor Runtime Configuration Update

There are some parameters needed to be update periodically in the Image sensor to sense the image pixels correctly. These Parameters are as follows:

1. Exposure Time Control:
In high ambient Light (Light Brightness): Exposure time is very small (~= 10us) to avoid the pixel saturation (Over Exposure).

In Low Ambient Light (Dark Brightness): Exposure time is increased to allow the sensor to collect the Light as much as can to read the image with the best resolution.
Which means (we configure the imager sensor to transmit 30 Frame per Sec -> time=1/Freq = 1/30 = 33.3msec)

So, During Initialization, the sensor can start with a default or initial exposure time = 10 msec until receive the new configuration from the SOC.

Range: [0.01 ms to 33.3 ms] (Default: 10 ms)

2. Amplification Gain:
To amplify the received pixels to be able to send the image pixels to the serializer.

In the high ambient light (Light Brightness) is very small (0 dB) because the original captured pixels in the signal are very clear and strong and to avoid the noise because if the signal is amplified so much, the noise will increase.

In the low ambient Light (Dark Brightness) is very high (~= 24 dB) to amplify the signal during the Dark.

So, Default amplification gain = 0 dB

MVC-110866 - During the run time, the software shall update the image sensor with and receive...

During the run time, the software shall update the image sensor with MVC-110865 - Image_Processing_AS_Exposure_Time and MVC-110867 - Image_Processing_AWB_Amplification_Gain received from the image processing feature periodically every MVC-110868 - IMG_SNSR_DYNAMIC_UPDATE_PERIODICITY [33 msec] through MVC-110324 - SerDes_I2c_Config_Cmd

Status	Draft
Verification Criteria	

24.1.2.7 Main PMIC Initialization Configuration

MVC-110874 - The Software shall set its power error status to "No_Error" to the Main PMIC thr...

The Software shall set its power error status to "No_Error" to the Main PMIC through MVC-110869 - PmicErrorIn, to inform the Main PMIC that no activated error SOC fault according to the HSI MVC-93518 - HSI - PMIC interface

Status	Draft
Verification Criteria	

MVC-110873 - Following the setting of the Main PMIC error status to "No_Error", the software...

Following the setting of the Main PMIC error status to "No_Error", the software shall enable the error Monitoring Funtinality in the Main PMIC through MVC-110872 - Generic_I2c_Config_Cmd using its slave address MVC-110871 - MASTER_PMIC_I2C_SLAVE_ADDRESS [0x03] according to the HSI MVC-93518 - HSI - PMIC interface

Status	Draft
Verification Criteria	

MVC-118360 - dfgdgsdgsd

dfgdgsdgsd

Status	Draft
Verification Criteria	

24.1.3 Non-Functional Requirements

24.1.4 Interfaces

24.1.4.1 Hardware Interfaces

24.1.4.2 Input Interfaces

SerDes_Lock_Status

camPmicError_MFP

imagerError_MFP

24.1.4.2.1 Output Interfaces

SerDes_desEn

SerDes_PowerOverCoaxEn

SerDes_I2c_Config_Cmd

ImagerEn_MFP

fsyncTrigger_MFP

camHeatingEn_MFP

PmicErrorIn

Generic_I2c_Config_Cmd

24.1.4.3 Software Interfaces

24.1.4.3.1 Input Interfaces

Image_Processing_AS_Exposure_Time

Image_Processing_AWB_Amplification_Gain

24.1.4.3.2 Output Interfaces

24.1.5 Configuration Parameters

DESER_EN_STABILIZATION_DELAY [50 msec]

SER_LOCK_STABILIZATION_DELAY [100 msec]

SER_IMG_SNSR_MASTER_CLK [25 MHZ]

IMG_SNSR_Frame_PER_SEC [30 FPS]

IMG_SNSR_DYNAMIC_UPDATE_PERIODICITY [33 msec]

MASTER_PMIC_I2C_SLAVE_ADDRESS [0x03]

REMOTE_SER_I2C_SLAVE_ADDRESS [0x80]

CAM_PMIC_I2C_SLAVE_ADDRESS [0xE0]

IMG_I2C_SLAVE_ADDRESS [0x20]

CAM_EEPROM_I2C_SLAVE_ADDRESS [0xA0]

24.2 Ethernet Communication Management Function

24.2.1 Ethernet Physical Layer Configuration Feature

24.2.1.1 Scope

There are 2 wires or pins between the SOC and PHY layer directly for Control Data or Commands Transfer.
physmi.MDIO: Management Data Input / Output for Control Plane,
physmi.MDC: for Clock.
&
12 Pins for Data transfer -> High Speed Data Control Pins:
12 Pins RGMII between the MAC in SOC pins and external PHY

24.2.1.2 Functional Requirements

24.2.1.2.1 PHY Configuration

MVC-113670 - During the System state is in "Initialization state", The software shall configu...
During the System state is in "Initialization state", The software shall configure the SOC pin multiplexing for the Ethernet PHY layer control lines MVC-113668 - Ethernet_PHY_Reset, MVC-113665 - Ethernet_MAC_OSC , MVC-113666 - Ethernet_PHYSMI_MDC and MVC-113667 - Ethernet_PHYSMI_MDIO , MVC-113669 - Ethernet_PHYRGMII , MVC-113663 - Ethernet_PHY_gpio and MVC-113664 - Ethernet_PHY_Interrupt according to the HSI MVC-93530 - HSI - Ethernet PHY

Status	Ready for review
Verification Criteria	1st: Verify using Oscilloscope / Logic Analyzer that the designated SoC control lines transition to their correct electrical states upon system initialization. 2nd: Verify successful MDIO read/write communication with the PHY transceiver post-initialization without reporting configuration fault

24.2.1.2.1.1 PHY Self-Test Check

MVC-113672 - Following the PHY reset , The software shall read the PHY Startup self-test stat...
Following the PHY reset , The software shall read the PHY Startup self-test status for the Ethernet Physical Layer through MVC-113667 - Ethernet_PHYSMI_MDIO to verify that the physical layer self-test passed successfully.

Status	Ready for review
Verification Criteria	Verify through Logic Analyzer that the software reads the PHY Self-Test Status Register over MDIO interface following the PHY reset, and confirms a PASS status to complete successful initialization.

MVC-113681 - If the software receives "Startup_Pass" status from the Ethernet PHY layer throu...
If the software receives "Startup_Pass" status from the Ethernet PHY layer through MVC-113667 - Ethernet_PHYSMI_MDIO, the software shall provide MVC-113677 - Ethernet_PHY_Self_Status as "Ready" within a max response time of (PHY_SELF_STATUS_RESPONSE_TOUT, Default: 10msec) .

Status	Draft
Verification Criteria	Verify through Logic Analyzer that Software receive “Startup_Pass” from the PHY over MDIO, and the software provide the status as “Ready” within 10 ms of status reception.

MVC-113680 - If the software doesn't detect "Startup_Pass" from the Ethernet PHY layer throug...
If the software doesn't detect "Startup_Pass" from the Ethernet PHY layer through MVC-113667 - Ethernet_PHYSMI_MDIO, within (PHY_SELF_STATUS_FAILURE_TOUT, Default: 50msec), the software shall provide MVC-113677 - Ethernet_PHY_Self_Status as"Failed"

Status	Draft
Verification Criteria	Verify by Fault Injection on MDIO line that if “Startup_Pass” is not detected within 50 ms, the software sets the status “failed”

24.2.1.2.1.2 PHY - SMI Configurations

MVC-113683 - The software shall configure the Ethernet PHY polarity detection feature through...
The software shall configure the Ethernet PHY polarity detection feature through MVC-113667 - Ethernet_PHYSMI_MDIO according to MVC-93530 - HSI - Ethernet PHY

Status	Ready for review
Verification Criteria	Verify through Logic Analyzer on the MDIO bus during initialization that the software writes the designated polarity detection configuration values to the target PHY registers according to HSI.

MVC-113682 - The software shall configure the internal RX clock delay of PHY RGMII over accor...
The software shall configure the internal RX clock delay of PHY RGMII over MVC-113667 - Ethernet_PHYSMI_MDIO according to MVC-93530 - HSI - Ethernet PHY

Status	Ready for review
Verification Criteria	Verify via Oscilloscope measurement during initialization that the software configure the clock delay register/control line of PHY

MVC-113686 - The software shall support read and write access to clause 45 PHY registers over...
The software shall support read and write access to clause 45 PHY registers over MVC-113667 - Ethernet_PHYSMI_MDIO using Clause 22 indirect access mechanism according to MVC-93530 - HSI - Ethernet PHY

Status	Ready for review
Verification Criteria	Verify through Logic Analyzer that performing a write or read operation to a Clause 45 PHY register (e.g., MMD Device1, Register 0x0001) executes the 3-steps Clause 22 indirect frame sequence (Select MMD command then write Target Register Address command then read or write) as defined in IEEE 802.3.

Since the hardware interface supports Clause 22 framing with a 5-bit register address space (up to 32 registers), the software uses Clause 22 Register 13 (MMD Access Control) and Register 14 (MMD Access Address/Data) as a indirect access bridge to read/write Clause 45 registers with 16-bit extended address spaces (e.g., MMD devices).

Ex: To read or write a clause 45 PHY register such as Access MMD Device 3, Register Address 0x0010, the software will execute the standard 4-frame indirect access transaction over MDIO in the following seq:

1. **Step 1 (Select MMD):** Write to PHY Reg 13 with function set to `Address` (00b) and MMD number set to target device (e.g., MMD 3).
2. **Step 2 (Set Target Register Address):** Write to PHY Reg 14 with the target 16-bit register address value (e.g., `0x0010`).
3. **Step 3 (Read/Write Actual Data):** Perform Read or Write on PHY Reg 14 to transfer the actual payload data of the targeted Clause 45 register.

24.2.1.2.1.3 TC10 Ethernet Standard for Power saving Configuration Sequence

MVC-113692 - During the Ethernet Initialization, The software shall enable the Open Alliance...
During the Ethernet Initialization, The software shall enable the Open Alliance TC10 Sleep and Wakeup mechansim inside the Ethernet PHY over MVC-113667 - Ethernet_PHYSMI_MDIO according to the HSI and configure the INHIBIT signal over-write control through MVC-113693 - Ethernet _PHY_INHIBIT to allow the post-handshake power management.

Status	Ready for review
Verification Criteria	Verify via Analyzer during Ethernet initialization that the software executes the TC10 configuration write sequence via MDIO then configures the INHIBIT override register bits

The following sequence to configure the TC10 Sleep and Wakeup Mechansim

1. Enable the PHY Super Configuration Access To be able to Reconfigure the TC10 Protected Regs
2. Enable TC10 Sleep/Wakeup Control mechanism
3. Disable the PHY Super Configuration Access to prevent any corruption for the TC10 Protected Regs
4. Enable the over-writing for the INHIBIT signal to allow to be overwritten after successful sleep handshake
5. Disable the over-riding for the INHIBIT signal to allow the PHY reset it to low after successful sleep handshake to allow the SOC enter sleep mode after successful sleep handshake

After, successful TC10 sleep handshake, the PHY hardware resets the PHY_INHIBIT signal to inform the Master PMIC with the successful sleep handshake. So the PMIC will cutoff the power about the SOC as whole and only keep the PHY only powered with very low consumption for wakeup again.

24.2.1.2.1.4 PHY Detection & Polarity Variants Handling

MVC-113696 - After Startup, The software shall read the PHY identifier through to identify th...
After Startup, The software shall read the PHY identifier through MVC-113667 - Ethernet_PHYSMI_MDIO to identify the PHY Hardware Variant either 100BASE-T1 (TJA110X) or Gigabit PHY (TJA112X) and provide it through MVC-113695 - Ethernet_PHY_Type as follows:

PHY ID	PHY Type
B013h	100BASE-T1 (TJA110X)
B030h	Gigabit PHY (TJA112X)

Status	Ready for review
Verification Criteria	Verify via Logic Analyzer that upon startup, the software reads the PHY ID registers over MDIO and sets PHY_Type to "100BASE-T1" when a 100BASE-T1 PHY ID is returned or "1000BASE-T1" when a Gigabit PHY ID is returned

24.2.1.2.1.4.1 TJA112X PHY Variant

MVC-113700 - If the software identify the PHY Variant as a "Gigabit PHY / 1000BASE-T1" throug...
If the software identify the PHY Variant as a "Gigabit PHY / 1000BASE-T1" through MVC-113695 - Ethernet_PHY_Type the software shall enable the PHY polarity correction through MVC-113667 - Ethernet_PHYSMI_MDIO according to MVC-93530 - HSI - Ethernet PHY

Status	Ready for review
Verification Criteria	Verify via Logic Analyzer that when PHY_Type is "1000BASE-T1", the software writes the designated register pattern over MDIO to enable PHY polarity correction according to HSI.

24.2.1.2.1.5 TJA110X PHY Varinat

MVC-113698 - If the software identify the PHY Variant as a "100BASE-T1" through The software...
If the software identify the PHY Variant as a "100BASE-T1" through MVC-113695 - Ethernet_PHY_Type The software shall disable the PHY polarity detection through MVC-113667 - Ethernet_PHYSMI_MDIO according to MVC-93530 - HSI - Ethernet PHY

Status	Ready for review
Verification Criteria	Verify via Logic Analyzer that when PHY_Type is "100BASE-T1", the software writes the designated register pattern over MDIO to disable PHY polarity detection according to HSI.

24.2.1.2.2 MAC Cofiguration

MVC-113859 - During Ethernet Initialization, the software shall read the stored MAC address t...
During Ethernet Initialization, the software shall read the stored MAC address through MVC-113852 - Camera_MAC_Address and validate that its first 3 bytes (Organizational Unique Identifier-OUI) match the configured MEKRA-OUI of 98:D3:D2

Status	Draft
Verification Criteria	Verified implicitly through execution of (Positive Path: MVC-113858) and (Negative Path: MVC-113861)

MVC-113858 - If the retrieved MAC address match MEKRA_OUI, the software shall configure the E...
If the retrieved MAC address match MEKRA_OUI, the software shall configure the Ethernet MAC Controller (MAC0) with the validated MAC address and provide "MAC_Valid_Status" through MVC-113856 - Ethernet_MAC_Address_Validity_Status to the Diagnostic Feature.

Status	Draft
Verification Criteria	Verify via SW Measurement (e.g., CANoe / XCP / UDS) that when the first 3 bytes in the Camera_MAC_Address match MEKRA_OUI, the software sets the "Validity_Status" interface to "MAC_Valid".

MVC-113861 - If the retrieved MAC Address-OUI doesn't match MEKRA_OUI, the software shall pro...
If the retrieved MAC Address-OUI doesn't match MEKRA_OUI, the software shall provide an "MAC_Invalid_OUI"
through MVC-113856 - Ethernet_MAC_Address_Validity_Status to the Diagnostic Feature.

Status	Draft
Verification Criteria	Verify via SW Measurement (e.g., CANoe / XCP / UDS) that when Camera_MAC_Address doesn't match MEKRA_OUI, the software sets the "Validity_Status" interface to "MAC_InValid_OUI".

24.2.1.2.2.1 MAC speed Configuration

MVC-113870 - The software shall configure the Ethernet MAC Controller interface mode (MAC0) b...
The software shall configure the Ethernet MAC Controller interface mode (MAC0) based on the identified PHY variant MVC-113695 - Ethernet_PHY_Type

Status	Draft
Verification Criteria	Verify implicitly by execution MVC-113867 and MVC-113869

24.2.1.2.2.2 PHY Variants Handling

24.2.1.2.2.2.1 VAR_STANDARD_CAMERA (TJA110X)

MVC-113867 - If the software identify the as a "100BASE-T...
If the software identify the MVC-113695 - Ethernet_PHY_Type as a "100BASE-T1" (TJA110X) through MVC-113695 - Ethernet_PHY_Type, The software shall configure the Ethernet MAC Controller interface mode (MAC0) to operate in R GMII mode with 100Mbps bandwidth according to MVC-93530 - HSI - Ethernet PHY and provide the configured speed 100Mbps through MVC-113866 - Ethernet_MAC0_Configured_Speed

Status	Draft
Verification Criteria	Verify via SW Measurement (e.g., CANoe / XCP / UDS) that when PHY_Type is set to "100BASE-T1", the software updates MAC0_Configured Speed to 100Mbps and Verify via Ethernet Network Analyzer that data transmission on the physical Ethernet interface operates at 100 Mbps RGMII clock rate.

24.2.1.2.2.2.2 VAR_SURROUNDED_CAMERA (TJA112X)

MVC-113869 - If the software identify the as 1000Base-T1(TJA112X), The software shall configu...
If the software identify the MVC-113695 - Ethernet_PHY_Type as 1000Base-T1(TJA112X), The software shall configure the Ethernet MAC Controller interface mode (MAC0) to operate in RGMII mode with 1G bps bandwidth according to MVC-93530 - HSI - Ethernet PHY and provide the configured speed 1G bps through MVC-113866 - Ethernet_MAC0_Configured_Speed

Status	Draft
Verification Criteria	Verify via SW Measurement (e.g., CANoe / XCP / UDS) that when PHY_Type is set to "1000BASE-T1", the software updates MAC0_Configured Speed to 1G bps and Verify via Ethernet Network Analyzer that data transmission on the physical Ethernet interface operates at 1G bps RGMII clock rate.

24.2.1.2.2.3 MAC Mode Configuration

MVC-113878 - The software shall periodically read the Physical Link Status through according...
The software shall periodically read the Physical Link Status through MVC-113667 - Ethernet_PHYSMI_MDIO according to MVC-93530 - HSI - Ethernet PHY at a cyclic rate of 10 msec to monitor the Ethernet Physical Link status.

Status	Draft
Verification Criteria	Verify via Analyzer that PHY status read requests transmit every 10 ms (±1 ms).

24.2.1.2.2.4 Link Loss

MVC-113877 - If the software detect the Physical Link status is Link_Down for a tout higher t...
If the software detect the Physical Link status is Link_Down for a tout higher than Ethernet Cable Stability Tout of 50 msec, The software shall provide a "ETH_Link_Down" through MVC-113873 - Ethernet_Link_status to the Diagnostic Feature.

Status	Draft
Verification Criteria	Verify via Diagnostic/Measurement Tool (e.g., CANoe) that:Disconnecting the Ethernet cable for < 50 ms does NOT trigger a link failure status.Keeping the cable disconnected for > 50 ms causes the software to set Physical_Link_Status to LINK_DOWN.

24.2.1.2.2.5 Link Establishment & Full Duplex Configuration

MVC-113879 - If the software detect the Physical Link status is Link_Up for a max Ethernet Ca...
If the software detect the Physical Link status is Link_Up for a max Ethernet Cable Stability Tout of 50 msec, the software shall configure the MAC Controller to operate as Full duplex mode and provide a "ETH_Link_UP" through MVC-113873 - Ethernet_Link_status to the Diagnostic Feature.

Status	Draft
Verification Criteria	Verify via Diagnostic/Measurement Tool (e.g., CANoe) that when the Ethernet cable is reconnected and remains stable for ≥ 50 ms:Physical_Link_Status is set to LINK_UP.MAC0 is set to FULL_DUPLEX.

24.2.1.3 Non-Functional Requirements

24.2.1.4 Interfaces

24.2.1.4.1 Software Interfaces

24.2.1.4.1.1 Input Interfaces

Camera_MAC_Address

24.2.1.4.1.2 Output Interfaces

Ethernet_PHY_Self_Status

Ethernet_PHY_Type

Ethernet_MAC0_Configured_Speed

Ethernet_MAC_Address_Validity_Status

Ethernet_Link_status

24.2.1.4.2 Hardware Interfaces

24.2.1.4.2.1 Input Interfaces

Ethernet_PHY_Interrupt

24.2.1.4.2.2 Output Interfaces

Ethernet_PHY_gpio

Ethernet_PHYSMI_MDC

Ethernet_MAC_OSC

Ethernet_PHY_Reset

Ethernet _PHY_INHIBIT

24.2.1.4.2.3 Bidirectional Interfaces

Ethernet_PHYSMI_MDIO

Ethernet_PHYRGMII

24.2.1.5 Ethernet UDP/IP Protocol Stack Handling Feature

24.2.1.5.1 Scope

24.2.1.5.2 Functional Requirements

24.2.1.5.2.1 Host name

MVC-118490 - Hostname Format

When transmitting outgoing DHCP messages (DHCP Discover and DHCP Request), the Software shall generate and include the DHCP Hostname as a total 16-bytes by concatenating

- **Device_Type**; read from NVM Software Configuration Data Block (someIPsdConfigurationoption) at address (0x03C30004) with a max length (DEV_TYPE_LENGTH, 256 bytes) as ASCII.
- **Serial_Number**; read from NVM production Data Block at address (SERIAL_NUMBER_ADDRESS, 0x03C2001E) with length of (SERIAL_NUMBER_LENGTH, 8 bytes) as ASCII.

Status	Ready for review
Verification Criteria	Verify through Wireshark that DHCP Discover or Request frame contains the Hostname consists of configured Device_Type concatenated with Serial Number

MVC-118492 - Devicetype length = 7 Byte

When constructing the DHCP Hostname, if the length of Device_Type equals to (DEVICE_TYPE_MIN_LENGTH, Value: 7 Bytes), the Software shall append a 1-byte hyphen "-" as ASCII after Device_Type to form an (DEVICE_TYPE_DEFAULT_LENGTH, Value: 8 Bytes), and concatenate the Serial Number using the following format:

$$Hostname = DeviceType + " - " + SerialNumber$$

Status	Ready for review
Verification Criteria	Verify through Wireshark that if the length of "Device_Type" = 7 bytes , so DHCP Discover or Request frame contains the Hostname = Device_Type-Serial Number

MVC-118491 - Devicetype Length < 7 Byte

When constructing the DHCP Hostname, if the length of Device_Type is less than (DEVICE_TYPE_MIN_LENGTH, Value: 7 Bytes), the Software shall append a hyphen ("-") followed by ASCII '0' padding after Device_Type to form an (DEVICE_TYPE_DEFAULT_LENGTH, Value: 8 Bytes), and concatenate the Serial Number using the following format:

$$Hostname = DeviceType + " - " + Padding'0's + SerialNumber$$

Status	Ready for review
Verification Criteria	Verify through Wireshark that if the length of "Device_Type" < 7 bytes , so the DHCP Discover or Request frame contains the Device_Type length completed with hyphen and zeros in ASCII to be total length as 8 bytes ,and then concatenated with Serail number. example Device_Type = "cam" 3 bytes , Ser. No = 00 00 00 01 So Hostname = "cam-000000000001"

MVC-118493 - Devicetype Length >= 8 Byte

When constructing the DHCP Hostname, if the length of Device_Type bigger than or equals to (DEVICE_TYPE_DEFAULT_LENGTH, Value: 8 Bytes), the Software shall keep only the first 8 bytes of Device_Type and concatenate the Serial Number without inserting neither hyphen nor Zero's padding using the following format:

$$Hostname = (first8bytes)DeviceType + SerialNumber$$

Status	Ready for review
Verification Criteria	Verify through Wireshark that if the length of "Device_Type" >= 8 bytes , so the DHCP Discover or Request frame contains the 1st 8 bytes only from Device_Type concatenated with Serail number. example Device_Type = "backupcamera" 12 bytes , Ser. No = 00 00 00 01 So Hostname = "backupca00000001"

24.2.1.5.2.2 DHCP Sequence , Networking Phase



24.2.1.5.2.2.1 IPCONFIG State

MVC-118496 - IPCONFIG Entry

After Power up, If the Ethernet link status MVC-118488 - Ethernet_Link_status is UP the software shall start in the "IPCONFIG"

State as the initial state to execute the DHCP Client negotiation protocol messages.

Status	Ready for review
Verification Criteria	Power OFF the ECU, ensure the Ethernet link is connected and Up then power up the ECU, verify through Wireshark that the SOC shall start the DHCP Client process, DHCP messages start exchanging

MVC-118498 - Network Communication Restriction

During "IPCONFIG" State, the Software shall restrict the network communication strictly to DHCP client negotiation protocol messages.

Status	Ready for review
Verification Criteria	Verify through Wireshark that after power while the SOC in IPCONFIG state, the SOC transmit only DHCP frames until IP configuration completes

MVC-118502 - Fast DHCP feature - Valid IP address.

During "IPCONFIG" State, the Software shall check the save IP configuration in NVM (NVM fast DHCP Settings section), and if a valid IP address saved; not equal to (INVALID_UDP_IP, Default: 0.0.0.0), the Software shall bypass the broadcast discovery and directly transmit "DHCP REQUEST" message containing the retrieved IP address to confirm this IP configuration.

Status	Draft
Verification Criteria	Verify through Wireshark that in case of "Fast DHCP feature is enabled in the project" and the SOC has a valid IP address, the SOC transmits a " DHCP REQUEST" message without broadcasting "DHCP DISCOVER"

MVC-118503 - Full DHCP negotiation

During "IPCONFIG" State, the Software shall execute the full DHCP negotiation protocol messages by broadcasting a "DHCP DISCOVER" message and waiting until receiving a valid "DHCP OFFER" containing an IP from the master vehicle ECU

Status	Draft
Verification Criteria	Verify through Wireshark that the SOC transmit a broadcast frame "DHCP DISCOVER" message and remains in the "IPCONFIG" state waiting for an incoming "DHCP OFFER"

MVC-118500 - Transmit DHCP Request based on DHCP OFFER

When the software receive an IP from the master ECU included in "DHCP OFFER", the Software shall transmit a "DHCP REQUEST" message containing the offered IP address to confirm this offered IP address.

Status	Ready for review
Verification Criteria	Verify through Wireshark that SOC transmit a "DHCP REQUEST" message containing the exact offer IP address back to the network to confirm the offered IP configuration

MVC-118522 - When the Software successfully receives a "DHCP ACK" message from the master ECU...

When the Software successfully receives a "DHCP ACK" message from the master ECU, the Software shall parse it to extract its assigned IP address through MVC-118510 - ECU_SRC_IP_Address and its associated lease time, subnet mask through MVC-118521 - Network_Subnet_Mask and Gateway through MVC-118561 - Received_Network_Gateway_IP and save MVC-118510 - ECU_SRC_IP_Address in NVM (NVM fast DHCP Settings section)

Status	Draft
Verification Criteria	Tested implicitly with the Req MVC-118501 , that after the SOC parse the ACK message and extract IP , Lease time and subnet mask, then the state changed to "Operational" state to activate the app services as "SOME/IP-OfferService"

MVC-118501 - Processing "DHCP ACK"

The Software switch to "OPERATIONAL" State and activate additional IP application services, including "SOME/IP-OfferService" to advertise the available camera services after parsing the "DHCP ACK" message successfully.

Status	Ready for review
Verification Criteria	Verify through Wireshark that after the ECU transmit the "DHCP REQUEST", after the SOC received a valid "DHCP ACK" msg, the state changed to "OPERATIONAL" state and activate the application services including "SOME/IP-OfferService" start transmitting or offering on the network.

24.2.1.5.2.2.2 OPERATINOAL State

MVC-118506 - DNS Configurations

During the DHCP Client negotiation protocol Process, the Software shall not support Domain Name System (DNS) configurations for DHCPv4; including sending DNS parameter requests or processing received DNS options from the master ECU.

Status	Ready for review
Verification Criteria	Verify through Wireshark that the SOC transmit, receive and process DHCP messages and not request any DNS options and if the SOC receives any DNS options from the Network, it ignores them

MVC-118507 - IP Force Renewal

During "OPERATIONAL" State, when the Software receives a "DHCP FORCERENEW" message from the Master ECU, the Software shall process the message without requesting DHCP Authentication (overriding RFC 3203 authentication requirements) and transmit a "DHCP REQUEST" to the master ECU to initiate the IP address renewal sequence.

Status	Draft
Verification Criteria	Verify through Wireshark that when the SOC receive an unauthenticated "DHCP FORCERENEW" msg from the master ECU, the SOC transmits a "DHCP REQUEST" message to renew its ipaddress.

MVC-118505 - Different IP reassigned.

During "OPERATIONAL" State, When the Software receives a "DHCP ACK" message IP configuration containing a new IP configuration that differs from the currently assigned IP MVC-118510 - ECU_SRC_IP_Address , the Software shall switch to "IPCONFIG" State and stop all active operational services while keeping only the DHCP client active, as follow:

- SOME/IP Service Discovery
- SOME/IP Camera Service
- Shutter Control
- Heating Control

Status	Draft
Verification Criteria	Verify through Wireshark that during the SOC running in OPERATIONAL state, once the soc receives a "DHCP ACK" msg with a different IP, the SOC switch to "IPCONFIG" and stop explicitly 1. Transmission of SOME/IP OfferService notifications 2. SOME/IP Camera Service methods 3. Shutter and Heating operations. while maintaining the DHCP client as active.

MVC-118509 - IP Lease time Expiration.

During "OPERATIONAL" State, When the Software detects that the lease time of the currently assigned IP address expires, the Software shall do the following:

- Reset the saved IP address in NVM (Fast DHCP Settings Memory Map) as (INVALID_UDP_IP, 0.0.0.0)

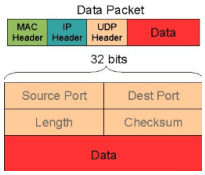
AND

- Switch to "IPCONFIG" State and stop all active operational services including (SOME/IP Service Discovery , SOME/IP Camera Service, Shutter and Heating Control) while keeping only the DHCP client active,

Status	Draft
Verification Criteria	Verify that When the lease time of the currently assigned IP address expired (which received in "DHCP ACK" msg from the Master ECU), the SoC switch back to IP CONFIG and reset its stored IP Address in NVM to 0.0.0.0 and stop all the operational services.

24.2.1.5.2.3 UDP Header Formatting

In this feature UDP / IP, the stack receive the RTP Packet then format UDP header then IPv4 Header then Ethernet Header.



MVC-118863 - Construct UDP Header

The Software shall construct the 8-bytes UDP header to encapsulate the application payload (either MVC-118850 - RTP Packet or Some/IP Packet) into a UDP datagram MVC-118489 - UDP_Datagram in accordance with IETF RFC 768 using the following header fields:

- **Source Port:** MVC-118852 - RTP_Src_Port
- **Destination Port:** MVC-118851 - RTP_Dest_Port
- **Length:** calculated as length of UDP header (8 bytes) + App payload length according to the standard RFC768
- **Checksum:** calculated according to the standard RFC768 .
- **Payload Data:** The application Packet either MVC-118850 - RTP Packet or SOME/IP Packet

Byte Offset	Byte 0	Byte 1	Byte 2	Byte 3
0	Source port		Destination port	
4	Length		Checksum	
8	Payload data			

Status	Ready for review
Verification Criteria	Verify through wireshark that the UDP packets containing Tagged Bitstream , configured UDP source port and Destination port and length and checksum according to standard RFC768

MVC-118865 - Parsing UDP Datagram

When the Software receives MVC-118548 - Encapsulated_Received_UDP_Datagrams from the lower layers (IP layer), the Software shall parse the 8-bytes UDP header, extract the Source port , Destination port, length and checksum fields, and deliver the payload to the upper layer in accordance with IETF RFC 768.

Status	Ready for review
Verification Criteria	Verify implicitly that the Software parses the 8-byte UDP header and forwards the payload correctly by observing the upper layer's response in Wireshark upon receiving a valid UDP datagram

MVC-118864 - Discard Invalid IP

The Software shall discard the received UDP Datagrams MVC-118548 - Encapsulated_Received_UDP_Datagramsif the source or destination IPs are invalid in accordance with IETF RFC 1122 Section 4.1.3.6.

Status	Draft

Verification Criteria	Verify implicitly that the Software discards UDP datagrams with invalid IP addresses by observing through Wireshark that no payload is delivered to the upper layer and no corresponding application response or action is generated

MVC-118866 - Port Demultiplexing

When the Software receives a UDP datagram MVC-118548 - Encapsulated_Received_UDP_Datagrams, the Software shall determine the owner application layer either RTP layer or ant other Application Layer (SOME/IP Layer) associated with the Destination port field and route the payload data to that layer.

Status	Ready for review
Verification Criteria	Verify implicitly via Wireshark that the Software inspects the Destination Port of a received UDP datagram and routes its payload to the matching upper layer, as evidenced by the corresponding upper layer (RTP or Application) executing its expected action upon receiving the payload.

MVC-118867 - UDP Checksum

The Software shall calculate the UDP checksum for all transmitted UDP datagrams MVC-118489 - UDP_Datagram , and shall verify the UDP checksum when receiving a UDP datagram MVC-118548 - Encapsulated_Received_UDP_Datagrams , discarding any packet with an invalid checksum in accordance with IETF RFC 1122 Section 4.1.3.4."

Status	Ready for review
Verification Criteria	Verify implicitly via Wireshark that the Software correctly handles UDP checksums by verifying that all transmitted datagrams contain a valid checksum field, and that received datagrams with invalid checksums are discarded, as evidenced by the upper layer remaining inactive and generating no response for invalid checksum packets

24.2.1.5.2.4 IPv4 Header Formatting

MVC-118869 - IPv4 Format

The Software shall implement the Internet Protocol Version 4 (IPV4) stack to format the IPv4 Header, transmit and receive the packets in accordance with IETF RFC 791

Status	Ready for review
Verification Criteria	Verify through Wireshark that the software formats, transmits, receives and parses IPv4 headers and packets according to RFC 791 specifications

MVC-118868 - Version Field Construction

The Software shall set the Version field of the IPv4 header as 4 Decimal (0b 0100) in all outgoing IPV4 datagrams in accordance with IETF RFC 791 Section 3.1

Status	Ready for review
Verification Criteria	Verify through Wireshark that all transmitted IPv4 datagrams have the Version field set to 4.

MVC-118871 - Version Field Validation

The Software shall discard any incoming IPV4 datagram containing a version field not equal to 4 Decimal (0b 0100) in accordance with IETF RFC 1122 Section 3.2.1.1

Status	Ready for review
Verification Criteria	Verify through Wireshark that in case of incoming frames with version field value != 4 , the software ignore and discard the, without any processing or any internal SW errors.

MVC-118870 - Checksum Construction

The Software shall calculate and set the Checksum field in IPV4 Header for all outgoing IPV4 datagrams in accordance with IETF RFC 791 Section 3.1.

Status	Ready for review
Verification Criteria	Verify through Wireshark that all transmitted IPv4 frames contain a valid IPV4 header checksum, calculated as defined in RFC 791

MVC-118872 - Checksum Validation

The Software shall verify that the checksum in the IPv4 header of all incoming IPv4 datagrams and ignore any received datagram with an invalid checksum accordance with IETF RFC 1122 Section 3.2.1.2

Status	Ready for review
Verification Criteria	Verify through Wireshark that in case of incoming frames with Corrupted header checksum, the software ignore and discard the, without any processing or any internal SW errors.

MVC-118874 - IP Addresses Fields Construction

The Software shall set the currently assigned IP of SOC MVC-118510 - ECU_SRC_IP_Address in the Source IP Address field as "32 bit" and set the IP of the subscribed Destination (Master ECU) in the Destination IP Address field as "32 bit" in accordance with the standard IETF RFC 791 Section 3.2 and IETF RFC 1122 Section 3.2.1.3 Addressing and IETF RFC 791 3.2 Internet Header Format.

Status	Ready for review
Verification Criteria	Verify through Wireshark that transmitted frame contains the assigned IP in the Source IP Field and IP of the subscribed ECU in the Destination IP field.

MVC-118873 - IP Addresses Fields Parsing

The Software shall parse and process the incoming frames only if the Destination IP field holds an IP Address match the Local assigned IP address MVC-118510 - ECU_SRC_IP_Address or a valid broadcast address or an active multicast group address and discard all other datagrams in accordance with IETF RFC 1122 Section 3.2.1.3. Addressing and IETF RFC 791 3.2 Internet Header Format.

Status	Ready for review
Verification Criteria	Verify through Wireshark that in case of incoming frames with a Destination IP match its local assigned IP, the Software parse it and passed it to the other layers in the stack. But in case of incoming frames with mismatch IP, the Software discard it without any further processing.

MVC-118876 - Time To Live Field - TTL Construction

The Software shall set the 8-bit Time-To-Live (TTL) field in the IPv4 header to the configured TTL in all transmitted datagrams in accordance with IETF RFC 791 Section 3.2 and IETF RFC 1122 Section 3.2.1.7.

Status	Draft
Verification Criteria	Verify through Wireshark that all the IPv4 frames transmitted from SOC contains a configured defined Time To Live in the TTL field

MVC-118875 - Time To Live Field - TTL Validation

The Software shall Parse and process the incoming IPv4 frames only if the frame has a TTL field with a value greater than (INVALID_TTL, Value: 0) in accordance with IETF RFC 791 Section 3.2 and IETF RFC 1122 Section 3.2.1.7.

Status	Draft
Verification Criteria	Verify through Wireshark that the SOC parse and route the packet to the upper layer only if the incoming IPv4 frame has TTL >0

MVC-118878 - MTU Construction

The Software shall set the Max Transmission Unit (MTU) for the IPv4 as (IPv4_MTU, Value: 1500 bytes) and provide it through MVC-118514 - IPv4_Configured_MTU

Status	Ready for review
Verification Criteria	Verify through Wireshark that the MTU field in all IPv4 frames transmitted or received set to 1500 bytes.

MVC-118877 - Network Prefix Calculating

The Software shall calculate the local network prefix (Network ID) for its assigned IPv4 address MVC-118510 - ECU_SRC_IP_Address using the received MVC-118521 - Network_Subnet_Mask , in accordance with IETF RFC 4632 Section 3.1. to identify the local subnet boundaries when transmitting frames to a Destination IP.

Status	Ready for review
Verification Criteria	Verify implicitly via Wireshark that the Software calculates the local network prefix correctly, as evidenced by transmitting ARP Requests directly for destination IPs within the local subnet, and routing through the default gateway for destination IPs outside the local subnet

$$LocalNetworkPrefix = itsassignedIPAddress \& NetworkSubnetmask$$

Ex:

Assigned IPv4 for SOC = 192.168.1.50

Subnet Mask = 255.255.255.0

Subscribed Destination IP = 192.168.1.200

1. calculate Local Network Prefix based on the Assigned IPv4 and Subnet Mask

Local Network ID = Assigned IP & Network Subnet Mask

= 192.168.1.50 & 255.255.255.0 = 192.168.1.0

2. to check the Destination for which the SOC will transmit the frame

calculate the Destination Network ID as

Destination Network ID = Destination IP & Network Subnet Mask

= 192.168.1.200 & 255.255.255.0 = 192.168.1.0

It means Dest. Network ID = Local Network ID -> it means that the SOC transmit the frames directly without needing to forward the frame firstly to gateway.

MVC-118882 - Mutlicast

The Software shall identify an IPv4 Multicast datagram if its Destination IPv4 address is within the Class D IPv4 Multicast Range (MIN_IPV4_MULTICAST_RANGE, Value: 224.0.0.0) to (MAX_IPV4_MULTICAST_RANGE, Value: 239.255.255.255) in accordance with IETF RFC 1112 Section 4.

Status	Ready for review
Verification Criteria	Verify through Wireshark that when the SOC receive a multicast IPv4 frame from the network, the SOC can parse the frame and pass it to the other layers in the stack. Also it can transmit a multicast frames.

MVC-118881 - Mutlicast

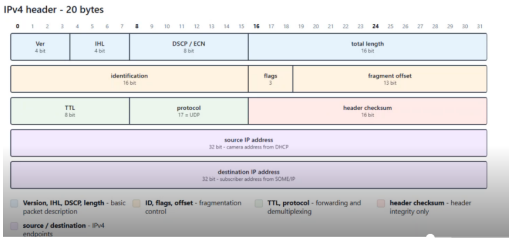
When the Software transmit an IPV4 multicast datagram over Ethernet, the Software shall construct the Destination Ethernet MAC address based on corresponding to the Destination IPv4 multicast address according to IETF RFC 1112 Section 6 and provide it through MVC-118534 - Destination_MAC

Status	Ready for review
Verification Criteria	Verify through Wireshark that when the SOC transmit a multicast frame , the SOC calculate the Dest. MAC Address mapped to the multicast Dest. IP correctly as defined in the standard

MVC-118884 - IPv4 Header Formatting

The Software shall construct the IPv4 packet by prepending a 20-byte IPv4 header to the UDP datagramMVC-118489 - UDP_Datagram , containing the Version, Total length, Time-To-Live (TTL), protocol (UDP), Header Checksum, Source IP Address and Destination IP address fields in accordance with RFC791 Section 3.1 and provide it through MVC-118547 - TX_IPv4_Packet

Status	Ready for review
Verification Criteria	Verify through Wireshark that all transmitted IPv4 packets contain a valid 20-byte IPv4 header containing correct fields values prior to the UDP datagram.



MVC-118885 - IPv4 Header Parsing

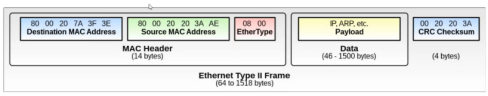
When receiving an IPV4 packet from Ethernet MVC-118855 - RX_IPv4_Packet , the Software shall parse the 20-byte IPv4 header to extract the Version, Total length, Time-To-Live (TTL), protocol (UDP), Header Checksum, Source IP Address and Destination IP address fields, and deliver the encapsulated UDP datagram through MVC-118548 - Encapsulated_Received_UDP_Datagrams to the upper layer upon successful validation in accordance with RFC791 Section 3.1.

Status	Ready for review
Verification Criteria	Verify through Wireshark that when the Software receive in IPv4 frame, the SOC can parse the IPv4 header and extract the data correctly.

MVC-118887 - Ethernet Header Encapsulation

The Software shall construct the Ethernet Header through MVC-118546 - TX_Ethernet_Frame by prepending the Ethernet Header to the MVC-118547 - TX_IPv4_Packet in accordance with IETF RFC 894; consisting of MVC-118534 - Destination_MAC , Source MAC Address and the Ethernet Type field as 0x800.

Status	Ready for review
Verification Criteria	Verify through Wireshark that all transmitted Ethernet frames encapsulate the IPv4 packet with a valid Ethernet Header containing the mapped/resolved Destination MAC address, the SOC's Source MAC address, and the EtherType field set to 0x800 as specified in IETF RFC 894



MVC-118886 - Ethernet Header Parsing

When receiving an Ethernet II Frame through MVC-118564 - RX_Ethernet_Frame, the Software shall parse the Ethernet Header and process the payload as an IPv4 packet only if the Ethernet Type field equals to 0x800s and the Destination MAC address matches the local physical MAC address or an active multicast MAC address in accordance with IETF RFC 894

Status	Ready for review
Verification Criteria	Verify through Wireshark that when receiving an Ethernet frame with EtherType 0x800 , the Software correctly extracts the IPv4 packet and passes it to the IPv4 layer, and discards any frame with any other EtherType or invalid Destination MAC

When transmitting an IPv4 packet, the Software shall determine that the destination is on the same local network if the result of applying the subnet mask to the destination IP address matches the source network address."

MVC-118889 - Direct Transmission to Destination

When transmitting an Ethernet Frame, the Software shall calculate the Destination Network ID based on the subscribed Destination IP address and MVC-118521 - Network_Subnet_Mask if the calculated Destination Network ID matches the Local Network ID, the Software shall transmit the frame MVC-118546 - TX_Ethernet_Frame directly to the destination over the local network.

Status	Ready for review
Verification Criteria	Verify through Wireshark that in case of calculated Local Network ID of Dest IP = Local Nework ID of SRC IP, the software forward/ transmit the frame directly to the Dest. without routing through a default gateway.

MVC-118888 - Transmission to Gateway

If the calculated Destination Network ID doesn't match the Local Network ID (the destination is outside the local subnet boundaries), the Software shall transmit the frame MVC-118546 - TX_Ethernet_Frame to the default gateway using the MVC-118561 - Received_Network_Gateway_IP

Status	Draft
Verification Criteria	Verify through Wireshark that When the destination IP address falls outside the Local subnet boundaries, the software routes the transmitted packet to the Default Gateway.

24.2.1.5.2.5 ICMP Management (Internet Control Management Protocol)

MVC-118892 - ICMP Echo Request

When the Software receives MVC-118564 - RX_Ethernet_Frame encapsulating an IPv4 Packet MVC-118855 - RX_IPv4_Packet in which the Protocol field set as (ICMP_PROTOCOL_TYPE, Value: 1), the Software shall identify this received frame as ICMP Echo Request and parse the ICMP header, validate the ICMP checksum and extract the identifier, sequence number and data payload in accordance with IETF RFC 792.

Status	Ready for review
Verification Criteria	Verify through Wireshark that when an ICMP Echo Request frame, the Software successfully validates the ICMP checksum and extracts the Identifier, Sequence Number, and Data payload field

MVC-118891 - ICMP Echo Reply

When the Software receives a valid ICMP Echo Request, the Software shall construct and transmit an ICMP Echo Reply encapsulated in an IPv4 packet MVC-118547 - TX_IPv4_Packet with a Protocol field set as (ICMP_PROTOCOL_TYPE, Value: 1), and wrapped in an Ethernet Frame MVC-118546 - TX_Ethernet_Frame, containing the received Identifier, Sequence Number, and Data payload, directed to the requester's IP and MAC addresses in accordance with IETF RFC 792.

Status	Ready for review
Verification Criteria	Verify through Wireshark by once the SOC receives an ICNP Echo Req , the SOC responds with an ICMP Echo Reply containing the exact same Identifier, Sequence Number, and Payload data as sent in the request, addressed to the requester's IP and MAC address.

24.2.1.5.2.6 ARP Management (Address Resolution Protocol)

MVC-118896 - ARP Reply Construction & Transmission

When the Software receives MVC-118564 - RX_Ethernet_Frame encapsulating an IPv4 Packet MVC-118855 - RX_IPv4_Packet in which the EtherType field set as (ETHER_TYPE_FIELD, Value: 0x806), the Software shall identify this received frame as ARP Request and parse the ARP header to validate the following in accordance with IETF RFC 826

- Hardware Type set as (HARDWARE_TYPE_FIELD, Value: 0x001),
- Protocol Type set as "IPv4" (ARP_PROTOCOL_TYPE, Value: 0x800),
- Hardware Size field set as (ARP_HW_TYPE, Value: 6),
- Protocol Size field set as (ARP_PROTOCOL_SIZE, Value: 4) and
- Operation Code Field set as "Request" (ARP_OPCODE_FIELD, Value: 1)

Status	Ready for review
Verification Criteria	Verify through Wireshark that upon receiving an Ethernet frame with EtherType 0x806, the Software correctly parses and validates the ARP Request header fields (Hardware Type = 0x001, Protocol Type = 0x800, Hardware Size = 6, Protocol Size = 4, and Opcode = 1) in accordance with IETF RFC 826."

MVC-118895 - ARP Reply Construction & Transmission

When Receiving a valid ARP Request with a destination address match the SOC's IP MVC-118510 - ECU_SRC_IP_Address , the Software shall construct and transmit an ARP Reply with Operation Code Field set as "Reply" (ARP_OPCODE_FIELD, Value: 2), and the SOC's MAC MVC-118533 - ECU_SRC_MAC as Target HW Address wrapped in an Ethernet frame MVC-118546 - TX_Ethernet_Frame transmitted to the requester's MAC address in accordance with IETF RFC 826.

Status	Ready for review
Verification Criteria	Verify implicitly via Wireshark that upon receiving a valid ARP Request targeting the SOC's IP address, the Software constructs and transmits an ARP Reply with Opcode set to 2, incorporating the SOC's MAC address as the Sender HW Address, and directed to the requester's MAC address

MVC-118898 - GARP Request

When the Software has assigned with a new IPv4 address MVC-118510 - ECU_SRC_IP_Address , the Software shall construct and transmit 3 Broadcast consecutive Gratuitous ARP Request messages (GARP) in which Sender IP field set as the new assigned IP for SOC MVC-118510 - ECU_SRC_IP_Address and Target MAC set as (BROADCAST_MAC, Value: FF:FF:FF:FF:FF:FF)
wrapped in an Ethernet frame MVC-118546 - TX_Ethernet_Frame to update neighboring ARP caches.

Status	Ready for review
Verification Criteria	Verify through Wireshark that when the SOC has assigned with a new IP, the SOC transmit 3 consecutive GARP msgs broadcast.

MVC-118897 - ARP Resolution and Cache Management

When the Software transmit an a IPV4 packet to local destination IP address whose MAC address is not present in the ARP cache
, the Software shall send an ARP Request for its IP address and temporarily store the latest transmitted packet for that destination.

Status	Ready for review
Verification Criteria	Verify through Wireshark that when an IPv4 packet is sent to an IP not in the ARP cache, the Software transmits an ARP Request for that IP and does not transmit the IPv4 packet until the ARP is resolved.

MVC-118900 - ARP Resolution and Cache Management

When the Software receive a valid ARP reply that resolves the target IP address, the Software shall update the ARP Cache with the resolved MAC address and immediately transmit the stored buffered packet to the MAC address over Ethernet.

Status	Ready for review
Verification Criteria	Verify through Wireshark that upon receiving a valid ARP Reply for the requested IP, the Software immediately transmits the buffered IPv4 packet encapsulated in an Ethernet frame using the resolved MAC address

24.2.1.5.3 Non-Functional Requirements

24.2.1.5.4 Interfaces

24.2.1.5.4.1 Software Interfaces

24.2.1.5.4.1.1 Input Interfaces

Ethernet_Link_status

RX_Ethernet_Frame

RX_IPv4_Packet

RTP Packet

RTP_Src_Port

RTP_Dest_Port

24.2.1.5.4.1.2 Output Interfaces

ECU_SRC_IP_Address

Network_Subnet_Mask

Received_Network_Gateway_IP

IPv4_Configured_MTU

Destination_MAC

ECU_SRC_MAC

UDP_Datagram

TX_IPv4_Packet

TX_Ethernet_Frame

Encapsulated_Received_UDP_Datagrams

24.2.1.5.4.1.3 Hardware Interfaces

24.2.1.5.4.1.4 Input Interfaces

24.2.1.5.4.1.5 Output Interfaces

24.2.1.5.4.2 Bidirectional Interfaces

24.2.1.6 Ethernet RTP Streaming Handling Feature

24.2.1.6.1 Scope

24.2.1.6.2 Functional Requirements

24.2.1.6.2.1 RTP Stream Routing and Socket Configuration

24.2.1.6.2.1.1 ISP0 / ISP1 Source IP Binding

MVC-117137 - ISP0 Source Port Valid Config

If the MVC-117284 - RTP_Src_Port1 retrieved from NVM is within the valid range from (RTP_MIN_VALID_PORT, Value: 1024) to (RTP_MAX_VALID_PORT, Value: 65535), The Software shall bind the local UDP socket to MVC-117284 - RTP_Src_Port1 f or transmitting the MVC-117136 - Tagged_H264_Video_Bitstream of ISP0.

Status	Draft
Verification Criteria	Verify through Wireshark during ISP0 video transmission that the Software transmits all UDP packets using the configured Source port in NVM if it falls within the valid range (1024 to 65535)

MVC-117138 - ISP0 Source Port Default Port as fallback

The Software shall bind the local UDP socket to the default port (UDP_SRC_DEFAULT_PORT, Default: 5004) for transmitting the MVC-117136 - Tagged_H264_Video_Bitstream of ISP0, If the MVC-117284 - RTP_Src_Port1 retrieved from NVM meet any one of the following conditions:

- MVC-117284 - RTP_Src_Port1is unconfigured, equals (RTP_UNCONFIGURED_PORT, Default: : 0xFFFF)

OR

- MVC-117284 - RTP_Src_Port1is outside the valid range; lower than (RTP_MIN_VALID_PORT, Default: 1024) or higher than (RTP_MAX_VALID_PORT, Default: 65535)

Status	Draft
Verification Criteria	Verify through Wireshark during ISP0 video transmission that the Software transmits all UDP packets using the default Source Port (5004) when rtpSourcePort1 in NVM = 0xFFFF or to an invalid value outside the range [1024–65535]

MVC-117140 - ISP1 Source Port Valid Config

If the MVC-117288 - RTP_Src_Port2retrieved from NVM is within the valid range from (RTP_MIN_VALID_PORT, Value: 1024) to (RTP_MAX_VALID_PORT, Value: 65535), The Software shall bind the local UDP socket to MVC-117288 - RTP_Src_Port2for transmitting the MVC-117136 - Tagged_H264_Video_Bitstream of ISP1.

Status	Draft
Verification Criteria	Verify through Wireshark during ISP1 video transmission that the Software transmits all UDP packets using the configured Source port in NVM if it falls within the valid range (1024 to 65535)

MVC-117139 - ISP1 Source Port Default Port as fallback

The Software shall bind the local UDP socket to the default port (ISP1_SRC_DEFAULT_PORT, Default: 5006) for transmitting the MVC-117136 - Tagged_H264_Video_Bitstream of ISP1, If the MVC-117288 - RTP_Src_Port2 retrieved from NVM meet any one of the following conditions:

- MVC-117288 - RTP_Src_Port2 is unconfigured, equals (RTP_UNCONFIGURED_PORT, Default: : 0xFFFF)

OR

- MVC-117288 - RTP_Src_Port2 is outside the valid range; lower than (RTP_MIN_VALID_PORT, Default: 1024) or higher than (RTP_MAX_VALID_PORT, Default: 655

Status	Draft
Verification Criteria	Verify through Wireshark during ISP1 video transmission that the Software transmits all UDP packets using the default Source Port (5006) when rtpSourcePort1 in NVM = 0xFFFF or to an invalid value outside the range [1024–65535]

MVC-117144 - ISP0/ISP1 Sockets - SOC IP Binding

The Software shall bind the local UDP socket corresponding to ISP0 and ISP1 to the active local IP address of the ECU MVC-119053 - ECU_SRC_IP_Address if the retrieved IP address is valid and not equals (UNCONFIGURED_IP, Default: 0.0.0.0)

Status	Draft
Verification Criteria	Verify through Wireshark during ISP0 and ISP1 video transmission that all transmitted UDP packets use a SOC's Source IP Address, provided it is a valid IP address and not equal to 0.0.0.0.

24.2.1.6.2.1.2 ISP0 Destination IP & Dynamic Port Calculation & Transmission Conditions

MVC-117145 - ISP0 Destination Port Calculation

The Software shall calculate the destination port1 for ISP0 MVC-117286 - RTP_Dest_Port1 by adding "rtpDestinationPort1" retrieved from NVM to the last octet of MVC-119053 - ECU_SRC_IP_Address

Status	Draft
Verification Criteria	Verify through Wireshark during ISP0 video transmission that the UDP Destination Port = the dest port1 saved in NVM + the last byte of the ECU source IP

$$DestinationPort = rtpDestinationport1 + (ECUIPAddress\&0xFF)$$

MVC-117908 - ISP0 Destination Port - Default Fallback

If "rtpDestinationPort1" retrieved from NVM is invalid, or unconfigured; equals (RTP_UNCONFIGURED_PORT, Value: : 0xFFFF), the Software shall use a default port (UDP_Dest_DEFAULT_PORT1, Value: 5005) as the base port to calculate the Destination Port1 MVC-117286 - RTP_Dest_Port1

Status	Draft
Verification Criteria	Verify through Wireshark when Destination port 1 in NVM = 0xFFFF that the transmitted UDP Destination Port is calculated using the default base port (5005) + the last byte of the ECU source IP

MVC-117291 - ISP0 Video Transmission - Destination1

The Software shall transmit the tagged video stream MVC-117136 - Tagged_H264_Video_Bitstream of ISP0 to the Destination IP MVC-117285 - RTP_Dest_IP1 retrieved from NVM using MVC-117286 - RTP_Dest_Port1, If all the following conditions are met:

- MVC-117285 - RTP_Dest_IP1 is valid and not equal to (UNCONFIGURED_IP, Value: 0.0.0.0)

AND

- MVC-117286 - RTP_Dest_Port1 is within the valid range from (RTP_MIN_VALID_PORT, Value: 1024) to (RTP_MAX_VALID_PORT, Value: 65535)

Status	Draft
Verification Criteria	Verify through Wireshark that the ISP0 video stream is transmitted to Dest IP1 on calculated Dest Port1 when the destination IP is valid (!= 0.0.0.0) and the calculated destination port is within [1024–65535]

Question: Stop transmission and transmit to the dynamically subscribed Client or transmit to a default Dest IP ????

MVC-117301 - ISP0 Video Transmission - Dynamic On-Demand

If MVC-117285 - RTP_Dest_IP1 retrieved from NVM equals to (UNCONFIGURED_IP, Value: 0.0.0.0), the Software shall stop the transmission and transmit the MVC-117136 - Tagged_H264_Video_Bitstream of ISP0 to the IP address and Port of the dynamically subscribed client when receiving a valid video stream request.

Status	Draft
Verification Criteria	Verify through Wireshark when Dest IP1 in NVM = 0.0.0.0 -> 1. No ISP0 video packets are transmitted before receiving a client request & 2. once receiving a video req from a client, the the ISP0 video stream is transmitted directly to the requesting client's IP address and Port

24.2.1.6.2.1.3 ISP1 Destination IP & Dynamic Port Calculation & Transmission Conditions

MVC-117293 - ISP1 Destination Port Calculation

The Software shall calculate the destination port1 for ISP1 MVC-117290 - RTP_Dest_Port2 by adding "rtpDestinationPort2" retrieved from NVM to the last octet of MVC-119053 - ECU_SRC_IP_Address

Status	Draft
Verification Criteria	Verify through Wireshark during ISP1 video transmission that the UDP Destination Port = the dest port2 saved in NVM + the last byte of the ECU source IP

$$DestinationPort = rtpDestinationport1 + (ECUIPAddress \& 0xFF)$$

MVC-117909 - ISP1 Destination Port - Default Fallback

If "rtpDestinationPort2" retrieved from NVM is invalid, or unconfigured; equals (RTP_UNCONFIGURED_PORT, Value: : 0xFFFF), the Software shall use a default port (UDP_Dest_DEFAULT_PORT2, Value: 5038) as the base port to calculate the Destination Port2 MVC-117290 - RTP_Dest_Port2

Status	Draft
Verification Criteria	Verify through Wireshark when Destination port2 in NVM = 0xFFFF that the transmitted UDP Destination Port is calculated using the default base port (5038) + the last byte of the ECU source IP

MVC-117292 - ISP1 Video Transmission - Destination2

The Software shall transmit the tagged video stream MVC-117136 - Tagged_H264_Video_Bitstream of ISP1 to the Destination IP MVC-117289 - RTP_Dest_IP2 retrieved from NVM using MVC-117290 - RTP_Dest_Port2, If all the following conditions are met:

- MVC-117289 - RTP_Dest_IP2 is valid and not equal to (UNCONFIGURED_IP, Value: 0.0.0.0)

AND

- MVC-117290 - RTP_Dest_Port2is within the valid range from (RTP_MIN_VALID_PORT, Value: 1024) to (RTP_MAX_VALID_PORT, Value: 6553)

Status	Draft
Verification Criteria	Verify through Wireshark that the ISP0 video stream is transmitted to Dest IP1 on calculated Dest Port1 when the destination IP is valid (!= 0.0.0.0) and the calculated destination port is within [1024–65535]

Question: Stop transmission and transmit to the dynamically subscribed Client or transmit to a default Dest IP ????

MVC-117299 - ISP1 Video Transmission - Dynamic On-Demand

If MVC-117289 - RTP_Dest_IP2 retrieved from NVM equals to (UNCONFIGURED_IP, Value: 0.0.0.0), the Software shall stop the transmission and transmit the MVC-117136 - Tagged_H264_Video_Bitstream of ISP1 to the IP address and Port of the dynamically subscribed client when receiving a valid video stream request.

Status	Draft
Verification Criteria	Verify through Wireshark when Dest IP1 in NVM = 0.0.0.0 -> 1. No ISP0 video packets are transmitted before receiving a client request & 2. once receiving a video req from a client, the the ISP0 video stream is transmitted directly to the requesting client's IP address and Port

24.2.1.6.2.1.4 Dual Streaming

Need a system req for it

MVC-117304 - Dual video Stream concurrency

When the Software receives two video streams one with MVC-117334 - ISP_Index as ISP0 and ISP1 concurrently, the Software shall transmit both video streams independently and simultaneously over their individually configured UDP endpoints (Source and Destination IP and port pairs for ISP0 and ISP1).

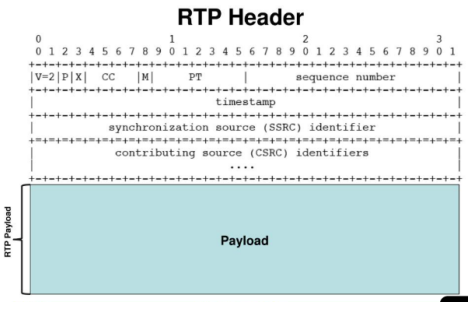
Status	Draft
Verification Criteria	Verify by simultaneously feeding input video streams to both ISP0 and ISP1, and through wireshark confirm that both video streams are transmitted concurrently and independently without packet collision or blocking or frame drops

24.2.1.6.3 RTP Protocol Serialization

MVC-117314 - RTP Packet Serialization - Network Byte Order

The Software shall serialize all the RTP Packet fields for the received tagged video stream MVC-117136 - Tagged_H264_Video_Bitstream; for both ISP0 and ISP1 through MVC-117334 - ISP_Index using Network Byte Order (Big-Endian) in accordance with RFC3550.

Status	Draft
Verification Criteria	Verify through Wireshark during active video streaming for ISP0, ISP1, and concurrent streams that all RTP header fields are formatted in Big-Endian byte order per RFC 3550."



MVC-117318 - RTP Packet Construction - Constant Header Fields

The Software shall construct the RTP Packet through MVC-118849 - RTP Packetfor the transmitted the tagged video stream MVC-117136 - Tagged_H264_Video_Bitstream for ISP0 or ISP1 through MVC-117334 - ISP_Index by prepending the constant RTP header fields defined in the standard RFC3550, Section 5.1 as follows:

- **Version Field** as (RTP_HDR_VER, Value: 0x2)
- **Padding Field** as (RTP_HDR_PADDING: Value: 0x0)
- **Extension Field** as : (RTP_HDR_EXT: Value: 0x0)
- **CSRC Count Field** as (RTP_HDR_COUNT: Value: 0x0)
- **Payload Type Field** as (RTP_HDR_PAYLOAD_TYPE: Value: 0x60 / 96 decimal)

Status	Draft
Verification Criteria	Verify through Wireshark for any active video stream that the static fields of the RTP header match RFC 3550 Section 5.1 (Version = 0x2, Padding = 0x0, Extension = 0x0, CSRC Count = 0x0, Payload Type = 0x60

MVC-117317 - Dynamic RTP Header Fields Structure

The Software shall construct the complete RTP Packet MVC-118849 - RTP Packet for the transmitted the tagged video stream MVC-117136 - Tagged_H264_Video_Bitstream for ISP0 or ISP1 through MVC-117334 - ISP_Index by combining the dynamic RTP Header fields (Marker bit, Sequence Number, Timestamp, SSRC) with the video payload according to the standard RFC3550, Section 5.1

Status	Draft
Verification Criteria	Verify through Wireshark during video streaming that each transmitted RTP packet contains valid dynamic header fields (Marker Bit, Sequence Number, Timestamp, and SSRC) prepended to the H.264 video payload

MVC-117320 - RTP-Sequence Number

The Software shall maintain the 16-bit RTP sequence Number field for each transmitted video stream MVC-117136 - Tagged_H264_Video_Bitstream in accordance with RFC3550, Section 5.1 as follows:

- **Initialization (intentionally deviating from RFC 3550):** the Sequence number shall start at (RTP_Seq_No_Initial_Value, Value: 0) rather than the randomized initial value specified in RFC 3550.
- **Sequential Incrementation:** the Sequence number shall be incremented by one (+1) for every consecutively transmitted RTP packet per stream.
- **Overflow Wrap-Around:** the Sequence number shall wrap around to (RTP_Seq_No_Initial_Value, Value: 0) in the subsequent packet to the Packet where the sequence number reach its max 16-bit (RTP_Seq_Max_Overflow_Value, Value: 65535/0xFFFF)

Status	Draft
Verification Criteria	Verify by triggering video streaming for both ISP0 and ISP1 simultaneously. Confirm through Wireshark that: The sequence number of the first transmitted RTP packet equals 0 for both streams. Each subsequent packet increments the sequence number by +1 independently per stream. When the sequence number reaches 65535 (0xFFFF), the next packet seamlessly wraps around and resets to 0."

MVC-117322 - RTP Packet - Timestamp

The Software shall maintain the 32-bit RTP Timestamp field for each transmitted video stream MVC-117136 - Tagged_H264_Video_Bitstream in accordance with RFC3550, Section 5.1 and RFC 6184, Section 5.1 as follows:

- **Time Base Reference:** The Timestamp represents a sampling time derived from (Reference_Clock_Rate, Value: 90 KHz) independent of the Synchronized Global Vehicle time (Synchronized in gPTP protocol).
- **Initialization:** Timestamp shall start at (RTP_TimeStamp_Initial_Value, Value: 0).
- **Frame level Persistence:** All RTP packets belonging to the same video frame shall hold an identical timestamp value.
- **Sequential Incrementation:** The timestamp shall be incremented by (Increment_Per_Frame, Value: 3000 ticks) between consecutive video frames based on the (Configured_Frame_Rate, Value: 30FPS).
- **Overflow Wrap-Around:** The timestamp shall wrap around to (RTP_TimeStamp_Initial_Value, Default: 0) in the subsequent packet after reaching its max 32-bit (RTP_TimeStamp_Overflow_Value, Value: 0xFFFFFFFF)

Status	Draft
Verification Criteria	Verify by capturing transmitted RTP packets for ISP0 and ISP1 using Wireshark under active 30 fps video streaming to confirm that: 1. The timestamp of the first video frame equals 0 & 2. All RTP packets belonging to the same video frame carry an identical timestamp value & 3. The timestamp value increments by exactly 3000 ticks between consecutive video frames. &4. when reaching 0xFFFFFFFF, the timestamp seamlessly wraps around to 0 on the subsequent frame reaching

$$SingleFrameTime = 1/30FPS$$

$$ClockRate = 90KHz = 90,000tickinonesec$$

$$Singleframeticksinoneclockof90,000Hz = \frac{90,000ticks/sec}{30frames/sec} = 3000ticks$$

MVC-117319 - RTP - SSRC

The Software shall generate a unique 32-bit SSRC (Synchronization Source) identifier for each active video stream MVC-117136 - Tagged_H264_Video_Bitstream for ISP0 or ISP1 through MVC-117334 - ISP_Index by combining the lower 24-bits of ECU MAC address with (ISP0_ID, Value: 0x00) for ISP0 or (ISP1_ID, Value: 0x01) for ISP1 as defined in rfc3550 section 5.1.

Status	Draft
Verification Criteria	Verify by triggering dual video streaming for both ISP0 and ISP1 simultaneously. Confirm through Wireshark that the SSRC value generated for ISP0 equals (Lower 24-bits of ECU MAC || 0x00) and for ISP1 equals (Lower 24-bits of ECU MAC || 0x01), ensuring both streams have strictly

MVC-119074 - Unique SSRC without Collision detection implementation

The Software shall maintain the assigned SSRC statically throughout the streaming session without executing RTCP collision detection or resolution procedures as defined in RFC 3550, Section 8.2.

Status	Draft
Verification Criteria	Verify through Wireshark during continuous video streaming that each stream maintains its initially assigned SSRC value statically across all transmitted RTP packets throughout the entire session, and confirm that no RTCP control packets or collision handling procedures are transmitted on the network

MAC Address = 6 bytes and SSRC= 4 bytes, So concatenate lower 3 bytes of MAC with 0x00 0r 0x01

24.2.1.6.3.1 RTP Packet - Marker

MVC-117323 - The Software shall set the 1-bit Marker (M) field to (RTP_Marker_Bit_End_Value,...

The Software shall set the 1-bit Marker (M) field to (RTP_Marker_Bit_End_Value, Default: 1) in the last RTP packet within a fragmented IDR NAL-unit and shall set it to (RTP_Marker_Bit_Default_Value, Default: 0) as the default state for each transmitted RTP packet, MVC-117136 - Tagged_H264_Video_Bitstream in accordance with RFC 6184, Section 5.1

Status	Draft
Verification Criteria	Verify through Wireshark during the active video streaming for ISP0 or ISP1 confirm that 1-bit Marker Field as 1 in the final RTP Packet and as 0 for all other transmitted Packets

For each compressed video frame (Access Unit) received for transmission on an active video stream, the software shall structure and transmit the RTP packets over the network in the following mandatory sequence:

1. **SPS Packet:** Transmit a single RTP packet carrying the H.264 SPS NAL unit (Type 7) with Marker Bit = `0`.
2. **PPS Packet:** Transmit a single RTP packet carrying the H.264 PPS NAL unit (Type 8) with Marker Bit = `0`.
3. **SEI Packet:** Transmit a single RTP packet carrying the H.264 SEI NAL unit (Type 6) prepended to the received tagged metadata payload, with Marker Bit = `0`.
4. **IDR Packets (Video Payload):** Split the IDR NAL unit (Type 5) into consecutive FU-A RTP packets (Type 28) if its size exceeds `Max_RTP_Payload_Size` (1460 bytes) as follows:
   - **First Fragment:** Set `S bit = 1` in FU Header, Marker Bit = `0`.
   - **Intermediate Fragments:** Set `S bit = 0` and `E bit = 0` in FU Header, Marker Bit = `0`.
   - **Final Fragment:** Set `E bit = 1` in FU Header, and set the RTP Header **Marker Bit = 1** to signal the end of the video frame

24.2.1.6.3.2 NAL unit Framing and Ordering

24.2.1.6.3.2.1 SPS & PPS Parameter Sets Construction

MVC-117327 - For each encoded video frame in the H.264 bitstream, the software shall append t...

The software shall append the SEI message inside a SEI NAL unit after the SPS and PPS NAL units and before the video payload NAL units.

Status	Draft
Verification Criteria	Verify via Packet/Stream Analyzer that within the H.264 stream structure, NAL units follow the sequence:SPS NAL Unit PPS NAL Unit SEI NAL Unit (Metadata) Video Payload NAL Unit(Compressed Image).

SPS NAL unit	PPS NAL unit	SEI NAL unit	IDR Video NAL unit
Sequence Parameters	Picture Parameters	Tagged Metadata	Encoded & compressed Image Frame

24.2.1.6.4 SPS and PPS Payload Construction

MVC-117339 - For each received frame through , the software shall construct the Sequence Para...

The software shall construct the Sequence Parameter Set (SPS) NAL Unit with (0x07) according to ITU-T H.264 H.264-Advanced Video Coding to define the pre-configured stream parameters (e.g. Profile, Level, Frame Dimensions)

Status	Draft
Verification Criteria	Verify via Ethernet Packet Sniffer (Wireshark) on the outgoing video stream that:An SPS NAL unit (type 0x07) is present and correctly defines the pre-configured stream parameters (Profile, Level, Resolution).

MVC-117341 - For each received frame through , the software shall construct the Picture Param...

The software shall construct the Picture Parameter Set (PPS) NAL Unit with (0x08) according to ITU-T H.264 H.264-Advanced Video Coding to define the picture parameter

Status	Draft
Verification Criteria	Verify via Ethernet Packet Sniffer (Wireshark) on the outgoing video bitstream that:A Picture Parameter Set (PPS) NAL unit with NAL unit type 0x08 is generated for each frame.The PPS payload is formatted according to the ITU-T H.264 specification and contains valid picture parameters (e.g., entropy coding mode, picture order count settings).

24.2.1.6.5 SEI NAL Unit Framing & Header

MVC-117330 - The software shall define the SEI NAL unit Header as (0x06) to identify the NAL...

The software shall define the SEI NAL unit Header as (0x06) to identify the NAL Type as SEI- Supplemental Enhancement Information (SEI Type 6) according to ITU-T H.264

Status	Draft
Verification Criteria	Verify via packet sniffer (Wireshark) that the first byte of the SEI NAL unit payload in the outgoing video stream has the NAL unit type field =0x06 (identifying SEI NAL type).

MVC-117328 - The software shall construct the SEI NAL unit by sequentially concatenating 1-by...

The software shall construct the complete SEI NAL unit by prefixing the 1-byte SEI NAL unit header (0x06) to the received SEI Metadata payload within MVC-117136 - Tagged_H264_Video_Bitstream

Status	Draft
Verification Criteria	Verify via a packet sniffer (Wireshark) that The SEI NAL unit payload transmitted with the exact byte order: [1-byte NAL Header (type 0x06)] + [SEI Protocol Wrapper / Header] + [SEI Metadata Payload].

MVC-117345 - For each running stream identified by its specific SSRC field, the software shal...

For each running stream identified by its specific SSRC field, the software shall structure and transmit the RTP packets according to RFC 6184 Section 1.3 as the following sequential order:

- SPS NAL unit payload (unit Type 7)
- PPS NAL unit payload (unit Type 8)
- SEI NAL unit Payload (unit Type 6, containing the tagged SEI metadata received through MVC-117136 - Tagged_H264_Video_Bitstream)
- Instantaneous Decoding Refresh (IDR) Video NAL Unit (unit Type 5).

Status	Draft
Verification Criteria	Verified through Wireshark that transmitted RTP packets for ISP0 and ISP1 in the active video streaming, confirm that the frame follows the packet header sequence: SPS (Type 7) followed by PPS (Type 8) followed by SEI (Type 6) followed by IDR (Type 5) and also the SEI NAL unit payload successfully contains the expected injected stream tagging metadata structure.

main req for IDR NAL unit.

When the size of an IDR NAL unit exceeds the maximum configured RTP payload limit `Max_RTP_Payload_Size` (Default: 1460 bytes, derived from Ethernet MTU 1500 - IP Header 20 - UDP Header 8 - RTP Header 12), the Software shall fragment the IDR NAL unit into sequential FU-A packets according to RFC 6184 Section 5.8 as follows:

1. Each intermediate FU-A fragment payload shall consist of a 1-byte FU Indicator, a 1-byte FU Header, and a data chunk of size (`Max_RTP_Payload_Size - 2 bytes`).
2. The Software shall repeat the fragmentation process sequentially until the entire IDR NAL unit is transmitted, with the final fragment carrying the remaining data chunk (which may be smaller than `Max_RTP_Payload_Size - 2 bytes`)."

Req2:

FU-A Packet Construction: "For each fragmented IDR chunk, the software shall construct the RTP payload by prepending a 1-byte FU Indicator and a 1-byte FU Header to the chunk data in accordance with RFC 6184 Section 5.8."

Verification Criteria:

"Verify via Wireshark that every fragmented RTP payload starts with a valid 1-byte FU Indicator and 1-byte FU Header followed by the chunk data."

MVC-117346 - When the size of IDR NAL unit exceeds the max configured RTP payload limit (MAX_...

When the size of IDR NAL unit exceeds the max configured RTP payload limit (MAX_RTP_PAYLOAD_SIZE, Value: 1460 bytes) based on the Ethernet MTU , the Software shall fragment and packetize the IDR NAL unit into sequential FU-A packets (Chunk) according to RFC 6184 Section 5.8 as follows:

- Each intermediate FU-A fragment payload (chunk) shall equal (MAX_RTP_PAYLOAD_SIZE, Value: 1460 bytes) - 2 bytes.
- The final chunk size shall carry the remaining payload bytes with a max (MAX_RTP_PAYLOAD_SIZE, Value: 1460 bytes) - 2 bytes

Status	Draft
Verification Criteria	Verify through Wireshark that any IDR NAL unit exceeding 1460 bytes is split into chunks where intermediate chunks are exactly 1458 bytes and the final chunk carries the remaining balance without data loss

$$MaxRTPPayloadSize = EthernetMTU - IPheader - UDPheader - RTPHeader$$

Example;
Ethernet MTU = e.g. 1500 bytes
IP header = e.g. 20 bytes
UDP Header = e.g. 8 bytes
RTP Header = e.g. 12 bytes
SO Max RTP Payload Size = 1500 - 20 - 8 - 12 = 1460 bytes

MVC-117351 - For each FU-A fragment, the software shall construct the RTP payload by prependi...

For each FU-A fragment, the software shall construct the RTP payload by prepending a 1-byte FU indicator and 1-byte FU Header to the IDR chunk where the chunk size equals (Max RTP Payload Size - 2 bytes)

Status	Draft
Verification Criteria	Verify through wireshark that the first two bytes of each FU-A RTP payload consist of a 1-byte FU Indicator followed by a 1-byte FU Header and the IDR payload chunk size in each FU-A packet exactly equals (Max RTP Payload Size - 2 bytes).

$$RTPChunkSize = MaxRTPPayloadSize - 2bytes$$

MVC-117362 - The Software shall calculate the max RTP Payload Size based on the standard prot...

The Software shall calculate the max RTP Payload Size based on the standard protocol header sizes (Ethernet MTU, IP header, UDP header, RTP Header) as defined in RFC 6184, RFC 3550.

Status	Draft
Verification Criteria	Verify that the calculated Max RTP Payload Size calculated correctly using standard network configuration and the total protocol header overhead is correctly subtracted from the configured Ethernet MTU

-

??????????????????????

Delete it : a video frame for The Software shall provide the local source port either MVC-117284 - RTP_Src_Port1 or MVC-117288 - RTP_Src_Port2 through MVC-118848 - RTP_Src_Port and the calculated destination port corresponding to the target stream either MVC-117286 - RTP_Dest_Port1 or MVC-117290 - RTP_Dest_Port2 through MVC-118847 - RTP_Dest_Port

Req: When transmitting a constructed RTP packet, the software shall set `RTP_Src_Port` and `RTP_Dest_Port` according to the active video stream as follows:

- For Stream 0 (ISP0): `RTP_Src_Port1` and `RTP_Dest_Port1`.
- For Stream 1 (ISP1): `RTP_Src_Port2` and `RTP_Dest_Port2`."

"Verify via Ethernet network sniffer (e.g., Wireshark) that:

1. RTP packets transmitted for Stream 0 (ISP0) use `RTP_Src_Port1` as the source port and `RTP_Dest_Port1` as the destination port.
2. RTP packets transmitted for Stream 1 (ISP1) use `RTP_Src_Port2` as the source port and `RTP_Dest_Port2` as the destination port."

24.2.1.6.6 Non-Functional Requirements

24.2.1.6.7 Interfaces

24.2.1.6.7.1 Software Interfaces

24.2.1.6.7.1.1 Input Interfaces

Tagged_H264_Video_Bitstream

ISP_Index

ECU_SRC_IP_Address

24.2.1.6.7.1.2 Output Interfaces

RTP_Src_Port1

RTP_Src_Port2

RTP_Src_IP

Invalid_UDP_Source_IP_Address

RTP_Dest_Port1

RTP_Dest_IP1

Invalid_UDP_Dest_Port1

Invalid_UDP_Dest_IP1

RTP_Dest_Port2

RTP_Dest_IP2

Invalid_UDP_Dest_Port2

Invalid_UDP_Dest_IP2

RTP Packet

RTP_Src_Port

RTP_Dest_Port

24.2.1.6.7.2 Hardware Interfaces

24.2.1.6.7.2.1 Input Interfaces

24.2.1.6.7.2.2 Output Interfaces

24.2.1.6.7.2.3 Bidirectional Interfaces

24.2.1.7 Ethernet Some-IP Protocol Management Feature

24.2.1.8 Scope

24.2.1.9 Functional Requirements

MVC-112934 - The software shall provide the camera ID as 0x00 through

The software shall provide the camera ID as 0x00 through MVC-112758 - Cam_ID [Prototype Value = 0x00]

Status	Draft
Verification Criteria	

24.2.1.10 Non-Functional Requirements

24.2.1.11 Interfaces

24.2.1.11.1 Software Interfaces

24.2.1.11.1.1 Input Interfaces

24.2.1.11.1.2 Output Interfaces

Cam_ID [Prototype Value = 0x00]

SomeIP_Master_Target_Timing_Offset

24.2.1.11.2 Hardware Interfaces

24.2.1.11.2.1 Input Interfaces

24.2.1.11.2.2 Output Interfaces

24.2.1.11.2.3 Bidirectional Interfaces

25 Time Synchronization Feature

25.1 Scope

25.2 Functional Requirements

25.2.1 Network Clock Alignment (gPTP protocol)

25.2.1.1 gPTP Operation Role

MVC-113095 - The software shall operate in gPTP slave mode to communicate with the Master Veh...

The software shall operate in gPTP slave mode to communicate with the Master Vehicle ECU to receive time synchronization messages and synchronize its local system time base MVC-113128 - gPTP_Global_Time

Status	Ready for review
Verification Criteria	Verify through Wireshark that the SOC operates in slave mode by receiving the gPTP msgs (Sync, Followup and Pdelay_Resp) and the SOC respond with Pdelay_Req frames on the Ethernet bus.

MVC-113374 - The Software shall transmit and receive all the gPTP protocol frames including (...

The Software shall transmit and receive all the gPTP protocol frames including (Sync, Follow_Up, Pdelay_Req, Pdelay_Resp, Pdelay_Resp_Follow_Up) through MAC0 controller.

Status	Ready for review
Verification Criteria	Verify through the Analyzer that the frames (Sync, Follow_Up, Pdelay_Req, Pdelay_Resp, Pdelay_Resp_Follow_Up) transmits and receives through MAC0 controller to and from PHY according to gPTP standard operation

MVC-113097 - When the SOC MAC layer receives a gPTP event frame from the PHY through , the so...
When the SOC MAC layer receives a gPTP event frame from the PHY through MVC-113090 - ETH_RGMII_Data , the software shall read the Hardware ingress timestamp from the MAC Controller (MAC0) and provide it through MVC-113098 - gPTP_MAC0_Ingress_Timestamp_t2

Status	Ready for review
Verification Criteria	Verify through the Logic analyzer that the SOC receives the sync frame followed by Followup frame over the RGMII Hardware Pin from the PHYand the SOC capture the master timestamp from these frames

MVC-113115 - The software shall parse the received "Sync" and "Follow_Up" frames on MAC0 to e...
The software shall parse the received "Sync" and "Follow_Up" frames on MAC0 to extract the header parameters and Precise Origin Grand Master timestamp_t1, consume MVC-113098 - gPTP_MAC0_Ingress_Timestamp_t2 and provide these extracted data through MVC-113114 - gPTP_MD_Sync_Data

Status	Ready for review
Verification Criteria	Verify that "gPTP_MD_Sync_Data" accurately captures the correct ingress timestamps(T2) & header parameters without dropping or data corruption

25.2.1.2 Propagation Delay Calculation

Actual Current Master Time = t1+ calculated Path delay + correction field

MVC-113104 - The software shall transmit "PdelayReq" frame every (Pdelay_Req_Periodic_Tout, D...
The software shall transmit "PdelayReq" frame every (Pdelay_Req_Periodic_Tout, Default: 1sec) to the PHY over MVC-113090 - ETH_RGMII_Data to calculate the Cable Propagation Delay between the SOC and the Master Vehicle ECU and provide it through MVC-113101 - gPTP_Path_Delay

Status	Ready for review
Verification Criteria	Verify through wireshark that SOC periodically transmits valid PdelayReq frames over the Ethernet bus. Upon receiving the PdelayResp and PdelayRespFollowup from the master, the SOC calculate the propagation delay value.

$$gPTP_Path_Delay = \frac{(T4 - T1) - (T3 - T2)}{2}$$

Where;
T1: Timestamp when PdelayReq was transmitted by the SOC (MAC0)
T2: Timestamp when PdelayReq was received by the Master ECU
T3: Timestamp when PdelayResp was transmitted by the Master ECU
T4: Timestamp when PdelayResp was received by the SOC (MAC0)

25.2.1.3 MAC0 - Grand Master Clock Synchronization

MVC-116386 - The software shall read the grand master clock to calculate the time offset betw...
The software shall read the grand master clock to calculate the time offset between its Local Clock in MAC0 controller and Grand Master clock through MVC-113089 - ETH_PHYSMI_MDIO, using the extracted Grand master timestamp_t1 from MVC-113114 - gPTP_MD_Sync_Data, MVC-113098 - gPTP_MAC0_Ingress_Timestamp_t2, and MVC-113101 - gPTP_Path_Delay and provide it through MVC-117461 - MAC0_Grandmaster_Clock_Offset

Status	Ready for review
Verification Criteria	Verify via MDIO bus trace that the SOC send write operations to update the PHY LTC registers to sync the PHY clock with the Grand Master clock

Time offset between the Grand Master Clock and Local SOC Clock.

$$gPTPOffset = (T2 - T1) - gPTPPathDelay - PHYInternalDelays$$

MVC-113123 - When the software detect that the time offset is lower than or equal (MAC0_Maste...
When the software detect that the time offset MVC-117461 - MAC0_Grandmaster_Clock_Offset is lower than or equal (MAC0_Master_Time_Offset_Threshold, Default: 100 nsec), the software shall provide the external sync status between the MAC0 and Grand Master clock as "Synchronized" through MVC-117462 - External_Sync_Status

Status	Ready for review
Verification Criteria	Verify that External_Sync_Status evaluates to "synchronized" if Time_Offset<=100nsec

MVC-117436 - After the adjustment; If the calculated is higher than (MAC0_MASTER_CLK_OFFSET_M...
After the adjustment; If the calculated MVC-117461 - MAC0_Grandmaster_Clock_Offset is higher than (MAC0_MASTER_CLK_OFFSET_MAX_THRESHOLD, Default: 100 ns), for (MAC0_MASTER_CLK_RETRIALS_CYCLES, Default: 3 retrials consecutive cycles), the software shall provide its external sync status between the MAC0 and Master CLK as "Unsynchronized" through MVC-117462 - External_Sync_Status

Status	Ready for review
Verification Criteria	Verify that if Time_Offset>=100nsec, the SOC try to sync for a max 3 trials then if still failed to sync, the External_Sync_Status evaluates to "Unsynchronized"

25.2.1.4 MAC0 - PHY Clock Synchronization

Target here
To align the Internal Clock in the SOC (on which the MAC layer depends) and the external PHY chip has its own clock (called: LTC- Timestamp counter).
So we have to sync between these clock so we call it internal clock because it is clock synchronization on the PCB Board level (between MAC in SOC & PHY)

This internal Clock Synchronization is done by (PPS.- Pulse Per Second) which means the MAC generate a digital HW pulse on a physical pin every periodic time.

This internal synchronization is done through hardware pin connected between the MAC & PHY (ENET0_PTP_PPS),

When the MAC generate this pulse on the pin, the MAC hard-lock the PHY and align the internal timers between the MAC & PHY

MVC-113373 - The software shall enable the phase lock functionality of PHY to MAC0 synchroniz...
The software shall enable the phase lock functionality of PHY to MAC0 synchronize and align the internal clock between the Ethernet MAC0 controller and Ethernet PHY layer.

Status	Ready for review
Verification Criteria	Verify using a Logic Analyzer or Oscilloscope connected to the ETE0_PTP_PPS pin that the software successfully toggles the PPS signal to lock the PHY and sync the internal timing of the Ethernet MAC0 controller with the Ethernet PHY layer

MVC-113449 - The software shall generate one digital HW pulse every 25 msec through MAC0 over...
The software shall generate one digital HW pulse every 25 msec through MAC0 over MVC-113092 - ETE0_PTP_PPS according to the HSI MVC-108355 - gPTP alternative function to maintain internal clock synchronization between the Ethernet MAC0 controller and Ethernet PHY counter (LTC).

Status	Ready for review
Verification Criteria	Verify via Logic Analyzer or Oscilloscope connected to the ETE0_PTP_PPS pin that the generated digital HW pulse occurs at a fixed period of 25 msec (± specified tolerance) and that the PHY Local Time Counter (LTC) captures its local timestamp synchronously on the rising edge of the pulse.

MVC-113455 - Following the rising edge of the PPS, the software shall read the captured MAC0...
Following the rising edge of the PPS, the software shall read the captured MAC0 local timestamp from the MAC0 controller and read the the PHY Local timestamp (LTC) through MVC-113089 - ETH_PHYSMI_MDIO, to calculate the time difference and provide it through MVC-113135 - MAC0_PHY_Clock_Offset

Status	Ready for review
Verification Criteria	Verify impilicity with req ID MVC-113456 and MVC-113457 that following the PPS rising edge, the software read the PHY local time stamp over MDIO and calculate the PHY MAC0 time difference , and updates "MAC0_PHY_Clock_Offset "

MVC-113456 - If the calculated is lower than or equal MAC0_PHY_OFFSET_MAX_THRESHOLD (Default:...
If the calculated MVC-113135 - MAC0_PHY_Clock_Offset is lower than or equal MAC0_PHY_OFFSET_MAX_THRESHOLD (Default: 200 ns), the software shall provide the internal sync status between the MAC0 and PHY as "Synchronized" through MVC-113452 - Internal_Sync_Status

Status	Ready for review
Verification Criteria	Verify that internal_Sync_Status evaluates to "synchronized" if Time_Offset<= 200nsec and no any change in the time base happen

MVC-113457 - If the calculated is higher than MAC0_PHY_OFFSET_MAX_THRESHOLD (Default: 200 ns...

If the calculated MVC-113135 - MAC0_PHY_Clock_Offset is higher than MAC0_PHY_OFFSET_MAX_THRESHOLD (Default: 200 ns), the software shall adjust the PHY LTC through MVC-113089 - ETH_PHYSMI_MDIO using MVC-113135 - MAC0_PHY_Clock_Offset to synchronize the PHY LTC with the MAC0 controller.

Status	Ready for review
Verification Criteria	Verify that internal_Sync_Status evaluates to "unsynchronized" if Time_Offset> 200nsec and the MAC0 adjust the PHY LTC through MDIO pin

MVC-113458 - After the adjustment; If the calculated is higher than (MAC0_PHY_OFFSET_MAX_THRE...

After the adjustment; If the calculated MVC-113135 - MAC0_PHY_Clock_Offset is higher than (MAC0_PHY_OFFSET_MAX_THRESHOLD, Default: 200 ns), for (MAC0_PHY_RETRIALS_CYCLES, Default: 3 retrials consecutive cycles), the software shall provide its internal sync status between the MAC0 and PHY as "Unsynchronized" through MVC-113452 - Internal_Sync_Status

Status	Ready for review
Verification Criteria	Verify that when Clock offset between MAC0 and PHY exceeds max threshold for 3 consecutive 25 msec PPS cycles adjustment attempts, Internal Sync Status transitions to "Unsynchronized"

25.2.1.5 gPTP Global Time Base Synchronization

MVC-116425 - When the software identifies and ,

When the software identifies MVC-113452 - Internal_Sync_Status and MVC-117462 - External_Sync_Status ,
as "Synchronized", the software shall calculate and provide the unified synchronized time base through MVC-113128 - gPTP_Global_Time using the Precise Origin Grand Master timestamp_t1 extracted from MVC-113114 - gPTP_MD_Sync_Data c orrected by the calculated MVC-113101 - gPTP_Path_Delay and the calculated clock offsets.

Status	Ready for review
Verification Criteria	Verify that when Internal_Sync_Status and external_sync_status are synched, the gptp_Global_Time holds the unique system clock

25.2.2 Frame and Camera Capture Synchronization - Fsync

GVTtime – Global Vechile Time : It is the global time in the whole vehicle, it is received through Ethernet by gPTP

Clock Adjustment

MVC-116451 - The software shall adjust the MAC1 local timer using the to synchronize and alig...

The software shall adjust the MAC1 local timer using the MVC-113135 - MAC0_PHY_Clock_Offset to synchronize and align the MAC1 local clock with the MVC-113128 - gPTP_Global_Time if the delta between the MVC-113128 - gPTP_Global_Time and MAC1 local clock is higher than (MAC1_Clock_Offset_Threshold, Default: 1usec)

Status	Draft
Verification Criteria	Verify that the once the MAC 0 provide the Clock offset to MAC1 , MAC1 matches its clock with gPTP_Global_Time

25.2.2.1 FSync Setup

MVC-113462 - The software shall generate the HW digital pulse to the SOW1 over according to t...

The software shall generate the HW digital pulse to the SOW1 over MVC-113091 - ETH_FSYNC_PULSE according to the HSI MVC-108360 - FSYNC trigger signal through the MAC1 PPS block with a cyclic period (FSYNC_Cyclic_Periodic, Default: 33.33 msec) based on the configured Frame rate 30 FPS).

Status	Draft
Verification Criteria	Verify using an oscilloscope or logic analyzer connected to the ETH_FSYNC_PULSE HW pin that the software continuously generates a cyclic hardware digital pulse with 33.33 ms periodicity.

MVC-116313 - The Software shall configure the camera module (SOW1) to initiate transmitting t...

The Software shall configure the camera module (SOW1) to initiate transmitting the first frame of the sequence
(FSYNC_Target_Frame_Rate, Default: 30 fps) when receiving the rising edge of the first pulse over MVC-113091 - ETH_FSYNC_PULSE

Status	Draft
Verification Criteria	Verify using a logic analyzer that upon the generation of the first rising edge on ETH_FSYNC_PULSE, the software commands the SOW1 camera module to immediately begin transmitting the first video frame."

MVC-117982 - The software shall compensate the POC cable delay while generating the Sync puls...

The software shall compensate the POC cable delay while generating the Sync pulse through to synchronize MAC1 with SOW1.

Status	Draft
Verification Criteria	

25.2.2.2 FSync Base Rasterization and Clock Alignment

means there are 1 line and video frames Lines : One line for pulse (Output from the SOC on which the SOC generate the pulses every 33.33msec)
And the SOW1 send the video frames to the SOC (they are input pins to SOW2),
Here the soc need to align the pulse with 30 video frames per sec to be as follow
For each pulse on the Pulse line, the SOW1 provide 1 frame to the SOW2 on the data lines for all 30 frames

MVC-113467 - The software shall set the default rasterization reference point for pulse gener...

The software shall set the default rasterization reference point for pulse generation over MVC-113091 - ETH_FSYNC_PULSE to the 1-second boundary of the Global Vehicle (Grand Master) time MVC-113128 - gPTP_Global_Time

Status	Draft
Verification Criteria	Verify using a logic analyzer/oscilloscope that the first pulse edge generated over ETH_FSYNC_PULSE aligns precisely with the integer 1-second boundary rollover of gPTP_Global_Time (GVTtime).

25.2.2.3 Dynamic FSync Phase Adjustment

MVC-116454 - the software shall calculate the required MAC1 phase drift adjustment relative t...

the software shall calculate the required MAC1 phase drift adjustment relative to the FSYNC_Cyclic_Periodic (Default: 33.33 msec based on the configured Frame rate 30 FPS) frame boundary of MVC-113128 - gPTP_Global_Time and reconfigure the rasterization period on the MVC-113091 - ETH_FSYNC_PULSE to apply the target offset to compensate for cable delays and phase drift without interrupting active camera streaming, if all the following conditions met:

- MVC-117462 - External_Sync_Status as "Synchronized".
  AND
- Receive the Phase alignment command with frame target timing reference timestamp from Master Vehicle ECU through MVC-116453 - SomeIP_Master_Target_Timing_Offset over SOME/IP.

Status	Draft
Verification Criteria	Verify that when SOMEIP_Target_Timing_Offset is updated, the software correctly calculates the dynamic phase drift relative to the 33.33 ms cyclic frame boundary. Verify using a logic analyzer/oscilloscope on the ETH_FSYNC_PULSE hardware pin that the software updates the rasterization period dynamically based on the calculated offset and cable delays, maintaining continuous streaming without frame drops."

MVC-116310 - If the software failed to synchronized with the Master vehicle ECU through as "U...

If the software failed to synchronized with the Master vehicle ECU through MVC-117462 - External_Sync_Status as "Unsynchronized", the software shall maintain MVC-113091 - ETH_FSYNC_PULSE generation using the last valid FSYNC_Cyclic_Periodic (Default: 33.33 msec based on the configured Frame rate 30 FPS) and shall hold any newly received frameSyncTimeRef updates without applying them until gPTP synchronozation is re-established.

Status	Draft
Verification Criteria	1. Verify via software trace logs that when gPTP_Site_Sync_Status transitions to 'Unsynchronized', the software continues generating ETH_FSYNC_PULSE at 30 fps using the last valid frameSyncTimeRef 2. Verify that once gPTP_Site_Sync_Status returns to 'Synchronized', the software automatically resumes applying the updated frameSyncTimeRef values to align the FSYNC pulse.

25.2.2.4 FSync Verification and Fallback Management

MVC-116287 - If the software synchronized with the Master vehicle ECU through as "Synchronize...

If the software synchronized with the Master vehicle ECU through MVC-117462 - External_Sync_Status as "Synchronized", The software shall verify that the time offset between the frame interval time raster and the MVC-117906 - Frame_Arrival_Timestamp at which receiving a SOW1 frame aligned to the MVC-116453 - SomeIP_Master_Target_Timing_Offset with a max tolerance (FSYNC_CAPTURE_TIMESTAMP_TOLERANCE, default: 80 usec)

Status	Draft
Verification Criteria	Verify via software execution traces that when gPTP_Site_Sync_Status is 'Synchronized' and the received Frame_Capture_Timestamp aligns with gPTP_Global_Time within+/- 40ms), the frame is accepted without setting synchronization errors.

25.2.2.5 FSync Variant Handling - Analog Camera Variant

MVC-116314 - In case of "Analog Camera Variant; the software shall disable MAC1 PPS block to...

In case of "Analog Camera Variant; the software shall disable MAC1 PPS block to disable generation of MVC-113091 - ETH_FSYNC_PULSE signal.

Status	Draft
Verification Criteria	Verify using a logic analyzer or oscilloscope on the ETH_FSYNC_PULSE pin that no pulse signals are generated or emitted during system operation(in case of analog variant)

25.3 Non-Functional Requirements

25.4 Interfaces

25.4.1.1 Software Interfaces

25.4.1.1.1 Input Interfaces

SomeIP_Master_Target_Timing_Offset

Frame_Camera_Timestamp

Frame_Arrival_Timestamp

25.4.1.1.2 Output Interfaces

gPTP_MAC0_Ingress_Timestamp_t2

gPTP_Path_Delay

gPTP_Global_Time

gPTP_MD_Sync_Data

MAC0_PHY_Clock_Offset

MAC0_Grandmaster_Clock_Offset

Internal_Sync_Status

External_Sync_Status

gPTP_Port_Sync_Capable

gPTP_Port_Sync_Status

gPTP_Port_Sync_Data

gPTP_Site_Sync_Status

25.4.1.2 Hardware Interfaces

25.4.1.2.1 Input Interfaces

25.4.1.2.2 Output Interfaces

ETE0_PTP_PPS

ETH_FSYNC_PULSE

25.4.1.2.2.1 Bidirectional Interfaces

ETH_RGMII_Data

ETH_PHYSMI_MDIO

26 Video and Image Processing Function

26.1 Video Stream Tagging Feature

26.1.1 Scope

H.264 Video Compression and Encoding Standard also called (AVS-Advanced Video Coding)

Received Video from the camera Imager is -> Raw video , Uncompressed
To send this video through the Ethernet Cable through the Vechile , Its size is very large may be (in Giga bits per second.)

SOC has internal module H.264 Encoder (Hardware Processor) -> Its function to compress these raw frames to decrease its size before it send, So the compressed video streams called "Bitstream" and written in H.264 format.

The H.264 Bitstream consists of units called (Network Abstraction Layer Units – NAL units), some of these units contain the compressed image and another units contain the encoder itself and another unit called SEI (Supplemental Enhancement Information) where we write additional information such as Frame ID, timestamp,….

Sequence:
In the Application Layer:

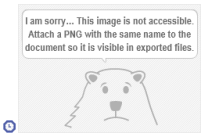
1. Camera Sensor(SOW1) -> provide the Raw video
2. The H.264 Encoder component inside the SOC ->
   a. Compress the raw video to "**H.264 Bitstream**".
   b. Create the Metadata like (GVTtime, Frame id…) in the format of H.264, to be called "**SEI Msg**"
   c. Inject these **"SEI msg"** inside the "**SEI NAL"** unit inside the Compressed H.264 Bitstream.
   d. So, we have the H.264 compressed Bitstream included the embedded metadata in NAL then packetize this compressed Bitstream with the meta data using the transport protocol (RTP) to form finally (Reat time Transport Packets).
3. The CV (Computer Vision) and HV (Hardware or Human Vision) applications
   a. Computer Vision- CV: means the software applications like
   (Advanced Driver Assistance Systems- ADAS) and Perception stack such as; Lane Detection , Pedestrian Detection, Emergency Braking
      i. They receive this compressed Bitstream , parse it , to abstract the metadata like Frame Id and timestamp.
   b. Human Vision-HV: such as the Central Display, Digital Rear View Mirrors, Video Recorders / DVR
      i. Dynamic Range parameters, Color Balance tags, Rendering Overlays -They receive this compressed Bitstream , parse it , to abstract the metadata like

In the Transport Layer:
This Bitstream go down to the Transport Layer where the **Real time transport protocol (RTP)**, the RTP packetize the SEI NAL unit into to be **"RTP Payload"** and add Header and footer to format finally the "**RTP packets** (RTP Header + RTP Payload[SEI NAL packets] + RTP Footer)"
Then to the UDP/IP Layer then physical layer (Ethernet).

26.1.2 Functional Requirements

26.1.2.1 SEI NAL Unit structure in H.264 Stream

MVC-112661 - For each encoded video frame, the software shall append the tagged metadata insi...

For each encoded video frame, the software shall append the tagged metadata inside a SEI NAL unit according to H.264 Specification 

T-REC-H.264-202408-I__PDF-E.pdf into the video bitstream.

Status	Draft
Verification Criteria	Verify though Wireshark on the RTP stream that for every encoded video frame: An H.264 SEI NAL unit containing the valid metadata is present and The SEI NAL unit is appended

For each received encoded

Add the tag metadata in SEI NAL unit in the frame then the tagged frame transferred to the RTP for packetization and transmission.

MVC-112686 - For each encoded video frame received through , the software shall append the ta...

For each encoded video frame received through MVC-112785 - Encoded Video Bitstream, the software shall append the tagged metadata inside a SEI NAL unit before the IDR video payload NAL units.

Status	Draft
Verification Criteria	Verify via Packet/Stream Analyzer that within the H.264 stream structure, NAL units follow the sequence:SPS NAL Unit PPS NAL Unit SEI NAL Unit (Metadata) Video Payload NAL Unit(Compressed Image).

SEI NAL unit	Video NAL units
Tagged Metadata	Encoded & compressed Image Frame

MVC-115083 - The software shall construct the SEI NAL unit by sequentially concatenating 1-by...

The software shall construct the payload in SEI NAL unit by sequentially concatenating the SEI protocol wrapper and the SEI Metadata payload.

Status	Ready for review
Verification Criteria	Verify via Wireshark that The SEI NAL unit payload transmitted with the exact order: [SEI Protocol Wrapper / Header] + [SEI Metadata Payload].

26.1.2.1.1 SEI NAL Unit Wrapper

MVC-118307 - The Software shall define Payload type as (SEI_PAYLOAD_TYPE, Default: 0x05) to i...

The Software shall define Payload type as (SEI_PAYLOAD_TYPE, Default: 0x05) to identify the payload as User Data Unregistered in the SEI NAL unit wrapper according to ISO/IEC 11578:1996 Annex A

Status	Ready for review
Verification Criteria	Verify through Wireshark that the payload type byte is set to 0x05 inside the NAL Header.

MVC-118310 - The Software shall define the payload size as (SEI_PAYLOAD_SIZE, Default: 74 byt...

The Software shall define the payload size as (SEI_PAYLOAD_SIZE, Default: 74 bytes) to represent the combined size of UUID (UUID_SIZE, Value: 16 bytes) and the Metadata Payload (METADATA_SIZE, Value: 58 bytes) in the SEI NAL unit wrapper according to ISO/IEC 11578:1996 Annex A

Status	Ready for review
Verification Criteria	Verify through Wireshark that the payload size field 74 bytes and it is next to payload type byte inside the NAL Header.

MVC-118309 - The Software shall define 16-bytes reserved for the UUID as the of system unique...

The Software shall define 16-bytes reserved for the UUID as the of system unique identifier followed immediately by the metadata payload according to ISO/IEC 11578:1996 Annex A.

Status	Ready for review
Verification Criteria	Verify through Wireshark that a 16-bytes reserved for UUID which is next to Payload size and followed by the metadata directly.

MVC-118263 - The Software shall construct the UUID of system unique Identifier in the SEI NAL...

The Software shall construct the UUID of system unique Identifier in the SEI NAL unit Wrapper as (SYS_UUID, Default: 0x534F5732-D911-8001-A101-7F2A67C8C580)

Status	Draft
Verification Criteria	Verify via wireshark that in the NAL Header: the UUID is added following the Payload Size Field, and exactly matches the value 0x534F5732-D911-8001-A101-7F2A67C8C580.

26.1.2.1.2 SEI NAL Unit Trailing Byte

MVC-112885 - The software shall append a 1-byte trailing field as (SEI_TRAILING_BYTE, Value:...

The software shall append a 1-byte trailing field as (SEI_TRAILING_BYTE, Value: 0x80) directly after the Metadata payload at the end of frame.

Status	Draft
Verification Criteria	Verify via stream analyzer that: a trailing byte 0x80 (0b 1000 0000) is appended after the final byte of the App Metadata payload and The total length of the constructed SEI NAL unit frame maintains proper RBSP byte alignment.

Add the 1-byte trailer byte to achieve the RBSP byte alignment at the end of the SEI NAL unit frame.

26.1.2.1.3 NAL Emulation Prevention

MVC-112689 - While the software constructs the SEI NAL unit, If the software detects two cons...

While the software constructs the SEI NAL unit, If the software detects two consecutive Zero bytes (0x00 0x00) followed by a byte 0x00 or 0x01, or 0x02 or 0x03, the software shall insert an emulation Prevention Code (EMULATION_PREVENT_CODE, Default: 0x03) after the second Zero byte according to the standard ITU_T H.264 standard.

Status	Ready for review
Verification Criteria	Inject a raw payload containing test byte patterns such as (0x00 0x00 0x03) into the NAL unit constructor and verify via Ethernet Packet Sniffer that: An emulation prevention byte 0x03 is correctly inserted after the second zero byte (resulting in 0x00 0x00 0x03 0x0x). Standard H.264 video decoders successfully parse the generated NAL units without start code emulation errors.

The receiving master ECU (Decoder side) is expected to remove the byte of emulation Prevention Code as 0x03 while parsing the received stream over the Ethernet to reconstruct the original payload data.

26.1.2.1.4 SEI Payload Structure and Placement

MVC-112886 - For each received frame through , the software shall construct the SEI metadata...

For each received frame through MVC-112785 - Encoded Video Bitstream, the software shall construct the SEI metadata payload as (METADATA_SIZE, Value: 58 bytes), according to the following memory layout; append it in the SEI NAL unit, merge it with the encoded video frame NAL units to format and provide the formatted SEI payload data to the downstream RTP layer through MVC-112783 - Tagged_H264_Video_Bitstream

Byte 0	Byte 1	Byte 2	Byte 3
CaptureTimestamp [0-3]			
CaptureTimestamp [4-7]			
FrameID			
Frame exposure time LPD		Frame gain Long LCG [0-1]	
Frame gain Long LCG [2-3]		Scene illuminance [0-1]	
Scene illuminance [2-5]			
Scene illuminance [6-7]		Error bitmask	CamID
CRC-16		Frame exposure time SPD	
Frame gain Long HCG		Frame gain Middle	
Frame gain Short		PreviewDataReadyTimestamp [0-1]	
PreviewDataReadyTimestamp [2-5]			

Byte 0	Byte 1	Byte 2	Byte 3
PreviewDataReadyTimestamp [6-7]		EncodeReadyTimestamp [0-1]	
EncodeReadyTimestamp [2-5]			
EncodeReadyTimestamp [6-7]		Encoder Frame Number [0-1]	
Encoder Frame Number [2-3]			

Status	Ready for review
Verification Criteria	Verify via logic analyzer that the Tagged_H264_Video_Bitstream output for every incoming frame: An SEI NAL unit is generated and merged with the frame's encoded NAL units, and The SEI metadata payload within the stream matches the exact 40-byte memory structure (verifying key offsets such as CaptureTimestamp at bytes 0–7, FrameID at bytes 8–11, and Padding at bytes 38–39). Data fields correctly mirror real-time metadata values extracted during frame capture and encoding.

26.1.2.1.4.1 Digital Camera Variant

MVC-112889 - In case of "Digital Camera Variant"; For each received frame through ; if the in...

In case of "Digital Camera Variant"; For each received frame through MVC-112785 - Encoded Video Bitstream; if the MVC-112786 - ROI_Index indicates "CV_index" or "HV_index", The software shall append the Computer Vision-CV and Human Vision-HV metadata SEI Payload inside the SEI NAL unit according to the defined memory layout as follows:

Metadata parameter	Description
MVC-112788 - Frame_Arrival_Timestamp	Timestamp when a full frame is received from the sensor and stored in raw buffer
MVC-112776 - Frame_ID	ID of frame received from the camera sensor
MVC-112775 - Frame_Exposure time LPD	Total integration time of LPD pixel
MVC-112778 - Frame_Gain_Long_LCG	Digital gain of LPD LCG frame (integer 8 bit / fraction 8 bit)
MVC-112777 - Scene_illuminance	For prototype the used value of scene brightness as follow scene_brightness_estimate = Luminance (averaged to all pixels) / (Digital gain of LPD LCG frame * total integration time of LPD pixel)
MVC-112791 - Diag_Video_Stream_Error_Status [Prototype value = 0x00]	The Camera Error Status set as 0x00 for the prototype
MVC-112790 - Cam_ID [Prototype Value = 0x00]	Camera ID set as 0x00 for the prototype
MVC-112789 - Video_Stream_CRC [Prototype Value = 0x0000]	CRC set as 0x00 for the prototype

Status	Ready for review
Verification Criteria	Verify via stream analyzer during execution in "Digital Camera Variant" that: When ROI_Index = "CV_index" or "HV_index", the SEI NAL unit payload is generated with the 8 mapped signals in exact sequence. ETH_Sync_Captured_Event_Timestamp through Scene_illuminance dynamically reflect live sensor inputs. Diag_Video_Stream_Error_Status, Cam_ID, and Video_Stream_CRC contain 0x00, 0x00, and 0x0000 respectively. When ROI_Index is set to any value other than "CV_index" or "HV_index", the CV/HV SEI metadata payload is not appended.

26.1.2.1.4.2 Analog Camera Variant

MVC-112890 - In case of "Analog Camera Variant"; For each received frame through ; if the ind...

In case of "Analog Camera Variant"; For each received frame through MVC-112785 - Encoded Video Bitstream; if the MVC-112786 - ROI_Index indicates "Analog_index", The software shall append the analog metadata SEI payload inside the SEI NAL unit according to the defined memory layout as follows:

Metadata parameter	Description
MVC-112788 - Frame_Arrival_Timestamp	Timestamp when a full frame is received from the sensor and stored in raw buffer
MVC-112776 - Frame_ID	ID of frame received from the camera sensor
MVC-112791 - Diag_Video_Stream_Error_Status [Prototype value = 0x00]	The Camera Error Status set as 0x00 for the prototype
MVC-112790 - Cam_ID [Prototype Value = 0x00]	Camera ID set as 0x00 for the prototype

Status	Draft
Verification Criteria	Verify via stream analyzer during execution in "Analog Camera Variant" that: When ROI_Index = "Analog_index", the analog metadata SEI payload is generated with items 1 up to 4 in exact sequence. Cam_ID and Diag_Video_Stream_Error_Status = 0x00 during prototype operation. ETH_Sync_Captured_Event_Timestamp and Frame_ID match the incoming frame capture timestamp and software frame counter. When ROI_Index is != "Analog_index", the analog SEI metadata payload is not appended.

26.1.2.1.4.3 Common Variants

MVC-112892 - For each received frame through ; The software shall append the debugging and te...

For each received frame through MVC-112785 - Encoded Video Bitstream; The software shall append the debugging and testing metadata SEI payload inside the SEI NAL unit according to the defined memory layout as follows:

Metadata parameter	Description
MVC-112775 - Frame_Exposure time LPD	Total integration time of LPD pixel
MVC-112778 - Frame_Gain_Long_LCG	Digital gain of LPD LCG frame (integer 8 bit / fraction 8 bit)
MVC-112781 - Frame_Gain_Middle	Digital gain of SPD-LOF frame (integer 8 bit / fraction 8 bit)
MVC-112774 - Frame_Gain_Short	Digital gain of SPD-CDS frame (integer 8 bit / fraction 8 bit)
MVC-112777 - Scene_illuminance	For prototype the used value of scene brightness as follow scene_brightness_estimate = Luminance (averaged to all pixels) / (Digital gain of LPD LCG frame * total integration time of LPD pixel)
MVC-112780 - Frame_Preview_Data_Ready_Timestamp [Prototype Value = 0x00]	Timestamp(ns) when a full frame is processed from the ISP
MVC-112784 - Encoded_Frame_Ready_Timestamp	Timestamp when a full frame is encoded (ns)
MVC-112787 - Encoded_Frame_Total_No	Frame number of an encoder stream

Status	Ready for review
Verification Criteria	Verify via stream analyzer on the output video stream that for every incoming frame: The debugging and testing metadata SEI payload is appended in the exact sequence of items 1 up to 7. Frame_Preview_Data_Ready_Timestamp = 0x00 during prototype operation. Encoded_Frame_Ready_Timestamp and Encoded_Frame_Total_No correctly map to real-time encoding timestamps and incrementing frame count values.

MVC-112893 - For each received frame through , the software shall set all the unused reserved...

For each received frame through MVC-112785 - Encoded Video Bitstream, the software shall set all the unused reserved fields inside the SEI message payload to 0x00.

Status	Ready for review
Verification Criteria	Verify via stream Analyzer on the generated video stream that: All reserved or unused bytes/bits within the SEI message payload are set to 0x00 for every constructed frame.

26.1.3 Non-Functional Requirements

26.1.4 Interfaces

26.1.4.1 Software Interfaces

26.1.4.1.1 Input Interfaces

Encoded Video Bitstream

ROI_Index

Frame_Arrival_Timestamp

Diag_Video_Stream_Error_Status [Prototype value = 0x00]

Cam_ID [Prototype Value = 0x00]

Encoded_Frame_Ready_Timestamp

Encoded_Frame_Total_No

Video_Stream_CRC [Prototype Value = 0x0000]

Frame_ID

Frame_Exposure time LPD

Frame_Gain_Long_LCG

Scene_illuminance

Frame_Preview_Data_Ready_Timestamp [Prototype Value = 0x00]

Frame_Exposure time_SPD

Frame_Gain_Long_HCG

Frame_Gain_Middle

Frame_Gain_Short

26.1.4.1.2 Output Interfaces

Tagged_H264_Video_Bitstream

26.1.4.2 Hardware Interfaces

26.1.4.2.1 Input Interfaces

26.1.4.2.2 Output Interfaces

26.2 Video Stream Management Feature

26.2.1 Scope

26.2.2 Functional Requirements

MVC-113075 - When the software successfully finished encoding the received video frame based...

When the software successfully finished encoding the received video frame based on H.264 , the software shall read the current timestamp through MVC-115067 - gPTP_Global_Time to provide the time at which a frame is finished encoding through MVC-112702 - Encoded_Frame_Ready_Timestamp

Status	Draft
Verification Criteria	

MVC-113074 - When the software successfully finished encoding the received video frame based...

When the software successfully finished encoding the received video frame based on H.264 , the software shall provide the total number of the encoded frames incremently through MVC-112701 - Encoded_Frame_Total_No

Status	Draft
Verification Criteria	

MVC-113077 - The software shall provide the ROI Index of the received frame as "HV or CV or A...

The software shall provide the ROI Index of the received frame as "HV or CV or Analog" through MVC-112718 - ROI_Index

Status	Draft
Verification Criteria	

MVC-113076 - For each encoding H.264 frame, The software shall provide the 16-bit CRC as 0x00...

For each encoding H.264 frame, The software shall provide the 16-bit CRC as 0x00 through MVC-112717 - Video_Stream_CRC [Protoype Value = 0x0000]

Status	Draft
Verification Criteria	

26.2.3 Non-Functional Requirements

26.2.4 Interfaces

26.2.4.1 Software Interfaces

26.2.4.1.1 Input Interfaces

Uncompressed_YUV_Frame

gPTP_Global_Time

26.2.4.1.2 Output Interfaces

Encoded Video Bitstream

ROI_Index

ISP_Index

Encoded_Frame_Ready_Timestamp

Encoded_Frame_Total_No

Video_Stream_CRC [Protoype Value = 0x0000]

26.2.4.2 Hardware Interfaces

26.2.4.2.1 Input Interfaces

26.2.4.2.2 Output Interfaces

26.2.5 Configuration Parameters

26.3 Image Processing Feature

26.3.1 Scope

26.3.2 Functional Requirements

26.3.2.1 Analog Camera Variant

MVC-112924 - In case of "Analog Camera Variant"; For each Raw frame received from the camera...

In case of "Analog Camera Variant"; For each Raw frame received from the camera sensor, The ISP-Image Signal Processor shall count the number of received frames sequentially and provide the frame ID through MVC-112724 - Frame_ID.

Status	Draft
Verification Criteria	

26.3.2.2 Digital Camera Variant

MVC-112926 - In case of "Digital Camera Variant"; For each Raw frame received from the camera...

In case of "Digital Camera Variant"; For each Raw frame received from the camera sensor, The ISP-Image Signal Processor shall parse the camera sensor embedded data to extract the Frame ID, Total integration time of LPD pixel and Digital gain of LPD LCG frame and provide them through MVC-112724 - Frame_ID , MVC-112723 - Frame_Exposure time LPD and MVC-112726 - Frame_Gain_Long_LCG

Status	Draft
Verification Criteria	

MVC-116308 - In case of "Digital Camera Variant"; For each Raw frame received from the camera...

In case of "Digital Camera Variant"; For each Raw frame received from the camera sensor through **(interface:** Raw_Camera_Frame), The ISP-Image Signal Processor shall parse the camera sensor embedded data to extract the total integration time of SPD pixel, digital gain of LPD HCG frame, digital gain of SPD-LOF frame, digital gain of SPD-CDS and provide them through MVC-112727 - Frame_Exposure time_SPD , MVC-112723 - Frame_Exposure time LPD

Status	Draft
Verification Criteria	

MVC-112928 - In case of "Digital Camera Variant"; For each Raw frame received from the camera...

In case of "Digital Camera Variant"; For each Raw frame received from the camera sensor through **(interface:** Raw_Camera_Frame**),** The ISP-Image Signal Processor shall calculate the scene illuminance based on the Color filter array statistics (CFA statistics), MVC-112911 - Camera Sensor Digital Gain and MVC-112910 - Camera Sensor Integration Time and provide it through MVC-112725 - Scene_illuminance

Status	Draft
Verification Criteria	

$$SceneIlluminance = \frac{CFAStatistics}{DigitalGain * IntegrationTime}$$

MVC-116307 - For each Raw frame received from the camera module through , the ISP-image signa...

For each Raw frame received from the camera module through MVC-115069 - Raw_Camera_Frame , the ISP-image signal processing shall parse the frame to extract the embedded capture timestamp and provide it through MVC-113614 - Frame_Camera_Timestamp

Status	Draft
Verification Criteria	

MVC-112930 - When the ISP processed a full image frame successfully, the software shall read...

When the ISP processed a full image frame successfully, the software shall read the current timestamp through MVC-115070 - gPTP_Global_Time to provide the time at which a frame is ready through MVC-112728 - Frame_Preview_Data_Ready_Timestamp.

Status	Draft
Verification Criteria	

MVC-117448 - When the ISP receives a full image frame successfully, the software shall provid...

When the ISP receives a full image frame successfully, the software shall provide the time at which a valid frame received from the Camera sensor based on MVC-115070 - gPTP_Global_Time to provide the time MVC-117447 - Frame_Arrival_Timestamp

Status	Draft
Verification Criteria	

26.3.3 Non-Functional Requirements

26.3.4 Interfaces

26.3.4.1 Software Interfaces

26.3.4.1.1 Input Interfaces

Raw_Camera_Frame

gPTP_Global_Time

Camera Sensor Digital Gain

Camera Sensor Integration Time

26.3.4.1.2 Output Interfaces

Uncompressed_YUV_Frame

Frame_ID

Frame_Exposure time LPD

Frame_Gain_Long_LCG

Scene_illuminance

Frame_Preview_Data_Ready_Timestamp

Frame_Arrival_Timestamp

Frame_Exposure time_SPD

Frame_Gain_Long_HCG

Frame_Gain_Middle

Frame_Gain_Short

Frame_Camera_Timestamp

26.3.4.2 Hardware Interfaces

26.3.4.2.1 Input Interfaces

26.3.4.2.2 Output Interfaces

26.3.5 Configuration Parameters

26.4 Camera Calibration Management Feature

26.4.1 Scope

26.4.2 Functional Requirements

26.4.3 Non-Functional Requirements

26.4.4 Interfaces

26.4.4.1 Software Interfaces

26.4.4.1.1 Input Interfaces

26.4.4.1.2 Output Interfaces

Camera Sensor Digital Gain

Camera Sensor Integration Time

26.4.4.2 Hardware Interfaces

26.4.4.2.1 Input Interfaces

26.4.4.2.2 Output Interfaces

26.4.5 Configuration Parameters

27 System Diagnostic Feature

27.1 Scope

27.2 Functional Requirements

27.3 Non-Functional Requirements

27.4 Interfaces

27.4.1 Software Interfaces

27.4.1.1 Input Interfaces

ETH_Sync_MAC0_PHY_Int_Sync_Status

27.4.1.2 Output Interfaces

Diag_Video_Stream_Error_Status [Prototype value = 0x00]

27.4.2 Hardware Interfaces

27.4.2.1 Input Interfaces

27.4.2.2 Output Interfaces

27.5 Impact analysis on the operating environment